

Quantum Tutorial

Build a Multi-tenant SaaS with Quantum

Version 1.4.2-SNAPSHOT, 2026-09-21T22:14:39Z

Table of Contents

1. 1. Project setup	2
1.1. Getting Started: Your First Quantum Application.....	2
1.1.1. Prerequisites	2
1.1.2. Project Setup.....	2
1.1.3. Configuration	3
1.1.4. Your First Model	3
1.1.5. Repository.....	4
1.1.6. REST Resource	4
1.1.7. Running the Application	5
1.1.8. Testing Your API	5
1.1.9. What You Get Automatically.....	5
1.1.10. Next Steps	6
1.1.11. Optional: Enable Ontology Modules	6
2. 2. Platform overview	8
2.1. Overview: Building SaaS with Quantum.....	8
2.1.1. SaaS and Multi-Tenancy First.....	8
3. 3. Domain modeling fundamentals	9
3.1. Modeling with Functional Areas, Domains, and Actions	9
3.1.1. Prefer Annotations for Functional Mapping (recommended).....	9
3.1.2. DataDomain on Models	11
3.1.3. Persistence Repositories	12
3.1.4. Exposing REST Resources	12
3.1.5. Lombok in Models	12
3.1.6. Validation with Jakarta Bean Validation.....	13
3.1.7. Jackson vs Jakarta Validation Annotations.....	13
3.2. Jackson ObjectMapper in Quarkus and in Quantum.....	14
3.3. Validation Lifecycle and Morphia Interceptors	15
3.4. Functional Area/Domain in RuleContext Permission Language.....	16
3.5. StateGraphs on Models	18
3.6. State Graph (DOT).....	19
3.7. CompletionTasks and CompletionTaskGroups	20
3.7.1. When to use them.....	20
3.7.2. Core entities	21
3.7.3. What the framework manages vs. what your application manages	22
3.7.4. State model.....	23
3.7.5. REST API surface.....	24
3.7.6. Request and response payloads.....	25
3.7.7. Completion event stream	27

3.7.8. Typical orchestration flow	27
3.7.9. Best practices	28
3.8. References and EntityReference	29
3.9. Tracking References with @TrackReferences and Delete Semantics	29
3.9.1. Ontologies in Quantum: Modeling Relationships That Are Resilient and Fast	30
3.9.2. Integrating Ontology with Morphia, Permissions, and Multi-tenancy	39
4. 4. Multi-tenant model and realm separation	43
4.1. Multi-Tenancy Models	43
4.1.1. DEDICATED_REALM: One Tenant per Database	43
4.1.2. POOLED_REALM: Many Tenants in One Database	44
4.1.3. Freemium and Trial Tenants	44
4.1.4. Tenant Onboarding and Access Invites	44
4.1.5. Choosing a Tenancy Model: Three Options	53
4.1.6. The GitHub-Like Identity Model, Mapped	54
4.1.7. User-Definable Facet Segmentation & Coordinate SPI in Multi-Tenant Environments	54
5. 5. Domain rule context	58
6. DomainContext, RuleContext, and DataDomain	59
6.1. DataDomain	59
6.2. DomainContext	59
6.3. RuleContext	59
6.4. End-to-End Flow	59
6.5. Resolvers and Variables in Rule Filters	60
6.5.1. Business Scenario: Multi-Tenant Franchise & Dynamic Partner Segmentation	60
6.5.2. How the Feature Supports the Business Solution	60
6.6. Concrete example: building and using a resolver	61
6.6.1. 1) Implement the SPI	61
6.6.2. 2) How RuleContext uses resolvers	62
6.6.3. 3) Author a rule that consumes the variable	63
6.6.4. 4) End-to-end behavior	63
6.6.5. String literals vs. typed values in resolver variables	63
6.6.6. Using AccessListResolver with Ontology (optional)	65
6.6.7. Admitted vocabulary for rule predicates	66
7. 6. Building RESTful CRUD APIs	67
7.1. REST: Find, Get, List, Save, Update, Delete	67
7.1.1. Base Concepts	67
7.1.2. Example Resource	67
7.1.3. Authorization Layers in REST CRUD	67
7.1.4. Querying	69
7.1.5. Responses and Schemas	69
7.1.6. Error Handling	69
7.1.7. Simple filters (equals)	70

7.1.8. Advanced filters: grouping and AND/OR/NOT	70
7.1.9. IN lists	71
7.1.10. Sorting	71
7.1.11. Projections	71
7.1.12. End-to-end examples	71
7.2. CSV Export and Import	71
7.2.1. Export: GET /csv	72
7.2.2. Import: POST /csv (multipart)	74
7.2.3. Import with preview sessions	75
7.2.4. Enhanced Import with ImportProfile	76
7.2.5. RowValueResolver: Custom Per-Row Logic	77
7.2.6. Dynamic Attribute Import	79
8. Authentication and Authorization	81
8.1. JWT Provider	81
8.2. Pluggable Authentication	81
8.3. Creating an Auth Plugin (using the Custom JWT provider as a reference)	81
8.4. AuthProvider interface (what a provider must implement)	82
8.5. UserManagement interface (operations your plugin must support)	82
8.6. Leveraging BaseAuthProvider in your plugin	83
8.7. Implementing your own provider	84
8.8. CredentialUserIdPassword model and DomainContext	84
8.9. Quarkus OIDC out-of-the-box and integrating with common IdPs	86
8.10. Authorization via RuleContext	88
8.10.1. Using Ontology Edges in List Endpoints (optional)	88
9. 7. Query language and filtering	91
9.1. Query Language Reference	91
9.1.1. Basic Syntax	91
9.1.2. Common Patterns	92
9.1.3. Querying Dynamic Attributes	95
9.2. Hands-on: Relationships and Ontology-aware queries	110
9.2.1. Hydrate related data with expand(path)	110
9.2.2. Ontology: constrain by relationships with hasEdge(predicate, dst)	111
9.2.3. Inspect the planner with QueryGateway	111
10. 8. Authentication, permissions, and annotations	112
11. Authentication and Authorization	113
11.1. JWT Provider	113
11.2. Pluggable Authentication	113
11.3. Creating an Auth Plugin (using the Custom JWT provider as a reference)	113
11.4. AuthProvider interface (what a provider must implement)	114
11.5. UserManagement interface (operations your plugin must support)	114
11.6. Leveraging BaseAuthProvider in your plugin	115

11.7. Implementing your own provider	116
11.8. CredentialUserIdPassword model and DomainContext	116
11.9. Quarkus OIDC out-of-the-box and integrating with common IdPs	119
11.10. Authorization via RuleContext	121
11.11. Generating JWT Tokens for API Access	121
11.11.1. Token Anatomy	121
11.11.2. Generating User Tokens via Login	122
11.11.3. Service Tokens for System-to-System Integration	122
11.11.4. How Tokens Scope to Security Policies	126
11.11.5. Generating RSA Key Pairs	128
11.11.6. Configuration Reference	129
12. Permissions: Rule Bases, SecurityURIHeader, and SecurityURIBody	130
12.1. Introduction: Layered Enforcement Overview	130
12.2. Key Concepts	132
12.3. Rule Structure (Illustrative)	133
12.4. Matching Algorithm	134
12.5. Priorities	135
12.6. Grant-based vs Deny-based Rule Sets	136
12.7. Row Filter Computation (Data Scoping)	137
12.7.1. The pipeline	137
12.7.2. What is scoped by default	137
12.7.3. Available substitution variables (static)	137
12.7.4. Dynamic access lists (the <code>AccessListResolver</code> SPI)	138
12.7.5. Summary	139
12.8. Feature Flags, Variants, and Target Rules	139
12.9. Scoped Action Enablement and Dependency Tracking	142
12.9.1. Core idea: the same scoped action tuple stays canonical	143
12.9.2. The three answers returned by the framework	143
12.9.3. Runtime models	144
12.9.4. REST API	145
12.9.5. Manifest-driven dependency relationships	146
12.9.6. Resolver SPI	147
12.9.7. Built-in OSS dependency types	147
12.9.8. Missing manifests and unsupported dependency types	148
12.9.9. Example contributor	148
12.9.10. Relationship to policies, feature flags, and UI actions	149
12.9.11. Enterprise extensions	150
12.9.12. Guidance	150
12.10. Multiple Matching RuleBases	150
12.11. Identity and Role Matching	151
12.11.1. How roles are defined for an identity (role sources and resolution)	151

12.12. Example Scenarios	156
12.13. Operational Tips	156
12.14. Field-Level Policy (Property Exclusions)	157
12.14.1. Declaring exclusions on a rule	157
12.14.2. Layered enforcement — one policy source, every egress guarded	157
12.14.3. Limitations (v1)	158
12.15. How UIActions and DefaultUIActions are calculated	158
12.16. How This Integrates End-to-End	160
12.17. Administering Policies via REST (PolicyResource)	160
12.17.1. Model shape (Policy)	160
12.17.2. How rule-contributed filters work (andFilterString, orFilterString, joinOp)	161
12.17.3. AccessListResolvers (SPI) for list-based access	164
12.17.4. PrincipalContextPropertiesResolver (SPI) for custom principal properties	166
12.17.5. Endpoints	172
12.17.6. Examples	173
12.17.7. How changes affect rule bases and enforcement	174
12.18. Realm override (X-Realm) and Impersonation (X-Impersonate)	175
12.18.1. What they do (at a glance)	175
12.18.2. How the headers integrate with permission evaluation	176
12.18.3. Required credential configuration (CredentialUserIdPassword)	176
12.18.4. End-to-end behavior from SecurityFilter (reference)	177
12.18.5. Practical differences and use cases	177
12.18.6. Examples	178
12.19. Data domain assignment on create: DomainContext and DataDomainPolicy	179
12.19.1. The problem this solves (and why it matters)	179
12.19.2. Key concepts recap: DomainContext and DataDomain	179
12.19.3. The default policy (do nothing and it works)	180
12.19.4. Policy scopes: principal-attached vs. global	180
12.19.5. The policy map and matching	180
12.19.6. How the resolver works	181
12.19.7. When would you want a non-global policy?	182
12.19.8. Relation to tenancy models	182
12.19.9. Authoring tips	182
12.19.10. API pointers	183
12.19.11. Ontology in Permission Rules (optional)	183
12.19.12. Business Scenarios & Policy Enforcement Solutions	184
12.19.13. Label resolution SPI and hasLabel() in rules	188
12.20. Script Helpers reference (when enabled)	190
12.20.1. Security Annotations: FunctionalMapping and FunctionalAction	192
13. 9. Seed packs: Declarative tenant seeding	198
14. Seed packs and declarative tenant seeding	199

14.1. Introduction	199
14.1.1. The problem	199
14.1.2. Why this needs to be solved	199
14.1.3. How seed packs solve it	199
14.2. Why seed packs?	200
14.3. High-level flow	200
14.4. Seed packs vs BootstrapPacks	200
14.5. Manifest quick reference	201
14.6. Realm filtering	202
14.6.1. Include realms (explicit list)	202
14.6.2. Exclude realms	202
14.6.3. Backward compatibility	203
14.7. Programmatic usage	203
14.8. Extensibility hooks	204
14.9. Operational tips	204
14.10. Primary scenarios	204
14.11. Explicit examples	205
14.11.1. Example 1: Minimal manifest and NDJSON	205
14.11.2. Example 2: Applying packs in code	205
14.11.3. Example 3: Using an archetype	206
14.11.4. Example 4: Exact version and includes in a manifest	206
14.12. Troubleshooting	206
14.13. How seeds are applied automatically at startup	207
14.13.1. Configuring which realms receive seeds on startup	207
14.14. Transforms in depth	209
14.14.1. Creating your own transforms (example: DropIfTransform)	212
14.14.2. Custom variable resolvers for stringInterpolation	214
14.14.3. Seed record listeners (reacting to seed data)	216
14.15. Test walkthrough: SeedLoaderIntegrationTest	219
14.16. Archetypes explained	220
14.16.1. Example A: Define and apply an archetype in the same pack	220
14.16.2. Example B: Cross-pack archetype in a dedicated "editions" pack	221
14.16.3. Resolution and ordering details	221
14.16.4. Interaction with ApplySeedPacksChangeSet	221
14.16.5. Tenant provisioning with archetypes	222
14.17. REST API for seed packs	222
14.18. Using MorphiaSeedRepository (Morphia-backed seeding)	224
14.19. Dataset URLs and routing (file:// and s3://)	225
14.20. S3 Seed Source (optional module)	226
14.20.1. When to use S3 vs. filesystem	226
14.20.2. S3 layout conventions	226

14.20.3. Configuration and credentials	227
14.20.4. Usage examples	228
14.20.5. Defining datasets in the manifest (S3)	229
14.20.6. Troubleshooting	229
15. 10. Migrations	231
16. Database Migrations and Index Management	232
16.1. Overview	232
16.2. Semantic Versioning	232
16.3. Configuration	233
16.4. How change sets are discovered and executed	233
16.5. Authoring a change set	234
16.6. Example change sets in the framework	235
16.7. REST APIs to trigger migrations (MigrationResource)	235
16.8. Per-entity index management (BaseResource)	236
16.9. Global index management (MigrationService)	237
16.10. Validating versions at startup	237
16.11. Notes and best practices	237
16.11.1. Checksum vs. from/to versions	237
17. 11. Testing	239
17.1. Testing in Quantum: Security Contexts, Repos, and REST APIs	239
17.1.1. Testing Framework	239
17.1.2. Prerequisites and glossary	239
17.1.3. Pattern 1 — Extend BaseRepoTest	239
17.1.4. Pattern 2 — Try-with-resources SecuritySession	240
17.1.5. Pattern 3 — Scoped-call wrapper (ScopedCallScope)	241
17.1.6. Pattern 4 — @TestSecurity annotation (Quarkus)	241
17.1.7. Testing REST APIs vs. Repository Logic	243
17.1.8. Useful Quarkus Test features	243
17.1.9. Real-world tips	243
17.1.10. Summary	243
18. 12. Next steps	245

This tutorial takes you end-to-end through building a small SaaS application on the Quantum framework. It pulls in the core concepts and how-to content from the main index in a progressive, hands-on flow so you can learn by doing.

Audience: Mid-level Java developers building multi-tenant SaaS on Quarkus and MongoDB.

Prerequisites: JDK 17+, Maven, Docker (for local MongoDB), and basic familiarity with Quarkus.

Artifacts: The docs can be built as HTML/PDF via Maven; the code snippets compile against the Quantum modules published in this repo.

What you'll build: A small, tenant-aware service with domain models, REST CRUD, flexible queries, security, migrations, and seed packs for baseline data.

Each section starts with:

- Problem: What we are solving
- Why for SaaS: Why this matters in multi-tenant systems
- How Quantum helps: Concepts and features that address the problem
- Walkthrough: Code and configuration that build on prior sections

Throughout, we reuse and expand the same example to reinforce learning. The reference guide links back to these sections for deeper context.

Chapter 1. 1. Project setup

Problem: Getting a runnable project scaffolded with Quantum and Quarkus.

Why for SaaS: Consistent project structure and dependencies are critical to scale teams and environments.

How Quantum helps: Provides opinionated dependencies, conventions, and ready-made modules for multi-tenancy, security, persistence, and seeding.

Walkthrough: Follow the steps below to create and run your first service.

1.1. Getting Started: Your First Quantum Application

This section walks through creating a simple multi-tenant Product catalog to demonstrate core Quantum concepts.

1.1.1. Prerequisites

- Java 17+
- Maven 3.8+
- MongoDB (local or cloud)
- Basic Quarkus knowledge (see [Quarkus Foundation](#))

1.1.2. Project Setup

Create a new Quarkus project with Quantum dependencies:

```
mvn io.quarkus:quarkus-maven-plugin:create \
  -DprojectGroupId=com.example \
  -DprojectArtifactId=product-catalog \
  -DclassName="com.example.ProductResource" \
  -Dpath="/products"
```

Add Quantum dependencies to `pom.xml`:

```
<dependency>
  <groupId>com.e2eq.framework</groupId>
  <artifactId>quantum-framework</artifactId>
  <version>${quantum.version}</version>
</dependency>
```

`quantum-framework` is the compatibility bundle and includes the framework's server-side REST adapters. For a service with an explicit endpoint boundary, select the narrower modules instead. For example, add `com.end2endlogic:quantum-query-rest` only when that backend is intended to host

the existing `/api/query` API. SDK-only consumers should omit REST and repository modules so they do not expose or persist server-owned APIs.

1.1.3. Configuration

Create `.env` file (copy from template):

```
MONGODB_USERNAME=your-username
MONGODB_PASSWORD=your-password
MONGODB_DATABASE=product-catalog
MONGODB_HOST=localhost:27017
JWT_SECRET=your-secret-key
```

Basic `application.properties`:

```
# MongoDB
quarkus.mongodb.connection-
string=${MONGODB_CONNECTION_STRING:mongodb://localhost:27017}
quarkus.mongodb.database=${MONGODB_DATABASE:product-catalog}

# JWT Authentication
auth.provider=custom
auth.jwt.secret=${JWT_SECRET:change-me-in-production}
auth.jwt.expiration=60

# CORS for development
quarkus.http.cors=true
quarkus.http.cors.origins=http://localhost:3000
```

1.1.4. Your First Model

Create a Product model with multi-tenancy built-in:

```
package com.example.model;

import com.e2eq.framework.annotations.FunctionalMapping;
import com.e2eq.framework.model.persistent.base.BaseModel;
import dev.morphia.annotations.Entity;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.NoArgsConstructor;
import lombok.experimental.SuperBuilder;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import java.math.BigDecimal;

@Entity
```

```

@Data
@NoArgsConstructor
@SuperBuilder
@EqualsAndHashCode(callSuper = true)
@FunctionalMapping(area = "catalog", domain = "product")
public class Product extends BaseModel {

    @NotBlank
    @Size(max = 50)
    private String sku;

    @NotBlank
    @Size(max = 200)
    private String name;

    private String description;
    private BigDecimal price;
    private boolean active = true;
}

```

Key points: - Extends `BaseModel` for automatic `DataDomain`, audit fields, and ID management - `@FunctionalMapping` declares this model's business area and domain for security rules - Standard Jakarta validation annotations - Lombok reduces boilerplate

1.1.5. Repository

Create a repository interface:

```

package com.example.repository;

import com.e2eq.framework.model.persistent.morphia.MorphiaRepo;
import com.example.model.Product;

public interface ProductRepo extends MorphiaRepo<Product> {
    // Custom queries can be added here
}

```

1.1.6. REST Resource

Create a REST endpoint:

```

package com.example.resource;

import com.e2eq.framework.rest.resources.BaseResource;
import com.example.model.Product;
import com.example.repository.ProductRepo;
import jakarta.ws.rs.Path;

```

```
@Path("/products")
public class ProductResource extends BaseResource<Product, ProductRepo> {
    // Inherits all CRUD endpoints: GET, POST, PUT, DELETE
    // GET /products/list - paginated list with filtering
    // GET /products/id/{id} - get by ID
    // POST /products - create new product
    // PUT /products/set?id={id}&pairs=field:value - update fields
    // DELETE /products/id/{id} - delete product
}
```

1.1.7. Running the Application

Start your application:

```
./mvnw quarkus:dev
```

The application provides: - Swagger UI at <http://localhost:8080/q/swagger-ui/> - Dev UI at <http://localhost:8080/q/dev/>

1.1.8. Testing Your API

Create a product:

```
curl -X POST http://localhost:8080/products \
-H "Content-Type: application/json" \
-d '{
    "sku": "WIDGET-001",
    "name": "Super Widget",
    "description": "The best widget ever",
    "price": 29.99,
    "active": true
}'
```

List products:

```
curl "http://localhost:8080/products/list?limit=10&sort=+name"
```

Filter products:

```
curl "http://localhost:8080/products/list?filter=active:true&&price:>##20"
```

1.1.9. What You Get Automatically

With this minimal setup, Quantum provides:

Multi-tenancy: Each product is automatically tagged with the creator's DataDomain (tenant, org, owner)

Security: DataDomain filtering ensures users only see their own data by default

Validation: Jakarta Bean Validation runs before persistence

Audit Trail: Automatic createdBy, createdAt, lastUpdatedBy, lastUpdatedAt fields

Consistent APIs: Standard REST patterns across all resources

Query Language: Powerful filtering with the ANTLR-based query syntax

OpenAPI: Automatic API documentation

1.1.10. Next Steps

- Add authentication: [Authentication Guide](#)
- Create sharing rules: [Permissions Guide](#)
- Learn the query language: [Query Language](#)
- See real-world example: [Supply Chain Tutorial](#)

1.1.11. Optional: Enable Ontology Modules

You can adopt ontology incrementally. If you don't enable it, everything works as before.

Quick checklist

1) Add dependencies (app-level) - quantum-ontology-core, quantum-ontology-mongo, quantum-ontology-policy-bridge - Do not add quantum-ontology-rest unless this application intentionally hosts the ontology control-plane API. Normal consumers call quantum-ontology-service through its generated SDK.

2) Turn it on via config

```
e2eq.ontology.enabled=true
```

3) Provide an ontology registry (TBox) - Start with an in-memory registry (recommended initially). See [Ontologies in Quantum](#).

4) Wire data and indexes - Inject EdgeDao as a CDI bean (@Inject); indexes are ensured automatically at startup. - Indices: (tenantId, p, dst) and (tenantId, src, p)

5) Materialize edges on entity changes - Use OntologyMaterializer when sources or intermediates change (e.g., Order/Customer/Org/Address/Shipment in the e-commerce example).

6) Use edges in queries/policies - Wrap BSON via ListQueryRewriter or constrain by _id sets. See [Integrating Ontology](#).

Notes

- Keep it optional: only wire beans and create collections when `e2eq.ontology.enabled=true`.
- Multi-tenant: always pass `tenantId` from `RuleContext`.

Chapter 2. 2. Platform overview

Problem: Understanding the moving parts and how they fit together.

Why for SaaS: You'll be composing features (tenancy, security, APIs, data) across many modules and services.

How Quantum helps: A cohesive model with clear boundaries and integration points.

Walkthrough: Read the overview and keep it handy as you progress.

2.1. Overview: Building SaaS with Quantum

Quantum is a Quarkus-based framework that accelerates building multi-tenant SaaS platforms on MongoDB. It provides:

- Multi-tenancy primitives for tenant creation, isolation, and data sharing
- A domain-first programming model with Functional Areas, Functional Domains, and Actions
- Data security and contextual evaluation via DataDomain, DomainContext, and RuleContext
- Consistent REST resources for find/get/list/save/update/delete operations
- Pluggable authentication with a provided JWT module and extension points

This guide targets mid-level Java developers and follows a structure similar to Spring's reference docs. Use Maven to generate HTML/PDF: see docs module README for commands.

2.1.1. SaaS and Multi-Tenancy First

SaaS solutions require:

- Onboarding automation: programmatic tenant creation, freemium/trial flows
- Isolation with selective sharing
- Policy-driven access that adapts to user, org, tenant, and action
- Operational efficiency (observability, cost control, upgradeability)

Quantum's building blocks address these needs out-of-the-box while remaining flexible to fit your architecture.

Chapter 3. 3. Domain modeling fundamentals

Problem: Modeling your domain effectively for persistence and API exposure.

Why for SaaS: Cross-tenant scale and isolation drive modeling choices (IDs, natural keys, versioning, ownership).

How Quantum helps: Provides base models, annotations, and repository patterns to model tenant-safe entities.

Walkthrough: Build a minimal domain model and persist it.

3.1. Modeling with Functional Areas, Domains, and Actions

Quantum organizes your system around three core constructs:

- Functional Area: A broad capability area (e.g., Identity, Catalog, Orders, Collaboration).
- Functional Domain: A cohesive sub-area within an area (e.g., in Collaboration: Partners, Shipments, Tasks).
- Actions: The set of operations applicable to a domain (CREATE, UPDATE, VIEW, DELETE, ARCHIVE, plus domain-specific actions).

These constructs allow:

- Fine-grained sharing: Point specific functional areas to shared databases while others remain strictly segmented.
- Policy composition: Apply RuleContext decisions at the level of area/domain/action.

3.1.1. Prefer Annotations for Functional Mapping (recommended)

Starting with this version, you should use annotations to declare a model or resource's Functional Area/Domain and the Action being performed. The legacy `bmFunctionalArea()` and `bmFunctionalDomain()` methods are still supported for backward compatibility in this release, but they will be phased out soon.

- Class-level mapping: use `@FunctionalMapping(area = "<area>", domain = "<domain>")` on your model or resource class.
- Method-level action: use `@FunctionalAction("<ACTION>")` on your JAX-RS resource methods when the action is not implied by the HTTP verb.

Example: model annotated with `FunctionalMapping`

```
import dev.morphia.annotations.Entity;
```

```

import lombok.*;
import lombok.experimental.SuperBuilder;
import com.e2eq.framework.model.persistent.base.BaseModel;
import com.e2eq.framework.annotations.FunctionalMapping;

@Entity
@Data
@NoArgsConstructor
@SuperBuilder
@EqualsAndHashCode(callSuper = true)
@FunctionalMapping(area = "catalog", domain = "product")
public class Product extends BaseModel {
    private String sku;
    private String name;
    // No need to override bmFunctionalArea/bmFunctionalDomain when using
    @FunctionalMapping
}

```

Example: resource method annotated with FunctionalAction

```

import jakarta.ws.rs.*;
import jakarta.ws.rs.core.MediaType;
import com.e2eq.framework.annotations.FunctionalAction;

@Path("/products")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class ProductResource {

    // Action will default from HTTP verb (POST -> CREATE), but you can be explicit:
    @POST
    @FunctionalAction("CREATE")
    public Product create(Product payload) { /* ... */ return payload; }

    // GET will infer VIEW automatically when building the ResourceContext
    @GET
    @Path("/{id}")
    public Product get(@PathParam("id") String id) { /* ... */ return new Product(); }
}

```

How the framework uses these annotations

- **SecurityFilter:** If the matched resource class has `@FunctionalMapping`, it uses area/domain from the annotation. If the method has `@FunctionalAction`, it uses that value; otherwise, it infers the action from the HTTP method (GET=VIEW, POST=CREATE, PUT/PATCH=UPDATE, DELETE=DELETE). If annotations are absent, it falls back to the existing path- and convention-based logic.
- **MorphiaRepo.fillUIActions:** If the model class has `@FunctionalMapping`, its area/domain are

used to resolve allowed UI actions; otherwise, it falls back to the legacy `bmFunctionalArea()/bmFunctionalDomain()` methods.

- `PermissionResource`: When listing functional domains, it prefers `@FunctionalMapping` on entity classes and falls back to `bmFunctionalArea()/bmFunctionalDomain()` when missing.

Migration notes

- Preferred: add `@FunctionalMapping` to each model class (or resource class) and remove the `bmFunctionalArea()/bmFunctionalDomain` overrides.
- Transitional: you can keep the legacy methods; they will be used only if the annotation is not present.
- Future: `bmFunctionalArea` and `bmFunctionalDomain` will be removed in a future release; plan to migrate now.

See also: [Security Annotations: FunctionalMapping and FunctionalAction](#)

3.1.2. DataDomain on Models

All persisted models carry `DataDomain` (`tenantId`, `orgRefName`, `ownerId`, etc.) for rule-based filtering and cross-tenant sharing.

Example model:

```
import dev.morphia.annotations.Entity;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.NoArgsConstructor;
import lombok.experimental.SuperBuilder;
import com.e2eq.framework.model.persistent.base.BaseModel;

@Entity
@Data
@NoArgsConstructor
@SuperBuilder
@EqualsAndHashCode(callSuper = true)
public class Product extends BaseModel {
    private String sku;
    private String name;

    @Override
    public String bmFunctionalArea() { return "Catalog"; }

    @Override
    public String bmFunctionalDomain() { return "Product"; }
}
```

3.1.3. Persistence Repositories

Define a repository to persist and query your model. With Morphia:

```
import com.e2eq.framework.model.persistent.morphia.MorphiaRepo;

public interface ProductRepo extends MorphiaRepo<Product> {
    // custom queries can be added here
}
```

3.1.4. Exposing REST Resources

Expose consistent CRUD endpoints by extending BaseResource.

```
import com.e2eq.framework.rest.resources.BaseResource;
import jakarta.ws.rs.Path;

@Path("/products")
public class ProductResource extends BaseResource<Product, ProductRepo> {
    // Inherit find, get, list, save, update, delete endpoints
}
```

With this minimal setup, you get standard REST APIs guarded by RuleContext/DataDomain and enriched with UIAction metadata.

3.1.5. Lombok in Models

Lombok reduces boilerplate in Quantum models and supports inheritance-friendly builders.

Common annotations you will see:

- `@Data`: Generates getters, setters, toString, equals, and hashCode.
- `@NoArgsConstructor`: Required by frameworks that need a no-arg constructor (e.g., Jackson, Morphia).
- `@EqualsAndHashCode(callSuper = true)`: Includes superclass fields in equality and hash.
- `@SuperBuilder`: Provides a builder that cooperates with parent classes (useful for BaseModel subclasses).

Example:

```
@Data
@NoArgsConstructor
@SuperBuilder
@EqualsAndHashCode(callSuper = true)
public class Product extends BaseModel {
    private String sku;
```

```
private String name;  
}
```

Notes: - Prefer `@SuperBuilder` over `@Builder` when extending `BaseModel/UnversionedBaseModel`. - Keep `equals/hashCode` stable for collections and caches; include `callSuper` when needed.

3.1.6. Validation with Jakarta Bean Validation

Quantum uses Jakarta Bean Validation to enforce invariants on models at persist time (and optionally at REST boundaries).

Typical annotations:

- `@Size(min=3)`: String/collection length constraints.
- `@Valid`: Cascade validation to nested objects (e.g., `DataDomain` on models).
- `@NotNull`, `@Email`, `@Pattern`, etc., as needed.

Where validation runs:

- Repository layer via Morphia `ValidationInterceptor` (`prePersist`):
- Executes `validator.validate(entity)` before the document is written.
- If there are violations and the entity does not implement `InvalidSavable` with `canSaveInvalid=true`, an `E2eqValidationException` is thrown.
- If `DataDomain` is null and `SecurityContext` has a principal, `ValidationInterceptor` will default the `DataDomain` from the principal context.
- Optionally at REST boundaries: You may also annotate resource DTOs/parameters with Jakarta validation; Quarkus can validate them before the method executes.

3.1.7. Jackson vs Jakarta Validation Annotations

These two families of annotations serve different purposes and complement each other:

- Jackson annotations (`com.fasterxml.jackson.annotation.*`) control JSON serialization/deserialization.
- Examples: `@JsonIgnore`, `@JsonIgnoreProperties`, `@JsonProperty`, `@JsonInclude`.
- They do not enforce business constraints; they affect how JSON is produced/consumed.
- Jakarta Validation annotations (`jakarta.validation.*`) declare constraints that are evaluated at runtime.
- Examples: `@NotNull`, `@Size`, `@Valid`, `@Pattern`.

Correspondence and interplay:

- Use Jackson to hide or rename fields in API responses/requests (e.g., `@JsonIgnore` on transient/calculated fields such as `UIActionList`).
- Use Jakarta Validation to ensure incoming/outgoing models satisfy required constraints;

ValidationInterceptor runs before persistence to enforce them.

- It's common to annotate the same field with both families when you both constrain values and want specific JSON behavior.

3.2. Jackson ObjectMapper in Quarkus and in Quantum

How Quarkus creates ObjectMapper:

- Quarkus produces a CDI-managed ObjectMapper. You can customize it by providing a bean that implements `io.quarkus.jackson.ObjectMapperCustomizer`.
- You can also tweak common features via `application.properties` using `quarkus.jackson.*` properties.

Quantum defaults:

- The framework provides a `QuarkusJacksonCustomizer` that:
- Sets `DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES = true` (reject unknown JSON fields).
- Registers custom serializers/deserializers for `org.bson.types.ObjectId` so it can be used as `String` in APIs.

Snippet from the framework:

```
@Singleton
public class QuarkusJacksonCustomizer implements ObjectMapperCustomizer {
    @Override
    public void customize(ObjectMapper objectMapper) {
        objectMapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, true);
        SimpleModule module = new SimpleModule();
        module.addSerializer(ObjectId.class, new ObjectIdJsonSerializer());
        module.addDeserializer(ObjectId.class, new ObjectIdJsonDeserializer());
        objectMapper.registerModule(module);
    }
}
```

Customize in your app:

- Add another `ObjectMapperCustomizer` bean (order is not guaranteed; make changes idempotent):

```
@Singleton
public class MyJacksonCustomizer implements ObjectMapperCustomizer {
    @Override
    public void customize(ObjectMapper mapper) {
        mapper.findAndRegisterModules();
        mapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
        mapper.setSerializationInclusion(JsonInclude.Include.NON_NULL);
    }
}
```

```
}
}
```

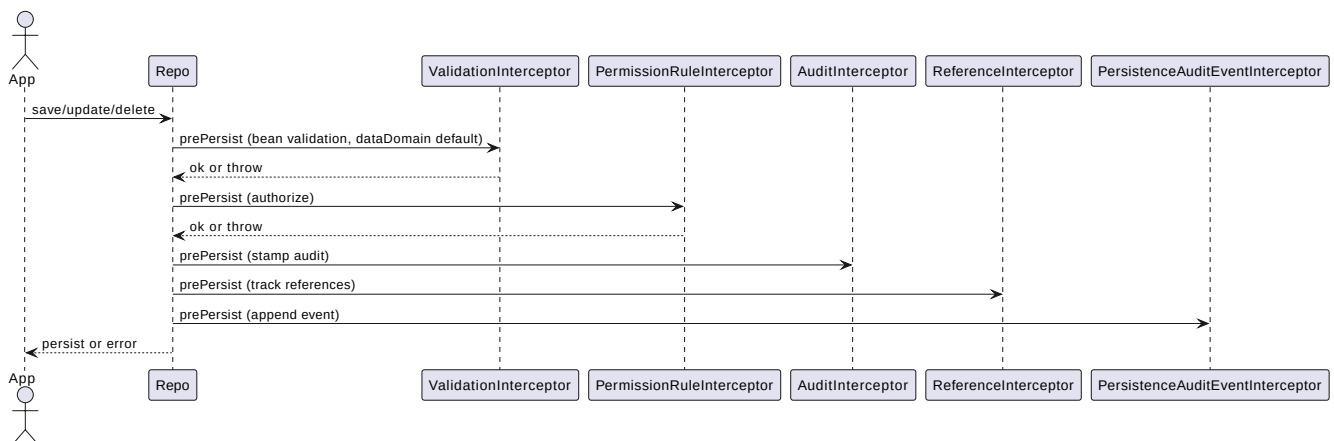
- Or set properties in application.properties:

```
# Fail if extraneous fields are present
quarkus.jackson.fail-on-unknown-properties=true
# Example date format and inclusion
quarkus.jackson.write-dates-as-timestamps=false
quarkus.jackson.serialization-inclusion=NON_NULL
```

When to adjust:

- Relax fail-on-unknown only for backward-compatibility scenarios; strictness helps catch client mistakes.
- Register modules (JavaTime, etc.) if your models include those types.

3.3. Validation Lifecycle and Morphia Interceptors



Morphia interceptors enhance and enforce behavior during persistence. Quantum registers the following for each realm-specific datastore:

Order	of registration	(see MorphiaDataStore):	1)	ValidationInterceptor	2)
PermissionRuleInterceptor			3)	AuditInterceptor	4)
PersistenceAuditEventInterceptor			4)	ReferenceInterceptor	5)

High-level responsibilities:

- ValidationInterceptor (prePersist):
- Defaults DataDomain from SecurityContext if missing.
- Runs bean validation and throws E2eqValidationException on violations unless the entity supports saving invalid states (InvalidSavable).
- PermissionRuleInterceptor (prePersist):
- Evaluates RuleContext with PrincipalContext and ResourceContext from SecurityContext.

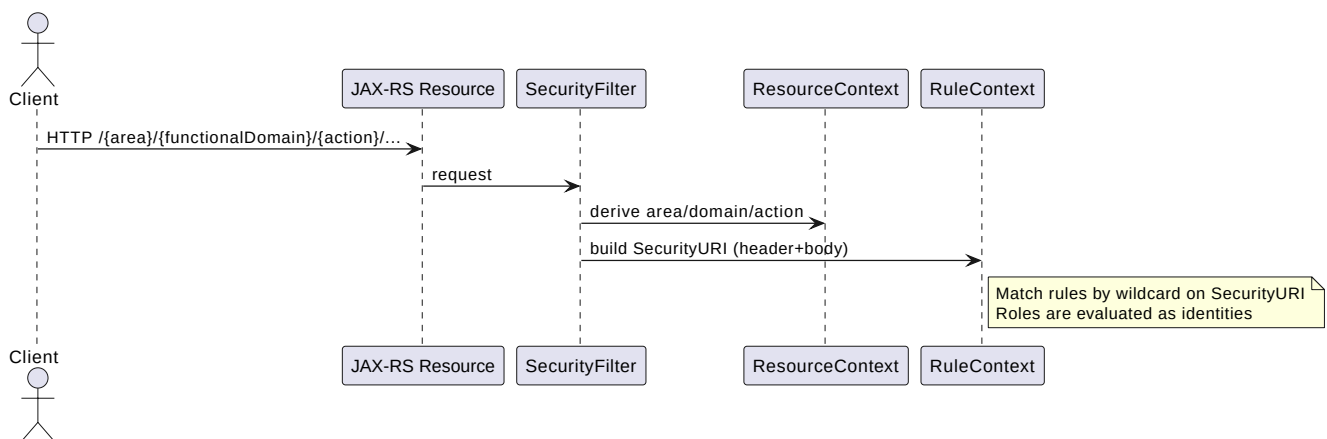
- Throws `SecurityCheckException` if the rule decision is not `ALLOW` (enforcing write permissions for save/update/delete).
- `AuditInterceptor` (`prePersist`):
 - Sets `AuditInfo` on creation and updates `lastUpdate` fields on modification; captures impersonation details if present.
- `ReferenceInterceptor` (`prePersist`):
 - For `@Reference` fields annotated with `@TrackReferences`, maintains back-references on the parent entities via `ReferenceEntry` and persists the parent when needed.
- `PersistenceAuditEventInterceptor` (`prePersist` when `@AuditPersistence` is present):
 - Appends a `PersistentEvent` with type `PERSIST`, date, `userId`, and version to the model's `persistentEvents` before saving.

When does validation occur?

- On every save/update path that hits persistence, `prePersist` triggers validation (and permission/audit/reference processing) before the document is written to MongoDB, guaranteeing constraints and policies are enforced consistently across all repositories.

3.4. Functional Area/Domain in RuleContext

Permission Language



Models express their placement in the business model via: - `bmFunctionalArea()`: returns a broad capability area (e.g., Catalog, Collaboration, Identity) - `bmFunctionalDomain()`: returns the specific domain within that area (e.g., Product, Shipment, Partner)

How these map into authorization and rules:

- `ResourceContext/DomainContext`: When a request operates on a model, the framework derives the functional area and domain from the model type (or resource) and places them on the current context alongside the action (`CREATE`, `UPDATE`, `VIEW`, `DELETE`, `ARCHIVE`). `RuleContext` consumes these to evaluate policies.
- `Permission language (SecurityURI-based matching)`: The framework derives area and functionalDomain from REST path segments using the convention: `/[area]/[functionalDomain]/[action]/...`. These values, together with action and identity, form

the SecurityURI (header + body) used by the rule engine. Rules match on SecurityURI fields using wildcard string comparison; there is no HTTP method/URL pattern matching.

- Permission language (query variables): The ANTLR-based query language exposes variables that can be referenced in filters:
- `${area}` corresponds to `bmFunctionalArea()`
- `${functionalDomain}` corresponds to `bmFunctionalDomain()` These can be used to author reusable filters or to record audit decisions by area/domain.
- Repository filters: `RuleContext` can contribute additional predicates that are area/domain-specific, enabling fine-grained sharing. For example, a shared Catalog area may allow cross-tenant VIEW, while a `Collaboration.Shipment` domain remains tenant-strict.

Examples

1) SecurityURI-based rule matching (Permissions)

```
- name: allow-catalog-product-reads
description: Allow USER and ADMIN to view products in the Catalog area
securityURI:
  header:
    identity: USER          # role treated as identity; author a second rule for
ADMIN if needed
    area: Catalog
    functionalDomain: Product
    action: view
  body:
    realm: system-com
    accountNumber: '*'
    tenantId: '*'
    dataSegment: '*'
    ownerId: '*'
    resourceId: '*'
effect: ALLOW
priority: 300
finalRule: false
```

2) Query variable usage (Filters)

You can reference the active area/domain in filter expressions (e.g., for auditing or conditional branching in custom rule evaluators):

```
# Constrain reads differently when operating in the Catalog area
(${area}:"Catalog" && dataDomain.orgRefName:"PUBLIC") ||
(${area}:"!Catalog" && dataDomain.tenantId:${pTenantId})
```

3) Model-driven mapping

Given a model like:

```
@Override public String bmFunctionalArea() { return "Collaboration"; }
@Override public String bmFunctionalDomain(){ return "Shipment"; }
```

- Incoming REST requests that operate on Shipment resources set area=Collaboration and functionalDomain=Shipment in the ResourceContext.
- RuleContext evaluates policies considering action + area + domain, e.g., deny cross-tenant UPDATE in Collaboration.Shipment, but allow cross-tenant VIEW in Collaboration.Partner if marked shared.

Notes

- Path convention: Use leading segments /{area}/{functionalDomain}/{action}/... so the framework can derive ResourceContext reliably. Extra segments after the first three are allowed; only the first three are used to compute area, domain, and action.
- Nonconformant paths: If the path has fewer than three segments, the framework sets an anonymous/default ResourceContext. In practice, rules will typically evaluate to DENY unless there is an explicit allowance for anonymous contexts.
- See also: the Permissions section for rule-base matching and priorities, and the DomainContext/RuleContext section for end-to-end flow.

3.5. StateGraphs on Models

StateGraphs let you restrict valid values and transitions of String state fields. They are declared on model fields with @StateGraph and enforced during save/update when the model class is annotated with @Stateful.

Key pieces: - @StateGraph(graphName="..."): mark a String field as governed by a named state graph. - @Stateful: mark the entity type as participating in state validation. - StateGraphManager: runtime registry that holds graphs and validates transitions. - StringState and StateNode: define the graph (states, initial/final flags, transitions).

Defining a state graph at startup:

```
@Startup
@ApplicationScoped
public class StateGraphInitializer {
    @Inject StateGraphManager stateGraphManager;
    @PostConstruct void init() {
        StringState order = new StringState();
        order.setFieldName("orderStringState");

        Map<String, StateNode> states = new HashMap<>();
        states.put("PENDING", StateNode.builder().state("PENDING").initialState(true)
            .finalState(false).build());
        states.put("PROCESSING", StateNode.builder().state("PROCESSING").initialState
            (false).finalState(false).build());
        states.put("SHIPPED", StateNode.builder().state("SHIPPED").initialState(false)
```

```

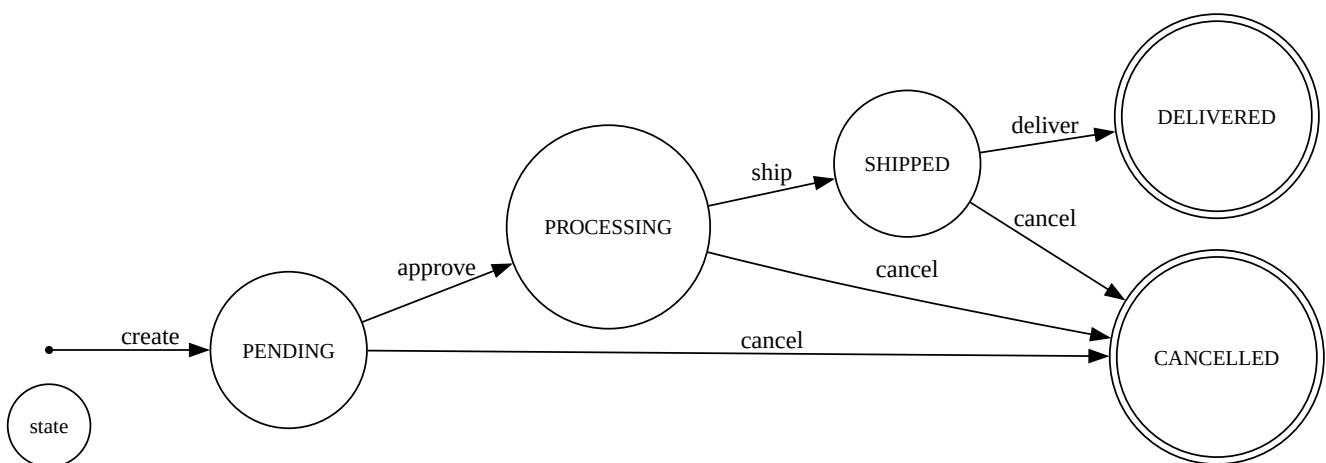
.finalState(false).build());
    states.put("DELIVERED", StateNode.builder().state("DELIVERED").initialState(
false).finalState(true).build());
    states.put("CANCELLED", StateNode.builder().state("CANCELLED").initialState(
false).finalState(true).build());
    order.setStates(states);

    Map<String, List<StateNode>> transitions = new HashMap<>();
    transitions.put("PENDING", List.of(states.get("PROCESSING"), states.get(
"CANCELLED")));
    transitions.put("PROCESSING", List.of(states.get("SHIPPED"), states.get("
CANCELLED")));
    transitions.put("SHIPPED", List.of(states.get("DELIVERED"), states.get(
"CANCELLED")));
    transitions.put("DELIVERED", null);
    transitions.put("CANCELLED", null);
    order.setTransitions(transitions);

    stateGraphManager.defineStateGraph(order);
}
}

```

3.6. State Graph (DOT)



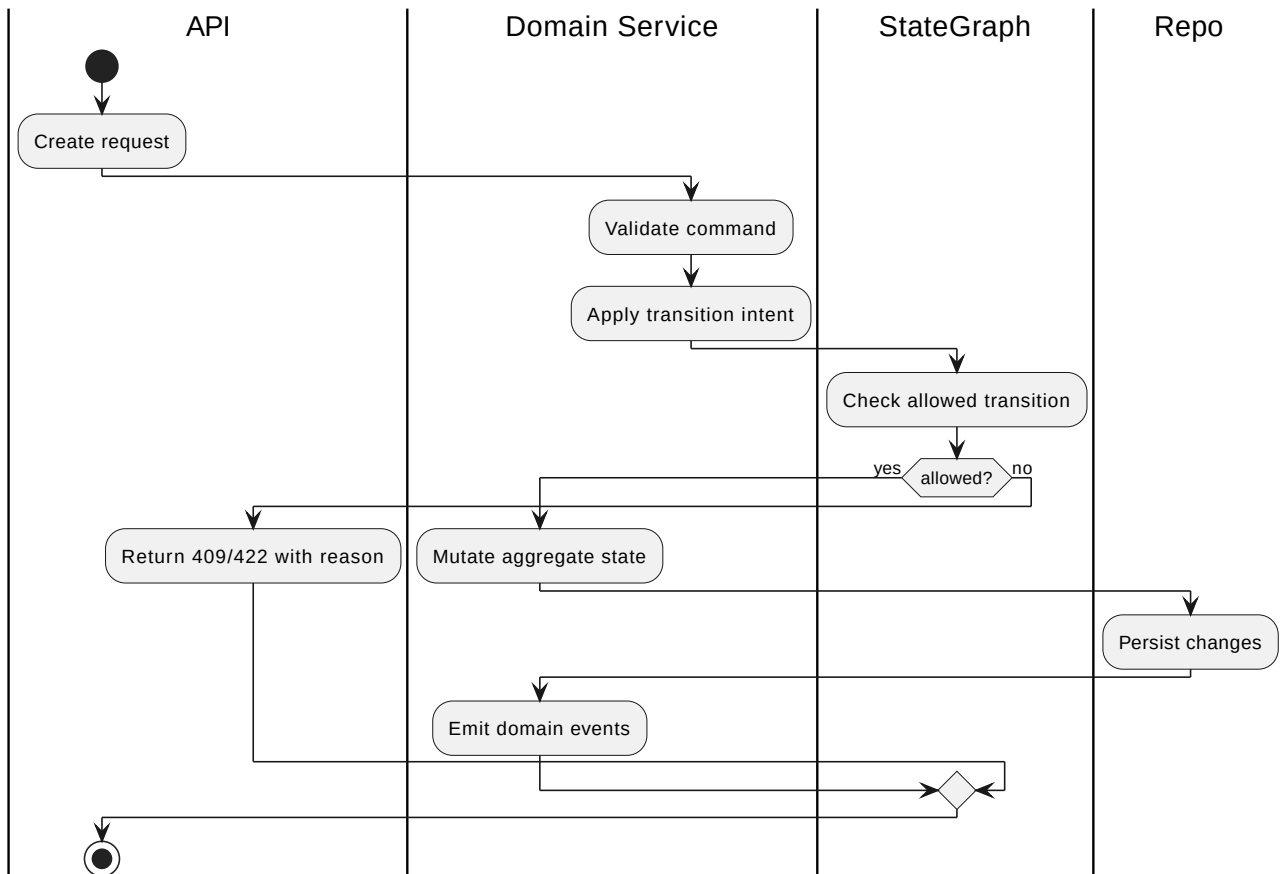
Using the graph in a model:

```

@Stateful
@Entity
@EqualsAndHashCode(callSuper = true)
public class Order extends BaseModel {
    @StateGraph(graphName = "orderStringState")
    private String status;

    @Override public String bmFunctionalArea() { return "Orders"; }
    @Override public String bmFunctionalDomain(){ return "Order"; }
}

```



How it affects save/update:

- On create: `validateInitialStates` ensures the field value is one of the configured initial states. Otherwise, `InvalidStateTransitionException` is thrown.
- On update: `validateStateTransitions` checks each `@StateGraph` field's old → new transition against the graph via `StateGraphManager.validateTransition()`. If invalid, save/update fails with `InvalidStateTransitionException`. This applies to full-entity saves and to partial updates via `repo.update(...pairs)` on that field.
- Utilities: `StateGraphManager.getNextPossibleStates(graphName, current)` and `printStateGraph(...)` can aid UIs.

3.7. CompletionTasks and CompletionTaskGroups

`CompletionTasks` and `CompletionTaskGroups` provide a small persistence-backed contract for tracking work that finishes later. They are useful when a request starts work immediately but the actual completion happens in a worker, another service, or a human-in-the-loop integration.

This section documents the concrete structures and runtime behavior, because other guides reference "completion tasks" as a pattern but do not define the payloads and lifecycle precisely.

3.7.1. When to use them

Use `CompletionTasks` when you need:

- A durable record of work that may outlive the initiating HTTP request

- A simple per-item status model with timestamps and outcome text
- A way to group related tasks under one business operation
- A lightweight event stream so clients can react when a task completes

Do not treat them as a workflow engine. The framework does not infer dependencies, retries, compensation rules, or aggregate group state transitions for you.

3.7.2. Core entities

Both entities extend `BaseModel`, so they inherit the common Quantum fields such as `id`, `refName`, `displayName`, `dataDomain`, `activeStatus`, `auditInfo`, and `version`.

The inherited fields matter here in two ways:

- `id` is the record identifier used by the REST endpoints and emitted in completion events.
- `refName`, `displayName`, and `dataDomain` are still available for your application-level correlation and multi-tenant scoping, even though they are not task-specific.

CompletionTask

```
@Entity("completionTask")
public class CompletionTask extends BaseModel {
    public enum Status { PENDING, RUNNING, SUCCESS, FAILED }

    @Reference
    CompletionTaskGroup group;

    String details;
    Status status;
    Date createdAt;
    Date completedDate;
    String result;
}
```

Field	Meaning	Set by
<code>group</code>	Optional Morphia reference to the parent <code>CompletionTaskGroup</code>	Client on create, or the <code>create/{groupId}</code> endpoint
<code>details</code>	Human-readable explanation of the unit of work, usually "what this task is doing"	Client/application
<code>status</code>	Task execution state: <code>PENDING</code> , <code>RUNNING</code> , <code>SUCCESS</code> , <code>FAILED</code>	Server sets <code>PENDING</code> on create; application or API updates afterward

Field	Meaning	Set by
<code>createdDate</code>	Timestamp when the task record was created	Server
<code>completedDate</code>	Timestamp written when the task is completed through the completion endpoint	Server
<code>result</code>	Outcome text, response summary, or error message	Application or completion endpoint caller

CompletionTaskGroup

```
@Entity("completionTaskGroup")
public class CompletionTaskGroup extends BaseModel {
    public enum Status { NEW, RUNNING, COMPLETE }

    String description;
    Status status;
    Date createdDate;
    Date completedDate;
}
```

Field	Meaning	Set by
<code>description</code>	Human-readable summary of the batch, workflow, or request that owns the tasks	Client/application
<code>status</code>	Coarse aggregate state: <code>NEW</code> , <code>RUNNING</code> , <code>COMPLETE</code>	Server sets <code>NEW</code> on create; application owns later transitions
<code>createdDate</code>	Timestamp when the group was created	Server
<code>completedDate</code>	Timestamp for when your application considers the group finished	Application

3.7.3. What the framework manages vs. what your application manages

The code-level behavior is intentionally small:

- `CompletionTaskGroupRepo.createGroup(...)` always sets `status=NEW` and `createdDate=now`.
- `CompletionTaskRepo.createTask(...)` always sets `status=PENDING` and `createdDate=now`.
- `CompletionTaskRepo.completeTask(...)` sets `status`, `completedDate=now`, and `result`.
- If the task belongs to a group, task completion emits an SSE message for that group.

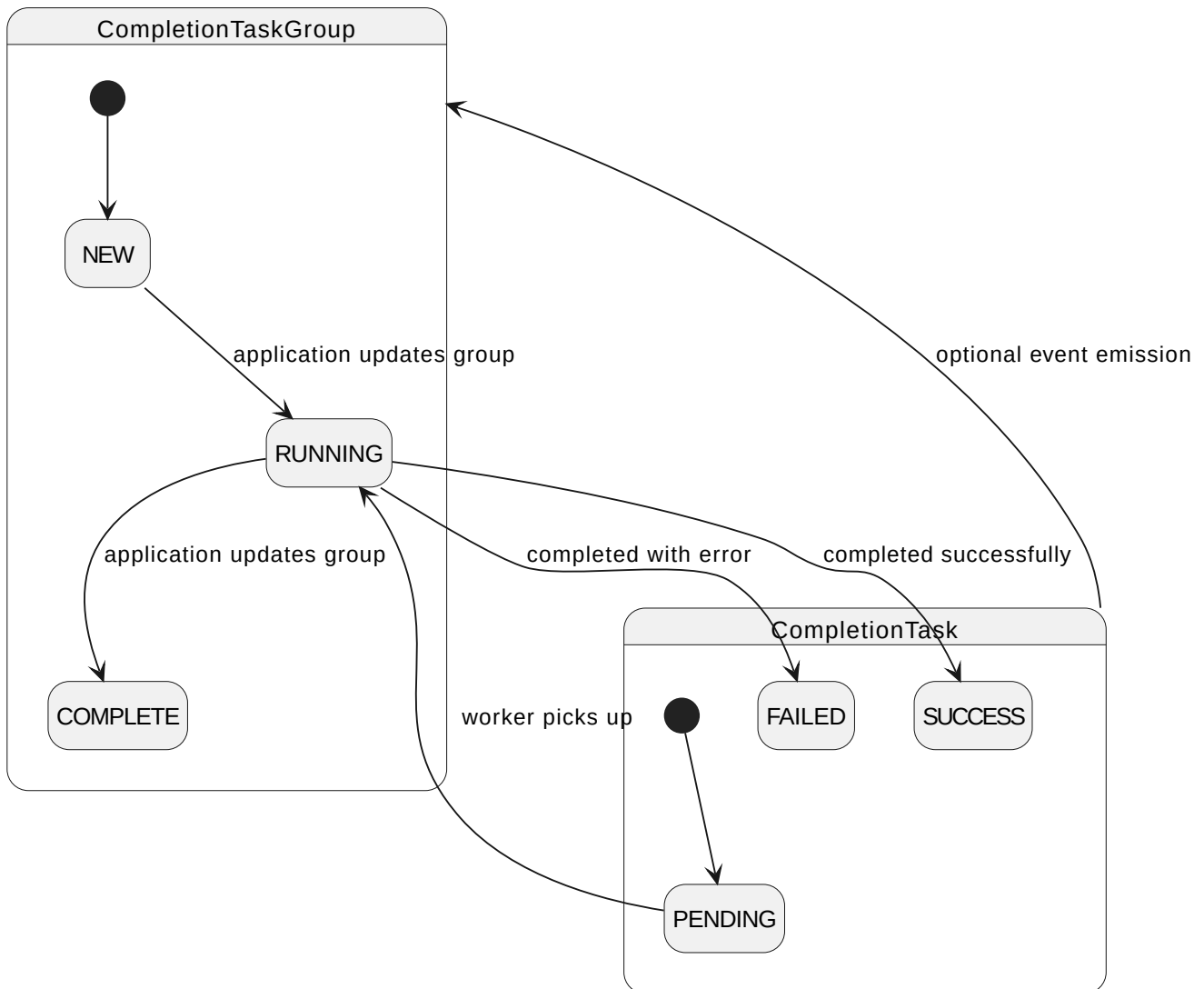
The framework does not automatically:

- Change a group from `NEW` to `RUNNING`

- Mark a group **COMPLETE** when all child tasks finish
- Count completed tasks or calculate percentages
- Retry failed tasks
- Validate that only terminal states are used with the completion endpoint

That means group rollup is application-owned. If you want `CompletionTaskGroup.status` to represent aggregate progress, update it explicitly from your worker or orchestration layer.

3.7.4. State model



Recommended conventions:

- Set a task to **RUNNING** when a worker begins processing it.
- Use **SUCCESS** and **FAILED** as the terminal task states.
- Set a group to **RUNNING** once at least one child task has begun.
- Set a group to **COMPLETE** only when your own aggregation logic decides the batch is finished.

3.7.5. REST API surface

The completion-task resources are small wrappers on top of the generic `BaseResource`, so they expose both task-specific endpoints and inherited read endpoints.

Task group endpoints

Method and path	Purpose	Notes
POST /integration/completionTaskGroup/create	Create a new task group	Server forces <code>status=NEW</code> and <code>createdDate</code>
GET /integration/completionTaskGroup/id/{id}	Fetch a group by Mongo <code>id</code>	Inherited from <code>BaseResource</code>
GET /integration/completionTaskGroup/refName/{refName}	Fetch a group by <code>refName</code>	Inherited from <code>BaseResource</code>
GET /integration/completionTaskGroup/subscribe/{id}	Subscribe to completion events for tasks in the group	Returns server-sent events

Task endpoints

Method and path	Purpose	Notes
POST /integration/completionTask/create/{groupId}	Create a task and attempt to attach it to the given group	Server forces <code>status=PENDING</code> and <code>createdDate</code>
PUT /integration/completionTask/complete/{id}?status=SUCCESS&result=...	Update a task status, set <code>completedDate</code> , and store <code>result</code>	Despite the method name, the code accepts any <code>CompletionTask.Status</code>
GET /integration/completionTask/id/{id}	Fetch a task by Mongo <code>id</code>	Inherited from <code>BaseResource</code>

Method and path	Purpose	Notes
GET /integration/completionTask/ refName/{refName}	Fetch a task by <code>refName</code>	Inherited from <code>BaseResource</code>

3.7.6. Request and response payloads

Creating a group

Minimal request:

```
{
  "refName": "catalog-import-20260327",
  "displayName": "Catalog Import 2026-03-27",
  "description": "Nightly catalog import for 250 SKUs"
}
```

Representative response:

```
{
  "id": "67e51d7c9d0f1d6df2f8ab12",
  "refName": "catalog-import-20260327",
  "displayName": "Catalog Import 2026-03-27",
  "description": "Nightly catalog import for 250 SKUs",
  "status": "NEW",
  "createdDate": "2026-03-27T04:12:14.118Z",
  "completedDate": null
}
```

Creating a task in a group

Minimal request:

```
{
  "refName": "sku-100045",
  "displayName": "Import SKU 100045",
  "details": "Import SKU 100045 from supplier feed"
}
```

Representative response:

```
{
  "id": "67e51df49d0f1d6df2f8ab13",
```

```

"refName": "sku-100045",
"displayName": "Import SKU 100045",
"details": "Import SKU 100045 from supplier feed",
"status": "PENDING",
"createdDate": "2026-03-27T04:14:12.091Z",
"completedDate": null,
"result": null,
"group": {
  "id": "67e51d7c9d0f1d6df2f8ab12"
}
}

```

Notes:

- The endpoint path requires a `groupId`.
- If the `groupId` does not resolve to an existing group, the current repository implementation still creates the task, but leaves `group` unset.
- The server ignores any client-supplied `status` or `createdDate` during creation and overwrites them.

Completing a task

Example:

```

curl -X PUT \

"http://localhost:8080/integration/completionTask/complete/67e51df49d0f1d6df2f8ab13?status=SUCCESS&result=Imported%20successfully"

```

Representative response:

```

{
  "id": "67e51df49d0f1d6df2f8ab13",
  "status": "SUCCESS",
  "completedDate": "2026-03-27T04:16:08.501Z",
  "result": "Imported successfully"
}

```

Important behavior:

- `completedDate` is always set by the completion endpoint.
- The endpoint name says "complete", but the implementation accepts any value in `CompletionTask.Status`.
- Updating the task does not automatically update the parent group status.

3.7.7. Completion event stream

`GET /integration/completionTaskGroup/subscribe/{groupId}` returns server-sent events. Each event is emitted when `CompletionTaskRepo.completeTask(...)` is called for a task in that group.

The event payload is plain text, not JSON. The current format is:

```
task:{taskId}:{status}
```

Example event data:

```
task:67e51df49d0f1d6df2f8ab13:SUCCESS
```

If you need richer event payloads, build that in your own orchestration layer or extend the resource.

3.7.8. Typical orchestration flow

Example creation flow:

```
CompletionTaskGroup group = completionTaskGroupRepo.createGroup(
    CompletionTaskGroup.builder()
        .refName("catalog-import-20260327")
        .displayName("Catalog Import 2026-03-27")
        .description("Nightly catalog import for 250 SKUs")
        .build()
);

for (Sku s : skus) {
    completionTaskRepo.createTask(
        CompletionTask.builder()
            .refName("sku-" + s.code())
            .displayName("Import SKU " + s.code())
            .details("Import SKU " + s.code() + " from supplier feed")
            .build(),
        group.getId().toString()
    );
}
```

Example worker progression:

```
CompletionTask task = completionTaskRepo.findOneByStatus(CompletionTask.Status.
PENDING);
if (task != null) {
    completionTaskRepo.update(task.getId(), "status", CompletionTask.Status.RUNNING);
    completionTaskGroupRepo.update(task.getGroup().getId(), "status",
CompletionTaskGroup.Status.RUNNING);
}
```

```

try {
    // ... do work ...
    completionTaskRepo.completeTask(
        task.getId().toString(),
        CompletionTask.Status.SUCCESS,
        "Imported successfully"
    );
} catch (Exception e) {
    completionTaskRepo.completeTask(
        task.getId().toString(),
        CompletionTask.Status.FAILED,
        e.getMessage()
    );
}
}

```

Example aggregate rollup owned by the application:

```

List<CompletionTask> tasks = completionTaskRepo.findByGroup(group);

boolean allTerminal = tasks.stream().allMatch(t ->
    t.getStatus() == CompletionTask.Status.SUCCESS ||
    t.getStatus() == CompletionTask.Status.FAILED
);
boolean anyRunning = tasks.stream().anyMatch(t ->
    t.getStatus() == CompletionTask.Status.RUNNING
);

if (allTerminal) {
    completionTaskGroupRepo.update(
        group.getId(),
        "status", CompletionTaskGroup.Status.COMPLETE,
        "completedDate", new Date()
    );
} else if (anyRunning) {
    completionTaskGroupRepo.update(
        group.getId(),
        "status", CompletionTaskGroup.Status.RUNNING
    );
}

```

3.7.9. Best practices

- Keep **details** short and stable so operators can quickly scan task lists.
- Use **result** for outcome summaries, error messages, or external correlation identifiers.
- Populate **refName** with a meaningful business key so tasks can be found without raw Mongo ids.

- Use `dataDomain` consistently so task and group records stay tenant-scoped.
- Treat group status as a denormalized summary owned by your orchestration code.
- Purge or archive old completed groups to avoid unbounded growth.

3.8. References and EntityReference

Morphia `@Reference` establishes relationships between entities: - One-to-one: a `BaseModel` field annotated with `@Reference`. - One-to-many: a `Collection<BaseModel>` field annotated with `@Reference`.

Example:

```
@Entity
public class Shipment extends BaseModel {
    @Reference(ignoreMissing = false)
    @TrackReferences
    private Partner partner;    // parent entity
}
```

`EntityReference` is a lightweight reference object used across the framework to avoid `DBRef` loading when only identity info is needed. Any model can produce one:

```
EntityReference ref = shipment.createEntityReference();
// contains: entityId, entityType, entityRefName, entityDisplayName (and optional realm)
```

REST convenience:

- `BaseResource` exposes `GET /entityref` to list `EntityReference` for a model with optional filter/sort.
- `Repositories` expose `getEntityReferenceListByQuery(...)`, and utilities exist to convert lists of `EntityReference` back to entities when needed.

When to use which:

- Use `@Reference` for strong persistence-level links where Morphia should maintain foreign references.
- Use `EntityReference` for UI lists, foreign-key-like pointers in other documents, events/audit logs, or cross-module decoupling without `DBRef` behavior.

3.9. Tracking References with @TrackReferences and Delete Semantics

`@TrackReferences` on a `@Reference` field tells the framework to maintain a back-reference set on the parent entity. The back-reference field is `UnversionedBaseModel.references` (a `Set<ReferenceEntry>`), which is calculated/maintained by the framework and should not be set by

clients.

What references contains:

- Each `ReferenceEntry` holds: `referencedId` (`ObjectId` of the child), `type` (fully-qualified class name of the child's entity), and `refName` (child's stable reference name).
- It indicates that the parent is being referenced by the given child entity. The set is used for fast checks and to enforce referential integrity.

How tracking works (save/update):

- `ReferenceInterceptor` inspects `@Reference` fields annotated with `@TrackReferences` during `prePersist`.
- When a child references a parent, a `ReferenceEntry` for the child is added to the parent's references set and the parent is saved to persist the back-reference.
- For `@Reference` collections, entries are added for each child-parent pair.
- If a `@Reference` is null but `ignoreMissing=false`, a save will fail with an `IllegalStateException` since the parent is required.

How it affects delete:

- During delete in `MorphiaRepo.delete(...)`:
- If `obj.references` is empty, the object can be deleted directly (after removing any references it holds to parents).
- If `obj.references` is not empty, the repo checks each `ReferenceEntry`. If any referring parent still exists, a `ReferentialIntegrityViolationException` is thrown to prevent breaking relationships.
- If all references are stale (referring objects no longer exist), the repo removes stale entries, removes this object's own reference constraints from parents, and performs the delete within a transaction.
- `removeReferenceConstraint(...)` ensures that, when deleting a child, its `ReferenceEntry` is removed from `parent.references` and the parent is saved, keeping back-references consistent.

Practical guidance:

- Annotate parent links with both `@Reference` and `@TrackReferences` when you need strong integrity guarantees and easy "who references me?" queries.
- Use `ignoreMissing=true` only for optional references; you still get back-reference tracking when not null.
- Expect HTTP delete to fail with a meaningful error if there are live references; remove or update those references first, or design cascading behavior explicitly in your domain logic.

3.9.1. Ontologies in Quantum: Modeling Relationships That Are Resilient and Fast



Looking for the short implementation plan? See `PROPOSAL.md` at the repository

root for a concise module-by-module checklist.

This section explains what an ontology is, how it differs from a traditional object model, and how the Quantum Ontology modules make it practical to apply ontology ideas to your domain models and queries. It also contrasts ontology-driven relationships with direct object references (for example, using `@Reference` or `EntityReference`).

What is an Ontology?

In software terms, an ontology is a formal, explicit specification of concepts and their relationships.

- **Concepts (Classes):** Named categories/types in your domain. Concepts can form taxonomies (is-a hierarchies), be declared disjoint, or be equivalent.
- **Relationships (Properties):** Named relationships between entities. Properties can have a domain (applies to X) and a range (points to Y). They may be inverse or transitive.
- **Axioms (Rules):** Constraints and entailment rules, including property chains such as: if $(A \xrightarrow{-p} B)$ and $(B \xrightarrow{-q} C)$ then we infer $(A \xrightarrow{-r} C)$.
- **Inference:** The process of deriving new facts (types, labels, edges) that were not explicitly stored but follow from axioms and known facts.

An ontology is not the data; it is the schema plus logic that gives your data additional meaning and enables consistent, automated inferences.

Ontology vs. Object Model

A conventional object model focuses on concrete classes, fields, and direct references between objects at implementation time. An ontology focuses on semantic types and relationships, with explicit rules that can derive new knowledge independent of how objects are instantiated.

Key differences:

- **Purpose** - Object model: Encapsulate data and behavior for application code generation and persistence. - Ontology: Encode shared meaning, constraints, and inference rules that remain stable as implementation details change.
- **Relationship handling** - Object model: Typically uses direct references or foreign keys; traversals are hard-coded and fragile to change. - Ontology: Uses named predicates (properties) and can infer additional relationships by rules (property chains, inverses, transitivity).
- **Polymorphism and evolution** - Object model: Polymorphism requires class inheritance in code; cross-cutting categories are awkward to add later. - Ontology: Entities can have multiple types/labels at once. New concepts and properties can be introduced without breaking existing data.
- **Querying** - Object model: Queries couple to concrete classes and field paths; changes force query rewrites. - Ontology: Queries target semantic relationships; reasoners can materialize edges that queries reuse, decoupling queries from implementation details.

Why prefer Ontology-driven relationships over `@Reference/EntityReference`

Direct references (`@Reference` or custom `EntityReference`) are simple to start but become restrictive as domains grow:

- **Tight coupling:** Code and queries couple to concrete field paths (`customer.primaryAddress.id`), making refactors risky.
- **Limited expressivity:** Hard to encode and reuse higher-order relationships (e.g., "partners of my supplier's parent org").
- **Poor polymorphism:**

References point to one collection/type; accommodating multiple target types requires extra code. - Performance pitfalls: Deep traversals cause extra queries, N+1 selects, or complex \$lookup joins.

Ontology-driven edges address these issues: - Decoupling via predicates: Use named predicates (e.g., `hasAddress`, `memberOf`, `supplies`) that remain stable while internal object fields change. - Inference for reachability: Property chains can materialize implied links ($A \xrightarrow{p} B \ \& \ B \xrightarrow{q} C \Rightarrow A \xrightarrow{r} C$), avoiding runtime multi-hop traversals. - Polymorphism-first: A predicate can connect heterogeneous types; type inferences (domain/range) remain consistent. - Query performance: Pre-materialized edges allow single-hop, index-friendly queries (in or eq filters) instead of ad-hoc multi-collection traversals. - Resilience to change: You can add or modify rules without rewriting data structures or touching referencing fields across models.

How Quantum supports Ontologies

Quantum provides three cooperating modules that make ontology modeling practical and fast:

- `quantum-ontology-core` (package `com.e2eq.ontology.core`)
- `OntologyRegistry`: Holds the `TBox` (terminology) of your ontology.
- `ClassDef`: Concept names and relationships (`parents`, `disjointWith`, `sameAs`).
- `PropertyDef`: Property names with optional domain, range, inverse flags, and transitivity.
- `PropertyChainDef`: Rules that define multi-hop implications (`chains` $\rightarrow$ implied property).
- `TBox`: Container for classes, properties, and property chains.
- `Reasoner` interface and `ForwardChainingReasoner`: Given an entity snapshot and the registry, computes inferences:
- New types/labels to assert on entities.
- New edges to add (implied by property chains, inverses, or other rules).
- `quantum-ontology-mongo` (package `com.e2eq.ontology.mongo`)
- `EdgeDao`: A thin DAO around an edges collection in Mongo. Each edge contains `tenantId`, `src`, predicate `p`, `dst`, `inferred` flag, `provenance`, and `timestamp`.
- `OntologyMaterializer`: Runs the `Reasoner` for an entity snapshot and upserts the inferred edges, so queries can be rewritten to simple in/in eq filters.
- `quantum-ontology-policy-bridge` (package `com.e2eq.ontology.policy`)
- `ListQueryRewriter`: Takes a base query and rewrites it using the `EdgeDao` to filter by the set of source entity ids that have a specific predicate to a given destination.
- This integrates ontology edges with `RuleContext` or policy decisions: policy asks for entities related by a predicate; the rewriter converts that into an efficient Mongo query.

These modules let you define your ontology (core), materialize derived relations (mongo), and leverage them in access and list queries (policy bridge).

Modeling guidance: from object fields to predicates

- Name relationships explicitly

- Define clear predicate names (hasAddress, memberOf, supplies, owns, assignedTo). Avoid encoding relationship semantics in field names only.
- Keep object model minimal and flexible
- Store lightweight identifiers (ids) as needed, but avoid deeply nested reference graphs that encode traversals in code.
- Model polymorphic relationships
- Prefer predicates that naturally connect multiple possible types (e.g., assignedTo can target User, Team, Bot) and rely on ontology type assertions to constrain where needed.
- Use property chains for common paths
- If business logic often traverses $A \rightarrow B \rightarrow C$, define a chain $p \sqsubseteq q \Rightarrow r$ and materialize r for faster queries and simpler policies.
- Capture inverses and transitivity
- For natural inverses (parentOf $\sqsubseteq$ childOf) or transitive relations (partOf, locatedIn), define them in the ontology so edges and queries stay consistent.
- Keep provenance
- Record why an edge exists (prov.rule, prov.inputs) so you can recompute, audit, or retract when inputs change.

Querying with ontology edges vs direct references

- Direct reference example (fragile/slow)
- Query: "Find Orders whose buyer belongs to Org X or its parents."
- With @Reference: requires joining $\text{Order} \rightarrow \text{User} \rightarrow \text{Org}$ and recursing org.parent; costly and tightly coupled to fields.
- Ontology edge example (resilient/fast)
- Define predicates: placedBy(order, user), memberOf(user, org), ancestorOf(org, org). Define chain placedBy $\sqsubseteq$ memberOf $\Rightarrow$ placedInOrg.
- Materialize edges: (order --placedInOrg--> org). Also make ancestorOf transitive.
- Query becomes: where order._id in EdgeDao.srcIdsByDst(tenantId, "placedInOrg", orgX).
- With transitivity, you can precompute ancestor closure or add a chain placedInOrg $\sqsubseteq$ ancestorOf $\Rightarrow$ placedInOrg to include parents automatically.

Migration: from @Reference to ontology edges

- Start by introducing predicates alongside existing references; do not remove references immediately.
- Materialize edges for hot read paths; keep provenance so you can reconstruct.
- Gradually update queries (list screens, policy filters) to use ListQueryRewriter with EdgeDao instead of deep traversals or \$lookup.
- Once stable, you can simplify models by removing rigid reference fields where unnecessary and rely on edges for read-side composition.

Performance and operational notes

- Indexing: Create compound indexes on edges: (tenantId, p, dst) and (tenantId, src, p) to support both reverse and forward lookups.
- Write amplification vs read wins: Materialization adds write work, but dramatically improves read latency and simplifies queries.
- Consistency: Re-materialize edges on relevant entity changes (source, destination, or intermediate) using `OntologyMaterializer`.
- Multi-tenancy: Keep tenantId in the edge key and filters; the provided `EdgeDao` methods include tenant scoping.

How this integrates with Functional Areas/Domains

- Functional domains often map to concept clusters in the ontology. Use `@FunctionalMapping` to aid discovery and apply policies per area/domain.
- Policies can refer to relationships semantically ("hasEdge placedInOrg OrgX") and rely on the policy bridge to turn this into efficient data filters.

Summary

- Ontology-powered relationships provide a stable, semantic layer over your object model.
- The Quantum Ontology modules let you define, infer, and query these relationships efficiently on MongoDB.
- Compared with direct `@Reference/EntityReference`, ontology edges are more expressive, resilient to change, and typically faster for complex list/policy queries once materialized.

Concrete example: Sales Orders, Shipments, and evolving to Fulfillment/Returns

This example shows how to use an ontology to model relationships around Orders, Customers, and Shipments, and how the model can evolve to include Fulfillment and Returns without breaking existing queries. We will:

- Define core concepts and predicates.
- Add property chains that materialize implied relationships for fast queries.
- Show how queries are rewritten using edges instead of deep object traversals.
- Evolve the model to support Fulfillment and Returns with minimal changes.

Core concepts (classes)

- Order, Customer, Organization, Shipment, Address, Region
- Later evolution: `FulfillmentTask`, `FulfillmentUnit`, `ReturnRequest`, `ReturnItem`, `RMA`

Key predicates (relationships)

- `placedBy(order, customer)`: who placed the order
- `memberOf(customer, org)`: a customer belongs to an organization (or account)

- `orderHasShipment(order, shipment)`: outbound shipment for the order
- `shipsTo(shipment, address)`: shipment destination
- `locatedIn(address, region)`: address is located in a Region
- `ancestorOf(org, org)`: organizational ancestry (transitive)

Property chains (implied relationships)

- `placedBy` $\sqsubseteq$ `memberOf` $\Rightarrow$ `placedInOrg`
- If `(order --placedBy-→ customer)` and `(customer --memberOf-→ org)`, then infer `(order --placedInOrg-→ org)`
- `orderHasShipment` $\sqsubseteq$ `shipsTo` $\Rightarrow$ `orderShipsTo`
- If `(order --orderHasShipment-→ shipment)` and `(shipment --shipsTo-→ address)`, infer `(order --orderShipsTo-→ address)`
- `orderShipsTo` $\sqsubseteq$ `locatedIn` $\Rightarrow$ `orderShipsToRegion`
- If `(order --orderShipsTo-→ address)` and `(address --locatedIn-→ region)`, infer `(order --orderShipsToRegion-→ region)`
- `placedInOrg` $\sqsubseteq$ `ancestorOf` $\Rightarrow$ `placedInOrg`
- Makes `placedInOrg` resilient to org hierarchy changes (`ancestorOf` is transitive). This is a common “closure” trick: re-assert the same predicate via chain to absorb hierarchy.

A minimal Java-style snippet to define this TBox

```
import java.util.*;
import com.e2eq.ontology.core.OntologyRegistry;
import com.e2eq.ontology.core.OntologyRegistry.*;

Map<String, ClassDef> classes = Map.of(
    "Order", new ClassDef("Order", Set.of(), Set.of(), Set.of()),
    "Customer", new ClassDef("Customer", Set.of(), Set.of(), Set.of()),
    "Organization", new ClassDef("Organization", Set.of(), Set.of(), Set.of()),
    "Shipment", new ClassDef("Shipment", Set.of(), Set.of(), Set.of()),
    "Address", new ClassDef("Address", Set.of(), Set.of(), Set.of()),
    "Region", new ClassDef("Region", Set.of(), Set.of(), Set.of())
);

Map<String, PropertyDef> props = Map.of(
    "placedBy", new PropertyDef("placedBy", Optional.of("Order"), Optional.of("Customer"), false, Optional.empty(), false),
    "memberOf", new PropertyDef("memberOf", Optional.of("Customer"), Optional.of("Organization"), false, Optional.empty(), false),
    "orderHasShipment", new PropertyDef("orderHasShipment", Optional.of("Order"), Optional.of("Shipment"), false, Optional.empty(), false),
    "shipsTo", new PropertyDef("shipsTo", Optional.of("Shipment"), Optional.of("Address"), false, Optional.empty(), false),
    "locatedIn", new PropertyDef("locatedIn", Optional.of("Address"), Optional.of("Region"), false, Optional.empty(), false),
);
```

```

    "ancestorOf", new PropertyDef("ancestorOf", Optional.of("Organization"), Optional.
of("Organization"), false, Optional.empty(), true), // transitive
    // implied predicates (no domain/range required, but you may add them for
validation)
    "placedInOrg", new PropertyDef("placedInOrg", Optional.of("Order"), Optional.of(
"Organization"), false, Optional.empty(), false),
    "orderShipsTo", new PropertyDef("orderShipsTo", Optional.of("Order"), Optional.of(
"Address"), false, Optional.empty(), false),
    "orderShipsToRegion", new PropertyDef("orderShipsToRegion", Optional.of("Order"),
Optional.of("Region"), false, Optional.empty(), false)
);

List<PropertyChainDef> chains = List.of(
    new PropertyChainDef(List.of("placedBy", "memberOf", "placedInOrg"),
    new PropertyChainDef(List.of("orderHasShipment", "shipsTo", "orderShipsTo"),
    new PropertyChainDef(List.of("orderShipsTo", "locatedIn", "orderShipsToRegion"),
    new PropertyChainDef(List.of("placedInOrg", "ancestorOf", "placedInOrg")
);

OntologyRegistry.TBox tbox = new OntologyRegistry.TBox(classes, props, chains);
OntologyRegistry registry = OntologyRegistry.inMemory(tbox);

```

Materializing edges for an Order

- Explicit facts for order O1:
- O1 placedBy C9
- C9 memberOf OrgA
- O1 orderHasShipment S17
- S17 shipsTo Addr42
- Addr42 locatedIn RegionWest
- OrgA ancestorOf OrgParent
- Inferred edges after running the reasoner for O1's snapshot:
- O1 placedInOrg OrgA
- O1 placedInOrg OrgParent (via closure with ancestorOf)
- O1 orderShipsTo Addr42
- O1 orderShipsToRegion RegionWest

How queries become simple and fast

- List Orders for Organization OrgParent (including children):
- Instead of joining Order → Customer → Org and recursing org.parent, run a single filter using materialized edges.

```
import com.mongodb.client.model.Filters;
```

```
import org.bson.conversions.Bson;
import com.e2eq.ontology.policy.ListQueryRewriter;

Bson base = Filters.eq("status", "OPEN");
Bson rewritten = rewriter.rewriteForHasEdge(base, tenantId, "placedInOrg", "OrgParent");
// Use rewritten in your Mongo find
```

- List Orders shipping to RegionWest:

```
Bson rewritten2 = rewriter.rewriteForHasEdge(Filters.empty(), tenantId,
"orderShipsToRegion", "RegionWest");
```

Why this is resilient

- If tomorrow Customer becomes AccountContact and the organization model gains Divisions and multi-parent org graphs, you only adjust predicates and chains.
- Queries that rely on placedInOrg or orderShipsToRegion remain unchanged and fast, because edges are re-materialized by OntologyMaterializer.

Evolving the model: add Fulfillment

New concepts

- FulfillmentTask: a unit of work to pick/pack/ship order lines
- FulfillmentUnit: a logical grouping (e.g., wave, tote, parcel)

New predicates

- fulfills(task, order)
- realizedBy(order, fulfillmentUnit)
- taskProduces(task, shipment)

New chains (implied)

- fulfills $\Rightarrow$ derived edge from task to order; combine with taskProduces to connect order to shipment without touching Order fields:
- fulfills $\square$ taskProduces $\Rightarrow$ orderHasShipment
- realizedBy $\square$ orderHasShipment $\Rightarrow$ fulfilledByUnit
- If (order --realizedBy- $\rightarrow$ fu) and (order --orderHasShipment- $\rightarrow$ s) $\Rightarrow$ (fu --fulfillsShipment- $\rightarrow$ s) or simply (order --fulfilledByUnit- $\rightarrow$ fu)

These chains let you introduce warehouse concepts without changing how UI filters orders by organization or ship-to region. Existing queries still operate via placedInOrg and orderShipsToRegion.

Evolving further: add Returns

New concepts

- ReturnRequest, ReturnItem, RMA

New predicates

- hasReturn(order, returnRequest)
- returnFor(returnItem, order)
- returnRma(returnRequest, rma)

New chains (implied)

- hasReturn $\Rightarrow$ openReturnOnOrg via placedInOrg:
- hasReturn $\square$ placedInOrg $\Rightarrow$ returnPlacedInOrg
- returnFor $\square$ orderShipsToRegion $\Rightarrow$ returnShipsToRegion

Example queries with new capabilities

- List Orders with open returns in OrgParent:

```
Bson r = rewriter.rewriteForHasEdge(Filters.empty(), tenantId, "returnPlacedInOrg",  
"OrgParent");
```

- List Returns associated to Orders shipping to RegionWest:

```
Bson r2 = rewriter.rewriteForHasEdge(Filters.empty(), tenantId, "returnShipsToRegion",  
"RegionWest");
```

Comparison with direct references (@Reference/EntityReference)

- With direct references you would encode fields like Order.customer, Order.shipments, Shipment.address, Address.region and then implement multi-hop traversals in code or \$lookup pipelines, rewriting them whenever you add Fulfillment or Returns.
- With ontology edges, you keep predicates stable and add property chains. Existing list and policy queries keep working and typically become faster due to single-hop filters on an indexed edges collection.

Operational tips for this scenario

- Ensure EdgeDao has indexes on (tenantId, p, dst) and (tenantId, src, p).
- Use OntologyMaterializer when Order, Shipment, Customer, Address, or org hierarchy changes to keep edges fresh.
- Keep provenance in edge.prov (rule, inputs) so you can recompute or retract edges when source data changes.

3.9.2. Integrating Ontology with Morphia, Permissions, and Multi-tenancy

This section focuses on integration and developer experience: how ontology edges flow into Morphia-based repositories and the permission rule language, while remaining fully multi-tenant and secure.

Big picture: where ontology fits

- Write path (materialization):
- Your domain code persists entities with minimal direct references.
- An `OntologyMaterializer` runs when entities change to derive and upsert edges into the edges collection (per tenant).
- Policy path (authorization and list filters):
- The permission rule language evaluates the caller's `SecurityContext/RuleContext` and produces logical filters.
- When a rule asks for a semantic relationship (`hasEdge`), we use `ListQueryRewriter` + `EdgeDao` to translate that into efficient Mongo filters over ids.
- Read path (queries):
- Morphia repos apply the base data-domain filters and the rewritten ontology constraint to queries, producing fast lists without deep joins.

Rule language: add `hasEdge()`

We introduce a policy function/operator to reference ontology edges directly from rules:

- Signature: `hasEdge(predicate, dstIdOrVar)`
- predicate: String name of the ontology predicate (e.g., `"placedInOrg"`, `"orderShipsToRegion"`).
- dstIdOrVar: Either a concrete id/refName or a variable resolved from `RuleContext` (e.g., `principal.orgRefName`, `request.region`).
- Semantics: The rule grants/filters entities for which an edge (`tenantId`, `src = entity_id`, `p = predicate`, `dst = resolvedDst`) exists.
- Composition: `hasEdge` can be combined with existing rule clauses (and/or/not) and other filters (states, tags, ownerId, etc.).

Example rule snippets (illustrative):

- Allow viewing Orders in the caller's org (including ancestors via ontology closure):
- `allow VIEW Order when hasEdge("placedInOrg", principal.orgRefName)`
- Restrict list to Orders shipping to a region chosen in request:
- `allow LIST Order when hasEdge("orderShipsToRegion", request.region)`

Under the hood, policy evaluation uses `ListQueryRewriter.rewriteForHasEdge(...)`, which converts `hasEdge` into a set of source ids and merges that with the base query.

Passing tenantId correctly

- Always resolve tenantId from RuleContext/SecurityContext (the same source your repos use for realm/database selection).
- EdgeDao and ListQueryRewriter already accept tenantId; never cross tenant boundaries when reading edges.
- Index recommendation (per tenant):
- (tenantId, p, dst)
- (tenantId, src, p)

Morphia repository integration patterns

The goal is zero-friction usage in existing repos without invasive changes.

Option A: Apply ontology constraints in code paths that already construct BSON filters.

- If your repo method builds a Bson filter before calling find(), wrap it through rewriter:

```
Bson base = Filters.and(existingFilters...);
Bson rewritten = hasEdgeRequested
    ? rewriter.rewriteForHasEdge(base, tenantId, predicate, dst)
    : base;
var cursor = datastore.getDatabase().getCollection(coll).find(rewritten);
```

Option B: Apply ontology constraints to Morphia Filter/Query via ids.

- When the repo uses Morphia's typed query API instead of BSON, pre-compute the id set and constrain by _id:

```
Set<String> ids = edgeDao.srcIdsByDst(tenantId, predicate, dst);
if (ids.isEmpty()) {
    return List.of(); // short-circuit
}
query.filter(Filters.in("_id", ids));
```

Option C: Centralize in a tiny helper for developer ergonomics.

- Provide one helper in your application layer, invoked wherever policies inject additional constraints:

```
public final class OntologyFilterHelper {
    private final ListQueryRewriter rewriter;
    public OntologyFilterHelper(ListQueryRewriter r) { this.rewriter = r; }

    public Bson ensureHasEdge(Bson base, String tenantId, String predicate, String dst)
    {
        return rewriter.rewriteForHasEdge(base, tenantId, predicate, dst);
    }
}
```

```
}  
}
```

Where to hook materialization

- On entity changes that are sources or intermediates for chains:
- Order (placedBy, orderHasShipment), Customer (memberOf), Shipment (shipsTo), Address (locatedIn), Organization (ancestorOf/parent), and any Fulfillment/Returns entities.
- Recommended patterns:
- On-save/on-update hooks in your service layer call `OntologyMaterializer.apply(...)` with the explicit edges known from the entity snapshot.
- For intermediates (e.g., `Address.region` changed), enqueue affected sources for recomputation; use provenance to locate impacted edges.
- Provide nightly/backfill jobs for recomputing edges across a tenant when ontology rules evolve.

Security and multi-tenant considerations

- Edge rows include `tenantId` and should be validated/filtered by tenant on every operation.
- Never trust a client-supplied predicate or destination id blindly; combine with rule evaluation and whitelist allowed predicates per domain if needed.
- For shared resources across tenants (rare), model cross-tenant permissions at the policy layer; don't reuse edges across tenants unless explicitly designed.

Developer workflow and DX checklist

- When writing a rule: use `hasEdge("<predicate>", <rhs>)` and rely on `RuleContext` variables for the destination when possible.
- When writing a list endpoint: read optional ontology filter hints from the policy layer; if present, apply `ensureHasEdge(...)` before `find()`.
- When changing domain relationships: update predicates/chains and re-materialize; list/policy code stays unchanged.
- When indexing a new tenant: include the edges indexes early and validate via a smoke test query using `ListQueryRewriter`.

Cookbook: end-to-end example with Orders + Org

- Policy: allow LIST Order when `hasEdge("placedInOrg", principal.orgRefName)`
- Request lifecycle: 1) Security filter builds `SecurityContext` and `RuleContext` with `tenantId` and `principal`. 2) Policy evaluation returns a directive to constrain by `hasEdge("placedInOrg", orgRefName)`. 3) Repo builds base filter (`state != ARCHIVED`, etc.). 4) Repo calls `OntologyFilterHelper.ensureHasEdge(base, tenantId, "placedInOrg", orgRefName)`. 5) Mongo executes a single-hop query using materialized edges; results respect both policy and multi-tenancy.

Migration notes for teams using @Reference

- Keep existing references for write-side integrity and local joins where simple.
- Introduce ontology edges on hot read paths first; update policy rules to hasEdge and verify results.
- Gradually replace deep \$lookup traversals with hasEdge-based rewrites.
- Ensure materialization hooks are deployed before removing data fields used as inputs to the ontology.

Chapter 4. 4. Multi-tenant model and realm separation

Problem: Isolating data per tenant while enabling selective sharing.

Why for SaaS: Security, compliance, and data residency require strong segregation and auditability.

How Quantum helps: Realm/tenant identifiers, context propagation, transforms, and repository filters.

Walkthrough: Configure realms and mark your models for tenancy.

4.1. Multi-Tenancy Models

Quantum supports multiple multi-tenant models for MongoDB deployments:

Every persisted `Realm` has a `deploymentType`:

- `DEDICATED` is the default. Realm records created before this field existed, and any record whose value is absent or null, resolve to `DEDICATED`.
- `SHARED` must be set explicitly before the realm can accept pooled tenant admissions.

The system-management realm catalog exposes this value so operators can see the database topology directly. Tenant lifecycle requests use `DEDICATED_REALM` and `POOLED_REALM`; those request values must agree with the realm's persisted `DEDICATED` or `SHARED` type.

Database migration `1.0.6` backfills existing system-realm catalog records. It updates only Realm documents where `deploymentType` is missing or null, setting them to `DEDICATED`; an explicitly configured `SHARED` value is preserved. Deployments should advance `quantum.database.version` to `1.0.6` and run the normal Quantum migration workflow rather than patching Realm documents individually.

4.1.1. `DEDICATED_REALM`: One Tenant per Database

- Each tenant is mapped to a dedicated MongoDB database within a cluster.
- Strong isolation at the database level; operational controls via MongoDB roles.
- Pros: Simplified backup/restore per tenant; reduced risk of data bleed.
- Cons: More databases to manage (indexes, connections), higher operational overhead.

How Quantum helps:

- `DataDomain` carries tenant identifiers (e.g., `tenantId`, `ownerId`, `orgRefName`) on each model.
- Repositories can resolve connections/DB selection per tenant, enabling routing to the appropriate database.

4.1.2. POOLED_REALM: Many Tenants in One Database

- Multiple tenants share a single database and collections.
- Isolation is enforced at the application layer using DataDomain filters.
- Pros: Fewer databases to manage; efficient index utilization and connection pooling.
- Cons: Strict discipline required to enforce filtering and access rules.

How Quantum helps:

- DataDomain is part of every persisted model, enabling programmatic, rule-based filtering.
- RuleContext and DomainContext inject tenant-aware filters into governed repositories and resources.
- Cross-tenant sharing can be modeled by specific DataDomain fields and RuleContext logic granting read access across tenants on a per-functional-area basis.

In both modes, realm means MongoDB placement and schema lifecycle while DataDomain means row visibility. **POOLED_REALM** is therefore safe only for models and repository paths that participate in governed DataDomain filtering. A shared database does not weaken or replace the policy boundary.

4.1.3. Freemium and Trial Tenants

- Programmatically create tenants to support self-service onboarding.
- Attach time-bound or capability-bound policies.
- Use scheduled jobs to convert/expire trials.

Quantum patterns:

- Tenant onboarding service creates a DataDomain scope and any default records.
- Policies are encoded in RuleContext checks to allow or restrict actions based on time, plan, or feature flags.

4.1.4. Tenant Onboarding and Access Invites

Quantum separates realm creation, tenant admission, user access, and the user-facing onboarding journey. Those are related, but they are not the same thing:

- Dedicated provisioning creates the tenant realm, runs realm migrations, and establishes the initial admin user.
- Pooled admission adds a tenant to an existing realm and does not create or initialize a database.
- Tenant onboarding workflow defines the steps a tenant user should complete before access is considered fully activated.
- Access invites bring an individual user into an existing tenant and can attach scoped access metadata.

This split helps teams support both admin-driven setup and self-service onboarding without forcing every tenant into a single path.

1. Tenant provisioning: create the tenant itself

The admin tenant provisioning API is exposed at:

- `POST /admin/tenants`
- `DELETE /admin/tenants/{realmId}`

Provisioning is intended for `admin` and `system` roles. The same endpoint supports both deployment topologies. Omitting `deploymentTopology` preserves the historical `DEDICATED_REALM` behavior. `POOLED_REALM` admission succeeds only when `placementRealmId` identifies an existing Realm explicitly marked `SHARED`; legacy and untyped realms are treated as `DEDICATED`.

Request fields:

Field	Purpose
<code>tenantDisplayName</code>	Human-readable label for the tenant
<code>tenantEmailDomain</code>	Source for the realm id; dots are converted to dashes
<code>orgRefName</code>	Organization reference used in the tenant <code>DomainContext</code> and <code>DataDomain</code>
<code>accountId</code>	Initial account number for the tenant
<code>adminUserId</code>	User id for the initial tenant admin
<code>adminSubject</code>	Optional stable subject override; defaults to <code>adminUserId</code>
<code>adminPassword</code>	Initial password for the admin user
<code>applicationId</code>	Application that owns the realm placement; may use the configured default
<code>deploymentTopology</code>	<code>DEDICATED_REALM</code> or <code>POOLED_REALM</code> ; when omitted, uses <code>quantum.tenant.provisioning.deployment-topology</code> (default <code>DEDICATED_REALM</code>)
<code>placementRealmId</code>	Existing target realm for <code>POOLED_REALM</code> ; when omitted, uses <code>quantum.tenant.provisioning.pooled-realm</code>
<code>seedArchetypes</code> or <code>archetypes</code>	Optional list of seed archetypes for dedicated realm creation. Pooled admission rejects archetypes until the seed contract can prove all writes are tenant-scoped.

Representative request:

```
{
  "tenantDisplayName": "Acme Logistics",
  "tenantEmailDomain": "acme.example",
  "orgRefName": "acme.example",
  "accountId": "1000000001",
  "adminUserId": "admin@acme.example",
  "adminSubject": "admin@acme.example",
  "adminPassword": "change-me-now",
  "seedArchetypes": ["DemoArchetype"]
}
```

```
}
```

Representative response:

```
{
  "realmId": "acme-example",
  "realmCreated": true,
  "userCreated": true,
  "appliedSeedArchetypes": ["DemoArchetype"],
  "warnings": []
}
```

What provisioning does:

- derives `realmId` from `tenantEmailDomain`
- writes a `Realm` catalog record in the system realm
- builds tenant `DomainContext` and `DataDomain`
- runs migrations in the new tenant realm
- creates or validates the initial admin user
- applies applicable base seed packs
- optionally applies requested seed archetypes
- applies indexes and verifies initialization

For `POOLED_REALM`, provisioning instead:

- derives a distinct tenant id from `tenantEmailDomain`
- resolves `placementRealmId` to an existing `Realm` catalog record
- creates a PARTICIPANT `RealmTenantMembership`
- assigns the tenant's users to the shared realm with the tenant's own `DomainContext` and `DataDomain`
- creates only the requested administrator identity and tenant-scoped realm assignment; it does not create baseline or demo identities
- skips realm migrations, base seed packs, archetypes, and index initialization
- never creates or drops the shared realm/database

The pooled realm must therefore be provisioned and migrated before admitting tenants. Every persisted model reachable by a pooled tenant must participate in governed `DataDomain` filtering for reads and writes. A user currently has one active tenant `DataDomain` assignment per realm; admitting the same identity into a second tenant in that realm fails closed until an explicit tenant-selection claim is available.

Example pooled admission:

```
{
  "tenantDisplayName": "Acme Logistics",
  "tenantEmailDomain": "acme.example",
  "orgRefName": "acme.example",
  "accountId": "1000000001",
  "adminUserId": "admin@acme.example",
  "adminPassword": "change-me-now",
  "deploymentTopology": "POOLED_REALM",
  "placementRealmId": "shared-logistics"
}
```

Provisioning is designed to be largely idempotent. If the same tenant or admin user already exists with matching attributes, the service proceeds and returns warnings instead of blindly recreating state. If existing state conflicts with the request, the service rejects the operation.

2. Tenant decommissioning: archive before topology-specific cleanup

Tenant deletion is a governed lifecycle operation, not a direct CRUD delete. The placement topology determines the cleanup boundary:

- **DEDICATED_REALM**: revoke tenant access, create and verify a database archive, then drop the tenant database and retire its realm catalog state.
- **POOLED_REALM**: revoke tenant access, export only rows matching the canonical tenant **DataDomain** identity (**orgRefName**, **accountNum**, **tenantId**, and **dataSegment**), then stamp every matching row with one immutable purge batch reference and one **purgeAfter** timestamp. The shared database and realm are never dropped.

MongoDB TTL cleanup is asynchronous and may remove documents from different collections at different times. Revoking tenant access before expiration is therefore the logical atomic barrier: users cannot observe a partially drained tenant. Before the workflow reports the pooled purge as scheduled, it must reconcile archive and expiration counts for every collection declared tenant-scoped.

Revocation verification includes already-issued tokens or sessions, not only future logins. Suspending a system-plane membership by itself is not a safe barrier for a stateless tenant plane. A deployment must install a revocation provider that invalidates active access (or waits behind an equivalent verified write/read fence) before archive and expiration proceed.

Before a pooled purge is planned, the persisted collection inventory must be reconciled with the collection ownership catalog. Every existing user collection must have exactly one scope:

- **TENANT_SCOPED**: selectable by the canonical tenant identity and eligible for tenant archive and expiration.
- **REALM_SHARED**: shared within the realm and excluded from every individual tenant purge.
- **SYSTEM_GLOBAL**: owned by the system management plane and excluded from tenant purge.

An existing collection without a scope declaration blocks the operation. Declaring a collection as shared or global also forbids tenant-selection and expiration field mappings for that collection.

The default Mongo registry is configured with comma-separated collection names:

```
quantum.tenant-storage.tenant-scoped-  
collections=orders,orderEvents,tenantFiles.files,tenantFiles.chunks  
quantum.tenant-storage.realm-shared-collections=realmConfiguration  
quantum.tenant-storage.system-global-collections=
```

Only MongoDB internal `system.*` namespaces are excluded automatically. GridFS and other blob collections are ordinary user collections for this purpose and must be classified explicitly. Reading the inventory does not create a Morphia datastore, collections, or indexes.

TTL indexes must exist on the configured `purgeAfter` field for every tenant-scoped collection. Realm bootstrap/migrations should install them; the default Mongo expiration provider also creates the absolute-time TTL index idempotently before it stamps rows. Large collections may be processed in resumable batches, but every batch uses the same purge reference and timestamp. Blob payloads owned by tenant rows are part of both archive and expiration coverage.

`ownerId` is not part of tenant deletion identity. It can narrow policy access, but using it for archive or purge selection can leave tenant-owned data behind. Direct `DELETE /admin/tenants/{realmId}` remains disabled until a governed control-plane workflow supplies archive verification, collection-scope metadata, and purge reconciliation.

Direct tenant deletion is disabled. Decommissioning must use the governed control-plane lifecycle. In particular, pooled tenant removal is a scoped DataDomain operation and must never drop its shared realm/database.

2. Tenant onboarding workflow: define the journey after invite/registration

Tenant onboarding workflow configuration is exposed at:

- `GET /onboarding/workflow/current`
- `POST /onboarding/workflow/current`

This is tenant-scoped configuration. The active realm is derived from the current security context, so the workflow is saved and read per tenant realm.

The persisted model is `TenantOnboardingWorkflow`, which includes:

- `workflowEnabled`
- `inviteRequired`
- `registrationRequired`
- `surveyRequired`
- `adminApprovalRequired`
- `autoAssignSurveyOnInvite`
- `defaultSurveyRefName`
- `defaultInviteEmailTemplateKey`

- `workflowDefinitionJson`
- `completionMessage`

`defaultInviteEmailTemplateKey` is the email-template key the onboarding flow should use for invite messages. The framework default is `tenant-onboarding-invite`, which can be satisfied by:

- a classpath template under `templates/email/tenant-onboarding-invite/`
- a realm-scoped override in `email_templates`
- a workflow update that points to a different key

By default, the workflow steps are generated in this order:

- `invite`
- `registration`
- `survey`
- `approval`
- `activation`

Each step can be enabled or omitted by toggling the corresponding booleans. When no custom workflow JSON is supplied, the framework generates a default `workflowDefinitionJson` using those flags.

Representative response:

```
{
  "refName": "default-tenant-onboarding",
  "displayName": "Tenant User Onboarding",
  "activeStatus": true,
  "inviteRequired": true,
  "registrationRequired": true,
  "surveyRequired": true,
  "adminApprovalRequired": true,
  "autoAssignSurveyOnInvite": true,
  "defaultSurveyRefName": "tenant-user-onboarding",
  "defaultInviteEmailTemplateKey": "tenant-onboarding-invite",
  "completionMessage": "Your tenant onboarding is complete. You can now sign in and begin using the workspace.",
  "steps": [
    { "key": "invite", "type": "access_invite", "required": true },
    { "key": "registration", "type": "registration_request", "required": true },
    { "key": "survey", "type": "survey", "required": true, "surveyRefName": "tenant-user-onboarding" },
    { "key": "approval", "type": "admin_approval", "required": true },
    { "key": "activation", "type": "account_activation", "required": true }
  ]
}
```

This workflow config answers "what steps should happen?" It does not by itself send invites or create tenant users.

3. Registration requests: capture onboarding data for approval

The registration-request endpoints are exposed at:

- `POST /onboarding/registrationRequest/create`
- `POST /onboarding/registrationRequest/approve`

This API is the registration step that the onboarding workflow can reference through the `registration_request` step type. It is useful when a tenant wants users or organizations to submit onboarding information before access is approved.

At a high level:

- `create` persists an `ApplicationRegistration`
- `approve` transitions an existing registration request through the repository approval logic

This is the structured "please review my onboarding submission" side of the journey, whereas access invites are the "you have been invited into this tenant" side.

4. Access invites: grant a user into an existing tenant

Access invites are exposed at:

- `GET /access/invites`
- `GET /access/invites/{refName}`
- `POST /access/invites`
- `POST /access/invites/{refName}/revoke`
- `POST /access/invites/accept`

The first four require an authenticated caller in the target tenant. Invite acceptance is `@PermitAll` because the recipient may not yet have tenant access.

The persisted `AccessInvite` model includes:

- `email` or `targetUserId`
- `invitedByUserId`
- `scopeRefs`
- `grantedRoles`
- `allowedFunctionalAreas`
- `allowedFunctionalDomains`
- `allowedActions`
- `inviteMessage`

- `expiresAt`
- `acceptedAt`
- `acceptedUserId`
- `status` of `PENDING`, `ACCEPTED`, `REVOKED`, or `EXPIRED`

Representative invite creation request:

```
{
  "email": "jane.doe@partner.example",
  "scopeRefNames": ["north-america"],
  "grantedRoles": ["user"],
  "allowedFunctionalAreas": ["integration"],
  "allowedFunctionalDomains": ["exchange", "workflow"],
  "allowedActions": ["view", "create"],
  "expiresInDays": 14,
  "inviteMessage": "Join the tenant workspace for partner onboarding."
}
```

Representative creation response:

```
{
  "refName": "invite-1775100000000",
  "email": "jane.doe@partner.example",
  "scopeRefNames": ["north-america"],
  "grantedRoles": ["user"],
  "allowedFunctionalAreas": ["integration"],
  "allowedFunctionalDomains": ["exchange", "workflow"],
  "allowedActions": ["view", "create"],
  "status": "PENDING",
  "inviteToken": "raw-token-returned-at-create-time-only"
}
```

Important invite behavior:

- either `email` or `targetUserId` is required
- only one active pending invite is allowed for the same email or target user
- tokens are stored as a hash; the raw token is only returned at invite creation time
- invites expire automatically based on `expiresAt`
- invites can be revoked before acceptance

Representative accept request:

```
{
  "realm": "acme-example",
  "token": "raw-token-returned-at-create-time-only",
}
```

```
"email": "jane.doe@partner.example",
"firstName": "Jane",
"lastName": "Doe",
"password": "choose-a-password"
}
```

Representative accept response:

```
{
  "userId": "jane.doe@partner.example",
  "email": "jane.doe@partner.example",
  "defaultRealm": "acme-example",
  "inviteRefName": "invite-1775100000000",
  "grantedScopes": ["north-america"]
}
```

What happens when an invite is accepted:

- the invite token is validated and checked for expiry
- the user identity is resolved from the authenticated user, request payload, invite target user, or invite email
- if the user does not already exist, a new credential is created and a password is required
- if the user already exists, the service ensures the user is authorized for the tenant realm
- a `UserProfile` is created if needed
- the invite status is changed to `ACCEPTED`

Invite extension points

The invite flow is intentionally extensible through two SPIs:

- `AccessInviteProvisioner`
- `AccessInviteNotificationService`

`AccessInviteProvisioner` can:

- validate requested scopes when the invite is created
- perform application-specific provisioning when the invite is accepted

`AccessInviteNotificationService` can:

- deliver the invite token through email, messaging, or another notification mechanism

The framework ships with no-op default beans for both, so applications can opt in to deeper behavior without rewriting the invite flow.

How these pieces fit together

A common pattern looks like this:

1. An admin provisions the tenant with `POST /admin/tenants`.
2. The tenant configures or accepts the default onboarding workflow with `GET/POST /onboarding/workflow/current`.
3. A tenant admin creates an access invite for a user with `POST /access/invites`.
4. The invited user accepts the invite with `POST /access/invites/accept`.
5. If the workflow requires registration, survey, or approval, those steps are completed before the tenant considers onboarding complete.

This separation gives you flexibility:

- use tenant provisioning for admin-created tenants
- use registration requests for self-service intake and approval
- use access invites for controlled user admission into an existing tenant
- use workflow configuration to decide which of those steps are mandatory in each tenant

4.1.5. Choosing a Tenancy Model: Three Options

With realm membership (see [design/realm-membership-adr.adoc](#)) the platform supports three deliberate configurations:

Option	Shape	When to use
One tenant, one realm (default)	<code>POST /admin/tenants</code> provisions a realm with a single OWNER membership.	The standard SaaS tenant: full physical isolation, independent lifecycle, own backups/residency. This is what a multi-tenant platform provisions per customer.
Shared collaboration realm	One OWNER org plus N PARTICIPANT memberships (<code>RealmTenantMembership.membershipRole</code>). All parties' data lives in one realm; visibility is shaped per org by DataDomain stamps plus relationship policies (<code>hasEdge</code>) and field-level exclusions.	Multi-party operations against shared entities — e.g. a fulfillment realm where carrier, supplier, and customer orgs collaborate on the same sales orders and fulfillment plans, each seeing their own slice.
GitHub-like multi-realm identity	Users hold ONE global identity and N per-realm role assignments (<code>UserRealmRole</code>): ADMIN in realm A, VIEWER in realm B.	Consultants, partners, and platform operators who work across several tenants without duplicate accounts.

4.1.6. The GitHub-Like Identity Model, Mapped

GitHub concept	Quantum primitive	Notes
Account	Credential (system realm)	One global identity; <code>defaultRealm</code> is the personal/default context.
Organization	Organization + Account entities	Carry <code>DataDomain</code> identity inside realms.
Org membership + role	<code>UserRealmRole</code> (userId, realmRefName, roles[], sponsoringOrgRefName)	ADMIN in A / VIEWER in B = two documents. <code>status</code> : active/invited/suspended.
Repository	Realm	The unit of placement and lifecycle (own database, migrations, seeds).
Repo transfer / ownership	<code>RealmTenantMembership.membershipRole = owner</code>	Exactly one owner per realm; owner holds lifecycle and billing authority.
Invitation	<code>AccessInvite (/access-invites)</code>	Existing invite flow grants a user into a realm; on acceptance a <code>UserRealmRole</code> records the granted roles.
Token scopes	JWT per-realm role claims	At issuance the IdP resolves <code>RealmMembershipService.rolesForUser(user, realm)</code> ; the tenant plane validates via JWKS and routes by claims — no callback to the control plane per request.

Resolution APIs live in `RealmMembershipService` (quantum-system): `membershipsForOrg`, `membersOfRealm`, `ownerOfRealm`, `realmsForUser`, `rolesForUser`. Embedded deployments resolve locally; remote (tier-2) deployments get the same surface backed by the control-plane client (Phase C).

4.1.7. User-Definable Facet Segmentation & Coordinate SPI in Multi-Tenant Environments

Traditional multi-tenancy models stop at isolating database collections by a single tenant ID. However, enterprise SaaS platforms often require multidimensional logical segmentation within and across tenants — such as isolating data by geographic region, operating subsidiary, business unit, partner channel, or sensitivity facet.

Quantum delivers this capability through the **Store-Free Coordinate SPI** (`DataDomainResolver`) and **User-Definable Facet Segmentation**, allowing a single server instance to support heterogeneous segmentation policies for different tenants.

Physical Realm Isolation vs. Logical Facet Segmentation

Quantum maintains a strict distinction between physical database placement and logical row-level segmentation:

- **Physical Realm Isolation:** Dictates MongoDB cluster and database routing (`DEDICATED` vs `SHARED`)

/POOLED). It acts as an inviolate physical admission gate.

- **Logical Facet Segmentation:** Dictates which rows, relationships, and fields within a realm are visible to a given principal based on multidimensional coordinates (e.g. `tenantId`, `orgRefName`, `department`, `region`, `classificationTier`).

Single Server Instance with Per-Tenant Heterogeneous Policies

A single Quantum server instance natively supports distinct, user-defined segmentation policies for different tenants without requiring separate runtime processes or re-deployments:

- **Tenant Isolation:** In dedicated realms, policies reside in the tenant's own database; in pooled realms, policies are isolated by `tenantId`.
- **Custom Dimension Modeling:** Tenant A can segment data by `department` and `complianceTier`, while Tenant B concurrently segments data by `region` and `partnerId`.
- **Dynamic Variable Injection:** Policies automatically bind contextual coordinates into `RuleContext` (`${pTenantId}`, `${orgRefName}`, `${ownerId}`, `${policyFilter}`, and custom facet variables), which are evaluated transparently across both single-collection queries and multi-hop aggregation pipelines.

Configuring User-Definable Facets

Facet definitions and policy rules are declared via `DataDomainComponentBinding` and `DataDomainPolicyEntry`:

Configuring Facet Bindings (`DataDomainComponentBinding`)

```
{
  "tenantId": "tenant-acme",
  "name": "OrderSegmentation",
  "facets": {
    "region": {
      "type": "FIELD",
      "targetPath": "shippingAddress.region"
    },
    "department": {
      "type": "FIELD",
      "targetPath": "departmentCode"
    },
    "classification": {
      "type": "VARIABLE",
      "targetPath": "principal.securityClearance"
    }
  }
}
```

Configuring Policy Constraints (`DataDomainPolicyEntry`)

```
{
  "tenantId": "tenant-acme",
```

```

"targetClass": "com.e2eq.sample.Order",
"action": "find",
"filter": "status:!\"ARCHIVED\" && region:${userRegion}",
"facetFilters": {
  "department": "${userDepartment}",
  "classification": "CONFIDENTIAL"
}
}

```

Coordinate Resolution Precedence and Defaults

When a request arrives at `QueryGatewayResource` or a governed `MorphiaRepo`, `DataDomainResolver` resolves coordinates using a strict three-tier precedence hierarchy:

```

flowchart TD
    Req["Incoming Query / Traversal Request"] --> T1["Tier 1: Tenant-Specific Policy?<br/>(systemDataDomainPolicies for Class + Action)"]
    T1 -- Yes --> ApplyTenant["Apply Tenant Policy & Bind Custom Facets"]
    T1 -- No --> T2["Tier 2: Global / System Policy Provider?"]
    T2 -- Yes --> ApplyGlobal["Apply Global Policy & Bind Standard Facets"]
    T2 -- No --> T3["Tier 3: Caller PrincipalContext Present?<br/>(tenantId, ownerId, orgRefName)"]
    T3 -- Yes --> ApplyPrincipal["Apply Principal Default Coordinates"]
    T3 -- No --> FailClosed["DataDomainResolution.Unresolvable<br/>(FAIL CLOSED - HTTP 500 / 403)"]

    ApplyTenant --> ExecuteQuery["Execute Query / Traversal Pipeline"]
    ApplyGlobal --> Execute
    ApplyPrincipal --> Execute

```

Resolution Defaults Table

Resolution Stage	Resolution Source	Default / Fallback Behavior
Tenant Policy Configured	<code>systemDataDomainPolicies</code> collection for tenant.	Employs tenant's custom <code>filter</code> and <code>facetFilters</code> . Variables are injected into <code>RuleContext</code> .
No Tenant Policy Configured	System Global Policy Provider.	Employs platform-wide default data domain rules if registered; otherwise falls back to Tier 3.
Fallback (Zero Policy Records)	Caller's <code>PrincipalContext</code> .	Resolves default coordinates directly from user credentials: <code>pTenantId = principal.tenantId</code> , <code>ownerId = principal.userId</code> , <code>orgRefName = principal.orgRefName</code> .

Resolution Stage	Resolution Source	Default / Fallback Behavior
Missing Security Credentials	None.	Yields <code>DataDomainResolution.Unresolvable</code> . Fails closed immediately with HTTP 500 (<code>SecurityContextMissing</code>) or HTTP 403.

Multi-Hop Traversal Isolation

During multi-hop traversals (e.g. `Customer` → `Order` → `Supplier` via `expand(...)`):

1. `DataDomainResolver.resolveForHop(targetClass, viaPath, context)` is invoked at each relationship boundary.
2. Target entity policies and row-level security constraints are applied to that hop's MongoDB `$lookup` pipeline.
3. Access constraints **accumulate monotonically**: a caller cannot hop across a foreign reference to bypass tenant boundaries or see records they are not authorized to view in the target domain.
4. References pointing to external tenants or unauthorized records are pruned silently, returning `null` or empty arrays to eliminate information leakage.

Chapter 5. 5. Domain rule context

Problem: Applying business rules based on user, org, account, and tenant context.

Why for SaaS: Entitlements and behavior vary by tenant, role, and plan.

How Quantum helps: Built-in domain rule context to consistently pass identity and policy inputs.

Walkthrough: Inject and use the context in services and repositories.

Chapter 6. DomainContext, RuleContext, and DataDomain

Quantum enforces multi-tenant isolation and sharing through contextual data carried on models and evaluated at runtime.

6.1. DataDomain

Every persisted model includes a DataDomain that describes ownership and scope, commonly including fields such as:

- `tenantId`: Identifies the tenant
- `orgRefName`: Organization unit reference within a tenant
- `ownerId`: Owning user or system entity
- `realm`: Optional runtime override for partitioning

These fields enable filtering, authorization, and controlled sharing of data between tenants or org units.

6.2. DomainContext

DomainContext represents the current execution context for a request or operation, typically capturing:

- current tenant/org/user identity
- functional area / functional domain
- the action being executed (e.g., CREATE, UPDATE, VIEW, DELETE, ARCHIVE)

It feeds downstream components (repositories, resources) to consistently apply filtering and policy decisions.

6.3. RuleContext

RuleContext encapsulates policy evaluation. It can:

- Enforce whether an action is allowed for a given model and DataDomain
- Produce additional filters and projections used by repositories
- Grant cross-tenant read access for specific functional areas (e.g., shared catalogs) while keeping others strictly isolated

6.4. End-to-End Flow

1. A REST request enters a BaseResource-derived endpoint.

2. The resource builds a DomainContext from the security principal and request parameters.
3. RuleContext evaluates permissions and returns effective filters.
4. Repository applies filters (DataDomain-aware) to find/get/list/update/delete.
5. The model's UIActionList can be computed to reflect what the caller can do next.

This pattern ensures consistent enforcement across all CRUD operations, independent of the specific model or repository.

6.5. Resolvers and Variables in Rule Filters

RuleContext can attach FILTERs (not only ALLOW/DENY) to repository queries using rule fields and filter strings. Variables inside those filter strings are populated from:

- PrincipalContext and ResourceContext standard variables: `principalId`, `pAccountId`, `pTenantId`, `ownerId`, `orgRefName`, `resourceId`, `action`, `functionalDomain`, `area`
- Dotted coordinate namespaces:
 - `${facet.<key>}` & `${coord.<key>}`: Resolved dynamically from tenant `DataDomainPolicyEntry.facetFilters` or contextual coordinate maps
 - `${principal.<attr>}`: Extracted from authenticated principal claims (`userId`, `defaultRealm`, `tenantId`, custom attributes)
 - `${dd.<attr>}`: Extracted from target `DataDomain` coordinates (`tenantId`, `orgRefName`, `accountNum`, `dataSegment`, `ownerId`)
- AccessListResolver SPI implementations: per-request computed Collections (e.g., customer IDs the caller can access)

Implementation highlights: - `AccessListResolver` has methods `key()`, `supports(...)`, `resolve(...)`. Resolvers are injected and invoked for each request; results are published as variables by key. - `MorphiaUtils.VariableBundle` carries both string variables and typed object variables (including Collections and arrays) to the query listener. - The `QueryToFilterListener` and in-memory `QueryToPredicateJsonListener` support single-variable IN clauses with or without brackets (`field:^[${var}]` or `field:^[${var}]`), expanding Collections/arrays into native `$in` filters and coercing types (`ObjectId`, numbers, booleans, dates).

6.5.1. Business Scenario: Multi-Tenant Franchise & Dynamic Partner Segmentation

A national retail brand operates diverse franchised store chains sharing a centralized inventory platform. Access is governed not by static roles, but by custom dynamic coordinates: store tiers (`facet.tier: ["FLAGSHIP", "REGIONAL"]`) and sales zones (`coord.salesZone: "NORTHEAST"`).

6.5.2. How the Feature Supports the Business Solution

Administrators define a single generic security policy:

```
tier:^[${facet.allowedTiers} && zone:${coord.salesZone} && status:AVAILABLE
```

The `DataDomainResolver` SPI binds the caller's credentials into typed facet collections and zone variables at request time. Bracketless `IN (tier:^[${facet.allowedTiers})` automatically unpacks collections into native MongoDB `$in` operators. Unauthenticated or foreign-tenant requests fail closed.

Authoring examples: - Constrain by principal domain:

+

```
orgRefName:${orgRefName} && dataDomain.tenantId:${pTenantId}
```

- Bracketless single-variable IN for customer visibility or facet tiers:

```
customerId:^[${accessibleCustomerIds} && tier:^[${facet.allowedTiers}
```

For the complete query language reference, see [Query Language](#).

6.6. Concrete example: building and using a resolver

This section shows how to implement a resolver that restricts access to orders by the set of `customerIds` the current user is allowed to see.

6.6.1. 1) Implement the SPI

Create a CDI bean that implements `AccessListResolver`. It decides when it applies and returns a Collection of values. The collection can be `ObjectId`, `String`, numbers, etc.

```
import com.e2eq.framework.securityrules.AccessListResolver;
import com.e2eq.framework.model.persistent.base.UnversionedBaseModel;
import com.e2eq.framework.model.securityrules.PrincipalContext;
import com.e2eq.framework.model.securityrules.ResourceContext;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.inject.Inject;
import org.bson.types.ObjectId;
import java.util.*;

@ApplicationScoped
public class CustomerAccessResolver implements AccessListResolver {

    @Inject CustomerAccessService service; // your app-specific service

    @Override
    public String key() {
        // This becomes the variable name available to rules: ${accessibleCustomerIds}
    }
}
```

```

        return "accessibleCustomerIds";
    }

    @Override
    public boolean supports(PrincipalContext pctx, ResourceContext rctx,
                           Class<? extends UnversionedBaseModel> modelClass) {
        // Optionally narrow by area/domain/action/model
        return rctx != null && "sales".equalsIgnoreCase(rctx.getArea())
            && "order".equalsIgnoreCase(rctx.getFunctionalDomain());
    }

    @Override
    public Collection<?> resolve(PrincipalContext pctx, ResourceContext rctx,
                                Class<? extends UnversionedBaseModel> modelClass) {
        // Return the set of customer ids for this user; could be ObjectId or String.
        // Example returns strings; the query listener will coerce 24-hex to ObjectId.
        return service.findCustomerIdsForUser(pctx.getUserId());
    }
}

```

Notes: - You can return `List<ObjectId>` directly if you prefer; no coercion needed then. - The resolver runs per request. Cache internally if the computation is expensive.

6.6.2. 2) How RuleContext uses resolvers

At query time, `RuleContext` discovers all `AccessListResolver` beans and calls `supports(...)`. For those that apply, it invokes `resolve(...)` and publishes the result into the variable bundle under the provided key(). Variables are available to the BI-API query via `${...}`.

Internally this uses `MorphiaUtils.VariableBundle` and `QueryToFilterListener` to carry both strings and typed objects/collections. Starting with this release you can reuse the exact same variable bundle for custom evaluation paths. The `RuleContext.resolveVariableBundle(...)` helper runs all matching `AccessListResolver` beans and returns the maps you can feed directly into `QueryPredicates` (for in-memory rules) or any other component that understands `${var}` tokens.

```

PrincipalContext pc = ...; // build from request
ResourceContext rc = ...;

MorphiaUtils.VariableBundle vars = ruleContext.resolveVariableBundle(pc, rc,
    UserProfile.class);

Predicate<JsonNode> predicate = QueryPredicates.compilePredicate(
    "customerId^[${accessibleCustomerIds}]",
    vars.strings,
    vars.objects
);

JsonNode order = QueryPredicates.toJsonNode(Map.of("customerId",
    "5f1e1a5e5e5e5e5e5e5e5e5e51"));

```

```
boolean allowed = predicate.test(order); // true when resolver returned that ObjectId
```

This allows the same resolver outputs to drive Morphia queries and client-side predicate checks without duplicating resolver logic.

6.6.3. 3) Author a rule that consumes the variable

Given the resolver above, a rule can attach an IN filter to constrain queries:

```
// andFilterString (example)
customerId:^([${accessibleCustomerIds}])
```

When executed: - If `accessibleCustomerIds` is a Collection/array, each element is type-coerced (ObjectId, number, date, boolean, or string) and used in `$in`. - If `accessibleCustomerIds` is a comma-separated string, it is split and each token is coerced similarly. - An empty collection results in an empty `$in` (matches none), effectively denying access via filtering, not via ALLOW/DENY.

6.6.4. 4) End-to-end behavior

- SecurityFilter sets ResourceContext (area/domain/action) per request.
- RuleContext evaluates rules for the principal and resource and gathers resolvers.
- The repository composes filters including the rule-provided IN clause with the access list.
- Only documents whose customerId is in the caller's resolved set are returned.

6.6.5. String literals vs. typed values in resolver variables

When an `AccessListResolver` returns a list of values that will be used in an `IN` clause (for example, `field:^([${var}])`), the engine attempts to coerce each element to an appropriate type so Mongo/Morphia filters are typed correctly:

- 24-hex string → ObjectId
- `true/false` → Boolean
- integer → Long
- decimal → Double
- ISO-8601 datetime → `java.util.Date`
- `yyyy-MM-dd` → `java.time.LocalDate`
- otherwise → String

This works well when your target field is an ObjectId, number, or date. However, string fields can contain values that look like other types (for example, a 24-hex string that resembles an ObjectId). In those cases you must force "treat as plain string" so no coercion occurs.

To do this, the framework provides a small wrapper type `StringLiteral`. If a resolver returns `StringLiteral` instances, the listener unwraps them to plain `String` values and skips coercion

entirely.

Example A: Resolver returns ObjectIds (typed)

```
@ApplicationScoped
public class CustomerAccessResolver implements AccessListResolver {
    public static final ObjectId ID1 = new ObjectId("5f1e1a5e5e5e5e5e5e51");
    public static final ObjectId ID2 = new ObjectId("5f1e1a5e5e5e5e5e5e52");

    @Override public String key() { return "accessibleCustomerIds"; }
    @Override public boolean supports(PrincipalContext p, ResourceContext r, Class<?
extends UnversionedBaseModel> m) {
        return r != null && "sales".equalsIgnoreCase(r.getArea()) && "order"
.equalsIgnoreCase(r.getFunctionalDomain()) && "view".equalsIgnoreCase(r.getAction());
    }
    @Override public Collection<?> resolve(PrincipalContext p, ResourceContext r,
Class<? extends UnversionedBaseModel> m) {
        return java.util.List.of(ID1, ID2); // typed values pass through as-is
    }
}
```

Rule:

```
customerId:^(${accessibleCustomerIds})
```

Result: `$in` with `List<ObjectId>` on `customerId`.

Example B: Resolver returns String literals (force raw strings)

```
@ApplicationScoped
public class CustomerCodeResolver implements AccessListResolver {
    @Override public String key() { return "accessibleCustomerCodes"; }
    @Override public boolean supports(PrincipalContext p, ResourceContext r, Class<?
extends UnversionedBaseModel> m) {
        return r != null && "sales".equalsIgnoreCase(r.getArea()) && "order"
.equalsIgnoreCase(r.getFunctionalDomain()) && "view".equalsIgnoreCase(r.getAction());
    }
    @Override public Collection<?> resolve(PrincipalContext p, ResourceContext r,
Class<? extends UnversionedBaseModel> m) {
        return java.util.List.of(
            com.e2eq.framework.model.persistent.morphia.StringLiteral.of(
                "5f1e1a5e5e5e5e5e5e5e51"),
            com.e2eq.framework.model.persistent.morphia.StringLiteral.of("CUST-42")
        );
    }
}
```

Rule:

```
customerCode:^(${accessibleCustomerCodes}]
```

Result: `$in` with `List<String>` on `customerCode` (even for hex-like strings).

Other types supported

Resolvers can also return numbers, booleans, and dates/datetimes. Already-typed elements (`Number`, `Boolean`, `java.util.Date`, `java.time.LocalDate`, `ObjectId`) are preserved. String elements are heuristically parsed into those types unless wrapped with `StringLiteral`.

Authoring tips:

- Prefer returning already-typed values when you know the target field type.
- Use `StringLiteral` when a value might be misinterpreted (for example, 24-hex or numeric-looking strings).
- For CSV strings published under a variable, the engine splits by comma and applies the same per-element coercion.

6.6.6. Using AccessListResolver with Ontology (optional)

When ontology is enabled, an `AccessListResolver` can compute ID lists from semantic edges (materialized in Mongo) and publish them as variables for use in rule filters.

Example resolver (conceptual)

```
@ApplicationScoped
public class OrdersByOrgResolver implements AccessListResolver {
    @Inject EdgeDao edgeDao; // from quantum-ontology-mongo
    @Override public String key() { return "idsByPlacedInOrg"; }
    @Override public boolean supports(PrincipalContext p, ResourceContext r, Class<?
extends UnversionedBaseModel> model) {
        return model.getSimpleName().equals("Order");
    }
    @Override public Collection<?> resolve(PrincipalContext p, ResourceContext r, Class
<? extends UnversionedBaseModel> model) {
        String tenantId = p.getDataDomain().getTenantId();
        String org = p.getDataDomain().getOrgRefName();
        return edgeDao.srcIdsByDst(tenantId, "placedInOrg", org);
    }
}
```

Rule filter usage

```
id:^(${idsByPlacedInOrg}
```

- Always scope by tenantId from RuleContext/PrincipalContext.
- This is optional and only active if you wire ontology components. For a deeper integration path, see [Integrating Ontology](#).

6.6.7. Admitted vocabulary for rule predicates

When a rule references an ontology predicate through a query helper such as `hasEdge("placedInOrg", ...)`, the predicate must be **admitted** for policy use, not merely declared in the TBox. As `RuleContext` registers a rule it fires a `RuleVocabularyCheck`, and `PolicyVocabularyGuard` (`quantum-ontology-policy-bridge`) validates the referenced predicates against the realm's admitted set. A predicate that is undeclared (**ABSENT**) or declared-but-not-admitted (**PROVISIONAL**) fails rule registration, naming the rule and the offending predicate.

Enforcement is fail-closed: if the realm's admitted set cannot be read it is treated as **UNAVAILABLE** and the guard refuses to downgrade a governed realm to "all admitted." A legacy realm (`admittedPredicates == null`) admits all declared predicates for back-compatibility. See [Ontology Modeling: provisional vs. admitted vocabulary tier](#) for the tier model, and the Quantum Enterprise **Ontology governance control plane** chapter for the admin promote/demote endpoints.

Chapter 7. 6. Building RESTful CRUD APIs

Problem: Exposing standardized CRUD endpoints with minimal boilerplate.

Why for SaaS: Consistency across services reduces cognitive load and accelerates delivery.

How Quantum helps: Resource scaffolding, conventions, and helpers for common CRUD.

Walkthrough: Create controllers/resources for your model and wire persistence.

7.1. REST: Find, Get, List, Save, Update, Delete

Quantum provides consistent REST resources backed by repositories. Extend `BaseResource` to expose CRUD quickly and consistently.

7.1.1. Base Concepts

- `BaseResource<T, R extends Repo<T>>` provides endpoints for:
- `find`: query by criteria (filters, pagination)
- `get`: fetch by id or refName
- `list`: list all within scope with paging
- `save`: create
- `update`: modify existing
- `delete`: delete or soft-delete/archival depending on model
- `UIActionList`: derive available actions based on current model state.
- `DataDomain` filtering is applied across all operations to enforce multi-tenancy.

7.1.2. Example Resource

```
import com.e2eq.framework.rest.resources.BaseResource;
import jakarta.ws.rs.Path;

@Path("/products")
public class ProductResource extends BaseResource<Product, ProductRepo> {
}
```

7.1.3. Authorization Layers in REST CRUD

Quantum combines static, identity-based checks with dynamic, domain-aware policy evaluation. In practice you will often use both:

1) Hard-coded permissions via annotations

- Use standard Jakarta annotations like `@RolesAllowed` (or the framework's `@RoleAllow` if

present) on resource classes or methods to declare role-based checks that must pass before executing an endpoint.

- These checks are fast and decisive. They rely on the caller's roles as established by the current `SecurityIdentity`.

Example:

```
import jakarta.annotation.security.RolesAllowed;

@RolesAllowed({"ADMIN", "CATALOG_EDITOR"})
@Path("/products")
public class ProductResource extends BaseResource<Product, ProductRepo> {
    // Only ADMIN or CATALOG_EDITOR can access all inherited CRUD endpoints
}
```

2) JWT groups and role mapping

- When using the JWT provider, the token's groups/roles claims are mapped into the Quarkus `SecurityIdentity` (see the Authentication guide).
- Groups in JWT typically become roles on `SecurityIdentity`; these roles are what `@RolesAllowed/@RoleAllow` checks evaluate.
- You can augment or transform roles using a `SecurityIdentityAugmentor` (see `RolesAugmentor` in the framework) to add derived roles based on claims or external lookups.

3) RuleContext layered authorization (dynamic policies)

- After annotation checks pass, `RuleContext` evaluates domain-aware permissions. This layer can:
- Enforce `DataDomain` scoping (tenant/org/owner)
- Allow cross-tenant reads for specific functional areas when policy permits
- Contribute query predicates and projections to repositories
- Think of `@RolesAllowed/@RoleAllow` as the coarse-grained gate, and `RuleContext` as the fine-grained, context-sensitive policy engine.

4) Quarkus SecurityIdentity and SecurityFilter

- Quarkus produces a `SecurityIdentity` for each request containing principal name and roles.
- The framework's `SecurityFilter` inspects the incoming request (e.g., JWT) and populates/augments the `SecurityIdentity` and the derived `DomainContext` used by `RuleContext` and repositories.
- `BaseResource` and underlying repos (e.g., `MorphiaRepo`) consume `SecurityIdentity/DomainContext` to apply permissions and filters consistently.

For detailed rule-base matching (URL, headers, body predicates, priorities), see the Permissions section.

7.1.4. Querying

- Use query parameters or a request body (depending on your API convention) to express filters.
- RuleContext contributes tenant-aware filters and projections automatically.
- See [Query Language](#) for the full BIAPIQuery syntax, including array filtering with elemMatch and IN-clause enhancements that accept resolver-provided lists.

7.1.5. Responses and Schemas

- Models are returned with calculated fields (e.g., actionList) when appropriate.
- OpenAPI annotations in your models/resources integrate with MicroProfile OpenAPI for schema docs.

7.1.6. Error Handling

- Validation errors (e.g., ImportRequiredField, Size) return helpful messages.
- Rule-based denials return appropriate HTTP statuses (403/404) without leaking cross-tenant metadata.

Query Language (ANTLR-based)

The find/list endpoints accept a filter string parsed by an ANTLR grammar (BIAPIQuery.g4). Use the filter query parameter to express predicates; combine them with logical operators and grouping. Sorting and projection are separate query parameters.

- Operators:
- Equals: '='
- Not equals: '!='
- Less than/Greater than: '<' / '>'
- Less-than-or-equal/Greater-than-or-equal: '<=' / '>='
- Exists (field present): '~' (no value)
- In list: '^' followed by [v1,v2,...]
- Boolean literals: true/false
- Null literal: null
- Logical:
- AND: '&&'
- OR: '||'
- NOT: '!' (applies to a single allowed expression)
- Grouping: parentheses '(' and ')'
- Values by type:
- Strings: unquoted or quoted with "..."; quotes allow spaces and punctuation

- Whole numbers: prefix with '#' (e.g., #10)
- Decimals: prefix with '.' (e.g., 19.99)
- Date: yyyy-MM-dd (e.g., 2025-09-10)
- DateTime (ISO-8601): 2025-09-10T12:30:00Z (timezone supported)
- ObjectId (Mongo 24-hex): 5f1e9b9c8a0b0c0d1e2f3a4b
- Reference by ObjectId: @@5f1e9b9c8a0b0c0d1e2f3a4b
- Variables:
 \${ownerId|principalId|resourceId|action|functionalDomain|pTenantId|pAccountId|rTenantId|rAccountId|realm|area}

7.1.7. Simple filters (equals)

```
# string equality
name:"Acme Widget"
# whole number
quantity:#10
# decimal number
price:##19.99
# date and datetime
shipDate:2025-09-12
updatedAt:2025-09-12T10:15:00Z
# boolean
active:true
# null checks
description:null
# field exists
lastLogin:~
# object id equality
id:5f1e9b9c8a0b0c0d1e2f3a4b
# variable usage (e.g., tenant scoping)
dataDomain.tenantId:${pTenantId}
```

7.1.8. Advanced filters: grouping and AND/OR/NOT

```
# Products that are active and (name contains widget OR gizmo), excluding discontinued
active:true && (name:*widget* || name:*gizmo*) && status:! "DISCONTINUED"

# Shipments updated after a date AND (destination NY OR CA)
updatedAt:>=2025-09-01 && (destination:"NY" || destination:"CA")

# NOT example: items where category is not null and not (price < 10)
category:!null && !(price:<##10)
```

Notes: - Wildcard matching uses ': **name:*widget** (prefix/suffix/contains). '?' matches a single character. - Use parentheses to enforce precedence; otherwise AND/OR follow standard left-to-right

with explicit operators.

7.1.9. IN lists

```
status:^[ "OPEN", "CLOSED", "ON_HOLD" ]
ownerId:^[ "u1", "u2", "u3" ]
referenceId:^[ @05f1e9b9c8a0b0c0d1e2f3a4b, @06a7b8c9d0e1f2a3b4c5d6e7f ]
```

7.1.10. Sorting

Provide a sort query parameter (comma-separated fields): - '-' prefix = descending, '+' or no prefix = ascending.

Examples:

```
# single field descending
?sort=-createdAt

# multiple fields: createdAt desc, refName asc
?sort=-createdAt,refName
```

7.1.11. Projections

Limit returned fields with the projection parameter (comma-separated): - '+' prefix = include, '-' prefix = exclude.

Examples:

```
# include only id and refName, exclude heavy fields
?projection=+id,+refName,-auditInfo,-persistentEvents
```

7.1.12. End-to-end examples

- GET `/products/list?skip=0&limit=50&filter=active:true&&name:*widget*&sort=-updatedAt&projection=+id,+name,-auditInfo`
- GET `/shipments/list?filter=(destination:"NY" | | destination:"CA")&&updatedAt:>=2025-09-01&sort=origin`

These features integrate with RuleContext and DataDomain: your filter runs within the tenant/org scope derived from the security context; RuleContext may add further predicates or projections automatically.

7.2. CSV Export and Import

These endpoints are inherited by every resource that extends BaseResource. They are mounted

under the resource's base path. For example, PolicyResource at /security/permission/policies exposes:

- GET /security/permission/policies/csv
- POST /security/permission/policies/csv
- POST /security/permission/policies/csv/session
- POST /security/permission/policies/csv/session/{sessionId}/commit
- DELETE /security/permission/policies/csv/session/{sessionId}
- GET /security/permission/policies/csv/session/{sessionId}/rows

Authorization and scoping:

- All CSV endpoints are protected by the same @RolesAllowed("user", "admin") checks as other CRUD operations.
- RuleContext filters and DataDomain scoping apply the same way as list/find; exports stream only what the caller may see, and imports are saved under the same permissions.
- In multi-realm deployments, include your X-Realm header as you do for CRUD; underlying repos resolve realm and domain context consistently.

7.2.1. Export: GET /csv

Produces a streamed CSV download of the current resource collection.

Query parameters and behavior:

fieldSeparator (default ",")

Single character used to separate fields. Typical values: `,`, `;`, `\t`.

requestedColumns (default refName)

Comma-separated list of model field names to include, in output order. If omitted, BaseResource defaults to refName. Nested list extraction is supported with the `[0]` notation on a single nested property across all requested columns (e.g., `addresses[0].city`, `addresses[0].zip`). Indices other than `[0]` are rejected. If the nested list has multiple items, multiple rows are emitted per record (one per list element), preserving other column values.

quotingStrategy (default QUOTE_WHERE_ESSENTIAL)

- QUOTE_WHERE_ESSENTIAL: quote only when needed (when a value contains the separator or quoteChar).
- QUOTE_ALL_COLUMNS: quote every column in every row.

quoteChar (default ")

The character used to surround quoted values.

decimalSeparator (default .)

Reserved for decimal formatting. Note: current implementation ignores this value; decimals are rendered using the locale-independent dot.

charsetEncoding (default UTF-8-without-BOM)

One of: `US-ASCII`, `UTF-8-without-BOM`, `UTF-8-with-BOM`, `UTF-16-with-BOM`, `UTF-16BE`, `UTF-16LE`. “with-BOM” values write a Byte Order Mark at the beginning of the file (UTF-8: `EF BB BF`; UTF-16: `FE FF`).

filter (optional)

ANTLR DSL filter applied server-side before streaming (see Query Language section). Reduces rows and can improve performance.

filename (default downloaded.csv)

Suggested download filename returned via Content-Disposition header.

offset (default 0)

Zero-based index of the first record to stream.

length (default 1000, use -1 for all)

Maximum number of records to stream from offset. Use `-1` to stream all (be mindful of client memory/time).

prependHeaderRow (optional boolean, default false)

When true, the first row contains column headers. Requires `requestedColumns` to be set (the default `refName` satisfies this requirement).

preferredColumnNames (optional list)

Overrides header names positionally when `prependHeaderRow=true`. The list length must be $\leq$ `requestedColumns`; an empty string entry means “use default field name” for that column.

Response:

- 200 OK with Content-Type: text/csv and Content-Disposition: attachment; filename="..."
- On validation/processing errors, the response status is 400/500 and the body contains a single text line describing the problem (e.g., “Incorrect information supplied: ...”). Unrecognized query parameters are rejected with 400.

Examples:

- Export selected fields with header, custom filename and filter

```
curl -H "Authorization: Bearer $JWT" \
  -H "X-Realm: system-com" \
  "https://host/api/products/csv?requestedColumns=id,refName,price&prependHeaderRow=true
  &filename=products.csv&filter=active:true&sort=+refName"
```

- Export nested list's first element across columns

```
# emits one row per address entry when more than one is present
```

```
curl -H "Authorization: Bearer $JWT" \  
"https://host/api/customers/csv?requestedColumns=refName,addresses[0].city,addresses[0]  
.zip&prependHeaderRow=true"
```

7.2.2. Import: POST /csv (multipart)

Consumes a CSV file (multipart/form-data) and imports records in batches. The form field name for the file is file.

Query parameters and behavior:

fieldSeparator (default ",")

Single character expected between fields.

quotingStrategy (default QUOTE_WHERE_ESSENTIAL)

Same values as export; controls how embedded quotes are recognized.

quoteChar (default ")

The expected quote character in the file.

skipHeaderRow (default true)

When true, the first row is treated as a header and skipped. Mapping is positional, not by header names.

charsetEncoding (default UTF-8-without-BOM)

The file encoding. "with-BOM" variants allow consuming a BOM at the start.

requestedColumns (required)

Comma-separated list of model field names in the same order as the CSV columns. This positional mapping drives parsing and validation. Nested list syntax `[0]` is allowed with the same constraints as export.

Behavior:

- Each row is parsed into a model instance using type-aware processors (ints, longs, decimals, enums, etc.).
- Bean Validation is applied; rows with violations are collected as errors and not saved; valid rows are batched and saved.
- For each saved batch, insert vs update is determined by refName presence in the repository.
- Response entity includes counts (importedCount, failedCount) and per-row results when available.
- Response headers:
 - X-Import-Success-Count: number of rows successfully imported.
 - X-Import-Failed-Count: number of rows that failed validation or DB write.
 - X-Import-Message: summary message.

Example (direct import):

```
curl -X POST \
  -H "Authorization: Bearer $JWT" \
  -H "X-Realm: system-com" \
  -F "file=@policies.csv" \

"https://host/api/security/permission/policies/csv?requestedColumns=refName,principalId,description&skipHeaderRow=true&fieldSeparator=,&quoteChar=\"&quotingStrategy=QUOTE_WHERE_ESSENTIAL&charsetEncoding=UTF-8-without-BOM"
```

7.2.3. Import with preview sessions

Use a two-step flow to analyze first, then commit only valid rows.

- POST /csv/session (multipart): analyzes the file and creates a session
 - Same parameters as POST /csv (fieldSeparator, quotingStrategy, quoteChar, skipHeaderRow, charsetEncoding, requestedColumns).
 - Returns a preview ImportResult including sessionId, totals (totalRows, validRows, errorRows), and row-level findings. No data is saved yet.
- POST /csv/session/{sessionId}/commit: imports only error-free rows from the analyzed session
 - Returns CommitResult with inserted/updated counts.
 - DELETE /csv/session/{sessionId}: cancels and discards session state (idempotent; always returns 204).
- GET /csv/session/{sessionId}/rows: page through analyzed rows
 - Query params:
 - skip (default 0), limit (default 50)
 - onlyErrors (default false): when true, returns only rows with errors
 - intent (optional): filter rows by intended action: INSERT, UPDATE, or SKIP

Notes and constraints:

- requestedColumns must reference actual model fields. Unknown fields or multiple different nested properties are rejected (only one nested property across requestedColumns is allowed when using [0]).
- Unrecognized query parameters are rejected with HTTP 400 to prevent silent misconfiguration.
- Very large exports should prefer streaming with sensible length settings or server-side filters to reduce memory and time.
- Imports run under the same security rules as POST / (save). Ensure the caller has permission to create/update the target entities in the chosen realm.

7.2.4. Enhanced Import with ImportProfile

ImportProfile provides reusable, configurable transformation and mapping rules for CSV imports. Profiles are persistent entities that can be created via REST and referenced during imports.

Key Features

- **Value Mapping:** Transform CSV values before type conversion (e.g., "Y" → **true**, "Active" → **ACTIVE**)
- **Lookup Resolution:** Resolve foreign references by looking up values in other collections
- **String Transformations:** Trim, case conversion, regex replacement
- **Intent Control:** Specify INSERT vs UPDATE per row via an intent column
- **Header Modifiers:** Mark fields as required (*), optional (?), calculated (~), or key (#)

Using a Profile

Create a profile first, then reference it during import:

```
# Create profile
curl -X POST \
  -H "Authorization: Bearer $JWT" \
  -H "Content-Type: application/json" \
  "https://host/api/integration/import-profiles" \
  -d @product-import-profile.json

# Import using profile
curl -X POST \
  -H "Authorization: Bearer $JWT" \
  -F "file=@products.csv" \
  "https://host/api/products/csv/session/profile?profileRefName=product-import-v1"
```

Profile with Session Flow

- POST `/api/{entity}/csv/session/profile` (multipart): analyzes the file using the specified profile
 - Query param: **profileRefName** - reference to an existing ImportProfile
 - Returns preview ImportResult including sessionId, totals, and row-level findings
- POST `/api/{entity}/csv/session/{sessionId}/commit`: imports valid rows (same as basic flow)

Example ImportProfile (JSON)

```
{
  "refName": "product-import-v1",
  "displayName": "Product Import Profile",
  "targetType": "com.example.Product",
  "columnMappings": [
    {
```

```

    "sourceColumn": "Status",
    "targetField": "status",
    "valueMappings": {
      "A": "ACTIVE",
      "I": "INACTIVE",
      "D": "DISCONTINUED"
    },
    "unmappedValueBehavior": "FAIL"
  },
  {
    "sourceColumn": "Category",
    "targetField": "categoryRefName",
    "lookup": {
      "lookupCollection": "Category",
      "lookupMatchField": "displayName",
      "lookupReturnField": "refName",
      "onNotFound": "FAIL"
    }
  }
],
"intentColumn": "_action",
"defaultIntent": "UPSERT"
}

```

Intent Column

When `intentColumn` is configured, each row can specify its own intent:

Intent	Behavior
INSERT	Insert new record. Fails if refName already exists.
UPDATE	Update existing record. Fails if refName does not exist.
UPSERT	Auto-detect: INSERT if new, UPDATE if exists.
SKIP	Skip this row entirely.



MERGE and **DELETE** are intentionally not supported for safety reasons.

For complete ImportProfile documentation, see [CSV Import Enhancements](#).

7.2.5. RowValueResolver: Custom Per-Row Logic

The `RowValueResolver` SPI allows arbitrary code execution for each row during import. Unlike static lookups, `RowValueResolvers` have access to all columns in the row and can invoke external services.

Key Features

- **Arc CDI Beans:** Full dependency injection support

- **Multi-Collection Updates:** Update multiple collections from a single CSV row
- **Full Row Access:** Read any column value, not just the mapped column
- **Context Awareness:** Access to realm, session, row number, and profile

Example Implementation

```
@ApplicationScoped
public class GeocodeResolver implements RowValueResolver {

    @Inject
    GeocodingService geocodingService;

    @Override
    public String getName() {
        return "geocodeResolver";
    }

    @Override
    public ResolveResult resolve(String inputValue, Map<String, Object> rowData,
    ImportContext context) {
        String street = (String) rowData.get("street");
        String city = (String) rowData.get("city");
        String state = (String) rowData.get("state");

        GeocodingResult result = geocodingService.geocode(
            String.format("%s, %s, %s", street, city, state));

        if (result != null && result.isValid()) {
            return ResolveResult.success(new GeoPoint(result.getLatitude(), result
            .getLongitude()));
        }
        return ResolveResult.nullValue();
    }
}
```

Configuration in ImportProfile

```
{
  "columnMappings": [
    {
      "sourceColumn": "street",
      "targetField": "location",
      "rowValueResolverName": "geocodeResolver"
    }
  ]
}
```

Result Types

Method	Description
<code>ResolveResult.success(value)</code>	Set the field to the resolved value
<code>ResolveResult.passthrough(original)</code>	Keep the original value unchanged
<code>ResolveResult.nullValue()</code>	Set the field to null
<code>ResolveResult.skip()</code>	Skip this row (not an error)
<code>ResolveResult.error(message)</code>	Mark the row as an error

For complete RowValueResolver documentation, see [CSV Import Enhancements](#).

7.2.6. Dynamic Attribute Import

For entities with dynamic attributes (schema-less, nested key-value structures), Quantum supports multiple import strategies.

Import Strategies

Strategy	Description
DOT_NOTATION	Use <code>dyn.setName.attrName[:type]</code> column headers. Best for spreadsheet-friendly imports.
JSON_COLUMN	Single JSON column containing all attributes. Best for programmatic imports.
VERTICAL	Multiple rows per entity with one attribute per row. Best for attribute-centric workflows.
HYBRID	Combines DOT_NOTATION and JSON_COLUMN in the same import.

DOT_NOTATION Example

```
name,sku,dyn.logistics.weight:Double,dyn.logistics.hazmat:Boolean,dyn.compliance.ce_mark:Boolean
"Widget Pro","WGT-001",2.5,false,true
"Gadget X","GDG-002",1.2,false,true
```

JSON_COLUMN Example

```
name,sku,dynamicAttributes
"Widget Pro","WGT-001","{"logistics":{"weight":2.5,"hazmat":false},"compliance":{"ce_mark":true}}"
```

Configuration in ImportProfile

```
{
  "refName": "product-import-dynamic",
  "dynamicAttributeStrategy": "DOT_NOTATION",
  "dynamicAttributePrefix": "dyn.",
  "enableDynamicAttributeDiscovery": true,
  "dynamicAttributeMergeStrategy": "MERGE"
}
```

Merge Strategies

Strategy	Behavior
REPLACE	Remove all existing attributes, replace with imported
MERGE	Keep existing attributes, update matching, add new
APPEND	Keep existing attributes, only add new (no updates)

For complete dynamic attribute import documentation, see [CSV Dynamic Attributes Design](#).

Chapter 8. Authentication and Authorization

Quantum integrates with Quarkus security while providing a pluggable approach to authentication. The repository includes a JWT provider module to get started quickly and an extension surface to replace or complement it.

8.1. JWT Provider

- Module: `quantum-jwt-provider`
- Purpose: Validate JWTs on incoming requests, populate the security principal, and surface tenant/org/user claims that feed `DomainContext`.
- Configuration: Standard Quarkus/MicroProfile JWT properties plus custom claim mappings as needed for `DataDomain`.

8.2. Pluggable Authentication

You can introduce alternative authentication mechanisms (e.g., API keys, SAML/OIDC front-channel tokens exchanged for back-end JWTs, HMAC signatures) by providing CDI beans that integrate with the security layer and emit the same normalized context consumed by `DomainContext/RuleContext`.

Typical steps:

1. Implement a request filter or identity provider that validates the token/credential.
2. Map identity and tenant claims into a principal model (`tenantId`, `orgRefName`, `userId`, `roles`).
3. Ensure `BaseResource` (and other entry points) can derive `DomainContext` from that principal.

8.3. Creating an Auth Plugin (using the Custom JWT provider as a reference)

An auth plugin is typically a CDI bean that:

- Extends `BaseAuthProvider` to inherit user-management helpers and persistence utilities.
- Implements `AuthProvider` to integrate with request-time authentication flows.
- Implements `UserManagement` to expose CRUD-style operations for users, passwords, and roles.

A concrete provider should:

- Be annotated as a CDI bean (e.g., `@ApplicationScoped`).
- Provide a stable `getName()` identifier (e.g., `"custom"`, `"oidc"`, `"apikey"`).
- Use config properties for secrets, issuers, token durations, and any external identity provider details.
- Build a Quarkus `SecurityIdentity` with the authenticated principal and roles.

8.4. AuthProvider interface (what a provider must implement)

Core methods:

- SecurityIdentity validateAccessToken(String token)
 - Parse and validate the incoming credential (JWT, API key, signature).
 - Return a SecurityIdentity with principal name and roles; throw a security exception for invalid tokens.
- String getName()
 - A short identifier for the provider; persisted alongside credentials and used in logs/metrics.
- LoginResponse login(String userId, String password)
 - Credential-based login. Return a structured response:
 - positiveResponse: includes SecurityIdentity, roles, accessToken, refreshToken, expirationTime, and realm/mongodbUrl if applicable.
 - negativeResponse: includes error codes/reason/message for clients to act on (e.g., password change required).
- LoginResponse refreshTokens(String refreshToken)
 - Validate the refresh token, mint a new access token (and optionally a new refresh token), and return a positive response.

Notes:

- Login flow should check force-change-password or equivalent flags and return a negative response when user interaction is required before issuing tokens.
- validateAccessToken should only accept valid, non-expired tokens and construct SecurityIdentity consistently with role mappings used across the platform.

8.5. UserManagement interface (operations your plugin must support)

Typical responsibilities include:

- User lifecycle
 - String createUser(String userId, String password, Set<String> roles, DomainContext domainContext, [optional] DataDomain)
 - void changePassword(String userId, String oldPassword, String newPassword, Boolean forceChangePassword)
 - boolean removeUserWithUserId(String userId)
 - boolean removeUserWithSubject(String subject)
- Role management

- `void assignRolesForUserId(String userId, Set<String> roles)`
- `void assignRolesForSubject(String subject, Set<String> roles)`
- `void removeRolesForUserId(String userId, Set<String> roles)`
- `void removeRolesForSubject(String subject, Set<String> roles)`
- `Set<String> getUserRolesForUserId(String userId)`
- `Set<String> getUserRolesForSubject(String subject)`
- Lookups and existence checks
 - `Optional<String> getSubjectForUserId(String userId)`
 - `Optional<String> getUserIdForSubject(String subject)`
 - `boolean userIdExists(String userId)`
 - `boolean subjectExists(String subject)`

Return values and exceptions:

- Throw `SecurityException` or domain-specific exceptions for invalid states (duplicate users, bad password, unsupported hashing).
- Return `Optional` for lookups that may not find a result.
- For removals, return `boolean` to communicate whether a record was deleted.

8.6. Leveraging `BaseAuthProvider` in your plugin

When you extend `BaseAuthProvider`, you inherit ready-to-use capabilities that reduce boilerplate:

- Impersonation controls
 - `enableImpersonationWithUserId` / `enableImpersonationWithSubject`
 - `disableImpersonationWithUserId` / `disableImpersonationWithSubject`
 - These set or clear an impersonation filter script and realm regex that downstream services can honor to act on behalf of another identity under controlled scope.
- Realm override helpers
 - `enableRealmOverrideWithUserId` / `enableRealmOverrideWithSubject`
 - `disableRealmOverrideWithUserId` / `disableRealmOverrideWithSubject`
 - Useful for multi-realm/tenant scenarios, enabling scoped cross-realm behavior.
- Persistence utilities
 - Built-in use of the credential repository to save, update, and delete credentials.
 - Consistent validation of inputs (non-null checks, non-blank checks).
 - Hashing algorithm guardrails to ensure only supported algorithms are used.

Best practices when deriving:

- Record the auth provider name in stored credentials as informational provenance. It tells

callers which provider resolved the credential or token; it is not an authentication routing or authorization binding.

- Reuse the role merge/remove patterns to avoid accidental role loss.
- Prefer emitting precise exceptions (e.g., `NotFound` for missing users, `SecurityException` for access violations).

8.7. Implementing your own provider

Checklist:

- Class design
 - `@ApplicationScoped` bean
 - extends `BaseAuthProvider`
 - implements `AuthProvider` and `UserManagement`
 - return a stable `getName()`
- Configuration
 - Externalize secrets (signing keys), issuers, token durations, and realm details via `MicroProfile Config`.
- `SecurityIdentity`
 - Consistently build identities with principal and roles; include useful attributes for auditing/telemetry.
- Tokens/credentials
 - For JWT-like tokens, implement robust parsing, signature verification, expiration checks, and claim validation.
 - For non-JWT credentials (API keys, HMAC), ensure replay protection and scope binding.
- Responses and errors
 - Use structured `LoginResponse` for both success and error paths.
 - Prefer idempotent user/role operations; validate inputs and surface actionable messages.

8.8. `CredentialUserIdPassword` model and `DomainContext`

This section explains how user credentials are represented, how those records tie to tenancy and realms, and how the server chooses the database (“realm”) for REST calls.

What the credential model represents

`userId`

The human-friendly login handle that users type. Must be unique within the applicable tenancy/realm scope.

subject

A stable, system-generated identifier for the principal. Tokens and internal references favor subject over userId because subjects do not change.

description, emailOfResponsibleParty

Optional metadata to describe the credential and provide an owner contact.

domainContext

The tenancy and organization placement of the principal. It contains:

- tenantId: Logical tenant partition.
- orgRefName: Organization/business unit within the tenant.
- accountId: Account or billing identifier.
- defaultRealm: The default database/realm used for this identity's operations.
- dataSegment: Optional partitioning segment for advanced sharding or data slicing.

roles

The set of authorities granted (e.g., USER, ADMIN). These become groups/roles on the SecurityIdentity.

issuer

An identifier for who issued the credential or tokens (useful for auditing and multi-provider setups).

passwordHash, hashingAlgorithm

The stored password hash and declared algorithm. Not exposed over REST. Providers verify passwords against this.

forceChangePassword

Flag that forces a password reset on next login; the login flow returns a structured negative response instead of tokens.

lastUpdate

Timestamp for auditing and token invalidation strategies.

area2RealmOverrides

Optional map to route specific functional areas to different realms than the default (e.g., "Reporting" → analytics-realm).

realmRegEx

Optional regex to limit or override which realms this identity may act in; also used by impersonation/override flows.

impersonateFilterScript

Optional script indicating the filter/scope applied during impersonation so actions are constrained.

authProviderName

Informational provenance naming the provider that resolved this credential or token (e.g., “custom”, “oidc”). Provider selection comes from an explicit request override or the ordered `auth.provider` configuration, never from this field.

How DomainContext selects the realm for REST calls

- For each authenticated request, the server derives or retrieves a DomainContext associated with the principal.
- The DomainContext.defaultRealm indicates which backing MongoDB database (“realm”) should be used by repositories for that request.
- If realm override features are enabled (e.g., through provider helpers or per-credential overrides), the system may route certain functional areas to alternate realms using area2RealmOverrides or validated by realmRegEx.
- The remainder of DomainContext (tenantId, orgRefName, accountId, dataSegment) is applied as scope constraints through permission rules and repository filters so reads and writes are automatically restricted to the correct tenant/org segment.

8.9. Quarkus OIDC out-of-the-box and integrating with common IdPs

Quarkus ships with first-class OpenID Connect (OIDC) support, enabling both service-to-service and browser-based logins.

What the Quarkus OIDC extension provides

- OIDC client and server-side adapters:
 - Authorization Code flow with PKCE for browser sign-in.
 - Bearer token authentication for APIs (validating access tokens on incoming requests).
 - Token propagation for downstream calls (forwarding or exchanging tokens).
- Token verification and claim mapping:
 - Validates issuer, audience, signature, expiration, and scopes.
 - Maps standard claims (sub, email, groups/roles) into the security identity.
- Multi-tenancy and configuration:
 - Supports multiple OIDC tenants via configuration, each with its own issuer, client id/secret, and flows.
- Logout and session support:
 - Front-channel and back-channel logout hooks depending on provider capabilities.

Integrating with common providers

- Works with providers like Keycloak, Auth0, Okta, Azure AD, Cognito, and enterprise IdPs exposing OIDC.

- Configure the issuer URL and client credentials. Quarkus discovers endpoints via the provider's .well-known/openid-configuration.
- For roles/permissions, map provider groups/roles claims to your platform roles in the identity.

OIDC vs OAuth vs OpenID (terminology and evolution)

OAuth 2.0

Authorization framework for delegated access (scopes), not authentication. Defines flows to obtain access tokens for APIs.

OpenID (OpenID 1.x/2.0)

Older federated identity protocol that preceded OIDC. It has been superseded by OpenID Connect.

OpenID Connect (OIDC)

An identity layer on top of OAuth 2.0. Adds standardized authentication, user info endpoints, ID tokens (JWT) with subject and profile claims, and discovery metadata. In practice, OIDC is the modern standard for SSO and user authentication; OAuth remains the authorization substrate underneath.

Summary

- OpenID → historical, replaced by OIDC.
- OAuth 2.0 → authorization framework.
- OIDC → authentication (identity) layer built on OAuth 2.0.

OIDC and SAML in relation to SSO

SAML (Security Assertion Markup Language)

XML-based federation protocol widely used in enterprises for browser SSO; uses signed XML assertions transported through browser redirects/posts.

OIDC

JSON/REST-oriented, uses JWTs, and is well-suited for modern SPAs and APIs.

Relationship: * Both enable SSO and federation across identity providers and service providers. * Many enterprise IdPs support both; OIDC is generally simpler for APIs and modern web stacks, while SAML is entrenched in legacy/enterprise SSO.

Bridging: * Gateways or identity brokers can translate SAML assertions to OIDC tokens and vice versa, allowing gradual migration.

Common customer IdP models and OIDC integration patterns

- Centralized IdP (single-tenant)
 - One organization-wide IdP issues tokens for all users.
 - Configure a single OIDC tenant in Quarkus; map groups/roles to application roles.
- Multi-tenant SaaS with per-tenant IdP (BYOID)

- Each customer brings their own IdP.
- Configure Quarkus OIDC multitenancy with per-tenant issuer discovery and client credentials.
- Tenant selection can be based on domain, request header, or path; the selected OIDC tenant performs login and token validation.
- Brokered identity
 - Use a broker that federates to multiple upstream IdPs (OIDC, SAML).
 - Quarkus integrates with the broker as a single OIDC client; the broker handles IdP routing and protocol translation.
- Hybrid API and web flows
 - Browser apps use Authorization Code flow with sessions; APIs use bearer token authentication.
 - The OIDC extension can handle both in the same application when properly configured.

8.10. Authorization via RuleContext

Authentication establishes identity; RuleContext enforces what the identity can do. For each action (CREATE, UPDATE, VIEW, DELETE, ARCHIVE), RuleContext can:

- Allow or deny the action
- Contribute additional filters (e.g., org scoping, functional-area specific sharing)
- Adjust UIActionList to reflect permitted next steps

This division of responsibilities keeps providers focused on identity while policies remain centralized in RuleContext.

8.10.1. Using Ontology Edges in List Endpoints (optional)

When ontology is enabled and edges are materialized, list endpoints can avoid deep joins or multi-collection traversals by rewriting queries based on semantic relationships.

Pattern A: Wrap BSON with ListQueryRewriter

```
Bson base = Filters.and(existingFilters...);
Bson rewritten = rewriter.rewriteForHasEdge(base, tenantId, "placedInOrg", orgRefName);
collection.find(rewritten).iterator();
```

Pattern B: Constrain Morphia query by IDs

```
Set<String> ids = edgeDao.srcIdsByDst(tenantId, "orderShipsToRegion", region);
if (!ids.isEmpty()) {
    query.filter(dev.morphia.query.filters.Filters.in("_id", ids));
}
```

```
}
```

Notes

- Always scope by `tenantId` from `DomainContext/RuleContext`.
- Index edges on `(tenantId, p, dst)` and `(tenantId, src, p)` to keep queries fast.
- See [Integrating Ontology](#) for more integration options.

OntologyAwareResource

`OntologyAwareResource` is an abstract base class in the framework (package `com.e2eq.ontology.policy.rest`, `quantum-ontology-policy-bridge`) that extends `BaseResource` to add ontology-aware list and detail endpoints. Resources extending it expose standard CRUD plus:

- **Ontology list endpoint:** `GET /{basePath}/ontology` — lists entities with optional ontology-based filtering (e.g. filter by related customer, project, or timesheet IDs via the ontology graph)
- **Ontology context endpoint:** `GET /{basePath}/id/{id}/ontology` — returns ontology edges for a single entity
- **Optional aggregation and expand:** when `feature.ontologyList.aggregation.enabled=true`, supports the `expand` query param to return combined attributes spanning collections (via MongoDB `$lookup`)

Extending OntologyAwareResource

1. Extend `OntologyAwareResource<T, R>` instead of `BaseResource<T, R>`.
2. Add `GET @Path("ontology")` — call `getOntologyList(headers, skip, limit, filter, sort, projection, expand?, constraints...)`.
3. Add `GET @Path("id/{id}/ontology")` — call `buildOntologyContextResponse(headers, id)`.
4. For ontology filters, pass `constraint(predicate, ids)` where `predicate` is the ontology property (e.g. `invoicesCustomer`) and `ids` are destination entity IDs.
5. Override `supportsExpandInOntologyList()` to return `true` if the entity has expandable references (e.g. `customerReference`, `projectReference`).

Configuration

Property	Default	Description
<code>feature.ontologyList.aggregation.enabled</code>	<code>false</code>	When <code>true</code> , resources with <code>supportsExpandInOntologyList()=true</code> can use aggregation for the <code>expand</code> query param

Dependencies

`OntologyAwareResource` uses `ListQueryRewriter`, `OntologyContextEnricherMongo` (`quantum-ontology-mongo`), and `MorphiaDataStoreWrapper` (`quantum-morphia-repos`). It is provided by `quantum-ontology-policy-bridge`; applications must include that module (and thus `quantum-ontology-mongo`

transitively) when using this base class.

Chapter 9. 7. Query language and filtering

Problem: Powerful, safe, and consistent querying across collections.

Why for SaaS: Tenants need flexible reporting and filtration without bespoke endpoints.

How Quantum helps: A uniform query language layer with server-side enforcement.

Walkthrough: Add query endpoints and test filters/security.

9.1. Query Language Reference

Quantum uses an ANTLR-based query language (BIAPIQuery.g4) for filtering, searching, and constraining data across all REST endpoints. This single, consistent syntax works everywhere: list APIs, permission rules, and access resolvers.

9.1.1. Basic Syntax

Operators

Operator	Symbol	Example
Equals	:	name:"Widget"
Not equals	:!	status:! "DELETED"
Less than	:<	price:<##100
Greater than	:>	quantity:>#50
Less than or equal	:≤	createdAt:≤2024-12-31
Greater than or equal	:>=	updatedAt:>=2024-01-01T00:00:00Z
Field exists	:~	description:~
In list	:^	status:^["ACTIVE","PENDING"]
Not in list	:!^	status:!^["DELETED","ARCHIVED"]

Value Types

Type	Prefix	Example
String	none or "..."	name:widget or name:"Super Widget"
Number (integer)	#	quantity:#42
Number (decimal)	##	price:##19.99
Date	none	shipDate:2024-12-25

Type	Prefix	Example
DateTime	none	createdAt:2024-12-25T10:30:00Z
Boolean	none	active:true
Null	none	description:null
ObjectId	none	id:507f1f77bcf86cd799439011
Reference	@@	parentId:@@507f1f77bcf86cd799439011
Variable	\${...}	ownerId:\${principalId}, tier:^\${facet.allowedTiers}, zone:\${coord.salesZone}



Dotted Variable Identifiers & Single-Variable IN: The query grammar admits dotted variable paths (`${facet.<key>}`, `${coord.<key>}`, `${principal.<attr>}`, `${dd.<attr>}`). When paired with the list membership operator (`:^`), a single variable resolving to a Java `Collection` or array (e.g. `allowedTiers = ["TIER_1", "TIER_2"]`) can be passed without brackets (`tier:^${facet.allowedTiers}`), and the query compiler will automatically unpack the elements into a native `$in` evaluation.

Logical Operators

Operator	Symbol	Example
AND	&&	active:true && price:>##10
OR		status:"ACTIVE" status:"PENDING"
NOT	!!	!!(price:<##5)
Grouping	()	(active:true featured:true) && price:>##0

9.1.2. Common Patterns

String Matching

```
# Exact match
name:"Super Widget"

# Wildcard matching
name:*widget*      # contains "widget"
name:widget*       # starts with "widget"
name:*widget       # ends with "widget"
name:w?dget        # single character wildcard

# Case sensitivity (depends on database collation)
name:"WIDGET"       # may or may not match "widget"
```

Text Search

```
# Full-text search (requires MongoDB text index)
text("priority escalation")

# Combine text search with other filters
text("priority escalation") && status:"OPEN"

# Valid: top-level text AND a grouped OR of non-text predicates
text("priority") && (status:"OPEN" || status:"PENDING")
```

Notes:

- `text(...)` maps to MongoDB `$text` queries and is case-insensitive by default.
- MongoDB allows only one `text(...)` clause per query.
- Text search must be backed by a text index on the target collection. If the collection has no text index, MongoDB returns an error (Quantum does not pre-check index existence).
- The lexer keyword `text` is also accepted as a normal field name or unquoted value, so queries such as `type:text` and `text:active` remain valid.
- In-memory evaluation (policy/tests) approximates `$text` with case-insensitive **whole-word** matching (tokenized contains-any). It does not implement stemming or stop words. Example: `text("cat")` matches `"the cat sat"` but not `"category"`.



MongoDB requires `$text` to be a top-level query operator. The following usages are **not allowed** and will be rejected at parse time:

Invalid Usage	Example	Reason
Inside NOT	<code>!!text("foo")</code>	MongoDB does not support <code>\$text</code> inside <code>\$not</code> or <code>\$nor</code>
Inside OR	<code>text("foo") status:"OPEN"</code>	MongoDB does not allow <code>\$text</code> inside <code>\$or</code>
Inside elemMatch	<code>tags:{text("foo")}</code>	MongoDB does not support <code>\$text</code> inside <code>\$elemMatch</code>
Multiple text clauses	<code>text("foo") && text("bar")</code>	Only one <code>\$text</code> operator is allowed per query

Valid usage: `text(...)` combined with `&&` (AND) at the top level is supported because MongoDB allows `$text` alongside other top-level conditions.

Numeric Ranges

```
# Price between 10 and 100
price:>=##10 && price:<=##100

# Quantity greater than 0
quantity:>#0
```

```
# Exact count
itemCount:#5
```

Date and Time Queries

```
# Orders from today
createdAt:>=2024-12-25

# Orders from last week
createdAt:>=2024-12-18 && createdAt:<2024-12-25

# Specific timestamp
updatedAt:2024-12-25T14:30:00Z

# Orders modified this year
updatedAt:>=2024-01-01T00:00:00Z
```

List Membership

```
# Status in specific values (IN)
status:^[ "ACTIVE", "PENDING", "PROCESSING" ]

# Exclude statuses (NOT IN)
status:!^[ "DELETED", "ARCHIVED" ]

# User IDs from a list (IN)
ownerId:^[ "user1", "user2", "user3" ]

# Exclude specific users (NOT IN)
ownerId:!^[ "user1", "user2" ]

# ObjectId list (IN)
categoryId:^[ @507f1f77bcf86cd799439011, @507f1f77bcf86cd799439012 ]

# ObjectId list (NOT IN)
categoryId:!^[ @507f1f77bcf86cd799439011, @507f1f77bcf86cd799439012 ]

# Mixed types (coerced automatically)
priority:^[ #1, #2, #3 ]

# Using variables (CSV expansion supported by access resolvers)
customerId:!^[ ${accessibleCustomerIds} ]
```

Null and Existence Checks

```
# Field has any value (not null)
```

```

description:~

# Field is null
description:null

# Field is not null
description:!null

# Field exists and is not empty string
description:~ && description:!""

```

9.1.3. Querying Dynamic Attributes

Models that implement `DynamicAttributeSupport` can store flexible, user-defined attributes organized into named sets. These can be queried using the `elemMatch` syntax for array fields.

Dynamic Attribute Structure

Dynamic attributes are stored as:

```

{
  "dynamicAttributeSets": [
    {
      "name": "logistics",
      "attributes": [
        { "id": "uuid-1", "name": "weight", "value": 25.5, "type": "NUMBER" },
        { "id": "uuid-2", "name": "hazmat", "value": false, "type": "BOOLEAN" }
      ]
    },
    {
      "name": "compliance",
      "attributes": [
        { "id": "uuid-3", "name": "certNumber", "value": "CERT-123", "type": "STRING" }
      ]
    }
  ]
}

```

Querying by Attribute ID and Value

Use `elemMatch` syntax `{...}` to query attributes within the array:

```

# Find records where a specific attribute (by ID) has a specific value
dynamicAttributeSets.attributes:{id:21f63b90-08b4-4280-a28d-f003f9c114b3 &&
value:"Conference hall"}

# Find records with a numeric attribute value

```

```
dynamicAttributeSets.attributes:{id:94138c39-2115-4681-8c18-aa0c0596b065 && value:#99}
```

```
# Find records with a decimal attribute value
```

```
dynamicAttributeSets.attributes:{id:uuid-1 && value:##25.5}
```

Querying by Attribute Name and Value

```
# Find records where an attribute named "weight" has value > 10
```

```
dynamicAttributeSets.attributes:{name:"weight" && value:>##10}
```

```
# Find records with a specific attribute name and string value
```

```
dynamicAttributeSets.attributes:{name:"certNumber" && value:"CERT-123"}
```

```
# Boolean attribute query
```

```
dynamicAttributeSets.attributes:{name:"hazmat" && value:true}
```

Combining Multiple Dynamic Attribute Conditions

```
# Multiple attributes must match (separate elemMatch conditions)
```

```
dynamicAttributeSets.attributes:{name:"weight" && value:>##10} &&
```

```
dynamicAttributeSets.attributes:{name:"hazmat" && value:false}
```

```
# OR within a single attribute query
```

```
dynamicAttributeSets.attributes:{value:"A" || value:"B"}
```

```
# Combined with regular field queries
```

```
status:"ACTIVE" && dynamicAttributeSets.attributes:{name:"priority" && value:"HIGH"}
```

Querying by Set Name

To filter by the set name itself:

```
# Records that have a "logistics" attribute set
```

```
dynamicAttributeSets:{name:"logistics"}
```

```
# Combine set name with attribute value
```

```
dynamicAttributeSets:{name:"logistics"} &&
```

```
dynamicAttributeSets.attributes:{name:"weight" && value:>##50}
```

Advanced Tags (Simple Key-Value)

For simpler tag-based filtering, use **advancedTags**:

```
# Find by tag name and value
```

```
advancedTags:{name:"priority" && value:"urgent"}
```

```
# Multiple tag conditions
advancedTags:{name:"region" && value:"US"} && advancedTags:{name:"tier" &&
value:"premium"}
```

Advanced Examples

Complex Business Logic

```
# Active products under $50 OR featured products at any price
(active:true && price:<##50) || featured:true

# Orders needing attention: overdue OR high-value pending
(dueDate:<2024-12-25 && status:! "COMPLETED") ||
(status:"PENDING" && totalAmount:>##1000)

# Products with inventory issues
(quantity:<=#5 && reorderPoint:>#5) || stockStatus:"OUT_OF_STOCK"
```

Multi-tenant Filtering

```
# User's own records
dataDomain.ownerId:${principalId}

# Organization-wide access
dataDomain.orgRefName:${orgRefName}

# Tenant-scoped with public sharing
dataDomain.tenantId:${pTenantId} || dataDomain.orgRefName:"PUBLIC"
```

Audit and Compliance

```
# Records modified by specific user
auditInfo.lastUpdatedBy:"john.doe"

# Changes in date range
auditInfo.lastUpdatedDate:>=2024-12-01 &&
auditInfo.lastUpdatedDate:<2024-12-31

# Created vs modified
auditInfo.createdDate:auditInfo.lastUpdatedDate # never modified
auditInfo.createdDate:!auditInfo.lastUpdatedDate # has been modified
```

Variables in Filters

Variables are resolved from the current security context and can be used in permission rules and access resolvers.

Standard Variables

Variable	Description
<code>\${principalId}</code>	Current user's ID
<code>\${pTenantId}</code>	Principal's tenant ID
<code>\${pAccountId}</code>	Principal's account ID
<code>\${pOrgRefName}</code>	Principal's organization
<code>\${realm}</code>	Current realm/database
<code>\${area}</code>	Current functional area
<code>\${functionalDomain}</code>	Current functional domain
<code>\${action}</code>	Current action (CREATE, UPDATE, etc.)

Custom Variables from Access Resolvers

```
// In your AccessListResolver
@Override
public String key() {
    return "accessibleCustomerIds"; // becomes ${accessibleCustomerIds}
}

@Override
public Collection<?> resolve(...) {
    return Arrays.asList("CUST001", "CUST002", "CUST003");
}
```

```
# Use in filter
customerId:^(${accessibleCustomerIds})
```

Performance Tips

Efficient Queries

```
# Good: Use indexed fields first
status:"ACTIVE" && createdAt:>=2024-01-01

# Better: Combine with specific values
status:"ACTIVE" && ownerId:${principalId} && createdAt:>=2024-01-01

# Avoid: Leading wildcards on large collections
name:*widget # can be slow on millions of records
```

Projection for Large Objects

```
# In REST calls, limit returned fields
GET /products/list?filter=active:true&projection=+id,+name,+price,-description
```

Integration with REST APIs

List Endpoints

```
# Basic filtering
GET /products/list?filter=active:true

# With sorting and pagination
GET /products/list?filter=price:>##10&sort=-createdAt&skip=20&limit=10

# Complex filter with projection
GET
/orders/list?filter=(status:"PENDING"||status:"PROCESSING")&&totalAmount:>##100&projection=+id,+status,+totalAmount,+customerName
```

Permission Rules

```
- name: user-own-records
  priority: 300
  match:
    method: [GET]
    url: /api/**
  effect: ALLOW
  andFilterString: "dataDomain.ownerId:${principalId}"
```

Access Resolvers

```
// Resolver returns customer IDs user can access
public Collection<?> resolve(...) {
    return customerService.getAccessibleIds(principalId);
}

// Used in permission rule
andFilterString: "customerId:^([${accessibleCustomerIds}])"
```

Error Handling

Common syntax errors and solutions:

```
# Wrong: Missing quotes for multi-word strings
name:Super Widget
```

```

# Right:
name:"Super Widget"

# Wrong: Incorrect number prefix
price:19.99
# Right:
price:##19.99

# Wrong: Invalid date format
createdDate:12/25/2024
# Right:
createdDate:2024-12-25

# Wrong: Unbalanced parentheses
(active:true && price:>##10
# Right:
(active:true && price:>##10)

# Wrong: text() inside NOT
!!text("search")
# Right: Use text() at top level only
text("search") && status:!"DELETED"

# Wrong: text() inside OR
text("search") || status:"ACTIVE"
# Right: Use text() with AND only
text("search") && (status:"ACTIVE" || status:"PENDING")

# Wrong: Multiple text() clauses
text("foo") && text("bar")
# Right: Combine search terms in a single text()
text("foo bar")

```

See Also

- [REST CRUD Querying](#)
- [Permission Rules](#)
- [Access Resolvers](#)

Execution engines and listeners

The BI-API query syntax is parsed once (via ANTLR) and can be executed by different "listeners" depending on the use case. Quantum ships with two primary implementations that share the same grammar and semantics:

- Morphia listener: converts a query into Mongo/Morphia Filters for database-side execution
- In-memory listener: converts a query into a Java Predicate over JSON data for Quarkus/GraalVM-friendly in-memory execution

Morphia: QueryToFilterListener

Use this when you want the database to perform the filtering. The listener walks the parse tree and produces a `dev.morphia.query.filters.Filter` which you can apply to Morphia queries. This is ideal for repository APIs and any endpoint where you want to leverage MongoDB indexes and avoid loading large data sets into memory.

Key characteristics: - Output type: Morphia Filter - Execution: database-side (MongoDB) - Semantics: identical to grammar (comparisons, IN/NIN, exists, null, regex with wildcards, elemMatch, boolean &&|||/!) - Text search: `text("...")` is supported when a collection has a text index; only one text clause is allowed per query, and it must be at the top level (cannot be inside OR, NOT, or elemMatch). - Variable expansion: supports `${vars}` and single-variable IN list expansion (e.g., `[$ids]` can expand to a collection/array or a comma-separated string)

Example:

```
import com.e2eq.framework.grammar.*;
import com.e2eq.framework.model.persistent.morphia.QueryToFilterListener;
import dev.morphia.query.filters.Filter;
import dev.morphia.query.filters.Filters;
import org.antlr.v4.runtime.*;
import org.antlr.v4.runtime.tree.ParseTreeWalker;
import org.apache.commons.text.StringSubstitutor;

String query = "(status:Assigned||status:Pending)&&displayName:*Route*";
var vars = java.util.Map.<String,String>of();

// Parse
CharStream cs = CharStreams.fromString(query);
BIAPIQueryLexer lexer = new BIAPIQueryLexer(cs);
CommonTokenStream tokens = new CommonTokenStream(lexer);
BIAPIQueryParser parser = new BIAPIQueryParser(tokens);
BIAPIQueryParser.QueryContext tree = parser.query();

// Build Morphia filter
QueryToFilterListener listener = new QueryToFilterListener(vars, new
StringSubstitutor(vars), /* modelClass */ null);
ParseTreeWalker.DEFAULT.walk(listener, tree);
Filter morphiaFilter = listener.getFilter();

// Use with Morphia query (example)
// datastore.find(MyEntity.class).filter(morphiaFilter).iterator().toList();
```

A few query examples (taken from `testQueryStrings.txt`):

- Equality: `field:"quotedString"`
- Comparisons: `field:>##12.56`, `field:≤#123`
- IN/NIN: `field:^[value1,value2]`, `field:!^[value1,value2]`

- Exists/Null: field:~, field:null
- elemMatch: arrayField: {(subField:<#12)| |(subField:>#15)}

In-memory (JsonNode): QueryToPredicateJsonListener

Use this when you need to evaluate queries in memory without reflection on POJOs. This implementation compiles a query into a `java.util.function.Predicate` over a Jackson `JsonNode`. It is Quarkus/GraalVM friendly, useful for: - Unit tests where you want to validate query behavior without a database - Post-filtering or pre-filtering of already-fetched data - Evaluating access rules or business logic against transient objects

Key characteristics: - Output type: `Predicate<JsonNode>` - Execution: in-memory - No runtime reflection: operates on `JsonNode` - Semantics and variable expansion match the Morphia listener

Convenience helpers exist in `QueryPredicates`:

```
import com.e2eq.framework.query.QueryPredicates;
import com.fasterxml.jackson.databind.JsonNode;
import java.util.function.Predicate;
import java.util.Map;

String query = "(status:Assigned||status:Pending)&&displayName:*Route*";
Predicate<JsonNode> p = QueryPredicates.compilePredicate(query, Map.of(), Map.of());

// Example data as a POJO or Map -> convert to JsonNode
record Ticket(String status, String displayName) {}
Ticket ticket = new Ticket("Assigned", "Route Exception in
Route:To[http://com.xxx/update]");
JsonNode node = QueryPredicates.toJsonNode(ticket);

boolean include = p.test(node); // true
```

Additional examples

- Equality and comparisons

```
var vars = Map.<String,String>of();
var objVars = Map.<String,Object>of();
Predicate<JsonNode> eq = QueryPredicates.compilePredicate("quantity:#42", vars,
objVars);
Predicate<JsonNode> gt = QueryPredicates.compilePredicate("price:>##19.99", vars,
objVars);

JsonNode product = QueryPredicates.toJsonNode(Map.of("quantity", 42, "price", 25.00));
assert eq.test(product);
assert gt.test(product);
```

- IN / NIN with variable expansion

```
var vars = Map.of("principalId", "66d1f1ab452b94674bbd934a");
Predicate<JsonNode> in =
QueryPredicates.compilePredicate("ownerId^[${principalId},value2]", vars, Map.of());
JsonNode doc = QueryPredicates.toJsonNode(Map.of("ownerId",
"66d1f1ab452b94674bbd934a"));
assert in.test(doc);
```

- elemMatch over arrays of objects

```
String q = "items:{(sku:abc||qty:>#10)&&price:<=##9.99}";
Predicate<JsonNode> em = QueryPredicates.compilePredicate(q, Map.of(), Map.of());
JsonNode order = QueryPredicates.toJsonNode(Map.of(
    "items", java.util.List.of(
        Map.of("sku","abc","qty", 5, "price", 9.99),
        Map.of("sku","xyz","qty", 12, "price", 8.50)
    ));
// Matches: first item by sku OR second item by qty with price cap
assert em.test(order);
```

- Regex with wildcards

```
Predicate<JsonNode> rx = QueryPredicates.compilePredicate("displayName:*Route*",
Map.of(), Map.of());
JsonNode ticket = QueryPredicates.toJsonNode(Map.of("displayName", "Route Exception in
Route:To[...]"));
assert rx.test(ticket);
```

Choosing the right listener

- Use Morphia (QueryToFilterListener) when:
 - You are filtering MongoDB collections and want the DB to do the work (indexing, pagination, scalability)
 - You need server-side performance and minimal memory footprint
- Use In-memory (QueryToPredicateJsonListener) when:
 - You run in Quarkus native image and want to avoid reflection on POJOs
 - You are writing unit tests or applying rules to transient/aggregated data
 - You need to evaluate a query over already materialized objects without another database round-trip

Both listeners aim to maintain parity with the grammar. If you observe mismatches, please file an issue with the query string, the evaluated data sample, and the expected vs actual results.

Query Field Validation

Quantum provides validation capabilities to detect references to non-existent fields before query execution, preventing runtime errors and providing early feedback.

Validating Listeners

Two validating listener implementations extend the standard listeners:

- `ValidatingQueryToFilterListener`: extends `QueryToFilterListener` for Morphia queries
- `ValidatingQueryToPredicateJsonListener`: extends `QueryToPredicateJsonListener` for in-memory predicates

Both validate field references against a model class during parsing and accumulate errors.

Example usage:

```
import com.e2eq.framework.model.persistent.morphia.ValidatingQueryToFilterListener;
import com.e2eq.framework.grammar.*;
import org.antlr.v4.runtime.*;
import org.antlr.v4.runtime.tree.ParseTreeWalker;

String query = "name:John AND invalidField:test";

BIAPIQueryLexer lexer = new BIAPIQueryLexer(CharStreams.fromString(query));
CommonTokenStream tokens = new CommonTokenStream(lexer);
BIAPIQueryParser parser = new BIAPIQueryParser(tokens);

ValidatingQueryToFilterListener listener =
    new ValidatingQueryToFilterListener(UserProfile.class);
ParseTreeWalker.DEFAULT.walk(listener, parser.query());

if (listener.hasValidationErrors()) {
    List<String> errors = listener.getValidationErrors();
    throw new IllegalArgumentException("Invalid query: " + errors);
}

Filter filter = listener.getFilter();
```

@ValidQueryFilter Annotation

For automatic validation in DTOs and models, use the `@ValidQueryFilter` annotation:

```
import com.e2eq.framework.annotations.ValidQueryFilter;
import jakarta.validation.constraints.NotNull;

public class SearchRequest {
    @NotNull
    @ValidQueryFilter(modelClass = UserProfile.class)
```

```

private String filterQuery;

// getters/setters
}

```

The annotation integrates with Jakarta Bean Validation and is automatically enforced by the `ValidationInterceptor` during entity persistence.

REST endpoint example:

```

@Path("/users")
public class UserResource {

    @GET
    public Response searchUsers(@Valid @BeanParam SearchParams params) {
        // filterQuery is validated before this method executes
        return Response.ok(userService.search(params.getFilterQuery())).build();
    }
}

public class SearchParams {
    @QueryParam("filter")
    @ValidQueryFilter(modelClass = UserProfile.class)
    private String filterQuery;
}

```

Persisted filter example:

```

public class SavedSearch extends BaseModel {
    private String name;

    @ValidQueryFilter(modelClass = Order.class)
    private String filterExpression;

    // When saved, ValidationInterceptor validates the filter
}

```

QueryFieldValidator Utility

For programmatic validation without listeners:

```

import com.e2eq.framework.query.QueryFieldValidator;

// Validate against a model class
QueryFieldValidator validator = QueryFieldValidator.forModelClass(UserProfile.class);

boolean isValid = validator.validateField("email"); // true

```

```

boolean isValid = validator.validateField("nonExistentField"); // false

if (validator.hasErrors()) {
    List<String> errors = validator.getErrors();
    // Handle validation errors
}

// Validate against aggregation pipeline schema
Map<String, Class<?>> schema = Map.of(
    "totalAmount", Double.class,
    "orderCount", Long.class,
    "customerName", String.class
);

QueryFieldValidator aggValidator = QueryFieldValidator.forAggregationSchema(schema);
aggValidator.validateField("totalAmount"); // true
aggValidator.validateField("invalidField"); // false

```

Benefits

- Early error detection: catch field reference errors before query execution
- Better error messages: specific feedback about which fields are invalid
- Development safety: prevent typos and refactoring issues
- API validation: validate user-provided query strings in REST endpoints
- Data integrity: prevent saving invalid filter expressions to the database

Expressing Ontology Constraints in Queries (optional)

The BIAPI query language primarily targets fields on documents. When using the ontology modules, there are two ways to incorporate ontology relationships into queries:

Option A: Variable from an AccessListResolver

- A resolver computes the set of IDs using EdgeDao (e.g., orders related by placedInOrg to the caller's org).
- Use an IN clause over id in your filter string:

```

# idsByPlacedInOrg is published by an AccessListResolver
id: ^${idsByPlacedInOrg}

```

Ontology operators: hasEdge, hasOutgoingEdge, and hasIncomingEdge

When the ontology module is enabled and edges are materialized in MongoDB, you can constrain results by semantic graph relationships using **hasEdge**, **hasOutgoingEdge**, and **hasIncomingEdge**.

Syntax

```
# Standard 2-argument form (relationship existence)
hasEdge(predicate, dst)
hasOutgoingEdge(predicate, dst)
hasIncomingEdge(predicate, src)

# Qualified 3-argument form (relationship existence WITH edge property constraints)
hasEdge(predicate, dst, { edgeFilter })
hasOutgoingEdge(predicate, dst, { edgeFilter })
hasIncomingEdge(predicate, src, { edgeFilter })
```

Arguments

- **predicate**: Relationship identifier. Can be an unquoted identifier (**placedInOrg**), a quoted string ("**placedInOrg**"), or a variable expression (**\${orgPredicate}**).
- **dst / src**: Target or source entity coordinate. Can be a string literal ("**OrgA**"), an **ObjectId**, a reference literal (**@507f1f77bcf86cd799439011**), or a variable placeholder (**\${orgRefName}**, **\${principalId}**).
- **{ edgeFilter }**: (Optional) A bracketed sub-query containing filtering expressions evaluated directly against the properties of the matching edge in the **edges** collection.

Edge Property Scoping and Mechanics

When querying an edge with property constraints:

1. **Automatic Field Scoping**: Any attribute referenced inside the **{ ... }** block that is not a reserved root edge attribute (**src**, **dst**, **p**, **dstType**, **dataDomain**, **_id**) is automatically rewritten to target dynamic edge properties stored under **props.<field>**. For example, **{ role:"PRIMARY" && active:true }** is compiled to **props.role == "PRIMARY" AND props.active == true**.
2. **MongoDB Compound Index Utilization**: Edge property filters are pushed down directly into MongoDB queries on the **edges** collection. They take advantage of compound indexes:
 - **idx_domain_p_dst_props**: **{ "dataDomain": 1, "p": 1, "dst": 1, "props": 1 }** (for **hasEdge / hasOutgoingEdge**)
 - **idx_domain_src_p_props**: **{ "dataDomain": 1, "src": 1, "p": 1, "props": 1 }** (for **hasIncomingEdge**)
3. **Supported Operators**: All standard BIAPI equality (**:**), inequality (**!=**), comparisons (**>**, **<**, **>=**, **<=**), list membership (**^**, **!^**), existence (**~**), and logical compositions (**&&**, **||**, **!!**) are supported on edge properties.
4. **Prohibited Operators**: Full-text search **text(...)** is prohibited inside **{ edgeFilter }** blocks because MongoDB does not permit **\$text** inside embedded document conditions or sub-filters; queries containing **text(...)** within edge filters are rejected at query validation time.

Defaults and Fallback Behavior

Element	Default Behavior	Rationale
Omitted { edgeFilter }	Matches any edge with the specified predicate and target/source, regardless of properties.	Preserves 100% backward compatibility for standard 2-argument calls.
Missing Edge Properties	Evaluates to false (no match). If an edge does not contain the queried property in its props map, the filter fails closed.	Predictable fail-closed semantics for dynamic schemas.
Tenant Scoping & Missing Security Context	Tenant ID is resolved from the active SecurityContext / RuleContext ({pTenantId}). If the query is executed without an active tenant or security context, the edge lookup returns empty results (_id: "none").	Inviolate multi-tenant data isolation; queries never leak edge relationships across realms or unauthenticated callers.
Dynamic Model Schema Validation	When query schema validation is enabled (ValidatingQueryToFilterListener), edge property keys bypass static entity model field validation.	Edge properties are open and dynamic; static entity models do not need predefined getters or schema mappings for edge property names.

Examples

Basic edge existence check (2-argument default)

```
# Find orders placed in an organization (finds source Order entities)
hasEdge("placedInOrg", ${orgRefName})

# Find locations accessible by a specific associate (finds target Location entities)
hasIncomingEdge(canAccessLocation, ${principalId})
```

Qualified edge filtering with exact property match

```
# Find accounts where user has a PRIMARY role
hasIncomingEdge("memberOf", ${principalId}, { role:"PRIMARY" })

# Find orders routed to a facility with an expedited service level
hasEdge("routedToFacility", "FAC-901", { serviceLevel:"EXPEDITED" })
```

Qualified edge filtering with multi-attribute constraints

```
# Find suppliers with valid certified relationships and confidence >= 0.90
hasEdge("suppliedBy", @@507f1f77bcf86cd799439011, { certified:true &&
```

```
confidence:>=##0.90 })
```

```
# Find assets with active status and priority in high tiers  
hasEdge("assignedToTeam", "TEAM-ALPHA", { active:true && tier:^[#1, #2] })
```

Combining edge traversal filters with root entity filters

```
# Orders in pending state placed in ORG-1 with a high-priority edge tag  
hasEdge("placedInOrg", "ORG-1", { priority:"HIGH" }) && status:"PENDING" &&  
total:>##500.00
```

Query Macros for Temporal & AI Confidence Governance

To simplify complex temporal and probabilistic filtering, the query preprocessor recognizes two high-level macros:

1. `@asOf(timestamp)`:

- Compiles to bitemporal range matching: `validFrom:<=ts || validFrom:null) && (validTo:>=ts || validTo:null` (or prefixed with `edges.` inside expansion/traversal contexts).
- Backed by compound index `idx_domain_temporal`.
- Strips surrounding quotes so ISO-8601 timestamps parse cleanly as ANTLR `DATE` / `DATETIME` tokens.
- **Business Scenario & Solution:** Point-in-time compliance audits (e.g. healthcare HIPAA audits or financial regulatory inquiries) can reproduce past authorization states without maintaining separate historical snapshot tables or running deletion cron jobs.

2. `@minConfidence(threshold)`:

- Compiles to confidence threshold filtering: `confidence:>=threshold` (or `edges.confidence:>=threshold`).
- Backed by compound index `idx_domain_confidence`.
- **Business Scenario & Solution:** AI and LLM graph derivation governance (e.g. anti-money laundering beneficial ownership links). Mission-critical operations require human-certified links (`confidence: 1.0`), while exploratory analysis accepts AI-inferred relationships (`@minConfidence(0.80)`).

Query Macro Examples

```
# Temporal Point-in-Time Audit: Did associate have active role on September 1st, 2026?  
hasEdge("assignedToBranch", "BRANCH-NY", { @asOf(2026-09-01T00:00:00Z) &&  
role:"MANAGER" })
```

```
# Probabilistic AI Governance: Only traverse high-confidence vendor relationships  
hasEdge("subsidiaryOf", "PARENT-CORP", { @minConfidence(0.85) &&  
assertionMethod:"LLM_INFERENCE" })
```

```
# Combining temporal validity, AI confidence, and security classification  
hasEdge("operatesInFacility", "FAC-901", {
```

```
@asOf(2026-09-20T00:00:00Z) &&  
@minConfidence(0.90) &&  
securityLabel:"CONFIDENTIAL"  
})
```

Root projection recap: `fields:[+...,-...]`

To include or exclude fields from the root documents:

- Syntax: `fields:[+f1,+f2,-f3]`
- Include mode: if any `+` is present, only the listed fields are included (with explicit `-` carving out exceptions).
- Exclude mode: if only `-` entries exist, all fields are included except those.
- Default `_id`: preserved unless explicitly specified (`+_id/-_id`).

Examples

```
# Include a few fields from the root and drop one  
fields:[+_id,+total,-internalNotes]  
  
# Combine with expansion (planner chooses aggregation mode)  
expand(customer) && fields:[+_id,+customer,+total]
```

For per-expansion projections and advanced traversal filters, see [Query Expansion](#).

9.2. Hands-on: Relationships and Ontology-aware queries

In addition to basic filters, the query layer supports relationship hydration and ontology-aware constraints. These features are MongoDB-first in this release and remain backward compatible: - If you don't use them, behavior is unchanged. - When you do, the planner may choose a Mongo aggregation under the hood.

9.2.1. Hydrate related data with `expand(path)`

Use `expand(path)` to materialize related entities referenced by your root documents. Paths are dotted and can include array wildcards `[*]`.

Examples:

```
# Single reference  
expand(customer)  
  
# Array of references in an items array  
expand(items[*].product)
```

```
# Nested arrays (bounded by the first non-reference boundary)
expand(patient.visits[*].diagnoses[*])
```

Combine with normal filters and projection:

```
q = "realm:acme && expand(customer, fields:[+name,+tier]) &&
fields:[+_id,+total,+customer.name]"
```

Notes: - Depth on mixed arrays/objects stops at the first non-reference boundary. - Unknown projection paths are hard errors.

See: [Query Expansion](#) for more patterns and semantics.

9.2.2. Ontology: constrain by relationships with `hasEdge(predicate, dst)`

When the ontology module is enabled and edges are materialized, you can filter by semantic relationships using `hasEdge`.

Examples:

```
# Orders placed in a specific organization
hasEdge("placedInOrg", ${orgRefName})

# Tickets routed to a region by id
hasEdge("routedToRegion", 507f1f77bcf86cd799439011)
```

Tenancy is always applied when resolving edges. See: - [Permissions: hasEdge rule](#) - [Ontology in queries](#)

9.2.3. Inspect the planner with `QueryGateway`

You can inspect how a query will execute (FILTER vs AGGREGATION) using the new `QueryGateway` facade.

```
import com.e2eq.framework.model.persistent.morphia.query.QueryGateway;
import com.e2eq.framework.model.persistent.morphia.query.QueryGatewayImpl;
import com.e2eq.framework.model.persistent.morphia.planner.PlannerResult;

QueryGateway gateway = new QueryGatewayImpl();
PlannerResult pr = gateway.plan("expand(customer) && status:active", Order.class);
// pr.getMode() == PlannerResult.Mode.AGGREGATION
// pr.getExpandPaths() == ["customer"]
```

For lower-level control, use `MorphiaUtils.convertToPlannedQuery(...)`. See: [Planner](#) and [QueryGateway](#).

Chapter 10. 8. Authentication, permissions, and annotations

Problem: Securely authenticating users and enforcing fine-grained authorization.

Why for SaaS: Multi-tenant apps must ensure tenant isolation and principle-of-least-privilege.

How Quantum helps: JWT integration, permissions model, and security annotations.

Walkthrough: Protect your APIs and add role/permission checks.

Chapter 11. Authentication and Authorization

Quantum integrates with Quarkus security while providing a pluggable approach to authentication. The repository includes a JWT provider module to get started quickly and an extension surface to replace or complement it.

11.1. JWT Provider

- Module: `quantum-jwt-provider`
- Purpose: Validate JWTs on incoming requests, populate the security principal, and surface tenant/org/user claims that feed `DomainContext`.
- Configuration: Standard Quarkus/MicroProfile JWT properties plus custom claim mappings as needed for `DataDomain`.

11.2. Pluggable Authentication

You can introduce alternative authentication mechanisms (e.g., API keys, SAML/OIDC front-channel tokens exchanged for back-end JWTs, HMAC signatures) by providing CDI beans that integrate with the security layer and emit the same normalized context consumed by `DomainContext/RuleContext`.

Typical steps:

1. Implement a request filter or identity provider that validates the token/credential.
2. Map identity and tenant claims into a principal model (`tenantId`, `orgRefName`, `userId`, `roles`).
3. Ensure `BaseResource` (and other entry points) can derive `DomainContext` from that principal.

11.3. Creating an Auth Plugin (using the Custom JWT provider as a reference)

An auth plugin is typically a CDI bean that:

- Extends `BaseAuthProvider` to inherit user-management helpers and persistence utilities.
- Implements `AuthProvider` to integrate with request-time authentication flows.
- Implements `UserManagement` to expose CRUD-style operations for users, passwords, and roles.

A concrete provider should:

- Be annotated as a CDI bean (e.g., `@ApplicationScoped`).
- Provide a stable `getName()` identifier (e.g., `"custom"`, `"oidc"`, `"apikey"`).
- Use config properties for secrets, issuers, token durations, and any external identity provider details.

- Build a Quarkus SecurityIdentity with the authenticated principal and roles.

11.4. AuthProvider interface (what a provider must implement)

Core methods:

- SecurityIdentity validateAccessToken(String token)
 - Parse and validate the incoming credential (JWT, API key, signature).
 - Return a SecurityIdentity with principal name and roles. Throw a security exception for invalid tokens.
- String getName()
 - A short identifier for the provider. Persisted alongside credentials and used in logs/metrics.
- LoginResponse login(String userId, String password)
 - Credential-based login. Return a structured response:
 - positiveResponse: includes SecurityIdentity, roles, accessToken, refreshToken, expirationTime, and realm/mongodbUrl if applicable.
 - negativeResponse: includes error codes/reason/message for clients to act on (e.g., password change required).
- LoginResponse refreshTokens(String refreshToken)
 - Validate the refresh token, mint a new access token (and optionally a new refresh token), and return a positive response.

Notes:

- Login flow should check force-change-password or equivalent flags and return a negative response when user interaction is required before issuing tokens.
- validateAccessToken should only accept valid, non-expired tokens and construct SecurityIdentity consistently with role mappings used across the platform.

11.5. UserManagement interface (operations your plugin must support)

Typical responsibilities include:

- User lifecycle
 - String createUser(String userId, String password, Set<String> roles, DomainContext domainContext, [optional] DataDomain)
 - void changePassword(String userId, String oldPassword, String newPassword, Boolean forceChangePassword)
 - boolean removeUserWithUserId(String userId)

- `boolean removeUserWithSubject(String subject)`
- Role management
 - `void assignRolesForUserId(String userId, Set<String> roles)`
 - `void assignRolesForSubject(String subject, Set<String> roles)`
 - `void removeRolesForUserId(String userId, Set<String> roles)`
 - `void removeRolesForSubject(String subject, Set<String> roles)`
 - `Set<String> getUserRolesForUserId(String userId)`
 - `Set<String> getUserRolesForSubject(String subject)`
- Lookups and existence checks
 - `Optional<String> getSubjectForUserId(String userId)`
 - `Optional<String> getUserIdForSubject(String subject)`
 - `boolean userIdExists(String userId)`
 - `boolean subjectExists(String subject)`

Return values and exceptions:

- Throw `SecurityException` or domain-specific exceptions for invalid states (duplicate users, bad password, unsupported hashing).
- Return `Optional` for lookups that may not find a result.
- For removals, return `boolean` to communicate whether a record was deleted.

11.6. Leveraging `BaseAuthProvider` in your plugin

When you extend `BaseAuthProvider`, you inherit ready-to-use capabilities that reduce boilerplate:

- Impersonation controls
 - `enableImpersonationWithUserId` / `enableImpersonationWithSubject`
 - `disableImpersonationWithUserId` / `disableImpersonationWithSubject`
 - These set or clear an impersonation filter script and realm regex that downstream services can honor to act on behalf of another identity under controlled scope.
- Realm override helpers
 - `enableRealmOverrideWithUserId` / `enableRealmOverrideWithSubject`
 - `disableRealmOverrideWithUserId` / `disableRealmOverrideWithSubject`
 - Useful for multi-realm/tenant scenarios, enabling scoped cross-realm behavior.
- Persistence utilities
 - Built-in use of the credential repository to save, update, and delete credentials.
 - Consistent validation of inputs (non-null checks, non-blank checks).
 - Hashing algorithm guardrails to ensure only supported algorithms are used.

Best practices when deriving:

- Record the auth provider name in stored credentials as informational provenance. It tells callers which provider resolved the credential or token; it is not an authentication routing or authorization binding.
- Reuse the role merge/remove patterns to avoid accidental role loss.
- Prefer emitting precise exceptions (e.g., `NotFound` for missing users, `SecurityException` for access violations).

11.7. Implementing your own provider

Checklist:

- Class design
 - `@ApplicationScoped` bean
 - extends `BaseAuthProvider`
 - implements `AuthProvider` and `UserManagement`
 - return a stable `getName()`
- Configuration
 - Externalize secrets (signing keys), issuers, token durations, and realm details via `MicroProfile Config`.
- `SecurityIdentity`
 - Consistently build identities with principal and roles; include useful attributes for auditing/telemetry.
- Tokens/credentials
 - For JWT-like tokens, implement robust parsing, signature verification, expiration checks, and claim validation.
 - For non-JWT credentials (API keys, HMAC), ensure replay protection and scope binding.
- Responses and errors
 - Use structured `LoginResponse` for both success and error paths.
 - Prefer idempotent user/role operations; validate inputs and surface actionable messages.

11.8. `CredentialUserIdPassword` model and `DomainContext`

This section explains how user credentials are represented, how those records tie to tenancy and realms, and how the server chooses the database (“realm”) for REST calls.

What the credential model represents

userId

The human-friendly login handle that users type. Must be unique within the applicable tenancy/realm scope.

subject

A stable, system-generated identifier for the principal. Tokens and internal references favor subject over userId because subjects do not change.

description, emailOfResponsibleParty

Optional metadata to describe the credential and provide an owner contact.

domainContext

The tenancy and organization placement of the principal. It contains:

- tenantId: Logical tenant partition.
- orgRefName: Organization/business unit within the tenant.
- accountId: Account or billing identifier.
- defaultRealm: The default database/realm used for this identity's operations.
- dataSegment: Optional partitioning segment for advanced sharding or data slicing.

roles

The set of authorities granted (e.g., USER, ADMIN). These become groups/roles on the SecurityIdentity.

issuer

An identifier for who issued the credential or tokens (useful for auditing and multi-provider setups).

passwordHash, hashingAlgorithm

The stored password hash and declared algorithm. Not exposed over REST. Providers verify passwords against this.

forceChangePassword

Flag that forces a password reset on next login; the login flow returns a structured negative response instead of tokens.

lastUpdate

Timestamp for auditing and token invalidation strategies.

area2RealmOverrides

Optional map to route specific functional areas to different realms than the default (e.g., "Reporting" → analytics-realm).

realmRegEx

Optional regex to limit or override which realms this identity may act in; also used by impersonation/override flows.

impersonateFilterScript

Optional script indicating the filter/scope applied during impersonation so actions are constrained.

authProviderName

Informational provenance naming the provider that resolved this credential or token (e.g., “custom”, “oidc”). Provider selection comes from an explicit request override or the ordered **auth.provider** configuration, never from this field.

How DomainContext selects the realm for REST calls

- For each authenticated request, the server derives or retrieves a DomainContext associated with the principal.
- The DomainContext.defaultRealm indicates which backing MongoDB database (“realm”) should be used by repositories for that request.
- If realm override features are enabled (e.g., through provider helpers or per-credential overrides), the system may route certain functional areas to alternate realms using area2RealmOverrides or validated by realmRegEx.
- The remainder of DomainContext (tenantId, orgRefName, accountId, dataSegment) is applied as scope constraints through permission rules and repository filters so reads and writes are automatically restricted to the correct tenant/org segment.

Typical flow

1. Login

- A user authenticates with userId/password (or other mechanism).
- On success, a token is returned alongside role information; the principal is associated with a DomainContext that includes the defaultRealm.

2. Subsequent REST calls

- The token is validated; the server reconstructs SecurityIdentity and DomainContext.
- Repositories choose the datastore for defaultRealm and enforce tenant/org filters using the DomainContext values.
- If the request targets a functional area with a defined override, the operation may route to a different realm for that area alone.

3. UI implications

- The client does not need to know which realm is selected; it simply calls the API. The server ensures the correct database is used based on DomainContext and any configured overrides.

Best practices

- Keep userId immutable once established; use subject for internal joins and token subjects.
- Always attach the correct DomainContext when creating users to avoid cross-tenant leakage.
- Use realm overrides deliberately for well-isolated areas (e.g., analytics, archiving) and document them for operators.

11.9. Quarkus OIDC out-of-the-box and integrating with common IdPs

Quarkus ships with first-class OpenID Connect (OIDC) support, enabling both service-to-service and browser-based logins.

What the Quarkus OIDC extension provides

- OIDC client and server-side adapters:
 - Authorization Code flow with PKCE for browser sign-in.
 - Bearer token authentication for APIs (validating access tokens on incoming requests).
 - Token propagation for downstream calls (forwarding or exchanging tokens).
- Token verification and claim mapping:
 - Validates issuer, audience, signature, expiration, and scopes.
 - Maps standard claims (sub, email, groups/roles) into the security identity.
- Multi-tenancy and configuration:
 - Supports multiple OIDC tenants via configuration, each with its own issuer, client id/secret, and flows.
- Logout and session support:
 - Front-channel and back-channel logout hooks depending on provider capabilities.

Integrating with common providers

- Works with providers like Keycloak, Auth0, Okta, Azure AD, Cognito, and enterprise IdPs exposing OIDC.
- Configure the issuer URL and client credentials. Quarkus discovers endpoints via the provider's .well-known/openid-configuration.
- For roles/permissions, map provider groups/roles claims to your platform roles in the identity.

OIDC vs OAuth vs OpenID (terminology and evolution)

OAuth 2.0

- Authorization framework for delegated access (scopes), not authentication. Defines flows to obtain access tokens for APIs.

OpenID (OpenID 1.x/2.0)

- Older federated identity protocol that preceded OIDC. It has been superseded by OpenID Connect.

OpenID Connect (OIDC)

- An identity layer on top of OAuth 2.0. Adds standardized authentication, user info endpoints, ID tokens (JWT) with subject and profile claims, and discovery metadata.
- In practice, OIDC is the modern standard for SSO and user authentication; OAuth remains the authorization substrate underneath.

Summary

- OpenID → historical, replaced by OIDC.
- OAuth 2.0 → authorization framework.
- OIDC → authentication (identity) layer built on OAuth 2.0.

OIDC and SAML in relation to SSO SAML (Security Assertion Markup Language)::

- XML-based federation protocol widely used in enterprises for browser SSO.
- Uses signed XML assertions transported through browser redirects/posts.

OIDC

- JSON/REST-oriented, uses JWTs, and is well-suited for modern SPAs and APIs.
- Relationship:
 - Both enable SSO and federation across identity providers and service providers.
 - Many enterprise IdPs support both; OIDC is generally simpler for APIs and modern web stacks, while SAML is entrenched in legacy/enterprise SSO.
- Bridging:
 - Gateways or identity brokers can translate SAML assertions to OIDC tokens and vice versa, allowing gradual migration.

Common customer IdP models and OIDC integration patterns

- Centralized IdP (single-tenant):
 - One organization-wide IdP issues tokens for all users.
 - Configure a single OIDC tenant in Quarkus; map groups/roles to application roles.
- Multi-tenant SaaS with per-tenant IdP:
 - Each customer brings their own IdP (BYOID).
 - Configure Quarkus OIDC multitenancy with per-tenant issuer discovery and client credentials.
 - Tenant selection can be based on domain, request header, or path; the selected OIDC tenant performs login and token validation.
- Brokered identity:
 - Use a broker (e.g., a central identity layer) that federates to multiple upstream IdPs (OIDC, SAML).
 - Quarkus integrates with the broker as a single OIDC client; the broker handles IdP routing and protocol translation.
- Hybrid API and web flows:
 - Browser apps use Authorization Code flow with sessions; APIs use bearer token authentication.
 - Quarkus OIDC extension can handle both in the same application when properly configured.

Best practices

- Prefer OIDC for new integrations; use SAML through a broker if enterprise constraints require it.
- Normalize roles/claims server-side so downstream authorization (RuleContext, repositories) sees consistent group names regardless of IdP.
- Use token exchange or client credentials for service-to-service calls; do not reuse end-user tokens where not appropriate.
- For multi-tenant OIDC, secure tenant resolution logic and validate issuer/tenant binding to prevent mix-ups.

11.10. Authorization via RuleContext

Authentication establishes identity; RuleContext enforces what the identity can do. For each action (CREATE, UPDATE, VIEW, DELETE, ARCHIVE), RuleContext can:

- Allow or deny the action
- Contribute additional filters (e.g., org scoping, functional-area specific sharing)
- Adjust UIActionList to reflect permitted next steps

This division of responsibilities keeps providers focused on identity while policies remain centralized in RuleContext.

11.11. Generating JWT Tokens for API Access

Applications frequently need machine-to-machine access that does not go through an interactive login flow. The framework provides two mechanisms: **short-lived user tokens** generated via the login endpoint, and **long-lived service tokens** generated via the dedicated service-token endpoint. Both produce standard RS256-signed JWTs that carry identity, roles, and scope claims consumed by the `SecurityFilter` and the permission system.

11.11.1. Token Anatomy

Every JWT produced by the framework contains the following claims:

Claim	Example	Purpose
<code>sub</code>	<code>a1b2c3d4-...</code> (UUID)	The subject — a stable identifier for the principal. For user tokens, this is the user's <code>subject</code> from the credential store. For service tokens, a newly generated UUID.
<code>iss</code>	<code>https://myapp.example.com/issuer</code>	Issuer — must match <code>mp.jwt.verify.issuer</code> in the consuming application's config for the token to be accepted.
<code>aud</code>	<code>b2bi-api-client</code>	Audience — the intended recipient. Auth tokens use <code>b2bi-api-client</code> ; refresh tokens use <code>b2bi-api-client-refresh</code> .

Claim	Example	Purpose
groups	["admin", "user"]	Roles embedded in the token. These are merged with roles from the credential store and user group membership at validation time.
scope	authToken	Distinguishes auth tokens (authToken) from refresh tokens (refreshToken).
iat	1707849600	Issued-at timestamp (seconds since epoch).
exp	1707853200	Expiration timestamp (seconds since epoch).

The token is signed with the application's RSA private key using the RS256 algorithm. By default, the key is loaded from `privateKey.pem` on the classpath (provided by the `quantum-default-keys` module during development). In production, configure `quantum.jwt.private-key-location` to point at an externally managed key. Consumers validate the signature using the corresponding public key (`publicKey.pem` or the configured `quantum.jwt.public-key-location`).

11.11.2. Generating User Tokens via Login

The standard way to obtain a token is through the login endpoint:

```
curl -sS -X POST \
  'http://localhost:8080/auth/login?userId=admin@example.com&password=secret'
```

Response:

```
{
  "accessToken": "eyJhbGciOiJSUzI1NiIsInR5cCI6I..",
  "refreshToken": "eyJhbGciOiJSUzI1NiIsInR5cCI6I.."}

```

The access token expires after the configured duration (`com.b2bi.jwt.duration` in seconds). Use the refresh token to obtain a new access token without re-authenticating:

```
curl -sS -X POST \
  -H 'Authorization: Bearer <refresh-token>' \
  'http://localhost:8080/auth/refresh'
```

This flow is appropriate for interactive clients (UIs, CLI tools) that can handle periodic token refresh.

11.11.3. Service Tokens for System-to-System Integration

For integrations that cannot perform interactive login — MCP servers, cron jobs, webhook handlers, CI/CD pipelines, partner APIs — the framework provides **service tokens**: long-lived JWTs with a dedicated credential record.

Generating a Service Token

```
curl -sS -X POST \
  -H 'Authorization: Bearer <admin-access-token>' \
  -H 'Content-Type: application/json' \
  http://localhost:8080/auth/service-token \
  -d '{
    "roles": ["user"],
    "expirationSeconds": 31536000,
    "description": "MCP server integration token"
  }'
```

Alternatively, you can use the **Quantum CLI** tool to generate service tokens interactively or via command-line arguments.

Using the Quantum CLI

The Quantum CLI (`quantum-cli` module) provides a `generate-token` command that handles authentication and token generation in a single step.

Build the CLI:

```
mvn clean package -pl quantum-cli -am
```

The executable JAR is located at `quantum-cli/target/quarkus-app/quarkus-run.jar`.

Usage:

```
java -jar quantum-cli/target/quarkus-app/quarkus-run.jar \
  --user admin@company.com \
  --roles admin,system \
  --expires -1 \
  --description "MCP Server Token" \
  --url http://localhost:8080
```

Command Options:

- `-u`, `--user`: (Required) The User ID to authenticate as.
- `-p`, `--password`: (Required) The password for authentication. You will be prompted for it if not provided.
- `-r`, `--roles`: Comma-separated list of roles to embed in the token.
- `-e`, `--expires`: Expiration in seconds. Use `-1` for a non-expiring token.
- `-d`, `--description`: Optional description for the service token.
- `--url`: The base URL of the Quantum API (defaults to `http://localhost:8080`).

Response:

```
{
  "accessToken": "eyJhbGciOiJSUzI1NiIsInR5cCI6I..",
  "issuer": "https://myapp.example.com/issuer",
  "subject": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "credentialType": "SERVICE_TOKEN"
}
```

Request Parameters

Field	Type	Required	Description
<code>roles</code>	string array	Yes	Roles to embed in the token and store on the credential (e.g., <code>["user"]</code> , <code>["admin"]</code> , <code>["agent-read"]</code>).
<code>expirationSeconds</code>	long	No	Seconds until expiry. When <code>null</code> , the token is issued for approximately 100 years (effectively non-expiring).
<code>description</code>	string	No	Human-readable label for the credential (e.g., <code>"Nightly sync job"</code> , <code>"Claude MCP integration"</code>).

What Happens on the Backend

When you call `POST /auth/service-token`, the framework:

1. **Validates the caller**—the endpoint requires `admin` or `system` role (`@RolesAllowed({"admin", "system"})`). Only administrators can create service tokens.
2. **Looks up the caller's credential**—from the JWT `sub` claim, the framework finds the caller's `CredentialUserIdPassword` in the system realm.
3. **Creates a new `SERVICE_TOKEN` credential** with:
 - `userId: svc:<uuid>` (auto-generated, not user-controlled)
 - `subject`: A new UUID
 - `credentialType: SERVICE_TOKEN`
 - `parentCredentialSubject`: The caller's subject (for audit trail)
 - `roles`: From the request
 - `domainContext`: Inherited from the caller's credential (same realm, tenant, org)
 - `issuer`: The configured `mp.jwt.verify.issuer`
4. **Saves the credential** to MongoDB.
5. **Links it to the caller's UserProfile** via `additionalCredentialRefs` so the token can be traced to its creator.
6. **Signs and returns the JWT.**

Using a Service Token

The service token is used like any other JWT — pass it in the **Authorization** header:

```
curl -sS -X GET \  
  -H 'Authorization: Bearer <service-token>' \  
  http://localhost:8080/api/query/rootTypes
```

The **SecurityFilter** validates the token, looks up the **SERVICE_TOKEN** credential by its **sub** claim, and builds a **PrincipalContext** with the credential's **DomainContext**, roles, and realm — exactly as it would for a user token.

Example: Configuring an MCP Server with a Service Token

Generate a token for your MCP integration:

```
# Login as admin first  
ACCESS_TOKEN=$(curl -sS -X POST \  
  'http://localhost:8080/auth/login?userId=admin@mycompany.com&password=admin-secret'  
 \  
  | jq -r '.accessToken')  
  
# Generate a service token with read-only roles  
SERVICE_TOKEN=$(curl -sS -X POST \  
  -H "Authorization: Bearer $ACCESS_TOKEN" \  
  -H 'Content-Type: application/json' \  
  http://localhost:8080/auth/service-token \  
  -d '{  
    "roles": ["user"],  
    "expirationSeconds": 31536000,  
    "description": "Claude MCP integration - read-only"  
  }' | jq -r '.accessToken')  
  
echo "Service token: $SERVICE_TOKEN"
```

Then configure Claude Desktop to use it via a bridge that includes the token:

```
{  
  "mcpServers": {  
    "quantum": {  
      "command": "node",  
      "args": ["/path/to/quantum-mcp-bridge/dist/index.js"],  
      "env": {  
        "QUANTUM_BASE_URL": "https://api.mycompany.com",  
        "QUANTUM_AUTH_TOKEN": "<paste-service-token-here>"  
      }  
    }  
  }  
}
```

```
}
```

For the native MCP server (when the Quantum app itself serves `/mcp`), authentication depends on your deployment. In development, the MCP endpoint may be accessible without a token if you configure it appropriately. In production, configure your MCP client or reverse proxy to include the service token as a Bearer header.

11.11.4. How Tokens Scope to Security Policies

A token does not grant unlimited access. Every API call passes through the same security pipeline regardless of whether it was made by a human user, an MCP client, or a service integration.

Token-to-Policy Flow

```
JWT arrives in Authorization header
|
v
SecurityFilter parses JWT
|
v
Extract "sub" claim -> look up CredentialUserIdPassword in system realm
|
v
Build PrincipalContext:
- userId (from credential)
- roles (merged from TOKEN + CREDENTIAL + USERGROUP)
- dataDomain (tenantId, orgRefName, accountNum, ownerId)
- defaultRealm (from credential's DomainContext)
- area2RealmOverrides (optional per-area realm routing)
|
v
Determine ResourceContext:
- area (from @FunctionalMapping, e.g., "integration")
- functionalDomain (e.g., "query")
- action (from @FunctionalAction or HTTP method, e.g., "find")
|
v
RuleContext.checkRules(principalContext, resourceContext):
- Match rules by identity (role or userId)
- Match by area, functionalDomain, action
- Apply DataDomain constraints (tenantId, orgRefName, etc.)
- Return ALLOW / DENY with winning rule name
|
v
If DENY -> HTTP 403 with decision details
If ALLOW (EXACT) -> proceed, no data filters
If ALLOW (SCOPED) -> proceed with data-level filters applied at repo layer
```

Role Merging

When a token is validated, the effective roles come from three sources:

1. **TOKEN:** The `groups` claim in the JWT itself.
2. **CREDENTIAL:** The `roles` array stored on the `CredentialUserIdPassword` document in MongoDB.
3. **USERGROUP:** Roles inherited from `UserGroup` membership, resolved via the user's `UserProfile` in the target realm.

The union of all three sources becomes the effective role set for the request.

Data Scoping via DataDomain

The credential's `DomainContext` determines which tenant, organization, and account the token operates within. This translates to a `DataDomain` that is applied as query filters at the repository layer:

- **tenantId:** Isolates data by tenant. A service token created by an `acme-corp` admin can only access `acme-corp` data.
- **orgRefName:** Scopes to an organization within the tenant.
- **accountNum:** Scopes to an account.
- **ownerId:** Set to the credential's `userId`; used for owner-based access rules.

These constraints are enforced automatically by Morphia repositories and cannot be bypassed by the token holder.

Realm Override with X-Realm

If the service token's credential has `authorizedRealms` or a matching `realmRegex`, the caller can pass `X-Realm: <target-realm>` to operate in a different realm. The `SecurityFilter` validates that the credential is authorized for the target realm, then rebuilds the `PrincipalContext` with the target realm's `DomainContext`.

```
curl -sS -X POST \  
  -H 'Authorization: Bearer <service-token>' \  
  -H 'X-Realm: acme-corp-staging' \  
  -H 'Content-Type: application/json' \  
  http://localhost:8080/api/agent/execute \  
  -d '{"tool": "query_find", "arguments": {"rootType": "Order", "query":  
"status:OPEN"}}'
```

Limiting Service Token Scope

To follow the principle of least privilege:

- **Assign minimal roles** — give the service token only the roles it needs. If it only reads data, do not include `admin`.

- **Use per-tenant agent configuration** — configure `quantum.agent.tenant.<realm>.enabledTools` to restrict which gateway tools the agent can call.
- **Set `maxFindLimit`** — cap query results with `quantum.agent.tenant.<realm>.maxFindLimit`.
- **Use `expirationSeconds`** — even for long-lived tokens, set an expiration. A 1-year token (31536000 seconds) is preferable to an effectively infinite one.
- **Monitor and audit** — the `parentCredentialSubject` on `SERVICE_TOKEN` credentials links every service token to the admin who created it. Token validation logs include the `userId` and `realm` for each request.

11.11.5. Generating RSA Key Pairs

The framework signs tokens with an RSA key pair. To generate a new key pair:

```
# Generate a 2048-bit RSA private key in PKCS#8 format
openssl genpkey -algorithm RSA -out privateKey.pem -pkeyopt rsa_keygen_bits:2048

# Extract the public key
openssl rsa -pubout -in privateKey.pem -out publicKey.pem
```



The private key must be in PKCS#8 PEM format (begins with `-----BEGIN PRIVATE KEY-----`). PKCS#1 format (`-----BEGIN RSA PRIVATE KEY-----`) is not supported. If you have a PKCS#1 key, convert it:

```
openssl pkcs8 -topk8 -inform PEM -outform PEM -nocrypt -in rsa_private.pem -out
privateKey.pem
```

Where to Place the Keys

There are three options, from simplest to most production-ready:

Option 1: Use the default keys (development only)

Add the `quantum-default-keys` module to your application POM. This places a pre-generated key pair on the classpath automatically — no files to copy.

```
<dependency>
  <groupId>com.end2endlogic</groupId>
  <artifactId>quantum-default-keys</artifactId>
  <version>${quantum.version}</version>
</dependency>
```



The default keys are published in the open-source repository. Never use them in production.

Option 2: Classpath keys (simple deployments)

Place `privateKey.pem` and `publicKey.pem` in `src/main/resources/` of your application module. The framework loads them from the classpath at startup and caches them in memory.

Option 3: External keys via configuration (recommended for production)

Store the keys outside the application artifact and configure their locations in `application.properties`:

```
quantum.jwt.private-key-location=file:/etc/quantum/keys/privateKey.pem
quantum.jwt.public-key-location=file:/etc/quantum/keys/publicKey.pem

# SmallRye JWT validation also needs the public key path
mp.jwt.verify.publickey.location=/etc/quantum/keys/publicKey.pem
```

This approach allows key rotation without rebuilding or redeploying the application. See [Key Pair Management](#) for the full configuration reference.

11.11.6. Configuration Reference

Property	Default	Description
<code>auth.provider</code>	(required)	Comma-separated list of provider names (e.g., <code>custom,custom,oidc</code>). See Multiple Auth Providers .
<code>mp.jwt.verify.publickey.location</code>	(required)	Classpath location of the RSA public key PEM file (e.g., <code>publicKey.pem</code>).
<code>mp.jwt.verify.issuer</code>	(required)	Expected issuer claim in JWTs. Must match the issuer used during token generation.
<code>mp.jwt.verify.audiences</code>	(required)	Comma-separated list of accepted audience values (e.g., <code>b2bi-api-client,b2bi-api-client-refresh</code>).
<code>com.b2bi.jwt.duration</code>	<code>10000</code>	Access token duration in seconds for the custom JWT provider.
<code>auth.jwt.secret</code>	(required)	Secret for HMAC-based operations. Use an environment variable: <code>\${JWT_SECRET}</code> .
<code>auth.jwt.expiration</code>	<code>15</code>	Fallback access token expiration in minutes.
<code>auth.jwt.refresh-expiration</code>	<code>30</code>	Refresh token expiration in minutes.
<code>quantum.jwt.private-key-location</code>	(unset — defaults to classpath <code>privateKey.pem</code>)	Path to the RSA private key for token signing. Supports <code>file:</code> , <code>classpath:</code> , or bare classpath resource names. See Key Pair Management .
<code>quantum.jwt.public-key-location</code>	(unset — defaults to classpath <code>publicKey.pem</code>)	Path to the RSA public key for token verification. Same prefix conventions as the private key property. NOTE: also update <code>mp.jwt.verify.publickey.location</code> to match.

Chapter 12. Permissions: Rule Bases, SecurityURIHeader, and SecurityURIBody

This section explains how Quantum evaluates permissions for REST requests using rule bases that match on a SecurityURI composed of a header and a body. The header includes identity, area, functionalDomain, and action. The body includes realm, accountNumber, tenantId, dataSegment, ownerId, and resourceId. It also covers how identities and roles (as found on userProfile or credentialUserIdPassword) are matched, how priority works, and how multiple matching rule bases are evaluated.

12.1. Introduction: Layered Enforcement Overview

Quantum evaluates "can this identity do X?" through three complementary layers. Understanding them in order helps you pick the right tool for the job and combine them safely:

1. REST API annotations (top layer, code-level)

- What: JAX-RS/Jakarta Security annotations on resource methods, for example, `@RolesAllowed("ADMIN")`, `@PermitAll`, `@DenyAll`, `@Authenticated`.
- Purpose: Coarse-grained, immediate gates right at the endpoint. Ideal for baseline protections (for example, only ADMIN may call `/admin/**`) and for non-dynamic constraints that rarely change.
- Pros: Simple, fast, visible in code reviews.
- Cons: Hard-coded; changing access requires a code change, build, and deploy. No data-aware scoping (for example, cannot express tenant/domain filters).

2. Feature flags (exposure and variants)

- What: Turn capabilities on/off per environment or cohort and select variants (A/B, multivariate). See "Feature Flags, Variants, and Target Rules" below.
- Purpose: Control who even sees or can reach a capability during rollout (by tenant, role, geography, plan), independently of authorization. Flags answer "is the feature ON and which variant?"; they do not by themselves prove the caller is authorized.
- Pros: Reversible, environment-aware, safe rollout and experimentation.
- Cons: Not a substitute for authorization; must be paired with roles/permission rules for enforcement.

3. Permission Rules with SecurityURI (fine-grained, dynamic)

- What: Declarative rule bases authored against a SecurityURI composed of header (identity, area, functionalDomain, action) and body (realm, accountNumber, tenantId, dataSegment, ownerId, resourceId). Rules decide ALLOW or DENY and may use an optional postconditionScript for additional checks. See sections "Key Concepts", "Matching Algorithm", and examples below.
- Purpose: Express least-privilege, data-aware policies that evolve without code changes (data-driven authoring).

- Pros: Dynamic, auditable, supports role-based matching and simple attribute scoping via the SecurityURI body.
- Cons: Requires governance of rulebases and careful priority management.

How roles, Functional Areas, and Functional Domains fit in

- Roles: Used by both layers (1) and (3). Annotations directly reference roles. In the rule engine, roles are evaluated by treating each role as an identity: rules authored for a role name (for example, "ADMIN") are considered alongside rules authored for the user's own userId. Effective roles are resolved by merging IdP roles with roles on the user record; see "How roles are defined for an identity" below.
- Functional Area/Domain: Derived from the URL convention `{area}/{functionalDomain}/{action}` as parsed by `SecurityFilter.determineResourceContext`. Author policies using these fields to target business capabilities rather than raw URLs or ad-hoc headers.
- DataDomain: When rules ALLOW an action, they can attach data-scope filters (tenant/org/owner) so downstream reads/writes are constrained to the caller's domain.

Choosing the right approach

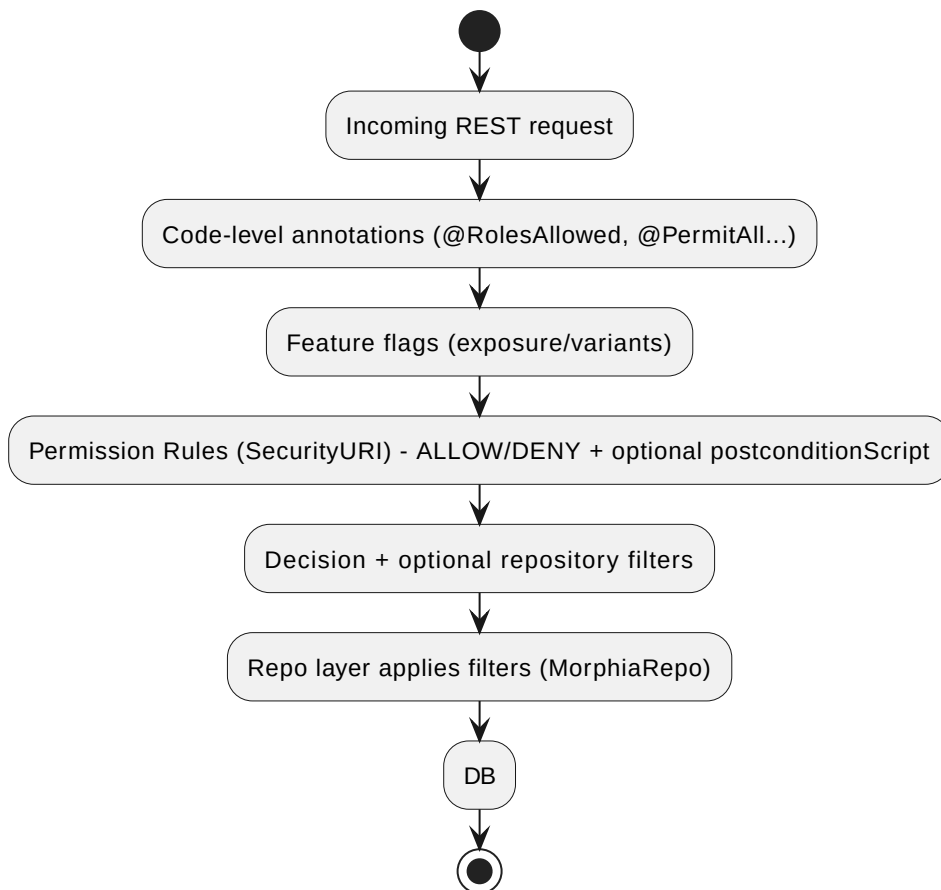
- Use annotations for stable, coarse gates you want visible in code (for example, admin-only endpoints, health endpoints with `@PermitAll`).
- Use feature flags to manage rollout/exposure and variants across environments and cohorts.
- Use permission rules to encode fine-grained, data-aware authorization and to evolve policy without redeploying.

Compare and contrast

- Annotation-based controls are compile-time and hard-code policy into the service; changing them requires code changes.
- Permission Rules and the Rule Language are data-driven and user-changeable (with proper governance), enabling rapid, auditable policy changes and DataDomain scoping.
- In practice: apply annotations as the first gate, evaluate feature flags to determine exposure/variant, then evaluate permission rules to decide ALLOW/DENY and attach scopes. This layered approach yields both safety and agility.



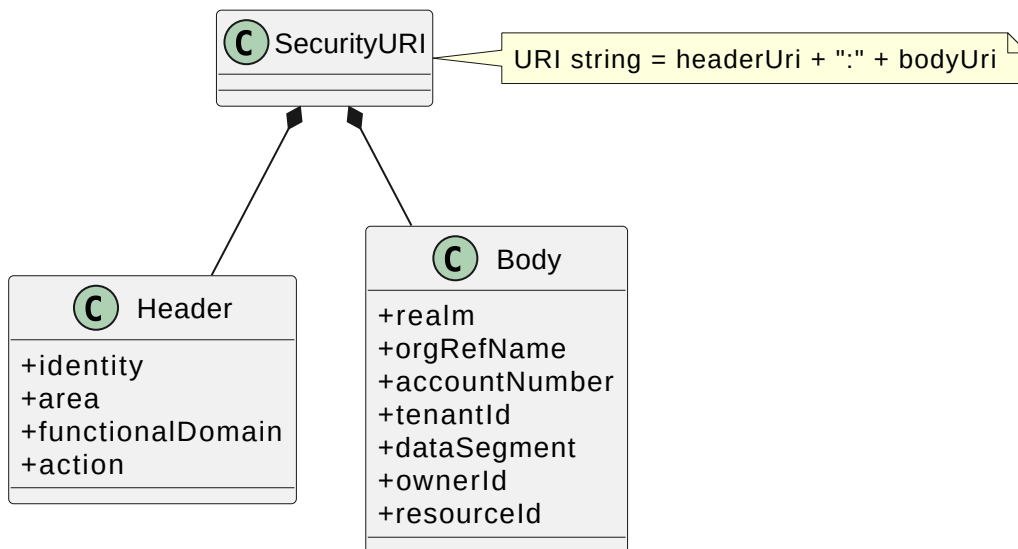
Ontology- and label-aware policy helpers are available to `postconditionScript` when the optional ontology module is enabled. Helpers include `hasEdge`, `hasAnyEdge`, `hasAllEdges`, `relatedIds` for graph checks; `hasLabel` for label checks; and `isA/noViolations` for type/validation contexts. See the "Script Helpers reference" section for details. When data access must be restricted to lists resolved outside the rule engine (for example, from an external ACL service), use `AccessListResolvers` (SPI) and reference their outputs from `andFilterString/orFilterString`.



12.2. Key Concepts

- Identity: The authenticated principal, typically originating from JWT or another provider. It includes:
 - `userId` (or `credentialUserIdPassword` username)
 - roles (authorities/groups)
 - `tenantId`, `orgRefName`, optional realm, and other claims that contribute to `DomainContext`
- `userProfile`: A domain representation of the user that adds human information such as first name, last name, email address, phone number and provides a linkage back to identity, roles, and policy decorations (feature flags, plans, expiration, etc.).
- Rule Base (Permission Rule): A declarative rule with matching criteria and an effect (ALLOW or DENY). Criteria are authored against `SecurityURI` and may include:
 - `SecurityURIHeader` fields: identity (`userId` or role name), area, `functionalDomain`, action
 - `SecurityURIBody` fields: realm, `accountNumber`, `tenantId`, `dataSegment`, `ownerId`, `resourceId`
 - Optional `postconditionScript` evaluated with `pcontext/rcontext` for additional checks
 - Priority: integer used to sort rule evaluation (lower numbers evaluated first)
 - Effect: ALLOW or DENY; ALLOWs may be paired with repository-level scoping using the `SecurityURI` body (e.g., `DataDomain` constraints)

12.3. Rule Structure (Illustrative)



```
- name: allow-catalog-product-reads
description: Allow USER and ADMIN to view products in the Catalog area
securityURI:
  header:
    identity: USER          # or a specific userId; roles are treated as
identities
    area: Catalog
    functionalDomain: Product
    action: view
  body:
    realm: system-com
    accountNumber: '*'
    tenantId: '*'
    dataSegment: '*'
    ownerId: '*'
    resourceId: '*'
  postconditionScript:
  effect: ALLOW
  priority: 300
  finalRule: false

- name: default-deny
description: Fallback deny when nothing else matches
securityURI:
  header:
    identity: '*'
    area: '*'
    functionalDomain: '*'
    action: '*'
  body:
    realm: '*'
    accountNumber: '*'
    tenantId: '*'
```

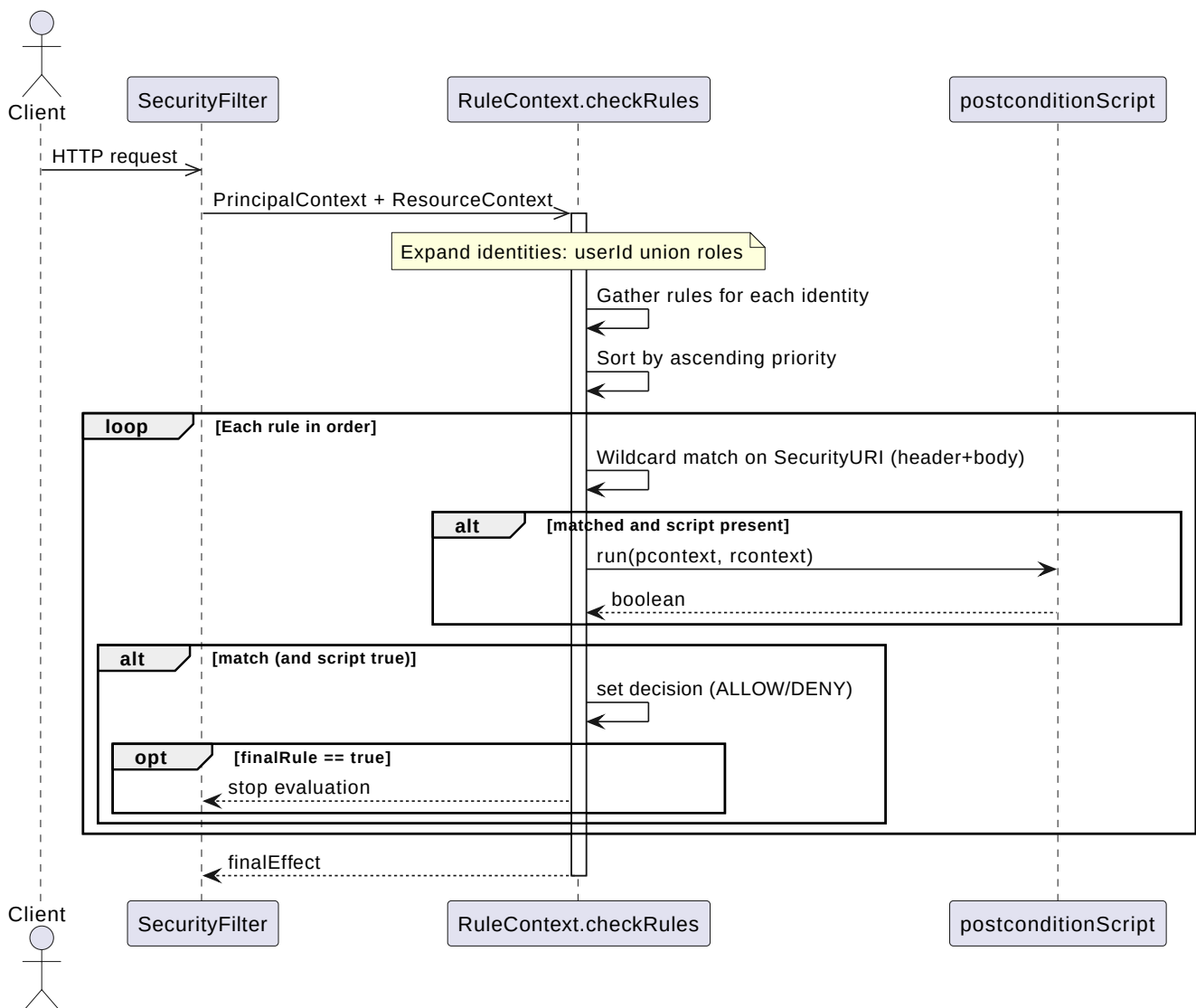
```

dataSegment: '*'
ownerId: '*'
resourceId: '*'
effect: DENY
priority: 10000
finalRule: true

```

- The securityURI.header section corresponds to SecurityURIHeader: identity (userId or role name), area, functionalDomain, and action. Functional area/domain/action are typically derived from the URL convention `/[{area}]/[{functionalDomain}]/[{action}]` by SecurityFilter.
- The securityURI.body section corresponds to SecurityURIBody: realm, accountNumber, tenantId, dataSegment, ownerId, and resourceId. These values are matched using simple string equality with support for the wildcard '*'.
- Optional postconditionScript may be provided and is executed as JavaScript with pcontext and rcontext bindings; the rule only applies if the script evaluates to true.

12.4. Matching Algorithm



The engine evaluates permission rules using SecurityURI wildcard matching. At a high level:

1. Build ResourceContext

- SecurityFilter derives area, functionalDomain, and action from the request path (or REST annotations if present) and sets the ResourceContext.

2. Expand identities

- Build a set of identities consisting of the caller's userId plus each effective role. Each of these identities is treated as a potential match target for rules.

3. Gather candidate rules

- For each identity, collect rules authored for that identity. This forms the candidate set. There is no separate HTTP method/URL or rolesAny/rolesAll matching.

4. Sort by priority

- Order candidates by ascending priority (lower numbers are evaluated first).

5. URI wildcard comparison

- For each rule in priority order, compare the caller's expanded SecurityURIs to the rule's securityURI using case-insensitive wildcard comparison on the full URI string (header + body). Asterisks (*) in rules match any value.

6. Postcondition script (optional)

- If the rule specifies postconditionScript, execute it as JavaScript with pcontext (principal) and rcontext (resource) variables bound. The rule applies only if the script returns true.

7. Apply effect and finalRule

- When a rule matches (and any script returns true), set the response's finalEffect to the rule's effect (ALLOW or DENY). If finalRule is true, stop evaluating further rules; otherwise continue.

8. Default decision

- If no rule determines a decision, the system returns the default final effect configured by the caller of the check (typically DENY).

12.5. Priorities

- Lower integer = higher priority. Example: priority 1 overrides priority 10.
- Use tight scopes with low priority for critical protections (e.g., denials), and broader ALLOWs with higher numeric priority.
- Recommended ranges:
 - 1–99: global deny rules and emergency blocks
 - 100–499: domain/area-specific critical rules
 - 500–999: standard ALLOW policies
 - 1000+: defaults and catch-alls

12.6. Grant-based vs Deny-based Rule Sets

Grant-based rule sets start with a default decision of DENY and then incrementally add ALLOW scenarios through explicit rules. This model is fail-safe by default: any URL, action, or functional area that does not have a matching ALLOW rule remains inaccessible. As new endpoints or capabilities are added to the system, users will not gain access until an explicit ALLOW is authored. This is the recommended posture for security-sensitive systems and multi-tenant platforms.

Deny-based rule sets start with a default decision of ALLOW and then add DENY scenarios to carve away disallowed cases. In this model, new functionality is exposed by default unless a DENY is added. While convenient during rapid prototyping, this posture risks accidental exposure as the surface area grows.

Practical implications:

- Change management: Grant-based requires adding ALLOWs when shipping new features; Deny-based requires remembering to add new DENYs.
- Auditability: Grant-based policies make it easy to enumerate what is permitted; Deny-based requires proving the absence of permissive gaps.
- Safety: In merge conflicts or partial deployments, Grant-based tends to fail closed (DENY), which is usually safer.

Example defaults:

- Grant-based (recommended):

```
- name: default-deny
  priority: 10000
  securityURI:
    header: { identity: '*', area: '*', functionalDomain: '*', action: '*' }
    body:   { realm: '*', accountNumber: '*', tenantId: '*', dataSegment: '*',
ownerId: '*', resourceId: '*' }
  effect: DENY
  finalRule: true
```

- Deny-based (use with caution):

```
- name: default-allow
  priority: 10000
  securityURI:
    header: { identity: '*', area: '*', functionalDomain: '*', action: '*' }
    body:   { realm: '*', accountNumber: '*', tenantId: '*', dataSegment: '*',
ownerId: '*', resourceId: '*' }
  effect: ALLOW
  finalRule: true
```

Tip: Even in a deny-based set, author low-number DENY rules for critical protections. In most

production systems, prefer the grant-based model and layer specific ALLOWs for each capability.

12.7. Row Filter Computation (Data Scoping)

Matching decides *whether* an action is allowed. Once the effect is **ALLOW**, the framework derives the row filter that scopes *which records* a list/query returns. This is the data-scoping half of enforcement, and it is computed per-request from the principal, the matched rules, and (optionally) dynamic access lists.

12.7.1. The pipeline

1. A **BaseResource** list/query calls **MorphiaRepo.getFilterArray(userFilters, modelClass)**. The caller's own query filters are the starting set; security filters are **AND-ed on top** of them.
2. **RepoSecurityFilterBuilder** short-circuits when **SecurityContext.isIgnoringRules()** is set — returning the filters **unscoped**. This **ignoreRules** path is reserved for system-internal/unauthenticated lookups (e.g. credential resolution at login, infrastructure reads). Otherwise it requires both a **PrincipalContext** and a **ResourceContext** and **fails closed** (throws) if either is missing.
3. **RuleContext.getFilters(filters, pcontext, rcontext, modelClass)** runs **checkRules**; if the final effect is not **ALLOW** the base filters are returned unchanged. Otherwise it resolves a variable bundle and, for each matched **ALLOW** rule, interpolates the rule's **andFilterString/orFilterString** via **MorphiaUtils.convertToFilter(...)**, combining them with the rule's **joinOp** (AND/OR) and de-duplicating.
4. The resulting **Filter[]** is AND-ed into the Morphia query.



The **realm** is not a row filter — **getRealmId(principal)** returns the principal's **defaultRealm**, which selects **which database** the query runs against (**realm == database**). Row filters scope records *within* that database.

12.7.2. What is scoped by default

There is exactly **one** scope filter baked into the default/system rules:

```
dataDomain.tenantId:${pTenantId}
```

A normal authenticated query therefore gets **tenant isolation only**. Scoping by organization, account, or data segment is **not automatic** — you must author rules that reference the corresponding variables (see below).

12.7.3. Available substitution variables (static)

MorphiaUtils.createStandardVariableMapFrom registers these scalars from the principal/resource context. They resolve from the principal's **DataDomain** and **DomainContext**, which are populated from the authenticated token's claims.

Variable	Source
<code>\${pTenantId}</code>	<code>DataDomain.tenantId</code> (used by the default rule)
<code>\${pAccountId}</code>	<code>DataDomain.accountNum</code>
<code>\${orgRefName}</code>	<code>DataDomain.orgRefName</code> (note: no <code>p</code> prefix)
<code>\${ownerId}</code>	<code>DataDomain.ownerId</code>
<code>\${defaultRealm}</code>	<code>PrincipalContext.defaultRealm</code> (drives X-Realm scoping)
<code>\${principalId}</code>	<code>PrincipalContext.userId</code>
<code>\${resourceId},</code> <code>\${action},</code> <code>\${functionalDomain},</code> <code>\${area}</code>	<code>ResourceContext</code>
<code>\${dcTenantId},</code> <code>\${dcOrgRefName},</code> <code>\${dcAccountId},</code> <code>\${dcDataSegment}</code>	the <code>DomainContext</code> (X-Realm-aware; may differ from <code>DataDomain</code>)
<code>\${systemTenantId}</code>	<code>DataDomain.tenantId</code> (legacy alias)

To scope by org and account in addition to tenant, author a rule whose filter combines them:

```
- name: scope-by-org-and-account
  priority: 500
  securityURI:
    header: { identity: '*', area: '*', functionalDomain: '*', action: 'view' }
    body:   { realm: '*', tenantId: '*', accountNumber: '*', orgRefName: '*' }
  andFilterString: "dataDomain.orgRefName:${orgRefName}"
  orFilterString:  "dataDomain.accountId:${pAccountId}"
  joinOp: AND
  effect: ALLOW
```



Because every `${...}` value originates in the token, a token that omits tenant/org/account claims makes the corresponding filters unresolvable — even the default `${pTenantId}` filter. A centralized issuer **MUST** project the principal's `DomainContext` (realm/tenant/org/account) into token claims for this machinery to scope correctly.

12.7.4. Dynamic access lists (the `AccessListResolver` SPI)

Some scoping is not a static scalar but a **computed collection** — for example, "the set of location IDs this principal may see". These are supplied by `AccessListResolver` implementations rather than by the standard variable map. A resolver whose `key()` returns `accessibleLocationIds` makes the variable `${accessibleLocationIds}` available, referenced with the in-list operator:

```
locationId:^[${accessibleLocationIds}]
```

The list can be derived however the resolver chooses — a territory assignment, a `PrincipalContextPropertiesResolver` custom property, or by **walking the ontology graph**. The ontology path is `ListQueryRewriter` (in `quantum-ontology-policy-bridge`), which derives accessible IDs from edges such as `hasEdge("canSeeLocation", ...)`. This is the ontology↔rule seam: **ontology edges → access list → `${accessibleLocationIds}` → rule filter**. Predicate references used this way are validated at rule-load time by `RuleVocabularyValidator`.

Two behaviors distinguish dynamic lists from the static scalars:

- **`supports()` gating.** A resolver is only invoked when its `supports(pcontext, rcontext, modelClass)` returns true for the *current request*’s context (not the rule’s scope). Be permissive, or filter by `modelClass`, or the variable will be unresolvable for valid requests.
- **Skip-on-unresolved.** If a rule’s filter references a variable that cannot be resolved for the current model class (e.g. `${accessibleLocationIds}` on a `UserProfile` query), `getFilters` **skips that rule’s filter** rather than erroring — so a Location access rule does not wrongly constrain unrelated queries.

12.7.5. Summary

Scope	Kind	How it applies
Realm	database selection	<code>getRealmId</code> → <code>defaultRealm</code> picks the DB
Tenant	static, default	built-in <code>dataDomain.tenantId:\${pTenantId}</code>
Org / Account / Segment	static, opt-in	author a rule referencing <code>\${orgRefName}</code> / <code>\${pAccountId}</code> / <code>\${dcDataSegment}</code>
Location (and similar)	dynamic, resolver-backed	<code>AccessListResolver</code> (often ontology-edge-backed) supplies <code>\${accessibleLocationIds}</code> ; requires a rule and a registered resolver

12.8. Feature Flags, Variants, and Target Rules

Feature flags complement permission rules by controlling whether a capability is active for a given principal, cohort, or environment. Permissions answer “may this identity perform this action?”; feature flags answer “is this capability turned on, and which variant applies?” Use them together to achieve safe rollouts and fine-grained authorization.

Model reference: `com.e2eq.framework.model.general.FeatureFlag` with key fields:

- `enabled`: master on/off
- `type`: `BOOLEAN` or `MULTIVARIATE`
- `variants`: list of variant keys for multivariate experiments
- `targetRules`: cohort targeting rules
- `environment`: e.g., `dev`, `staging`, `prod`
- `jsonConfiguration`: arbitrary configuration for the feature (e.g., rollout %, UI copy, limits)

Example: Boolean flag for a new export API with environment-specific targeting

```
{
  "refName": "EXPORT_API",
  "description": "Enable CSV export endpoint",
  "enabled": true,
  "type": "BOOLEAN",
  "environment": "prod",
  "targetRules": [
    { "attribute": "role", "operator": "equals", "values": ["BETA"] },
    { "attribute": "tenantId", "operator": "in", "values": ["T100", "T200"] }
  ],
  "jsonConfiguration": { "rateLimitPerMin": 60 }
}
```

Example: Multivariate flag to roll out Search v2 to 10% of users and all members of a beta role

```
{
  "refName": "SEARCH_V2",
  "description": "New search implementation",
  "enabled": true,
  "type": "MULTIVARIATE",
  "variants": ["control", "v2"],
  "environment": "prod",
  "targetRules": [
    { "attribute": "role", "operator": "equals", "values": ["BETA"], "variant": "v2"
  },
    { "attribute": "userId", "operator": "hashMod", "values": ["10"], "variant": "v2"
  }
  ],
  "jsonConfiguration": { "defaultVariant": "control" }
}
```

Notes on TargetRules:

- attribute: a property from identity/userProfile (e.g., userId, role, tenantId, location, plan).
- operator: equals, in, contains, startsWith, regex, or domain-specific operators like hashMod for percentage rollouts.
- values: comparison values; semantics depend on operator.
- variant: when type is MULTIVARIATE, selects which variant applies when the rule matches.

How feature flags complement Permission Rule Context:

- The evaluation of a request can enrich the Rule Context (SecurityURI or userProfile) with resolved feature flags and variants (e.g., userProfile.features["SEARCH_V2"] = "v2").
- Permission rules can then require a feature to be present before ALLOWing an action:

```

- name: allow-export-when-flag-on
  description: Allow ADMIN and REPORTER identities to view export when feature flag is
on
  securityURI:
    header:
      identity: ADMIN      # treat roles as identities
      area: Reports
      functionalDomain: Export
      action: view
    body:
      realm: system-com
      accountNumber: '*'
      tenantId: '*'
      dataSegment: '*'
      ownerId: '*'
      resourceId: '*'
  postconditionScript: userProfile?.features?.EXPORT_API === true
  effect: ALLOW
  priority: 300
  finalRule: false

- name: allow-export-when-flag-on-reporter
  description: Same as above but for REPORTER role
  securityURI:
    header:
      identity: REPORTER
      area: Reports
      functionalDomain: Export
      action: view
    body:
      realm: system-com
      accountNumber: '*'
      tenantId: '*'
      dataSegment: '*'
      ownerId: '*'
      resourceId: '*'
  postconditionScript: userProfile?.features?.EXPORT_API === true
  effect: ALLOW
  priority: 300
  finalRule: false

```

Alternatively, systems may surface feature decisions via headers (e.g., X-Feature-SEARCH_V2: v2) so that rules or postconditionScript can read them directly from the request context.

Business usage examples for TargetRules and their correlation to Permission Rules:

- Progressive rollout by tenant TargetRule tenantId in [T100, T200] → Permission adds ALLOW for endpoints guarded by that flag so only those tenants can call them during rollout.
 - Role-based beta access: TargetRule role equals BETA → Permission requires both the BETA

feature flag and standard role checks (e.g., USER/ADMIN) to ALLOW sensitive actions.

- Plan/entitlement tiers: TargetRule plan in [Pro, Enterprise] → Permission rules enforce additional data-domain constraints (e.g., export size limits) while the flag simply turns the feature on for eligible plans.

Guidance: Feature Flags vs Permission Rules

- Put into Feature Flags:
 - Gradual, reversible rollouts; A/B or multivariate experiments; UI/behavior switches.
 - Environment gates (dev/staging/prod) and cohort targeting (tenants, beta users, geography).
 - Non-security configuration values in jsonConfiguration (limits, thresholds, copy) that do not change who is authorized.
- Put into Permission Rules:
 - Durable authorization logic: roles, identities, functional area/domain/action, and DataDomain constraints.
 - Compliance and least-privilege decisions where fail-closed behavior is required.
 - Enforcement that remains valid after a feature is fully launched (even when the flag is removed).

Recommendation

Use a grant-based permission posture (default DENY) and let feature flags decide which cohorts even see or can reach new capabilities. Then author explicit ALLOW rules for those capabilities, conditioned on both role and feature presence.

12.9. Scoped Action Enablement and Dependency Tracking

For the dedicated guide covering the runtime API, manifest model, SPI contracts, and built-in resolver types, see [Scoped Action Enablement](#).

Permission checks answer one important question: "may this identity perform **area** / **functionalDomain** / **action**?" In practice, teams also need to answer a second question: "why is this action still unavailable?"

Typical reasons include:

- policy denies the action
- a feature flag exists but is disabled
- a required setting has not been configured
- a prerequisite entity or runtime dependency is missing

Quantum now provides a framework-level dependency tracking layer for these cases. The implementation [guide](#) for this work lives in [docs/design/SCOPED_ACTION_ENABLEMENT_IMPLEMENTATION_GUIDE.md](#), and the runtime contracts are

implemented in the `quantum-action-enablement-models` and `quantum-action-enablement-quarkus` modules.

12.9.1. Core idea: the same scoped action tuple stays canonical

The unit of evaluation is still the same tuple already used by the policy engine:

```
area / functionalDomain / action
```

Examples:

- `integration / exchange / create`
- `integration / workflow / create`
- `system / action-enablement / check`

This is intentional. The framework does not introduce a second authorization primitive just to model readiness. Instead, it composes additional dependencies around the same scoped action that policy already understands.

Capabilities are still useful, but at a different level:

- A scoped action is the atomic framework-level unit.
- A capability is an application-defined composition over one or more scoped actions.
- Settings, feature flags, and runtime prerequisites are attached to the scoped action requirement manifest.

That structure is what lets a UI or shell explain "this action is blocked because the policy denies it" versus "this action is allowed, but the required secret or filesystem is not ready yet."

12.9.2. The three answers returned by the framework

Scoped action enablement separates three concerns that are often blurred together:

Field	Meaning
<code>allowed</code>	Whether the existing permission rule stack allows the scoped action for the evaluated identity and data domain
<code>enabled</code>	Whether rollout and administrative toggles, especially feature flags, say the action should be exposed
<code>ready</code>	Whether operational prerequisites such as settings, entities, or runtime dependencies are satisfied

An action is considered usable only when all three are true:

```
usable = allowed && enabled && ready
```

This distinction matters because the remediation path is different:

- `allowed=false` usually means a policy or role problem
- `enabled=false` usually means a feature flag or administrative rollout problem
- `ready=false` usually means setup, configuration, or runtime dependencies are incomplete

12.9.3. Runtime models

The core request and response models are intentionally small:

```
public class ScopedActionRef {
    String area;
    String functionalDomain;
    String action;
}

public class DependencyCheckRef {
    String type;
    String refName;
    Map<String, Object> config;
}

public class ScopedActionRequirement {
    ScopedActionRef scopedAction;
    String displayName;
    String description;
    List<DependencyCheckRef> dependencies;
}

public class ScopedActionEnablementStatus {
    ScopedActionRef scopedAction;
    boolean allowed;
    boolean enabled;
    boolean ready;
    boolean usable;
    List<EnablementBlocker> blockers;
}
```

An `EnablementBlocker` is the machine-readable explanation of why evaluation failed. Each blocker includes:

- `impact`: one of `ALLOWED`, `ENABLED`, or `READY`
- `type`: the dependency type, such as `permission` or `feature-flag`
- `code`: a stable blocker code, such as `permission-denied`
- `message`: a human-readable explanation
- `severity`: typically `warn` or `error`

- **metadata**: additional structured context

12.9.4. REST API

The runtime API is exposed at:

- **POST** /system/actions/enablement/check
- **GET** /system/actions/enablement/manifest

This resource itself is annotated as:

- **area=system**
- **functionalDomain=action-enablement**
- **action=check|view**

Example request:

```
{
  "identity": "tenant-admin",
  "realm": "acme",
  "orgRefName": "acme-org",
  "tenantId": "acme-tenant",
  "scope": "api",
  "actions": [
    {
      "area": "integration",
      "functionalDomain": "guided_integration",
      "action": "create"
    }
  ]
}
```

Representative response:

```
{
  "results": [
    {
      "scopedAction": {
        "area": "integration",
        "functionalDomain": "guided_integration",
        "action": "create"
      },
      "allowed": true,
      "enabled": false,
      "ready": false,
      "usable": false,
      "blockers": [
        {

```

```

        "impact": "ENABLED",
        "type": "feature-flag",
        "code": "feature-flag-disabled",
        "message": "Feature flag 'guided-integrations' is disabled.",
        "severity": "warn"
    },
    {
        "impact": "READY",
        "type": "setting-present",
        "code": "setting-missing",
        "message": "Configuration setting 'quantum.integration.shared-storage.root'
is not set.",
        "severity": "error"
    }
]
}
]
}

```

This is the key user-facing value: the response tells you whether the action is blocked by authorization, rollout state, or operational readiness, instead of forcing the UI to guess.

12.9.5. Manifest-driven dependency relationships

The relationship from settings and feature flags to functional area/domain/action lives in a manifest of `ScopedActionRequirement` entries. Each manifest entry says:

- which scoped action is being described
- which dependencies must be satisfied for that action to be usable
- optional display-oriented metadata such as `displayName` and `description`

Applications contribute these manifests via CDI:

```

public interface ScopedActionRequirementContributor {

    default int priority() {
        return 100;
    }

    Collection<ScopedActionRequirement> requirements();
}

```

The framework aggregates contributors through `ScopedActionRequirementRegistry`:

```

public interface ScopedActionRequirementRegistry {

    Optional<ScopedActionRequirement> find(ScopedActionRef ref);
}

```

```
List<ScopedActionRequirement> list();
}
```

This is how you express relationships such as:

- action `integration / guided_integration / create` depends on feature flag `guided-integrations`
- action `integration / exchange / create` depends on a tenant filesystem being provisioned
- action `system / some-admin-action / save` depends on a specific config property being present

12.9.6. Resolver SPI

Each dependency type is implemented by an `ActionDependencyResolver`:

```
public interface ActionDependencyResolver {

    String supportsType();

    DependencyResolutionResult evaluate(
        DependencyCheckRef dependency,
        EnablementEvaluationContext context
    );
}
```

`EnablementEvaluationContext` carries the evaluation inputs:

- identity
- realm
- roles
- scope
- `DataDomain`
- the target `ScopedActionRef`

This lets each resolver examine dependencies in the correct tenant, realm, and user context.

12.9.7. Built-in OSS dependency types

The open-source framework currently provides these dependency resolvers:

Dependency type	Purpose	Typical impact
<code>permission</code>	Evaluate the scoped action through the existing <code>RuleContext</code> and permission model	<code>ALLOWED</code>
<code>feature-flag</code>	Require that a <code>FeatureFlag</code> exists and is enabled in the active realm	<code>ENABLED</code>

Dependency type	Purpose	Typical impact
setting-present	Require that a named configuration property exists and is non-blank	READY
entity-exists	Require that a referenced entity exists in the current realm	READY

Representative blocker codes include:

- permission-denied
- feature-flag-missing
- feature-flag-disabled
- setting-name-missing
- setting-missing
- entity-model-unresolved
- entity-missing

12.9.8. Missing manifests and unsupported dependency types

The framework intentionally does not fail open here.

If no manifest entry is registered for a scoped action:

- evaluation falls back to a synthetic requirement that checks `permission`
- the response also includes a `manifest-missing` blocker with `impact=READY`

If a manifest references a dependency type for which no resolver is registered:

- the response includes `unsupported-dependency-type`
- the blocker is attached with `impact=READY`

This makes incomplete wiring visible instead of silently treating an action as operationally ready.

12.9.9. Example contributor

At the framework level, the built-in manifest contributor registers the enablement API itself with a permission dependency. Application modules then add their own manifests.

A typical application contributor looks like this:

```
@ApplicationScoped
public class MyActionEnablementManifestContributor
    implements ScopedActionRequirementContributor {

    @Override
    public Collection<ScopedActionRequirement> requirements() {
        return List.of(
```

```

        ScopedActionRequirement.builder()
            .scopedAction(ScopedActionRef.builder()
                .area("integration")
                .functionalDomain("guided_integration")
                .action("create")
                .build())
            .displayName("Generate Guided Integration")
            .description("Requires policy, feature rollout, and baseline setup.")
            .dependencies(List.of(
                DependencyCheckRef.builder().type("permission").build(),
                DependencyCheckRef.builder()
                    .type("feature-flag")
                    .refName("guided-integrations")
                    .build(),
                DependencyCheckRef.builder()
                    .type("setting-present")
                    .refName("quantum.integration.shared-storage.root")
                    .build()
            ))
            .build()
    );
}
}

```

This is the mechanism that turns "settings map to capabilities" into something concrete. The requirement manifest records which settings, feature flags, and prerequisites attach to which **area** / **functionalDomain** / **action** tuple.

12.9.10. Relationship to policies, feature flags, and UI actions

Think of the layers this way:

1. **@FunctionalMapping** and **@FunctionalAction** define the canonical scoped action tuple in code.
2. Permission rules answer whether that tuple is authorized.
3. Feature flags answer whether the action is turned on for the current cohort or realm.
4. Scoped action requirements answer what else must be configured or provisioned before the action is actually usable.

This complements, rather than replaces, existing UI action derivation:

- **fillUIActions()** still answers what actions are allowed from the permission perspective
- scoped action enablement answers why an action should still be disabled or annotated in the UI

That makes it a better fit for:

- setup dashboards
- onboarding flows
- context-sensitive "why unavailable?" messaging

- shells or MCP clients that need structured blockers

12.9.11. Enterprise extensions

The framework module is intentionally generic. Enterprise modules can contribute additional dependency resolvers without changing the core contracts.

The current enterprise action-enablement module extends the system with higher-level runtime checks such as:

- `managed-secret-configured`
- `tenant-filesystem-provisioned`
- `standard-mounts-present`
- `workflow-runtime-reachable`

Those resolvers support operational questions such as:

- has the tenant filesystem been provisioned?
- do the standard mounts exist?
- is a required managed secret configured?
- is the workflow runtime reachable?

That deeper operational behavior belongs in the enterprise documentation. The important framework-level point is that enterprise modules plug into the same `ActionDependencyResolver` SPI and return the same `EnablementBlocker` structure.

12.9.12. Guidance

- Keep permission checks canonical. Do not duplicate authorization logic inside custom readiness resolvers.
- Use `feature-flag` dependencies for rollout state, not for durable authorization.
- Use `setting-present` and `entity-exists` only for generic prerequisites. Domain-heavy operational checks belong in application or enterprise resolvers.
- Keep blocker codes stable because UIs and automation may key off them.
- Treat capabilities as documentation and UX groupings over scoped actions, not as a substitute for `area` / `functionalDomain` / `action`.

12.10. Multiple Matching RuleBases

- Ordering: rules are evaluated in ascending priority (lower numbers first).
- Decision: when a rule matches, its effect (ALLOW or DENY) becomes the current decision.
- `finalRule`: if the matching rule has `finalRule: true`, evaluation stops immediately; otherwise, evaluation continues and a later rule may overwrite the decision.
- Default: if no rule matches decisively, the default effect supplied by the caller (typically DENY) is

returned.

12.11. Identity and Role Matching

- Roles-as-identities: the engine evaluates rules for the caller's `userId` and for each effective role by treating each role name as an identity.
- There are no `rolesAny/rolesAll` fields in rules; author separate rules for specific roles as needed.
- Optional `postconditionScript` can inspect attributes (for example, `tenantId`) via `pcontext` and `rcontext` when additional checks are needed.
- Time or plan-based conditions can be implemented inside `postconditionScript` or via feature flags.

12.11.1. How roles are defined for an identity (role sources and resolution)

Quantum composes the effective roles for a request by merging:

- Roles from the identity provider (JWT/`SecurityIdentity`)
- Roles configured on the user record (`CredentialUserIdPassword.roles`)

Source details:

- Identity Provider (JWT): roles commonly arrive via standard claims (for example, `groups`, `roles`, or provider-specific fields). Quarkus maps these into `SecurityIdentity.getRoles()`. In multi-realm setups, the realm in `X-Realm` can scope lookups but does not alter what the JWT asserts.
- Quantum user record: `com.e2eq.framework.model.security.CredentialUserIdPassword` has a `String[] roles` field stored per realm. This can be administered by Quantum to grant platform- or tenant-level roles.

Merge semantics (current implementation):

- Union: the effective role set is the union of JWT roles and `CredentialUserIdPassword.roles`. If either source is empty, the other source defines the set.
- Fallback: when neither source yields roles, the framework defaults to `ANONYMOUS`.
- Where implemented: `SecurityFilter.determinePrincipalContext` builds `PrincipalContext` with the merged roles.

Realm considerations:

- The user record is looked up by subject or `userId` in the active realm (default or `X-Realm`). If a realm override is provided, it is validated with `CredentialUserIdPassword.realmRegex`.
- Roles stored in a user record are realm-specific; JWT roles are whatever the IdP asserts for the token.

Operating models:

- Quantum-managed roles:

- IdP authenticates the user (subject, username). Authorization is primarily driven by roles stored in `CredentialUserIdPassword.roles`.
- Use when you want central, auditable role assignment within Quantum, independent of IdP groups.
- IdP-managed roles:
 - IdP carries authoritative roles/groups in the JWT. Keep `CredentialUserIdPassword.roles` minimal or empty.
 - Use when enterprises require IdP as the source of truth for access groups.
- Hybrid (recommended in many deployments):
 - Effective roles = JWT roles union `CredentialUserIdPassword.roles`.
 - Use JWT for enterprise groups (for example, `DEPT_SALES`, `ORG_ADMIN`) and Quantum roles for app-specific grants (for example, `REPORT_EXPORTER`, `BETA`).
 - This avoids IdP churn for application-local concerns while respecting org policies.

Examples:

- JWT-only:
 - `JWT.groups = [USER, REPORTER]`; user record roles = []
 - Effective roles = `[USER, REPORTER]`
- Quantum-only:
 - `JWT.groups = []`; user record roles = `[USER, ADMIN]`
 - Effective roles = `[USER, ADMIN]`
- Hybrid union:
 - `JWT.groups = [USER]`; user record roles = `[BETA, REPORT_EXPORTER]`
 - Effective roles = `[USER, BETA, REPORT_EXPORTER]`

Guidance and best practices:

- Keep role names stable and environment-agnostic; use realms/permissions to scope where needed.
- Avoid overloading roles for feature rollout; use Feature Flags for rollout and variants, and roles for durable authorization.
- When IdP is authoritative, ensure consistent claim mapping so `SecurityIdentity.getRoles()` contains the expected values; commonly via `groups` claim in JWT.
- Use grant-based permission rules and require the minimal set of roles (`rolesAny/rolesAll`) needed for each capability.

Cross-references:

- User model: `com.e2eq.framework.model.security.CredentialUserIdPassword.roles`
- Context: `com.e2eq.framework.model.securityrules.PrincipalContext.getRoles()`

- Filter logic: `com.e2eq.framework.rest.filters.SecurityFilter.determinePrincipalContext`

Populating `RoleSource` and `roleAssignments`

Quantum exposes structured provenance for each role via:

- `com.e2eq.framework.model.auth.RoleSource` enum with values: `USERGROUP`, `IDP`, `CREDENTIAL`
- `com.e2eq.framework.model.auth.RoleAssignment` record: `role + Set<RoleSource> sources`
- Login response model `com.e2eq.framework.model.auth.AuthProvider.LoginPositiveResponse` now includes a `roleAssignments` field next to plain `roles`.
- `com.e2eq.framework.model.securityrules.SecurityCheckResponse` also includes `roleAssignments`.

Defaults and compatibility:

- If you do nothing, both login and security check responses automatically derive `roleAssignments` from the flat `roles` set and mark each role with `CREDENTIAL`. This preserves backward compatibility.
- To provide accurate provenance, populate `roleAssignments` explicitly where you already know role sources.

JSON example (login success):

```
{
  "userId": "alice",
  "roles": ["user", "admin"],
  "roleAssignments": [
    {"role": "user", "sources": ["idp", "credential"]},
    {"role": "admin", "sources": ["usergroup"]}
  ],
  "accessToken": "...",
  "refreshToken": "...",
  "expirationTime": 1732100000,
  "mongodbUrl": "...",
  "realm": "system"
}
```

AuthProviders: what you need to do

Auth providers (for example, `CustomTokenAuthProvider`) should add provenance at two points:

1. During login (`login(userId, password)`)
2. During token refresh (`refreshTokens(refreshToken)`)

At each point build three role sets and then compute per-role sources:

- IDP roles → from `io.quarkus.security.identity.SecurityIdentity.getRoles()` for the current access/refresh token

- Credential roles → from `com.e2eq.framework.model.security.CredentialUserIdPassword.getRoles()` in the DB
- User group roles → gathered by resolving the user's `UserProfile` and its `UserGroup` memberships

Then build `List<RoleAssignment>` and pass the full constructor of `LoginPositiveResponse`.

Example (inside `CustomTokenAuthProvider.login(...)` after you create `identity` and locate the `credential`):

```
// 1) Collect source role sets
Set<String> idpRoles = (identity != null) ? new java.util.LinkedHashSet<>(identity
.getRoles()) : java.util.Set.of();
Set<String> credentialRoles = new java.util.LinkedHashSet<>(java.util.Arrays.asList
(credential.getRoles()));
Set<String> userGroupRoles = new java.util.LinkedHashSet<>();
try {
    var userProfileOpt = userProfileRepo.getBySubject(credential.getSubject());
    if (userProfileOpt.isPresent()) {
        var groups = userGroupRepo.findByUserProfileRef(userProfileOpt.get()
.createEntityReference());
        if (groups != null) {
            for (com.e2eq.framework.model.security.UserGroup g : groups) {
                if (g != null && g.getRoles() != null) {
                    java.util.Collections.addAll(userGroupRoles, g.getRoles());
                }
            }
        }
    }
} catch (Exception e) {
    io.quarkus.logging.Log.warn("Group role expansion failed; continuing without group
roles", e);
}

// 2) Union of all roles for the response
java.util.Set<String> allRoles = new java.util.LinkedHashSet<>();
allRoles.addAll(idpRoles);
allRoles.addAll(credentialRoles);
allRoles.addAll(userGroupRoles);

// 3) Build per-role source assignments
java.util.List<com.e2eq.framework.model.auth.RoleAssignment> roleAssignments =
allRoles.stream()
    .filter(java.util.Objects::nonNull)
    .map(role -> {
        java.util.EnumSet<com.e2eq.framework.model.auth.RoleSource> src =
            java.util.EnumSet.noneOf(com.e2eq.framework.model.auth.RoleSource.class);
        if (idpRoles.contains(role)) src.add(com.e2eq.framework.model.auth.RoleSource
.IDP);
        if (credentialRoles.contains(role)) src.add(com.e2eq.framework.model.auth
.RoleSource.CREDENTIAL);
```

```

        if (userGroupRoles.contains(role)) src.add(com.e2eq.framework.model.auth
.RoleSource.USERGROUP);
        return new com.e2eq.framework.model.auth.RoleAssignment(role, src);
    })
    .toList();

// 4) Use the full constructor that accepts roleAssignments
return new com.e2eq.framework.model.auth.AuthProvider.LoginResponse(
    true,
    new com.e2eq.framework.model.auth.AuthProvider.LoginPositiveResponse(
        userId,
        identity,
        allRoles,           // Set<String>
        roleAssignments,    // List<RoleAssignment>
        accessToken,
        refreshToken,
        expirationTime,
        mongoDbUrl,
        realm
    )
);

```

Notes and tips:

- You may centralize this logic in `IdentityRoleResolver` by adding a method that returns `Map<String, EnumSet<RoleSource>>` (role → sources) and reuse it across login and `/check`.
- If you cannot resolve a user record (for example, token-only identities), populate only `IDP` sources.
- Avoid adding synthetic roles like `ANONYMOUS` to `roleAssignments`—the structure should cover actual grants.
- Consider caching group-derived roles per user/request if lookups are expensive.

Security Check responses: enriching `roleAssignments`

`SecurityCheckResponse` defaults to marking all roles as `CREDENTIAL`. To provide accurate provenance during permission checks:

1. Near `PermissionResource.check(...)`, where roles are resolved, compute the same three source sets as above.
2. Build `List<RoleAssignment>` from the final role set used for `PrincipalContext`.
3. After calling the rule engine, overwrite the response's assignments:

```

SecurityCheckResponse resp = ruleContext.checkRules(pc, rc);
resp.setRoleAssignments(roleAssignments);
return Response.ok(resp).build();

```

This keeps the default behavior (no breaking changes) while enabling clients to see precisely how

each role was granted.

12.12. Example Scenarios

1. Public catalog browsing

- Request: GET /Catalog/Products/VIEW?search=widgets
- Identity: anonymous or role USER
- Rules:
 - allow-public-reads (priority 100) ALLOW + readScope orgRefName=PUBLIC
- Outcome: ALLOW; repository applies DataDomain filter orgRefName=PUBLIC

2. Tenant-scoped shipment update

- Request: PUT /Collaboration/Shipments/UPDATE
- Headers: x-tenant-id=T1
- Body: { dataDomain: { tenantId: "T1" }, ... }
- Identity: user in tenant T1 with roles [USER]
- Rules:
 - allow-collab-update (priority 300) requires body.dataDomain.tenantId == identity.tenantId and rolesAny USER, ADMIN ⇒ ALLOW
- Outcome: ALLOW; Rule contributes writeScope tenantId=T1

3. Cross-tenant admin read with higher priority

- Request: GET /api/partners
- Identity: role ADMIN (super-admin)
- Rules:
 - admin-override (priority 50) ALLOW
 - default-tenant-read (priority 600) ALLOW with tenant filter
- Outcome: admin-override wins due to higher precedence (lower number), allowing broader read

4. Conflicting ALLOW and DENY at same priority

- Two rules match with priority 200: one ALLOW, one DENY
- Resolution: DENY wins unless merge strategy configured to handle explicitly; recommended to avoid same-priority conflicts by policy.

12.13. Operational Tips

- Author specific DENY rules with low numbers to prevent accidental exposure.
- Author SecurityURI header (area/domain/action) as narrowly as needed for sensitive domains.
- Prefer SecurityURI body fields and postconditionScript to refine matches without over-broad

area/domain/action patterns.

- Log matched rule names and applied scopes for auditability.

12.14. Field-Level Policy (Property Exclusions)

Row filters (`andFilterString` / `orFilterString`) control **which documents** a principal can see. Field-level policy controls **which fields of those documents** — the common "the carrier may see the shipment but not the price" requirement.

12.14.1. Declaring exclusions on a rule

`Rule.excludedFields` carries dotted property paths. The exclusions apply to any principal/resource the rule matches, composing with the same matching algorithm as row filters:

```
Rule.Builder()
  .withName("carrier-fulfillment-view")
  .withSecurityURI(uri)                // who/what this rule matches
  .withEffect(RuleEffect.ALLOW)
  .withAndFilterString("hasEdge('assignedTo', ${principal.orgRefName})")
  .withExcludedFields(List.of("pricing.unitPrice", "pricing.margin"))
  .build();
```

Semantics are **deny-wins**: the union of `excludedFields` across **all** matched ALLOW rules applies. A field excluded by any applicable rule is excluded, even if another matched rule does not mention it. Paths that do not exist on a queried model are ignored (nothing to protect). Excluded fields are **omitted** from results, not nulled-and-present — clients should treat absence under an active field policy as "restricted", and the REST layer signals this with the `X-Field-Policy` response header listing the applied exclusions.

12.14.2. Layered enforcement — one policy source, every egress guarded

The resolved exclusion set comes from one place (`RuleContext.getExcludedFieldPaths`) and is enforced wherever data leaves the platform. The query language is a multi-datasource contract, so enforcement is **per egress**, not per database:

Egress	Mechanism	Notes
MongoDB reads	Projection injection in <code>MorphiaRepo.buildFindOptions</code>	Excluded paths never leave the database. Caller INCLUDE projections have policy paths dropped (exclusion wins); caller EXCLUDE projections are unioned with policy paths.
MongoDB updates	Preserve-hidden-fields gate in <code>MorphiaRepo.merge</code>	A principal whose policy hides a field cannot blind-overwrite it: the stored value is restored (via <code>FieldPolicyEnforcer.copyPath</code>) before persisting.

Egress	Mechanism	Notes
In-memory / hierarchical / custom datasources	<code>FieldPolicyEnforcer.mask(result, paths)</code> at result materialization	For non-Mongo backends this is the datastore-level enforcement. Implementing the query contract against a new datasource requires wiring the enforcer — it is part of the adapter contract, exactly like honoring row-filter strings.
REST serialization	<code>FieldPolicyResponseInterceptor</code> (WriterInterceptor)	Defense-in-depth fed by the same resolved set: catches computed getters, reference-expanded fields, and responses assembled outside a secured repo. Adds the <code>X-Field-Policy</code> header when masking occurred.



A serializer-only approach (e.g. Jackson `@JsonView`) was considered and rejected as the primary mechanism: views are compile-time classes while Quantum policies are tenant-authorable runtime data, and serializer enforcement guards only the REST door while the reasoner, MCP surface, exports, and events read through repositories. The serializer layer is kept — as the last lock, driven by the same rule-resolved path set.

12.14.3. Limitations (v1)

- Wildcard paths (`pricing.*`) are not yet supported — list explicit paths.
- The write gate applies on `merge` (partial update); full-document `save` of a **new** entity is not field-filtered (the creator authored the values).
- Field exclusions are resolved per (principal, resource) pair, like row filters; per-**instance** visibility derives from relationship row filters (`hasEdge`), not from per-document field lists.

12.15. How UIActions and DefaultUIActions are calculated

When the server returns a collection of entities (for example, `userProfiles`), each entity may expose two action lists:

- **DefaultUIActions**: the full set of actions that conceptually apply to this type of entity (e.g., `CREATE`, `UPDATE`, `VIEW`, `DELETE`, `ARCHIVE`). Think of this as the “menu template” for the type.
- **UIActions**: the subset of actions the current user is actually permitted to perform on that specific entity instance right now.

Why they can differ per entity:

- Entity attributes: state or flags (e.g., `archived`, `soft-deleted`, `immutable`) can remove or alter available actions at instance level.

- Permission rule base: evaluated against the current request, identity, and context to allow or deny actions.
- DataDomain membership: tenant/org/owner scoping can further restrict actions if the identity is outside the entity's domain.

How the server computes them:

1. Start with a default action template for the entity type (DefaultUIActions).
2. Apply simple state-based adjustments (for example, suppress CREATE on already-persisted instances).
3. Evaluate the permission rules with the current identity and context:
 - Consider roles, functional area/domain, action intent, SecurityURI body fields, and any rule-contributed scopes.
 - Resolve DataDomain constraints to ensure the identity is permitted to act within the entity's domain.
4. Produce UIActions as the allowed subset for that entity instance.
5. Return both lists with each entity in collection responses.

How the client should use the two lists:

- Render the full DefaultUIActions as the visible set of possible actions (icons, buttons, menus) so the UI stays consistent.
- Enable only those actions present in UIActions; gray out or disable the remainder to signal capability but lack of current permission.
- This approach avoids flicker and keeps affordances discoverable while remaining truthful to the user's current authorization.

Example:

- You fetch 25 userProfiles.
- DefaultUIActions for the type = [CREATE, VIEW, UPDATE, DELETE, ARCHIVE].
- For a specific profile A (owned by your tenant), UIActions may be [VIEW, UPDATE] based on your roles and domain.
- For another profile B (in a different tenant), UIActions may be [VIEW] only.
- The UI renders the same controls for both A and B, but only enables the actions present in each item's UIActions list.

Operational considerations:

- Keep action names stable and documented so front-ends can map to icons and tooltips consistently.
- Prefer small, composable rules that evaluate action permissions explicitly by functional area/domain to avoid surprises.
- Consider server-side caching of action evaluations for list views to reduce latency, respecting

identity and scope.

12.16. How This Integrates End-to-End

- BaseResource extracts identity and headers to construct DomainContext.
- Rule evaluation uses SecurityURI (header + body) matching with optional postconditionScript and identity/userProfile to reach a decision and derive scope filters.
- Repositories (e.g., MorphiaRepo) apply the filters to queries and updates, ensuring DataDomain-respecting access.

12.17. Administering Policies via REST (PolicyResource)

The PolicyResource exposes CRUD-style REST APIs for creating and managing policies (rule bases) that drive authorization decisions. Each Policy targets a principalId (either a specific userId or a role name) and contains an ordered list of Rule objects. Rules match requests using SecurityURIHeader and SecurityURIBody and then contribute an effect (ALLOW/DENY) and optional repository filters.

- Base path: /security/permission/policies
- Auth: Bearer JWT (see Authentication); resource methods are guarded by @RolesAllowed("user", "admin") at the BaseResource level and your own realm/role policies.
- Multi-realm: pass X-Realm header to operate within a specific realm; otherwise the default realm is used.

12.17.1. Model shape (Policy)

A Policy extends FullBaseModel and includes: - id, refName, displayName, dataDomain, archived/expired flags (inherited) - principalId: userId or role name that this policy attaches to - description: human-readable summary - rules: array of Rule entries

Rule fields (key ones):

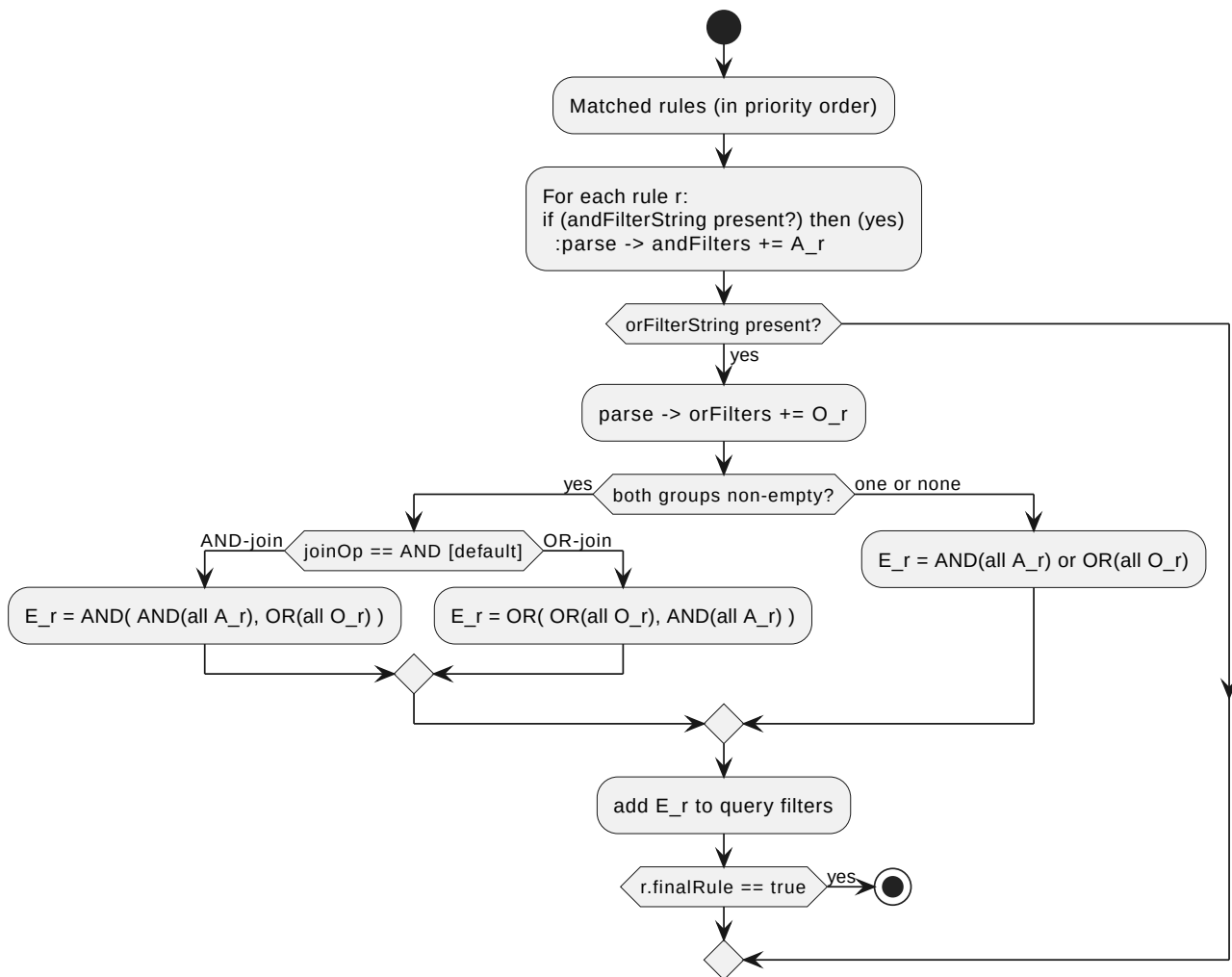
- name, description
- securityURI.header: identity, area, functionalDomain, action (supports wildcard "**")
- securityURI.body: realm, orgRefName, accountNumber, tenantId, ownerId, dataSegment, resourceId (supports wildcard "**")
- effect: ALLOW or DENY
- priority: integer; lower numbers evaluated first
- finalRule: boolean; stop evaluating when this rule applies
- andFilterString / orFilterString: ANTLR filter DSL snippets injected into repository queries (see Query Language section)
- joinOp: how to combine the andFilterString group with the orFilterString group when both are

present; defaults to AND

12.17.2. How rule-contributed filters work (andFilterString, orFilterString, joinOp)

When an ALLOW rule matches, it can contribute repository-level filters that restrict which documents are visible or mutable. The fields are:

- andFilterString: a filter expression added to the AND group.
- orFilterString: a filter expression added to the OR group.
- joinOp: when both groups are present, specifies how to join them. Values: AND or OR. Default: AND.



Composition algorithm (implemented in `RuleContext.getFilters`):

1. Collect all matched rules (in priority order; skipping NOT_APPLICABLE script results). For each rule:
 - If `andFilterString` is set, parse it into a Morphia Filter and add to the and-group.
 - If `orFilterString` is set, parse it into a Morphia Filter and add to the or-group.
2. If both groups are non-empty for the current rule:

- If joinOp == AND (default): effective = AND(AND(all and-group), OR(all or-group)).
 - If joinOp == OR: effective = OR(OR(all or-group), AND(all and-group)).
3. If only one group is non-empty:
 - Use AND(all and-group) or OR(all or-group) as the effective filter.
 4. Append the effective filter(s) to the query's filter list. If the rule has finalRule: true, stop accumulating more filters.
 5. Deduplicate filters by string form before returning.

Examples:

- AND only

```
- name: tenant-scope
  securityURI:
    header: { identity: USER, area: sales, functionalDomain: order, action: view }
    body:   { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
            '*', dataSegment: '*', resourceId: '*' }
    andFilterString: "dataDomain.tenantId:${pcontext.dataDomain.tenantId}"
    effect: ALLOW
    priority: 200
```

Resulting Morphia: AND(eq("dataDomain.tenantId", <caller-tenant>))

- OR only

```
- name: visibility-by-segment
  securityURI:
    header: { identity: USER, area: sales, functionalDomain: order, action: view }
    body:   { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
            '*', dataSegment: '*', resourceId: '*' }
    orFilterString: "dataDomain.dataSegment:^(PUBLIC|INTERNAL)"
    effect: ALLOW
    priority: 210
```

Resulting Morphia: OR(in("dataDomain.dataSegment", [PUBLIC, INTERNAL]))

- AND + OR with joinOp: AND (default)

```
- name: own-or-public
  securityURI:
    header: { identity: USER, area: security, functionalDomain: userProfile, action:
view }
    body:   { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
            '*', dataSegment: '*', resourceId: '*' }
    andFilterString: "dataDomain.ownerId:${principalId}"
    orFilterString: "dataDomain.dataSegment:^(PUBLIC)"
```

```
joinOp: AND
effect: ALLOW
priority: 220
```

Effective: AND(eq(ownerId, principalId), OR(eq(dataSegment, 'PUBLIC')))

- AND + OR with joinOp: OR

```
- name: owner-or-segment
  securityURI:
    header: { identity: USER, area: files, functionalDomain: document, action: view }
    body: { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
'*', dataSegment: '*', resourceId: '*' }
    andFilterString: "dataDomain.ownerId:${principalId}"
    orFilterString: "tags:^[shared]"
    joinOp: OR
    effect: ALLOW
    priority: 230
```

Effective: OR(eq(ownerId, principalId), AND(eq(tags, 'shared')))

Placeholders and variables:

The following standard variables are available in andFilterString/orFilterString:

Variable	Source	Description
<code>\${principalId}</code>	PrincipalContext.userId	The authenticated user's ID
<code>\${pAccountId}</code>	PrincipalContext.dataDomain.accountNum	The principal's account number
<code>\${pTenantId}</code>	PrincipalContext.dataDomain.tenantId	The principal's tenant ID
<code>\${ownerId}</code>	PrincipalContext.dataDomain.ownerId	The data owner ID
<code>\${orgRefName}</code>	PrincipalContext.dataDomain.orgRefName	The organization reference name
<code>\${defaultRealm}</code>	PrincipalContext.defaultRealm	The current realm (changes with X-Realm override)
<code>\${resourceId}</code>	ResourceContext.resourceId	The target resource ID
<code>\${action}</code>	ResourceContext.action	The action being performed
<code>\${functionalDomain}</code>	ResourceContext.functionalDomain	The functional domain
<code>\${area}</code>	ResourceContext.area	The functional area
<code>\${dcTenantId}</code>	DomainContext.tenantId	Tenant ID from the current domain context

Variable	Source	Description
<code>\${dcOrgRefName}</code>	DomainContext.orgRefName	Organization reference name from domain context
<code>\${dcAccountId}</code>	DomainContext.accountId	Account ID from domain context
<code>\${dcDataSegment}</code>	DomainContext.dataSegment	Data segment number from domain context
<code>\${facet.<key>}</code>	DataDomainPolicyEntry.facetFilters	Tenant-configured facet filters (e.g. <code>\${facet.allowedTiers}</code>)
<code>\${coord.<key>}</code>	DataDomainResolution coordinates	Custom coordinate variables (e.g. <code>\${coord.salesZone}</code>)
<code>\${principal.<attr>}</code>	PrincipalContext attributes	Authenticated principal attributes and claims
<code>\${dd.<attr>}</code>	DataDomain fields	Target DataDomain partition coordinates



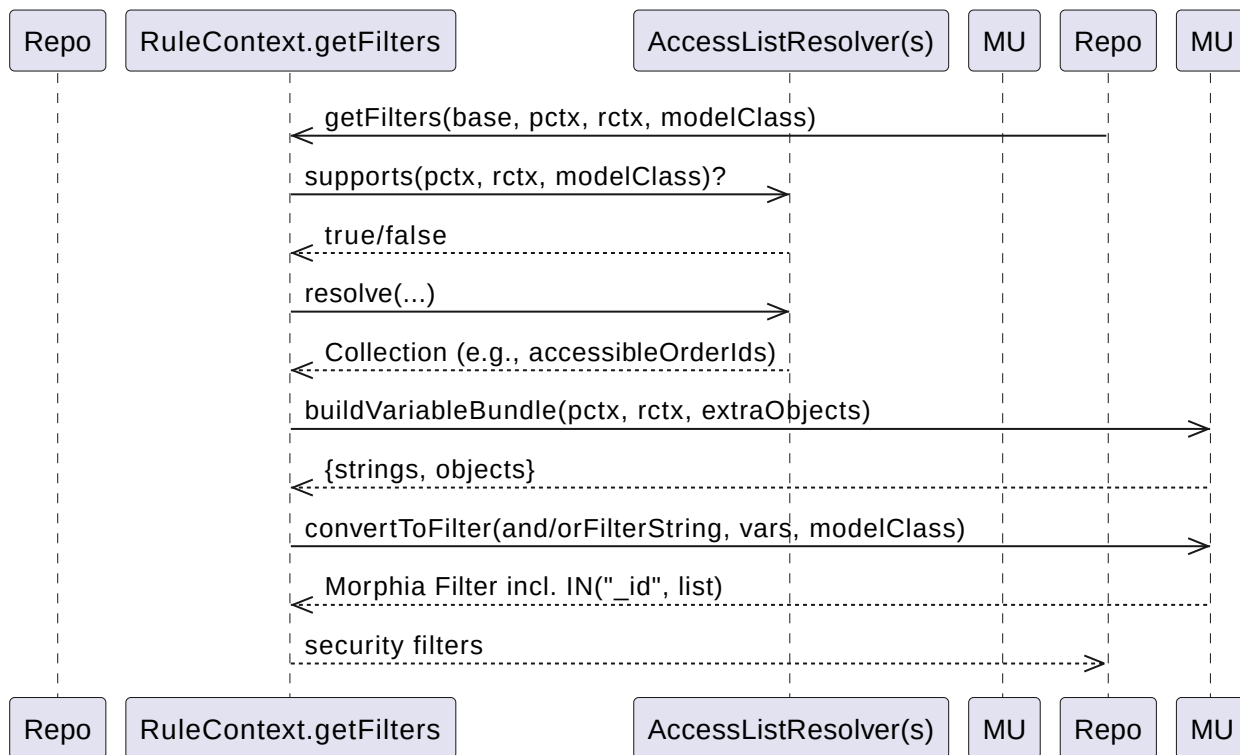
Dotted Variable Identifiers & Single-Variable IN: Variables with dotted paths (`${facet.tier}`, `${coord.salesZone}`) resolve in a strict hierarchy. When evaluated with the list membership operator (`:^`), single variables resolving to a Java Collection (e.g. `allowedTiers = ["FLAGSHIP", "REGIONAL"]`) can be passed without brackets (`tier:^${facet.allowedTiers}`), automatically unpacking into native MongoDB `$in` clauses.

X-Realm integration: When `X-Realm` is used, `${defaultRealm}` automatically reflects the target realm. This enables permission filters to dynamically scope data based on the active realm:

```
- name: scope-by-realm
  securityURI:
    header: { identity: USER, area: '*', functionalDomain: '*', action: view }
    body:   { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
'*', dataSegment: '*', resourceId: '*' }
    andFilterString: "dataDomain.tenantId:${defaultRealm}"
    effect: ALLOW
    priority: 200
```

- The filter DSL is described in the Query Language guide; it maps to Morphia `dev.morphia.query.filters.Filters` under the hood via `MorphiaUtils.convertToFilter`.

12.17.3. AccessListResolvers (SPI) for list-based access



AccessListResolvers let you plug in computed collections (IDs, codes, emails, etc.) at request time and reference them from `andFilterString/orFilterString`. This is ideal for ACL-style list checks or integrating with external systems that decide which resources a user may access.

How it works (as implemented):

- SPI: `com.e2eq.framework.securityrules.AccessListResolver`
- `key()`: the variable name published to the filter variable bundle (e.g., `"accessibleCustomerIds"`).
- `supports(pctx, rctx, modelClass)`: return true when this resolver applies for the current request and repository model type.
- `resolve(pctx, rctx, modelClass)`: return a `Collection<?>` which will be available as `${<key>}` in filter strings.
- `RuleContext.getFilters` discovers all `AccessListResolver` beans via CDI, calls `supports(...)`, and for those that apply it puts `key() → resolve(...)` into the "extraObjects" map. `MorphiaUtils.buildVariableBundle` merges these into the variable set used by `convertToFilter`.
- In your filter DSL, use:
 - IN with resolver-provided lists: `field: ^${key}`
 - Equality with resolver-provided scalars: `field: ${key}`
 - You can also reference nested principals: `${pcontext.dataDomain.tenantId}`, `${principalId}`, etc.

Examples:

- Restrict Orders to the set of ids returned by a resolver

```
- name: orders-by-acl
  securityURI:
```

```

header: { identity: USER, area: sales, functionalDomain: order, action: view }
body:   { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
'*', dataSegment: '*', resourceId: '*' }
andFilterString: "_id:^(accessibleOrderIds)"
effect: ALLOW
priority: 200

```

- Use resolver that returns customer codes and OR with a public segment

```

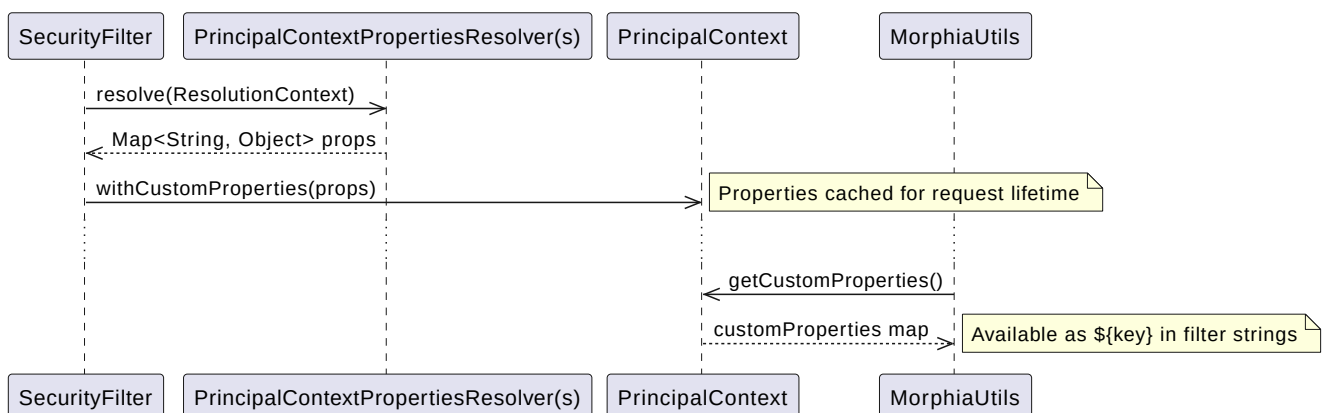
- name: customer-code-or-public
securityURI:
  header: { identity: USER, area: crm, functionalDomain: customer, action: view }
  body:   { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
'*', dataSegment: '*', resourceId: '*' }
andFilterString: "code:^(visibleCustomerCodes)"
orFilterString:  "dataDomain.dataSegment:[PUBLIC]"
joinOp: OR
effect: ALLOW
priority: 210

```

Reference implementation (tests/examples): - RuleContext.getFilters collects resolvers and exposes them to the filter StringSubstitutor via MorphiaUtils.buildVariableBundle. - See also: quantum-framework/src/test/java/com/e2eq/framework/securityrules/TestCustomerAccessResolver.java and TestStringAccessResolver.java for sample resolvers.

Guidance: - Keep keys stable and document them; they form your contract between resolver authors and rule authors. - Always scope resolver queries by tenant/realm from PrincipalContext/ResourceContext. - Return empty collections instead of null. An empty IN list yields no matches, which is safe by default.

12.17.4. PrincipalContextPropertiesResolver (SPI) for custom principal properties



PrincipalContextPropertiesResolver allows applications to enrich PrincipalContext with custom, domain-specific properties during authentication. Unlike AccessListResolver (which is invoked per-query and scoped to specific resources), these properties are resolved once when the principal context is built and are available throughout the request lifecycle.

Use cases:

- **Associate/Sales Rep binding:** Link a UserProfile to an Associate entity and expose `associateId` for filtering
- **Territory assignments:** Expose `accessibleTerritoryIds` based on the user's role assignments
- **Organizational hierarchies:** Expose `accessibleLocationIds` via ontology relationships
- **Custom attributes:** Any domain-specific property needed for security filtering

How it works:

- SPI: `com.e2eq.framework.model.securityrules.PrincipalContextPropertiesResolver`
- `priority()`: Execution order (lower = earlier). Default: 1000.
- `resolve(ResolutionContext)`: Return a `Map<String, Object>` of properties to add to `PrincipalContext`.
- `ResolutionContext` provides access to:
 - `isAnonymous()`, `getPrincipalName()`, `getIdentityRoles()`: Security identity info
 - `getUserId()`, `getSubjectId()`: User identifiers
 - `getRealm()`, `getDataDomain()`, `getDomainContext()`: Realm and domain context
 - `getHeader(name)`: HTTP request headers
 - `getJwtClaim(name)`: JWT token claims
 - `hasCredentials()`, `getCredentialUserId()`, `getCredentialRoles()`: Database credentials
- `SecurityFilter` discovers all resolvers via CDI, invokes them in priority order, and merges results into `PrincipalContext.customProperties`.
- `MorphiaUtils.createStandardVariableMapFrom` includes custom properties, making them available as `${key}` in filter strings.

Example implementation:

```
package com.myapp.security;

import com.e2eq.framework.model.securityrules.PrincipalContextPropertiesResolver;
import com.e2eq.framework.model.securityrules.ResolutionContext;
import com.e2eq.ontology.repo.OntologyEdgeRepo;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.inject.Inject;
import java.util.*;

@ApplicationScoped
public class AssociatePropertiesResolver implements PrincipalContextPropertiesResolver
{

    @Inject
    OntologyEdgeRepo edgeRepo;
```

```

@Override
public int priority() {
    return 100; // Run early
}

@Override
public Map<String, Object> resolve(ResolutionContext context) {
    Map<String, Object> properties = new HashMap<>();

    // Skip for anonymous users
    if (context.isAnonymous()) {
        return properties;
    }

    String userProfileId = context.getSubjectId().orElse(context.getUserId());

    // Find Associate linked to this UserProfile via hasUser relationship
    // Associate.hasUser -> UserProfile (functional/1:1)
    Optional<String> associateId = edgeRepo.singleSrcIdByDst(
        context.getDataDomain(), "hasUser", userProfileId);

    if (associateId.isPresent()) {
        properties.put("associateId", associateId.get());

        // Get territories this associate can access
        Set<String> territoryIds = edgeRepo.dstIdsBySrc(
            context.getDataDomain(), "assignedToTerritory", associateId.get());
        properties.put("accessibleTerritoryIds", territoryIds);

        // Get locations via inferred canSeeLocation edge
        Set<String> locationIds = edgeRepo.dstIdsBySrc(
            context.getDataDomain(), "canSeeLocation", associateId.get());
        properties.put("accessibleLocationIds", locationIds);
    }

    return properties;
}
}

```

Using custom properties in rules:

```

# Filter locations to those the associate can see
- name: associate-location-access
  securityURI:
    header: { identity: sales-rep, area: sales, functionalDomain: location, action:
view }
    body: { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
'*', dataSegment: '*', resourceId: '*' }
    andFilterString: "dataDomain.tenantId:${pTenantId}"
    orFilterString: "_id:^(accessibleLocationIds)"

```

```
effect: ALLOW
priority: 200
```

```
# Filter orders by territory
- name: territory-order-access
  securityURI:
    header: { identity: sales-rep, area: sales, functionalDomain: order, action: view
  }
    body: { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
  '*', dataSegment: '*', resourceId: '*' }
    andFilterString: "dataDomain.tenantId:${pTenantId} AND
territoryId:^[${accessibleTerritoryIds}"
  effect: ALLOW
  priority: 200
```

Accessing properties programmatically:

```
// In application code or postconditionScript
PrincipalContext pctx = SecurityContext.getPrincipalContext().get();

// Get a single property
String associateId = (String) pctx.getCustomProperty("associateId");

// Get with type safety
Set<String> locationIds = pctx.getCustomProperty("accessibleLocationIds", Set.class);

// Get all custom properties
Map<String, Object> allProps = pctx.getCustomProperties();
```

Comparison: PrincipalContextPropertiesResolver vs AccessListResolver

Aspect	PrincipalContextPropertiesResolver	AccessListResolver
When invoked	Once during authentication (eager)	Per query during rule evaluation (lazy)
Context available	SecurityIdentity, JWT claims, HTTP headers, Credentials	PrincipalContext, ResourceContext, ModelClass
Caching	Cached in PrincipalContext for request lifetime	Per-request via RuleContext
Use case	User-level properties constant across all queries	Resource-specific access lists that vary by model/action
Performance	Single invocation per request	Invoked per repository call

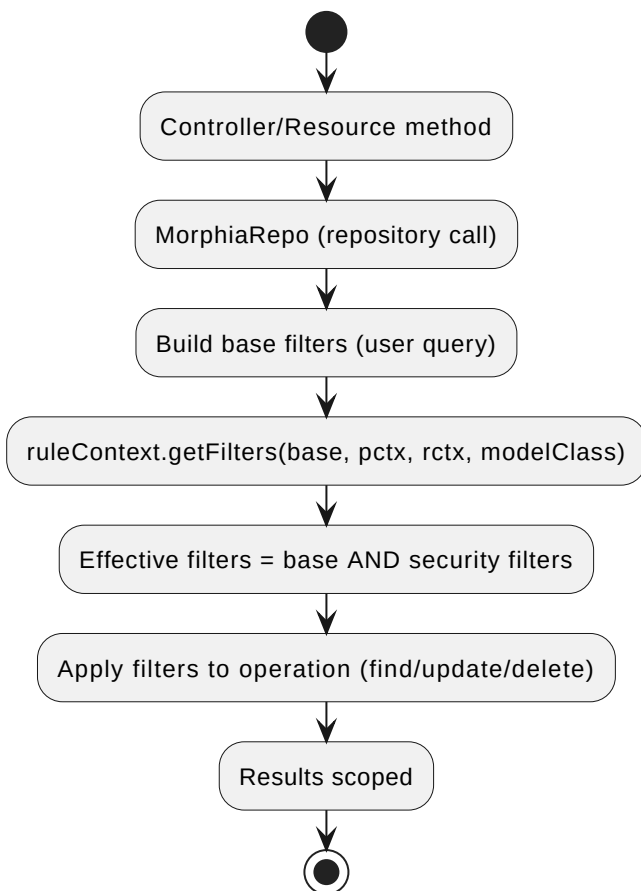
Guidance:

- Use PrincipalContextPropertiesResolver for properties that apply to the user regardless of what

resource they're accessing (e.g., associate ID, territory assignments).

- Use `AccessListResolver` when the access list depends on the specific resource type or action being performed.
- Both mechanisms can coexist and complement each other.
- Return empty maps (not null) from resolvers; resolver failures are logged and skipped, not propagated.
- Properties should not override built-in `PrincipalContext` fields (`userId`, `roles`, etc.).

Where filters are applied (MorphiaRepo methods):



- `getList` / `getListByQuery` / `find`: `RuleContext.getFilters` augments the base filters with the effective security filters before executing the query.
- `count`: same as read paths, ensuring counts reflect scoped visibility.
- `save (create)`: rules typically do not inject filters for inserts; writes are validated by separate preconditions or `postconditionScript`. If your policy encodes write-scope, author `UPDATE/DELETE` rules to guard modifications rather than `CREATE`, unless your domain enforces ownership/tenant on insert.
- `update` / `merge` / `set` / `bulk set`: the security filters are ANDed into the target selection so only documents visible under the policy are affected.
- `delete (by id or by query)`: security filters are applied to the selection to prevent deleting outside the allowed scope.

Implementation reference:

```
// Repositories call into RuleContext to augment filters
List<Filter> filters = ruleContext.getFilters(baseFilters,
    SecurityContext.getPrincipalContext().get(),
    SecurityContext.getResourceContext().get(),
    getPersistentClass());
Query<T> query = datastore.find(getPersistentClass()).filter(filters.toArray(new
Filter[0]));
```

Notes:

- Filters are applied regardless of whether the match decision was ALLOW or DENY; only ALLOW rules contribute filters. DENY rules decide the outcome but do not add filters.
- finalRule: true stops evaluating later rules for both decision and filter contribution.
- If no ALLOW rules match, repositories may still execute with caller-provided filters, but upstream permission checks should have DENIED the action; by convention, most endpoints call checkRules and short-circuit DENY before hitting the database.

Example impact per MorphiaRepo method:

- save: no security filters are injected into the insert operation; enforce ownership/tenant fields via model validation and/or postconditionScript.
- find/findById: adds security filters; if the requested id is outside scope, the result is empty.
- getList/getListByQuery: adds security filters to user-supplied query; scope cannot be broadened by the client.
- update/merge: AND the security filters into the update selector; records outside scope are unaffected.
- delete: AND the security filters into the delete selector; out-of-scope records are not removed.

Example payload:

```
{
  "refName": "defaultUserPolicy",
  "displayName": "Default user policy",
  "principalId": "user",
  "description": "Users can act on their own data; deny dangerous ops in security area",
  "rules": [
    {
      "name": "view-own-resources",
      "description": "Limit reads to owner and default data segment",
      "securityURI": {
        "header": { "identity": "user", "area": "*", "functionalDomain": "*", "action": "*" },
        "body": { "realm": "*", "orgRefName": "*", "accountNumber": "*", "tenantId": "*", "ownerId": "*", "dataSegment": "*", "resourceId": "*" }
      },
    },
  ],
}
```

```

    "andFilterString": "
dataDomain.ownerId:${principalId}&&dataDomain.dataSegment:#0",
    "effect": "ALLOW",
    "priority": 300,
    "finalRule": false
  },
  {
    "name": "deny-delete-in-security",
    "securityURI": {
      "header": { "identity": "user", "area": "security", "functionalDomain": "*",
"action": "delete" },
      "body": { "realm": "*", "orgRefName": "*", "accountNumber": "*", "tenantId":
"*, "ownerId": "*", "dataSegment": "*", "resourceId": "*" }
    },
    "effect": "DENY",
    "priority": 100,
    "finalRule": true
  }
]
}

```

12.17.5. Endpoints

All endpoints are relative to `/security/permission/policies`. These are inherited from `BaseResource` and are consistent across entity resources.

- GET `/list`
 - Query params: skip, limit, filter, sort, projection
 - Returns a `Collection<Policy>` with paging metadata; respects X-Realm.
- GET `/id/{id}` and GET `/id?id=...`
 - Fetch a single Policy by id.
- GET `/refName/{refName}` and GET `/refName?refName=...`
 - Fetch a single Policy by refName.
- GET `/count?filter=...`
 - Returns a `CounterResponse` with total matching entities.
- GET `/schema`
 - Returns JSON Schema for Policy.
- POST `/`
 - Create or upsert a Policy (if id is present and matches an existing entity in the selected realm, it is updated).
- PUT `/set?id=...&pairs=field:value`
 - Targeted field updates by id. pairs is a repeated query parameter specifying field/value pairs.

- PUT /bulk/setByQuery?filter=...&pairs=...
 - Bulk updates by query. Note: ignoreRules=true is not supported on this endpoint.
- PUT /bulk/setByIds
 - Bulk updates by list of ids posted in the request body.
- PUT /bulk/setByRefAndDomain
 - Bulk updates by a list of (refName, dataDomain) pairs in the request body.
- DELETE /id/{id} (or /id?id=...)
 - Delete by id.
- DELETE /refName/{refName} (or /refName?refName=...)
 - Delete by refName.
- CSV import/export endpoints for bulk operations:
 - GET /csv – export as CSV (field selection, encoding, etc.)
 - POST /csv – import CSV into Policies
 - POST /csv/session – analyze CSV and create an import session (preview)
 - POST /csv/session/{sessionId}/commit – commit a previously analyzed session
 - DELETE /csv/session/{sessionId} – cancel a session
 - GET /csv/session/{sessionId}/rows – page through analyzed rows
- Index management (admin only):
 - POST /indexes/ensureIndexes/{realm}?collectionName=policy

Headers:

- Authorization: Bearer <token>
- X-Realm: realm identifier (optional but recommended in multi-tenant deployments)

Filtering and sorting:

- filter uses the ANTLR-based DSL (see REST CRUD > Query Language)
- sort uses comma-separated fields with optional +/- prefix; projection accepts a comma-separated field list

12.17.6. Examples

- Create or update a Policy

```
curl -X POST \
  -H "Authorization: Bearer $JWT" \
  -H "Content-Type: application/json" \
  -H "X-Realm: system-com" \
  https://host/api/security/permission/policies \
```

```
-d @policy.json
```

- List policies for principalId=user

```
curl -H "Authorization: Bearer $JWT" \  
      -H "X-Realm: system-com" \  
  
"https://host/api/security/permission/policies/list?filter=principalId:'user'&sort=+refName&limit=50"
```

- Delete a policy by refName

```
curl -X DELETE \  
      -H "Authorization: Bearer $JWT" \  
      -H "X-Realm: system-com" \  
      "https://host/api/security/permission/policies/refName/defaultUserPolicy"
```

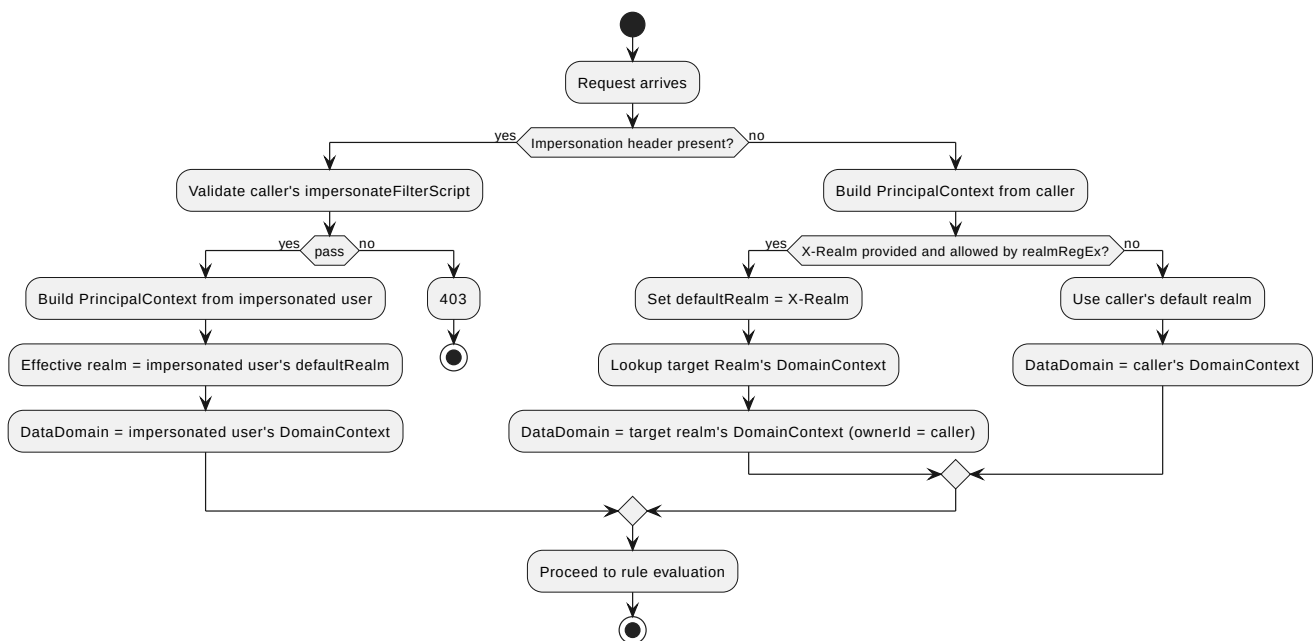
12.17.7. How changes affect rule bases and enforcement

- Persistence vs. in-memory rules:
- PolicyResource updates the persistent store of policies (one policy per principalId or role with a list of rules).
- RuleContext is the in-memory evaluator used by repositories and resources to enforce permissions. It matches SecurityURIHeader/Body, orders rules by priority, and applies effects and filters.
- Making persisted policy changes effective:
- On startup, migrations (see InitializeDatabase and AddAnonymousSecurityRules) typically seed default policies and/or programmatically add rules to RuleContext.
- When you modify policies via REST, you have two options to apply them at runtime:
 1. Implement a reload step that reads policies from PolicyRepo and rehydrates RuleContext (for example, RuleContext.clear(); then add rules built from current policies).
 2. Restart the service or trigger whatever policy-loader your application uses at boot.
- Tip: If you maintain a background watcher or admin endpoint to refresh policies, keep it tenant/realm-aware and idempotent.
- Evaluation semantics (recap):
- Rules are sorted by ascending priority; the first decisive rule sets the outcome. finalRule=true stops further processing.
- andFilterString/orFilterString contribute repository filters through RuleContext.getFilters(), constraining result sets and write scopes.
- principalId can be a concrete userId or a role; RuleContext considers both the principal and all associated roles.

- Safe rollout:
- Create new policies with a higher numeric priority (lower precedence) first, test with GET /schema and dry-run queries.
- Use realm scoping via X-Realm to stage changes in a non-production realm.
- Prefer DENY with low priority numbers for critical protections.

See also: - Permissions: Matching Algorithm, Priorities, and Multiple Matching RuleBases (sections above) - REST CRUD: Query Language and generic endpoint behaviors

12.18. Realm override (X-Realm) and Impersonation (X-Impersonate)



This section explains how to use the request headers X-Realm, X-Impersonate-, and X-Acting-On-Behalf-Of- alongside permission rule bases. These headers influence which realm (database) a request operates against and, in the case of impersonation, which identity's roles are evaluated by the rule engine.

12.18.1. What they do (at a glance)

- X-Realm: Overrides the target realm (MongoDB database) used by repositories for this request. Your own identity and roles remain the same. **The DataDomain is updated to the target realm's default DomainContext** (while preserving your userId as the ownerId). This ensures that data created via X-Realm is properly scoped to the target realm's tenant/org structure.
- X-Impersonate-Subject or X-Impersonate-UserId: Causes the request to run as another identity. The effective permissions become those of the impersonated identity (potentially more or less than your own). This is analogous to sudo on Unix or to "simulate a user/role" for troubleshooting.
- X-Acting-On-Behalf-Of-Subject or X-Acting-On-Behalf-Of-UserId: Records that the caller is acting on behalf of another party (for audit/tracking purposes). Unlike impersonation, this does NOT

change permissions—the caller retains their own identity and roles.

Only one of X-Impersonate-Subject or X-Impersonate-UserId may be supplied per request. Supplying both results in a 400/IllegalArgumentException.

12.18.2. How the headers integrate with permission evaluation

- Rule matching and effects (ALLOW/DENY) still follow the standard algorithm described earlier.
- With X-Realm (no impersonation):
 - The PrincipalContext.defaultRealm is set to the header value (after validation), and repositories operate in that realm.
 - **The DataDomain is updated** from the target realm's DomainContext (org, tenant, account, dataSegment), while preserving your userId as the ownerId.
 - Your own roles and identity remain intact; the rule base is evaluated for your identity and roles but with the target realm's data context.
 - Permission filter strings can use `${defaultRealm}` to dynamically scope queries based on the active realm.
- With impersonation:
 - The PrincipalContext is rebuilt from the impersonated user's credential. The effective roles used by the rule engine include the impersonated user's roles; the platform also merges in the caller's security roles from Quarkus SecurityIdentity. This means permissions can be a superset; design policy rules accordingly.
 - The effective realm for the request is set to the impersonated user's default realm (not the X-Realm header). If you passed X-Realm, it is still validated (see below) but not used to override the impersonated default realm in the current implementation.
 - The DataDomain is set from the impersonated user's DomainContext.
- With Acting-On-Behalf-Of:
 - The caller retains their own identity, roles, realm, and DataDomain.
 - The X-Acting-On-Behalf-Of-Subject or X-Acting-On-Behalf-Of-UserId is recorded in the PrincipalContext for audit purposes.
 - This is useful for service-to-service calls or when a support agent needs to perform actions that should be attributed to a customer.

12.18.3. Required credential configuration (CredentialUserIdPassword)

Two fields on CredentialUserIdPassword govern whether a user may use these headers:

- realmRegex (for X-Realm):
 - A wildcard pattern ("*" matches any sequence; case-insensitive) listing the realms a user is allowed to target with X-Realm.
 - If X-Realm is present but realmRegex is null/blank or does not match the requested realm, the server returns 403 Forbidden.

- Examples:
- "*" → allow any realm
- "acme-*" → allow realms that start with acme-
- "dev|stage|prod" is not supported as-is; use wildcards like "dev*" and "stage*" or a combined pattern like "(dev|stage|prod)" only if you store a true regex. The current validator replaces "with "." and matches case-insensitively.
- impersonateFilterScript (for X-Impersonate-*):
- A JavaScript snippet executed by the server (GraalVM) that must return a boolean. It receives three variables: username (the caller's subject), userId (caller's userId), and realm (the requested realm or current DB name).
- If the script evaluates to false, the server returns 403 Forbidden for impersonation.
- If the script is missing (null) and you attempt impersonation, the server rejects the request with 400/IllegalArgumentException.

Example impersonation script (allow only company admins to impersonate in dev realms):

```
// username = caller's subject, userId = caller's userId, realm = requested realm (or
current)
(username.endsWith('@acme.com') && realm.startsWith('dev-'))
```

Tip: Manage these two fields via your auth provider's admin APIs or directly through CredentialRepo in controlled environments.

12.18.4. End-to-end behavior from SecurityFilter (reference)

The SecurityFilter constructs the PrincipalContext/ResourceContext before rule evaluation:

- X-Realm is read and, if present, validated against the caller's credential.realmRegEx.
- If impersonation headers are present:
 - The caller's credential.impersonateFilterScript is executed. If it returns true, the impersonated user's credential is loaded and used to build the PrincipalContext.
 - The final PrincipalContext carries the impersonated user's defaultRealm and roles (merged with the caller's SecurityIdentity roles), and may copy area2RealmOverrides from the impersonated credential.
 - Without impersonation, the PrincipalContext is built from the caller's credential; X-Realm, when valid, sets the defaultRealm for this request.

12.18.5. Practical differences and use cases

- Realm override (X-Realm):
- Who you are does not change; only where you act changes. Your permissions (as determined by policies attached to your identity/roles) are applied against data in the specified realm.
- **DataDomain changes:** The DataDomain is updated from the target realm's DomainContext (tenant, org, account), ensuring new records are created with the correct scoping.
- Use cases:
- Multi-tenant admin tooling that needs to inspect or repair data in customer realms.

- Reporting or backfills where the same service is pointed at different tenant databases per request.
- Creating data in a target realm with proper DataDomain stamping.
- Impersonation (X-Impersonate-*):
 - Who you are (for authorization purposes) changes. You act with the impersonated identity's permissions; depending on your configuration, additional caller roles may be merged.
 - The DataDomain is set from the impersonated user's credential.
- Use cases:
 - Temporary elevation to an admin identity (sudo-like) for break-glass operations.
 - Simulate what a given role/identity can see/do for troubleshooting or customer support.
 - Testing user-specific permission rules without logging in as that user.
- Acting-On-Behalf-Of (X-Acting-On-Behalf-Of-*):
 - Who you are does NOT change; you retain your own identity, roles, realm, and DataDomain.
 - The "acting on behalf of" information is recorded in the PrincipalContext for audit trails.
- Use cases:
 - Service-to-service calls where the downstream service needs to know the original requestor.
 - Support agents performing actions that should be audited as being done "on behalf of" a customer.
- Compliance and audit requirements where the chain of delegation must be recorded.

Caveats: - Never set a permissive impersonateFilterScript in production. Keep it restrictive and auditable. - When using both X-Realm and impersonation in one call, be aware that the effective realm will be the impersonated user's default realm; X-Realm is not applied in the impersonation branch in the current implementation. - realmRegex must be populated for any user who needs realm override; leaving it blank effectively disables X-Realm for that user. - Acting-On-Behalf-Of headers do NOT grant any additional permissions; they are purely for audit purposes.

12.18.6. Examples

- List policies in a different realm using your own identity

```
curl -H "Authorization: Bearer $JWT" \
  -H "X-Realm: acme-prod" \
  "https://host/api/security/permission/policies/list?limit=20&sort=+refName"
```

- Simulate another user by subject while staying in their default realm

```
curl -H "Authorization: Bearer $JWT" \
  -H "X-Impersonate-Subject: 3d8f4e7b-...-idp-subject" \
  "https://host/api/security/permission/policies/list?limit=20"
```

- Attempt impersonation with a realm hint (validated by script; effective realm = impersonated default)

```
curl -H "Authorization: Bearer $JWT" \  
-H "X-Realm: dev-acme" \  
-H "X-Impersonate-UserId: tenant-admin" \  
"https://host/api/security/permission/policies/list?limit=20"
```

Security outcomes in all cases continue to be driven by your rule bases (Policy rules) matched against the effective `PrincipalContext` and `ResourceContext`.

12.19. Data domain assignment on create: `DomainContext` and `DataDomainPolicy`

This section explains how Quantum decides which `dataDomain` is stamped on newly created records, why this decision is necessary in a multi-tenant system, what the default behavior is, and how you can override it globally or per Functional Area / Functional Domain. It also describes the `DataDomainResolver` interface and the default implementation provided by the framework.

12.19.1. The problem this solves (and why it matters)

In a multi-tenant platform you must ensure each new record is written to the correct data partition so later reads/updates can be scoped safely. If the `dataDomain` is wrong or missing, you risk leaking data across tenants or making your own data inaccessible due to mis-scoping.

Historically, Quantum set the `dataDomain` of new entities to match the creator's credential (i.e., the principal's `DomainContext` → `DataDomain`). That default is sensible in many cases, but real systems often need more specific behavior per business area or type. For example: - You may centralize HR records in a single org-level domain regardless of who created them. - Sales invoices for EU customers must live under an EU data segment. - A specific product area might always write into a shared catalog domain separate from the author's tenant.

These needs require a simple, deterministic way to override the default per Functional Area and/or Functional Domain.

12.19.2. Key concepts recap: `DomainContext` and `DataDomain`

`DomainContext` (on credentials/realm)

captures the principal's scoping defaults (realm, org/account/tenant identifiers, data segment). At request time this is materialized into a `DataDomain`.

`DataDomain`

is what gets stamped onto persisted entities and later used by repositories to constrain queries and updates.

If you do nothing, new records inherit the principal's `DataDomain`.

12.19.3. The default policy (do nothing and it works)

Out of the box, Quantum preserves the existing behavior: if no policy is configured, the resolver falls back to the authenticated principal's DataDomain. This guarantees compatibility with existing applications.

Concretely: - ValidationInterceptor checks if an entity being persisted lacks a dataDomain. - If missing, it calls DataDomainResolver.resolveForCreate(area, domain). - The DefaultDataDomainResolver first looks for overrides (credential-attached or global); if none match, it returns the principal's DataDomain from the current SecurityContext.

12.19.4. Policy scopes: principal-attached vs. global

You can define overrides at two levels: - Principal-attached (per credential): attach a DataDomainPolicy to a CredentialUserIdPassword. The SecurityFilter places this policy into the PrincipalContext, so it applies only to records created by that principal. This is useful for VIP service accounts or specific partners. - Global policy: an application-wide DataDomainPolicy provided by GlobalDataDomainPolicyProvider. If present, this applies when the principal has no specific override for the matching area/domain.

Precedence: principal-attached policy wins over global policy; if neither applies, fall back to the principal's credential domain.

12.19.5. The policy map and matching

A DataDomainPolicy is a small map of rules: Map<String, DataDomainPolicyEntry> policyEntries, keyed by "<FunctionalArea>:<FunctionalDomain>" with support for "*" wildcards. The resolver evaluates keys in this order:

1. area:domain (most specific)
2. area:*
3. *:domain
4. : (global catch-all)
5. Fallback to principal's domain if no entry yields a value

Each DataDomainPolicyEntry has a resolutionMode: - FROM_CREDENTIAL (default): use the principal's credential domain (i.e., the historical behavior). - FIXED: use the first DataDomain listed in dataDomains on the entry.

Example policy definitions (illustrative JSON):

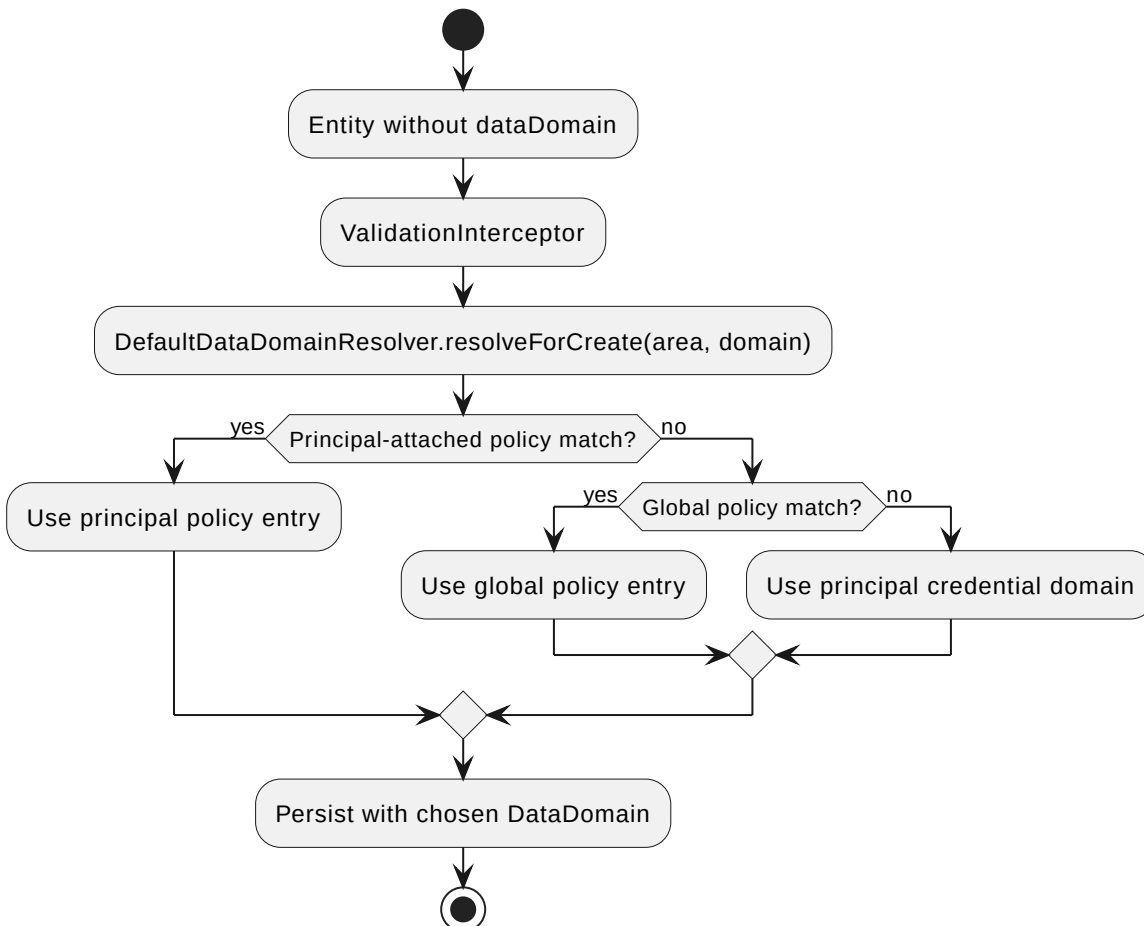
```
{
  "policyEntries": {
    "Sales:Invoice": { "resolutionMode": "FIXED", "dataDomains": [ {"orgRefName":
"ACME", "tenantId": "eu-1", "dataSegment": "INVOICE"} ] },
    "Sales:*":      { "resolutionMode": "FROM_CREDENTIAL" },
    "*:HR":         { "resolutionMode": "FIXED", "dataDomains": [ {"orgRefName":
"GLOBAL", "tenantId": "hr", "dataSegment": "HR"} ] },
```

```

    ".*": { "resolutionMode": "FROM_CREDENTIAL" }
  }
}

```

Behavior of the above: - Sales:Invoice records always go to the fixed EU invoices domain. - Any other Sales:* creation uses the creator's credential domain. - All HR records go to a central HR domain. - Otherwise, default to the creator's domain.



12.19.6. How the resolver works

Interfaces and default implementation:

```

public interface DataDomainResolver {
    DataDomain resolveForCreate(String functionalArea, String functionalDomain);
}

@ApplicationScoped
public class DefaultDataDomainResolver implements DataDomainResolver {
    @Inject GlobalDataDomainPolicyProvider globalPolicyProvider;
    public DataDomain resolveForCreate(String area, String domain) {
        DataDomain principalDD = SecurityContext.getPrincipalDataDomain()
            .orElseThrow(() -> new IllegalStateException("Principal context not providing a
data domain"));
        List<String> keys = List.of(areaOrStar(area)+":"+areaOrStar(domain), areaOrStar

```

```

(area)+":*", ".*:"+areaOrStar(domain), ".*:*");
// 1) principal attached policy from PrincipalContext
DataDomain fromPrincipal = resolveFrom(policyFromPrincipal(), keys, principalDD);
if (fromPrincipal != null) return fromPrincipal;
// 2) global policy
DataDomain fromGlobal = resolveFrom(globalPolicyProvider.getPolicy().orElse(null),
keys, principalDD);
if (fromGlobal != null) return fromGlobal;
// 3) default fallback
return principalDD;
}
}

```

Integration point: - ValidationInterceptor injects DataDomainResolver and calls it in prePersist when an entity's dataDomain is null. - SecurityFilter propagates a principal's attached DataDomainPolicy (if any) into the PrincipalContext so the resolver can see it.

12.19.7. When would you want a non-global policy?

Here are a few concrete scenarios: - Centralized HR: All HR Employee records are written to a shared HR domain regardless of the team creating them. This supports a shared-service HR model without duplicating HR data per tenant. - Regulated invoices: In the Sales:Invoice domain for EU, you must write under a specific EU tenantId/dataSegment to satisfy data residency. Other Sales domains can keep default behavior. - Shared catalog: The Catalog:Item domain is a cross-tenant shared catalog maintained by a core team. Writes should go to a canonical catalog domain even when initiated by tenant-specific users. - VIP account override: A particular integration user should always write to a staging domain for testing purposes, while all others use defaults. Attach a small policy to just that credential.

12.19.8. Relation to tenancy models

The policy mechanism supports both siloed and pooled tenancy: - Siloed tenancy: Most domains default to FROM_CREDENTIAL (each tenant writes to its own partition). Only a few shared services (e.g., HR, catalog) use FIXED to centralize data. - Pooled tenancy: You may lean on FIXED policies more often to route writes into pooled/segment-specific domains (e.g., region, product line), while still enforcing read/write scoping via permissions.

Because the resolver always validates through the principal context and falls back safely, you can introduce overrides gradually without destabilizing existing flows.

12.19.9. Authoring tips

- Start with no policy and verify your default flows. Add entries only where necessary.
- Prefer specific keys (area:domain) for clarity; use wildcards sparingly.
- Keep FIXED DataDomain objects minimal and valid for your deployment (orgRefName, tenantId, and dataSegment as needed).
- Document any global policy so teams know which areas are centralized.

12.19.10. API pointers

- `CredentialUserIdPassword.dataDomainPolicy`: optional per-credential overrides (propagated to `PrincipalContext`).
- `GlobalDataDomainPolicyProvider`: holds an optional in-memory global policy (null by default).
- `DataDomainPolicyEntry.resolutionMode`: `FROM_CREDENTIAL` (default) or `FIXED`.
- `DataDomainResolver` / `DefaultDataDomainResolver`: the extension point and default behavior.

12.19.11. Ontology in Permission Rules (optional)

If you enable the ontology modules, you can author rules that constrain access by semantic relationships, not field paths. This keeps policies stable as your object model evolves.

Key idea

- Materialize edges in Mongo using `OntologyMaterializer` (e.g., `placedInOrg`, `orderShipsToRegion`).
- During rule evaluation, translate a semantic constraint like "has edge `placedInOrg` to `OrgX`" into a set of IDs, and combine that with your query.

How to use

- Preferred: author a rule with a semantic hint and let the application translate it via `ListQueryRewriter`.
- Minimal change path: publish a variable via an `AccessListResolver` and use an IN filter over `_id`.

Example (resolver + IN filter)

- Add an `AccessListResolver` that returns order IDs for which (`tenantId`, `p="placedInOrg"`, `dst=orgRefName`) exists.
- In your rule's AND filter string (query language), use: `id: ^${idsByPlacedInOrg}`

See also

- Ontology overview and examples: [Ontologies in Quantum](#)
- Integration with Morphia and multi-tenancy: [Integrating Ontology](#)

Operational notes

- Ontology is optional. Enable it per service when the config flag and dependencies are present.
- Always scope edge queries by `tenantId` sourced from `RuleContext`.
- Index edges on (`tenantId`, `p`, `dst`) and (`tenantId`, `src`, `p`) for performance.

Rule language: add `hasEdge()`

We introduce a policy function/operator to reference ontology edges directly from rules. This lets policies constrain access by semantic relationships instead of field paths.

Signature

- `hasEdge(predicate, dstIdOrVar)`
- `hasIncomingEdge(predicate, srcIdOrVar)`

Parameters

- `predicate`: String name of the ontology predicate (e.g., "placedInOrg", "orderShipsToRegion").
- `dstIdOrVar` / `srcIdOrVar`: Either a concrete id/refName or a variable resolved from RuleContext (e.g., `principal.orgRefName`, `request.region`, `${principalId}`).

Semantics

- **hasEdge**: The rule grants/filters entities (sources) for which an edge exists: (tenantId, src = entity_id, p = predicate, dst = resolvedDst).
- **hasIncomingEdge**: The rule grants/filters entities (targets) for which an edge exists: (tenantId, src = resolvedSrc, p = predicate, dst = entity_id). Useful for finding entities that are destinations of a relationship (e.g., "find locations reachable FROM an associate").
- Multi-tenant safety: tenantId is always taken from RuleContext/DomainContext.

Composition

- `hasEdge` and `hasIncomingEdge` can be combined with existing rule clauses (and/or/not) and other filters (states, tags, ownerId, etc.).

Examples

- Allow viewing Orders in the caller's org (including ancestors via ontology closure):
- allow VIEW Order when `hasEdge("placedInOrg", principal.orgRefName)`
- Restrict list to Orders shipping to a region chosen in request:
- allow LIST Order when `hasEdge("orderShipsToRegion", request.region)`
- Allow viewing Locations reachable from the current user:
- allow VIEW Location when `hasIncomingEdge("canAccessLocation", ${principalId})`



In `andFilterString` and `orFilterString`, use standard BI-API syntax: - 2-argument existence check: `hasEdge(predicate, dst)` and `hasIncomingEdge(predicate, src)`. - 3-argument qualified check: `hasEdge(predicate, dst, { edgeFilter })` and `hasIncomingEdge(predicate, src, { edgeFilter })`. - Query macros: `@asOf(timestamp)` for bitemporal validity and `@minConfidence(threshold)` for probabilistic trust. In `postconditionScript`, use the JavaScript helper counterparts: `hasEdge(predicate, dst, propsOrFn)` and `hasIncomingEdge(predicate, src, propsOrFn)`.

12.19.12. Business Scenarios & Policy Enforcement Solutions

Business Scenario 1: Healthcare Temporary On-Call Staffing & Regulatory Audits

In a hospital network, physicians hold temporary appointments (e.g. "Acting Chief of Emergency Medicine" from Monday 08:00 to Friday 17:00). Policies must guarantee zero-latency expiration without relying on deletion batch cron jobs, while preserving historical audit trails for HIPAA compliance.

Solution in Permission Rules: Using the `@asOf(ts)` macro inside a qualified edge filter ensures that access expires automatically the moment `now >= validTo`, and enables point-in-time compliance audits:

```
- name: emergency-department-chart-access
  priority: 100
  securityURI:
    header: { identity: PHYSICIAN, area: clinical, functionalDomain: charts, action:
view }
    body: { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
'*', dataSegment: '*', resourceId: '*' }
  effect: ALLOW
  andFilterString: >
    hasEdge("actingChiefOf", "DEPT-EMERGENCY", { @asOf(2026-09-20T22:00:00Z) &&
status:"ACTIVE" })
```

Business Scenario 2: Aerospace Defense Project Classification Guard

In defense environments, the association between an engineering contractor and a classified stealth program (`PROJECT-TITAN`) has a security classification of `SECRET` with an `ITAR` handling compartment. An uncleared sub-contractor querying their project dashboard must not discover that `PROJECT-TITAN` exists or is linked to their team.

Solution in Permission Rules: Using Mandatory Access Control (MAC) edge properties pushed into the compound index scan (`idx_domain_security`), the relationship is withheld at the database level:

```
- name: defense-classified-project-view
  priority: 105
  securityURI:
    header: { identity: CONTRACTOR, area: engineering, functionalDomain: projects,
action: view }
    body: { realm: '*', orgRefName: '*', accountNumber: '*', tenantId: '*', ownerId:
'*', dataSegment: '*', resourceId: '*' }
  effect: ALLOW
  andFilterString: >
    hasEdge("assignedToProject", "PROJECT-TITAN", {
      securityLabel: "SECRET" &&
      compartments: ^ ["ITAR"]
    })
```

Under the hood

- Policy evaluation resolves `dstIdOrVar` / `srcIdOrVar` against `RuleContext` (for example, `principal.orgRefName` → "OrgP").
- The list/filter query is rewritten using `ListQueryRewriter.rewriteForHasEdge(...)` or `rewriteForHasIncomingEdge(...)`, which turns the predicate and resolved ID into a set of target/source IDs and merges it with the base query efficiently.
- MongoDB compound indexes `idx_domain_p_dst_props`, `idx_domain_temporal`, `idx_domain_security`, and `idx_domain_confidence` accelerate edge lookups directly at the datastore layer.

Full end-to-end example (implementation pattern)

1) Author a rule (illustrative YAML/pseudocode)

```
- name: list-orders-by-org
  priority: 100
  securityURI:
    header:
      identity: USER
      area: sales
      functionalDomain: order
      action: list
    body:
      realm: '*'
      accountNumber: '*'
      tenantId: '*'
      dataSegment: '*'
      ownerId: '*'
      resourceId: '*'
  effect: ALLOW
  # Semantic constraint expressed via ontology helper in postconditionScript
  postconditionScript: hasEdge("placedInOrg", pcontext?.dataDomain?.orgRefName) ===
true
```

Another example using `hasIncomingEdge`:

```
- name: view-reachable-locations
  priority: 110
  securityURI:
    header:
      identity: USER
      area: core
      functionalDomain: location
      action: view
    body:
      realm: '*'
      accountNumber: '*'
      tenantId: '*'
      dataSegment: '*'
      ownerId: '*'
```

```

    resourceId: '*'
    effect: ALLOW
    # Find locations that the current user has a "canAccessLocation" edge to
    andFilterString: "hasIncomingEdge(canAccessLocation, ${principalId})"

```

2) Evaluate policy and apply constraint in the repository/list path

```

import com.e2eq.ontology.policy.ListQueryRewriter;
import com.e2eq.ontology.repo.OntologyEdgeRepo;
import com.mongodb.client.model.Filters;
import org.bson.conversions.Bson;

// Injected once per service when ontology is enabled (Quarkus CDI)
@jakarta.inject.Inject OntologyEdgeDao edgeDao;
ListQueryRewriter rewriter = new ListQueryRewriter(edgeDao);

// Inside your list method, after building the base filter
String tenantId = ruleContext.getRealmId(pctx, rctx); // or from DomainContext
String predicate = "placedInOrg";
String orgRefName = pctx.getDataDomain().getOrgRefName(); // resolves
principal.orgRefName

Bson base = Filters.and(existingFilters...);
Bson rewritten = rewriter.rewriteForHasEdge(base, tenantId, predicate, orgRefName);
var results = datastore.getDatabase().getCollection("orders").find(rewritten).
iterator();

// Example for hasIncomingEdge in code
Bson locationRewritten = rewriter.rewriteForHasIncomingEdge(locationBase, tenantId,
"canAccessLocation", principalId);

```

3) Morphia-typed queries alternative

```

// If you're using Morphia's typed Query API
Set<String> ids = edgeDao.srcIdsByDst(tenantId, "placedInOrg", orgRefName);
if (ids.isEmpty()) {
    return java.util.List.of(); // short-circuit
}
query.filter(dev.morphia.query.filters.Filters.in("_id", ids));

```

Developer requirements and checklist

- Enable ontology (optional feature): set `e2eq.ontology.enabled=true` in your app and add module dependencies.
- Provide a TBox (OntologyRegistry) and run `OntologyMaterializer` on entity changes to keep edges up to date.
- Inject `EdgeDao` as a CDI bean (`@Inject`); indexes are ensured automatically at startup.

- Always pass `tenantId` from `RuleContext/DomainContext`; never cross tenants.
- When using variables on the RHS (`dstIdOrVar`), ensure the `RuleContext` exposes them (for example, `principal.orgRefName` or `request.region`).
- Monitor provenance (`edge.prov`) and re-materialize edges when intermediate nodes change.

Notes

- `hasEdge()` is a policy function; it is evaluated before constructing database filters. It is not part of the core BI-API query grammar.
- If ontology is disabled, skip the rewrite (return base) or configure a no-op implementation so policies that include `hasEdge` are rejected early with a clear error message.

12.19.13. Label resolution SPI and `hasLabel()` in rules

Labels are a lightweight way to attach semantic markers to principals and resources, and then use them in rule scripts. Quantum provides a pluggable label resolution SPI and a script helper to check labels during policy evaluation.

Why labels? - Decouple policy from rigid fields (e.g., “VIP”, “HARD_DELETE_DISABLED”, “B2B”). - Compute labels from multiple sources (simple tags, `advancedTags`, dynamic attributes, or custom derivations).

Components

- `LabelResolver` (SPI):

```
public interface LabelResolver {
    boolean supports(Class<?> type);
    java.util.Set<String> resolveLabels(Object entity);
}
```

- `DefaultLabelResolver` (built-in): Applies to `UnversionedBaseModel`.
- Collects tags (`String[]`), `advancedTags.name`, and optionally any `dynamicAttributes` whose `isInheritable()==true`, using best-effort reflection.
- `LabelService`: Aggregates all `LabelResolver` beans via CDI and delegates to the first that supports the entity type. Also supports annotation-based extraction (see below).
- Annotations (optional, opt-in at your model):

```
@LabelSource(method = "computeLabels") // class-level method returning
Collection<String>, String[], or String
public class MyEntity { ... }

public class MyEntity {
    @LabelField // field can be Collection<String>, String[], or
    String
    private java.util.List<String> policyLabels;
}
```

```
}
```

Availability in rule scripts

During policy evaluation, RuleContext resolves labels for both principal and resource contexts and installs a script helper:

- hasLabel(label: String) → Boolean
- Returns true if the current resource context has the specified label.
- Label resolution uses LabelService (including custom resolvers and annotations) at evaluation time.

Examples (postcondition script)

```
// Allow if the resource is labeled VIP
hasLabel("VIP")

// Combine with other helpers when ontology is enabled
hasLabel("RESTRICTED") && !hasEdge("placedInOrg", "OrgX")
```

Extending labels for your domain

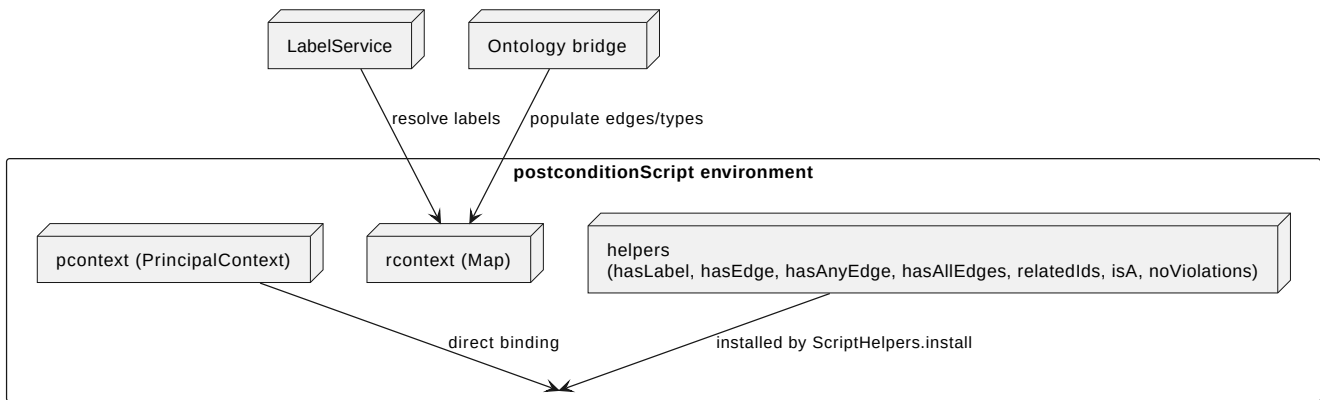
- Add a new resolver:

```
@jakarta.enterprise.context.ApplicationScoped
public class InvoiceLabelResolver implements LabelResolver {
    public boolean supports(Class<?> type) { return type.getName().endsWith("Invoice"); }
    public java.util.Set<String> resolveLabels(Object entity) {
        var out = new java.util.LinkedHashSet<String>();
        var inv = (com.example.Invoice) entity;
        if (inv.isHighValue()) out.add("HIGH_VALUE");
        if (inv.isPastDue()) out.add("PAST_DUE");
        return out;
    }
}
```

- Or annotate your model with @LabelSource/@LabelField to contribute labels without writing a resolver.

Notes - Labels are resolved best-effort; failures are swallowed to keep policy evaluation robust. - If multiple resolvers exist, the first that supports the type wins. Prefer narrow supports() checks. - Keep label names stable; treat them as part of your policy contract.

12.20. Script Helpers reference (when enabled)



Script helpers are small functions injected into the `postconditionScript` environment to make common policy checks concise. They are provided by `com.e2eq.ontology.policy.ScriptHelpers.install(...)` when the optional ontology/label modules are present. In all cases, `postconditionScript` also has direct access to:

- `pcontext`: the Java `PrincipalContext` for the caller
- `rcontext`: a lightweight map for resource-related helper data (labels, types, edges, violations) when populated by your application

Availability notes: - Labels are made available by `RuleContext` via `LabelService`. `hasLabel()` checks the resource's labels only. - Edges, types, and violations must be populated by your application into the `rcontext` map before calling `RuleContext.runScript` (typically via an ontology bridge). If absent, edge/type helpers return `false/empty`.

Helpers and examples:

- `isA(type: String) → Boolean`
- Purpose: Check whether the resource has a semantic type in `rcontext.types`.
- Example:

```
// Allow when the resource is of type "Invoice"
isA("Invoice")
```

- `hasLabel(label: String) → Boolean`
- Purpose: Check whether the resource has a policy label (labels resolved via `LabelService`).
- Example:

```
// Permit view if resource is labeled PUBLIC
hasLabel("PUBLIC")
```

- `hasEdge(predicate: String, dst: String|null) → Boolean`
- Purpose: True if an ontology edge with property `p=predicate` exists from the current resource to

dst. If dst is null, true if any dst exists for the predicate.

- Example:

```
// Require that this resource is placed in the caller's org
hasEdge("placedInOrg", pcontext?.dataDomain?.orgRefName)
```

- hasIncomingEdge(predicate: String, src: String|null) → Boolean
- Purpose: True if an ontology edge with property p=predicate exists from src to the current resource. If src is null, true if any src exists for the predicate.
- Example:

```
// Permit view if the current resource is a location reachable from the caller
hasIncomingEdge("canAccessLocation", principalId)
```

- hasAnyEdge(predicate: String, dsts: Collection<String>) → Boolean
- Purpose: True if there is at least one edge to any of the destinations.
- Example:

```
// Allow if the order ships to any of the selected regions
hasAnyEdge("orderShipsToRegion", ["NA", "EU"]) == true
```

- hasAllEdges(predicate: String, dsts: Collection<String>) → Boolean
- Purpose: True only if there is an edge for every destination in the list.
- Example:

```
// Require all compliance flags to be present
hasAllEdges("hasComplianceFlag", ["KYC", "AML"]) == true
```

- relatedIds(predicate: String) → List<String>
- Purpose: Return the list of dst ids for edges with property p=predicate.
- Example:

```
// Use in conjunction with a filter list variable (via AccessListResolver)
var projectIds = relatedIds("belongsToProject");
projectIds && projectIds.length > 0
```

- noViolations() → Boolean
- Purpose: True if rcontext.violations is empty. Helpful when upstream validators populate violations.
- Example:

```
// Deny action if any violations were detected earlier in the pipeline
noViolations() === true
```

Best practices: - Prefer simple boolean expressions; keep postconditionScript fast and side-effect free. - When using ontology helpers, always scope your edge materialization by tenantId to avoid cross-tenant leakage. - Combine helpers with SecurityURIBody scoping and repository filters for defense in depth.

Cross-references: - Ontology usage and hasEdge: see [Ontology in Permission Rules](#). - Label SPI and hasLabel: see [Label resolution SPI](#). - AccessListResolver for list variables in filters: see "AccessListResolvers (SPI) for list-based access" above.

12.20.1. Security Annotations: FunctionalMapping and FunctionalAction

Quantum uses annotations to declare a model or resource's functional area, domain, and actions for security evaluation. This replaces the legacy `bmFunctionalArea()` and `bmFunctionalDomain()` methods.

@FunctionalMapping

Use `@FunctionalMapping` on model classes or resource classes to declare their business placement:

```
import com.e2eq.framework.annotations.FunctionalMapping;

@Entity
@FunctionalMapping(area = "catalog", domain = "product")
public class Product extends BaseModel {
    // No need to override bmFunctionalArea/bmFunctionalDomain
}
```

```
@Path("/products")
@FunctionalMapping(area = "catalog", domain = "product")
public class ProductResource extends BaseResource<Product, ProductRepo> {
    // All methods inherit area/domain from class annotation
}
```

@FunctionalAction

Use `@FunctionalAction` on JAX-RS resource methods when the action differs from the HTTP verb default:

```
@Path("/products")
public class ProductResource {

    @POST
    @FunctionalAction("CREATE") // Explicit, though POST implies CREATE
```

```

public Product create(Product payload) {
    return productRepo.save(payload);
}

@GET
@Path("/{id}")
// No annotation needed - GET implies VIEW
public Product get(@PathParam("id") String id) {
    return productRepo.findById(id);
}

@PUT
@Path("/{id}/approve")
@FunctionalAction("APPROVE") // Custom action beyond standard CRUD
public Product approve(@PathParam("id") String id) {
    Product p = productRepo.findById(id);
    p.setStatus("APPROVED");
    return productRepo.save(p);
}
}

```

Default Action Mapping

When `@FunctionalAction` is not present, actions are inferred from HTTP methods:

HTTP Method	Default Action
GET	VIEW
POST	CREATE
PUT	UPDATE
PATCH	UPDATE
DELETE	DELETE

How the Framework Uses These Annotations

SecurityFilter

- Reads `@FunctionalMapping` from the matched resource class for area/domain
- Reads `@FunctionalAction` from the method, or infers from HTTP method
- Falls back to path-based parsing if annotations are missing

Scoped Action Enablement

- Reuses the same `area / functionalDomain / action` tuple as the dependency-tracking key for runtime action enablement
- Lets the framework answer more than "is this policy allowed?" and also report whether the action is feature-enabled and operationally ready

- Treats capabilities as an application-level composition concept built on top of one or more scoped actions, not as a new core security primitive

MorphiaRepo.fillUIActions

- Uses `@FunctionalMapping` on model classes to resolve allowed UI actions
- Falls back to legacy `bmFunctionalArea()/bmFunctionalDomain()` methods

PermissionResource

- Prefers `@FunctionalMapping` when listing functional domains
- Falls back to legacy methods when annotation is missing

Migration from Legacy Methods

Current (Legacy) Approach

```
@Entity
public class Product extends BaseModel {
    @Override
    public String bmFunctionalArea() {
        return "Catalog";
    }

    @Override
    public String bmFunctionalDomain() {
        return "Product";
    }
}
```

New (Recommended) Approach

```
@Entity
@FunctionalMapping(area = "catalog", domain = "product")
public class Product extends BaseModel {
    // Clean - no method overrides needed
}
```

Transitional Support

You can use both during migration: - If `@FunctionalMapping` is present, it takes precedence - If missing, legacy methods are used as fallback - Plan to remove legacy methods in future releases

Best Practices

Consistent Naming

Use lowercase, kebab-case for areas and domains:

```
@FunctionalMapping(area = "supply-chain", domain = "purchase-order")
@FunctionalMapping(area = "catalog", domain = "product")
@FunctionalMapping(area = "identity", domain = "user-profile")
```

Resource vs Model Annotations

- **Prefer model annotations** for consistency across all usage
- Use resource annotations only when the resource handles multiple model types
- Avoid duplicating annotations on both model and resource for the same entity
- Be consistent with naming because the same tuple may be used by policies, UI action derivation, and scoped action enablement manifests

Custom Actions

Define custom actions for business operations beyond CRUD:

```
@PUT
@Path("/{id}/publish")
@FunctionalAction("PUBLISH")
public Product publish(@PathParam("id") String id) { ... }

@POST
@Path("/{id}/duplicate")
@FunctionalAction("DUPLICATE")
public Product duplicate(@PathParam("id") String id) { ... }

@DELETE
@Path("/{id}/archive")
@FunctionalAction("ARCHIVE") // Soft delete vs hard DELETE
public void archive(@PathParam("id") String id) { ... }
```

Bypassing Data Scoping for System Operations

Some endpoints perform system-level operations that don't operate on tenant-scoped data entities. For example:

- Database migration operations (index creation, schema changes)
- System initialization and setup endpoints
- Administrative operations that don't query/modify tenant-scoped records

For these endpoints, permission rules may return a **SCOPED** decision with data-level constraints like `dataDomain.tenantId:${pTenantId}`. However, since there's no data entity to scope on, these constraints can't be meaningfully applied.

Use `bypassDataScoping = true` to mark such endpoints:

```

@POST
@Path("/indexes/applyAllIndexes/{realm}")
@RolesAllowed("admin")
@FunctionalMapping(area = "migration", domain = "indexes")
@FunctionalAction(value = "APPLY_ALL_INDEXES", bypassDataScoping = true)
public void applyAllIndexes(@PathParam("realm") String realm) {
    migrationService.applyAllIndexes(realm);
}

```

How It Works

When `bypassDataScoping = true`:

1. **Permission checks still apply** - The user must still be authorized (ALLOW decision) based on their roles and the permission rules
2. **SCOPED constraints are ignored** - Data-level filters (like `dataDomain.tenantId:${pTenantId}`) are bypassed because they're not applicable
3. **Audit logging** - The framework logs when data scoping is bypassed for transparency

Security Considerations



Only use `bypassDataScoping = true` for endpoints that genuinely do not operate on tenant-scoped data. Misuse could expose cross-tenant data access vulnerabilities.

Use Case	bypassDataScoping	Why
Database migrations	true	Index/schema operations don't involve tenant data
System initialization	true	Setup operations are realm-wide, not tenant-scoped
CRUD on tenant data	false (default)	Must respect tenant isolation
Report generation	false	Reports should only include user's accessible data

Example: MigrationResource

The `MigrationResource` class uses `bypassDataScoping` for all index management operations:

```

@Path("/system/migration")
@RolesAllowed({"admin"})
public class MigrationResource {

    @POST
    @Path("/indexes/applyIndexes/{realm}")
    @FunctionalMapping(area = "MIGRATION", domain = "INDEXES")
    @FunctionalAction(value = "APPLY_INDEXES", bypassDataScoping = true)
    public void applyIndexes(@PathParam("realm") String realm) {

```

```

        migrationService.applyIndexes(realm);
    }

    @POST
    @Path("/indexes/dropAllIndexes/{realm}")
    @FunctionalMapping(area = "MIGRATION", domain = "INDEXES")
    @FunctionalAction(value = "DROP_ALL_INDEXES", bypassDataScoping = true)
    public void dropIndexes(@PathParam("realm") String realm) {
        migrationService.dropAllIndexes(realm);
    }
}

```

Integration with Permission Rules

Annotations feed into permission rule matching:

```

- name: allow-catalog-reads
  priority: 300
  match:
    method: [GET]
    # Matches area/domain from @FunctionalMapping
    functionalArea: catalog
    functionalDomain: product
  rolesAny: [USER, ADMIN]
  effect: ALLOW

```

```

- name: admin-only-approval
  priority: 100
  match:
    method: [PUT]
    functionalArea: catalog
    functionalDomain: product
    # Matches @FunctionalAction("APPROVE")
    action: APPROVE
  rolesAll: [ADMIN]
  effect: ALLOW

```

See Also

- [Modeling: Functional Mapping](#)
- [Permission Rules](#)
- [DomainContext and RuleContext](#)

See also: [Permission Resource: Check APIs and Client Usage](#).

Chapter 13. 9. Seed packs: Declarative tenant seeding

Problem: Provisioning repeatable, versioned baseline data for tenants.

Why for SaaS: Every tenant must start with known-good defaults; upgrades must be idempotent and auditable.

How Quantum helps: Seed packs with manifests, datasets, transforms, includes, archetypes, and a registry.

Walkthrough: Create a seed pack, apply it programmatically, and verify idempotency.

Chapter 14. Seed packs and declarative tenant seeding

Quantum 1.2 introduces a seed-pack subsystem that lets applications publish versioned baseline content without hard-coding values in `ChangeSet` beans or maintaining a separate "seed" tenant.

14.1. Introduction

14.1.1. The problem

In multi-tenant SaaS platforms, every new tenant must start with a known-good baseline of data: code lists, roles, default settings, reference values, and sometimes product- or region-specific content. Traditionally this baseline is scattered across ad-hoc SQL/Mongo scripts, hand-written bootstrap code, or a "template" tenant that is copied forward. These approaches are hard to version, review, test, and repeat reliably across environments.

Compounding the issue, tenants evolve over time. As modules are upgraded, their baseline content must be updated too. Without a disciplined mechanism, teams risk drift between environments and tenants, brittle migrations, and non-idempotent provisioning that causes duplicates or corruption.

14.1.2. Why this needs to be solved

- Operational consistency: Provisioning should be predictable, repeatable, and safe to re-run.
- Developer velocity: Changes to baseline data should be reviewed like code and travel with the module that owns them.
- Compliance and audit: You need to know exactly which version of seed content was applied to which tenant and when.
- Composability: Different product editions or SKUs need different combinations of baseline content without forked scripts.

14.1.3. How seed packs solve it

Seed packs provide a declarative, versioned, and composable way to describe tenant baseline data:

- A manifest (`manifest.yaml`) declares datasets, natural keys, transforms, required indexes, and optional includes/archetypes.
- Datasets point to JSON/NDJSON files that are upserted using natural-key filters, making runs idempotent.
- Transforms inject tenant/realm identifiers and can rewrite references deterministically.
- Includes compose other packs with exact versions or semantic version ranges, enabling dependency management.
- Archetypes bundle a named set of packs to represent product tiers or verticals.
- A registry records checksums per dataset so unchanged data is skipped on subsequent runs.

Together, these features make seeding safe, observable, and maintainable across development, test, and production.

The next section expands on why seed packs are beneficial and how to use them effectively.

14.2. Why seed packs?

- **Versioned + reviewable:** seed packs are plain files (YAML + JSON/NDJSON) that live next to your module code. Pull requests show exactly which records changed.
- **Composable:** packs can depend on other packs and expose named *archetypes* for different product editions or verticals.
- **Pluggable sources:** load packs from the filesystem, object storage, or even a curated seed database by providing a custom `SeedSource`.
- **Tenant-aware:** transforms inject tenant identifiers and remap references before persisting.
- **Idempotent:** a `SeedRegistry` tracks checksums per dataset so provisioning can be re-run safely.

14.3. High-level flow

1. `SeedLoader` discovers manifests via the configured `SeedSource` implementations (for example the provided `FileSeedSource`).
2. A manifest (`manifest.yaml`) declares datasets, required indexes, transforms, optional includes, and archetypes.
3. During provisioning a migration invokes `SeedLoader.apply(...)` with the packs (or archetype) that should be materialised for the tenant.
4. Records are parsed, transformed, and upserted through a `SeedRepository` implementation. The default `MongoSeedRepository` writes to the tenant realm using natural-key filters.
5. The `SeedRegistry` (backed by `_seed_registry` via `MongoSeedRegistry`) records the checksum so unchanged datasets are skipped on later runs.

14.4. Seed packs vs BootstrapPacks

Seed packs and BootstrapPacks solve adjacent problems, but they are not interchangeable.

Question	Seed packs	BootstrapPacks
What they manage	Deterministic tenant data	Operational setup steps
Main representation	YAML manifest plus JSON/NDJSON datasets	Java pack definitions plus step handlers
Best use cases	Code lists, policies, defaults, baseline reference records	Calling services, validating prerequisites, external side effects, multi-step orchestration
Idempotency style	Natural keys plus dataset checksum tracking	Handler-driven behavior using apply modes and step policies

Use a seed pack when the desired end state is "these records should exist in the realm."

Examples:

- install baseline code lists

- create default security policies
- ship default surveys or catalog metadata

Use a BootstrapPack when the desired end state is "these setup actions should run in order."

Examples:

- invoke an application service to create a workspace
- call an external registration endpoint
- verify that a downstream dependency has been configured
- publish already-seeded workflow metadata to another platform

In practice, many products use both:

- seed packs establish the durable tenant data model
- BootstrapPacks orchestrate the operational work that sits around that data

See [Bootstrap Packs](#) for the runtime, SPI, startup, and admin API details.

14.5. Manifest quick reference

```
seedPack: logistics-core
version: 1.4.2
includes:
  - accounting-base@^1.1

datasets:
  - collection: codeLists
    file: datasets/codelists.ndjson
    naturalKey: [codeListName, code]
    upsert: true
    requiredIndexes:
      - name: uk_codeLists_name_code
        unique: true
        keys:
          codeListName: 1
          code: 1
    transforms:
      - type: tenantSubstitution
        config:
          tenantField: tenantId
          orgField: orgRefName
          ownerField: ownerId
          realmField: realmId

archetypes:
  - name: FulfillmentPlus
    includes:
```

- `logistics-core@^1.4`
- `shipping-defaults@~2`

14.6. Realm filtering

Seed packs can be restricted to specific realms using `includeRealms` or `excludeRealms` in the `scope` block of the manifest. This controls which realms a seed pack applies to during provisioning.

- `includeRealms` — the seed pack applies ONLY to the listed realms
- `excludeRealms` — the seed pack applies to ALL realms EXCEPT those listed
- If neither is set, the seed pack applies to all realms (default behavior)

These two fields are mutually exclusive — setting both causes a validation error at manifest load time.

Realm filtering composes with the scope `type`. The type-based check (`GLOBAL`, `PER_TENANT`, `TENANT_LIST`, etc.) runs first; if it passes, the realm filter is applied as an additional constraint.



Realm filtering requires `quantum.seed-pack.scopes.enabled=true` in your application properties. When scopes are disabled (the default), all seed packs apply to all realms regardless of scope configuration.

14.6.1. Include realms (explicit list)

Use `includeRealms` when a seed pack should only be applied to specific realms — for example, customer-specific configuration or region-specific reference data:

```
seedPack: acme-custom-config
version: 1.0.0
scope:
  type: GLOBAL
  includeRealms:
    - acme-com
    - acme-staging-com

datasets:
  - collection: codeLists
    file: datasets/acme-codes.ndjson
    naturalKey: [code]
```

This seed pack will only be applied when the target realm is `acme-com` or `acme-staging-com`. All other realms skip it.

14.6.2. Exclude realms

Use `excludeRealms` when a seed pack should apply everywhere except certain realms — for example, excluding test or system realms from production seed data:

```

seedPack: standard-permissions
version: 2.1.0
scope:
  type: GLOBAL
  excludeRealms:
    - test-system-com
    - dev-local-com

datasets:
  - collection: policy
    file: datasets/policies.ndjson
    naturalKey: [refName]
    upsert: true

```

This seed pack applies to every realm except `test-system-com` and `dev-local-com`.

14.6.3. Backward compatibility

The previously declared (but unused) `realms` field in the scope block is treated as an alias for `includeRealms`:

```

scope:
  type: GLOBAL
  realms: [acme-com]    # equivalent to includeRealms: [acme-com]

```

14.7. Programmatic usage

```

SeedLoader loader = SeedLoader.builder()
    .addSeedSource(new FileSeedSource("local", Paths.get("seed-packs")))
    .seedRepository(new MongoSeedRepository(mongoClient))
    .seedRegistry(new MongoSeedRegistry(mongoClient))
    .build();

SeedContext ctx = SeedContext.builder(realmId)
    .tenantId(tenantId)
    .orgRefName(orgRef)
    .accountId(accountId)
    .ownerId(ownerId)
    .build();

loader.apply(List.of(
    SeedPackRef.range("logistics-core", "^1.4"),
    SeedPackRef.of("oms-defaults")
), ctx);

```

Callers can also use `loader.applyArchetype("FulfillmentPlus", ctx)` to resolve an archetype defined

in any manifest.

14.8. Extensibility hooks

- Implement `SeedSource` to load manifests from custom storage (S3, Git, curated seed DB...).
- Register additional `SeedTransformFactory` instances with the builder to support bespoke transformations (for example JMESPath projections or deterministic `ObjectId` mapping).
- Swap in a different `SeedRepository/SeedRegistry` to write to alternative datastores or change the idempotency policy.

14.9. Operational tips

- Validate manifests in CI by running the loader against a disposable database.
- Keep seed pack versions aligned with module versions so upgrade paths are clear.
- Derive any `ObjectIds` deterministically from natural keys inside a transform so data can be re-applied without collisions.
- Use archetypes to model product tiers and optional modules: `TenantProvisioningService` can decide which archetype(s) to apply based on SKU.

14.10. Primary scenarios

1. Initial tenant provisioning
 - Apply one or more seed packs to bootstrap a brand-new tenant (realm) with baseline code lists, roles, and default settings.
 - Use `SeedPackRef.of("pack-name")` or `SeedPackRef.range("pack-name", "^1.4")` to control versions.
2. Updating a module to a new version
 - Publish a new seed pack version (e.g., `logistics-core 1.5.0`) with incremental dataset changes.
 - Re-run `loader.apply(...)` for the same tenant; unchanged datasets are skipped via `_seed_registry`, modified datasets are re-applied.
3. Idempotent re-apply during deployments
 - Safe to invoke on every startup/migration. Upserts are driven by `naturalKey` and `upsert: true`.
 - Keep natural keys stable; derive surrogate IDs deterministically in a transform if needed.
4. Selecting product tiers with archetypes
 - Define archetypes in a manifest to bundle multiple seed packs under a named edition.
 - Call `loader.applyArchetype("FulfillmentPlus", ctx)` to materialize the predefined stack for a tenant.
5. Composing packs with includes
 - Use includes to depend on base packs (e.g., `accounting-base@^1.1`) and extend with your

own datasets.

- Includes support exact (=1.2.3) and range (e.g., ^1.4, ~2) selectors via `SeedPackRef.parse("name@spec")`.

6. Partial refresh of specific datasets

- You can split large packs into multiple datasets and re-apply only the packs you want by passing a smaller list to `loader.apply(...)`.

7. Testing seed packs

- Add an integration test similar to `SeedLoaderIntegrationTest` that seeds into an ephemeral MongoDB and asserts collection state and `_seed_registry` entries.

14.11. Explicit examples

14.11.1. Example 1: Minimal manifest and NDJSON

```
seedPack: demo-seed
version: 1.0.0

datasets:
- collection: codeLists
  file: datasets/codeLists.ndjson
  naturalKey: [ code ]
  upsert: true
  requiredIndexes:
  - name: uk_codeLists_code
    unique: true
    keys:
      code: 1
  transforms:
  - type: tenantSubstitution
    config:
      tenantField: tenantId
      orgField: orgRefName
      accountField: accountId
      ownerField: ownerId
      realmField: realmId
```

Example NDJSON (datasets/codeLists.ndjson):

```
{"code": "NEW", "label": "New"}
{"code": "CLOSED", "label": "Closed"}
```

14.11.2. Example 2: Applying packs in code

```
SeedLoader loader = SeedLoader.builder()
```

```

.addSeedSource(new FileSeedSource("local", Paths.get("seed-packs")))
.seedRepository(new MongoSeedRepository(mongoClient))
.seedRegistry(new MongoSeedRegistry(mongoClient))
.build();

```

```

SeedContext ctx = SeedContext.builder("my-realm")
    .tenantId("tenant-123")
    .orgRefName("tenant-123")
    .accountId("acct-123")
    .ownerId("owner-123")
    .build();

loader.apply(List.of(
    SeedPackRef.of("demo-seed"),
    SeedPackRef.range("logistics-core", "^1.4")
), ctx);

```

14.11.3. Example 3: Using an archetype

```

archetypes:
- name: FulfillmentPlus
  includes:
  - logistics-core@^1.4
  - shipping-defaults@~2

```

Apply programmatically:

```

loader.applyArchetype("FulfillmentPlus", ctx);

```

14.11.4. Example 4: Exact version and includes in a manifest

```

seedPack: shipping-defaults
version: 2.3.0
includes:
- accounting-base@=1.1.2
- logistics-core@^1.5

datasets:
- collection: shippingMethods
  file: datasets/methods.json
  naturalKey: [ code ]

```

14.12. Troubleshooting

- Manifest parsing errors: Confirm manifest.yaml keys match SeedPackManifest fields; boolean

flags like `upsert` and `unique` must be proper booleans.

- Duplicate key or unique index violations: Check `naturalKey` and `requiredIndexes`; ensure transforms don't change key fields inconsistently.
- Nothing changes on re-run: The `_seed_registry` may have recorded the same checksum; bump version or change dataset content.
- File resolution issues: Ensure `FileSeedSource` base path points to the correct seed-packs directory and file names match.

14.13. How seeds are applied automatically at startup

The framework now applies seed packs via a dedicated `SeedStartupRunner`, independent of schema migrations. This runner discovers and applies the latest version of each seed pack for important realms (system/default/test) on application startup.

Key points:

- Discovery: `SeedStartupRunner` constructs a `SeedLoader` with a `FileSeedSource` pointing at the configured seed root. Set `quantum.seed.root` (for tests we default to `src/main/resources/seed-packs`). The source walks the directory tree and locates every `manifest.yaml` file.
- Selection: For each discovered seed pack name, the runner selects the latest semantic version and builds `SeedPackRef.exact(name, version)` for application.
- Execution: The runner builds a `SeedContext` for the target realm and calls `loader.apply(refs, context)`. Indexes declared in the manifest are created before data is upserted.
- Idempotency + repeatable: The `MongoSeedRegistry` stores a checksum per dataset in the realm's `_seed_registry` collection. If the checksum matches on a later run, the dataset is skipped; if it changes, the dataset is re-applied.
- Concurrency safety: The runner uses a Sherlock distributed lock per realm to prevent concurrent execution across nodes.

Configuration snippet:

```
# test profile uses a local seed root
quantum.seed.root=src/test/resources/seed-packs
# control seed runner behavior
quantum.seeds.enabled=true
quantum.seeds.apply.on-startup=true
```

14.13.1. Configuring which realms receive seeds on startup

By default, `SeedStartupRunner` applies seeds to the system, default, and test realms. You can customize this behavior using a combination of:

1. **Configuration property** (`quantum.seed-pack.apply.realms`): A comma-separated list of realm names to seed on startup.
2. **Realm flag** (`applySeedsOnStartup`): A boolean field on the Realm model/entity that, when set to `true`, includes that realm for automatic seeding.

The startup runner unions both sources: realms from the config property plus realms with the `applySeedsOnStartup` flag enabled.

Configuration examples:

```
# Explicit list of realms to seed
quantum.seed-pack.apply.realms=system-com,production-com,staging-com

# Disable config-based realms (only flag-based realms will be seeded)
quantum.seed-pack.apply.realms=none
```

Setting the flag on a Realm entity:

```
Realm realm = Realm.builder()
    .emailDomain("acme.com")
    .databaseName("acme-com")
    .domainContext(domainContext)
    .applySeedsOnStartup(true) // This realm will receive seeds on startup
    .build();
realmRepo.save(realm);
```

Or update an existing realm:

```
Optional<Realm> realmOpt = realmRepo.findByDatabaseName("acme-com", true, systemRealm
);
if (realmOpt.isPresent()) {
    Realm realm = realmOpt.get();
    realm.setApplySeedsOnStartup(true);
    realmRepo.save(realm);
}
```

Resolution logic:

- If `quantum.seed-pack.apply.realms` is configured with realm names (not "none" or empty), those realms are added to the list.
- The runner then queries the Realm collection in the system realm for all realms with `applySeedsOnStartup=true` and adds those to the list.
- The union of both sources (deduped) determines which realms receive seeds.
- If neither is configured, the default behavior applies: system, default, and test realms.

Notes:

- The Realm collection is always stored in the system realm (e.g., `system-com`).
- The `applySeedsOnStartup` flag defaults to `false` for all realms.
- This allows dynamic realm configuration without redeploying the application—just update the Realm entity in the database.

14.14. Transforms in depth

Transforms are small, composable functions that shape each dataset record just before it is written to the database. They let you keep dataset files generic and inject environment/tenant specifics or perform repeatable rewrites at apply time.

What a transform gets and returns: - Input: the current record (a Map), the SeedContext, and the Dataset definition - Output: the next record (Map) to be passed to the rest of the pipeline; return null or an empty map to drop the record

Where transforms are declared (manifest):

```
datasets:
- collection: codeLists
  file: datasets/codeLists.ndjson
  naturalKey: [ codeListName, code ]
  upsert: true
  transforms:
    - type: tenantSubstitution
      config:
        tenantField: tenantId
        orgField: orgRefName
        ownerField: ownerId
        accountField: accountNum
        realmField: realmId
    # Additional transforms can be added here and will execute in order
```

Execution semantics: - Ordering: transforms are executed top-to-bottom for each record - Short-circuit: if any transform returns null or an empty map, the record is skipped and no write occurs - Overwrite rules: a transform can set or overwrite fields on the record; when upsert=true, the final transformed record replaces the existing one matched by naturalKey - Interaction with naturalKey and indexes: transforms run before naturalKey validation and index creation; do not remove fields listed in naturalKey, otherwise an error will be thrown during write

Built-in transform types:

- tenantSubstitution: Injects identifiers from SeedContext into the record's dataDomain.* fields (tenant/org/owner/account) and sets realmId at the record root. Config keys in the manifest (defaults target dataDomain.*):
- tenantField: key inside dataDomain to receive tenantId (default: tenantId)
- orgField: key inside dataDomain to receive orgRefName (default: orgRefName)
- ownerField: key inside dataDomain to receive ownerId (default: ownerId)
- accountField: key inside dataDomain to receive account number (default: accountNum)
- realmField: root-level field to receive realmId (default: realmId)
- stringInterpolation: Performs variable substitution on string fields using {variableName} syntax. Walks through all string fields (including nested maps and lists) and replaces variable

references with resolved values. Config keys:

- fields: optional list of field names to interpolate; if omitted, all string fields are processed
- failOnMissing: if true, throws an exception when a variable cannot be resolved; if false (default), unresolved variables are left as-is

Built-in variables available from SeedContext: - `{realm}` or `{realmId}`: the realm/database identifier - `{tenantId}`: the tenant identifier - `{orgRefName}`: the organization reference name - `{accountId}`: the account identifier - `{ownerId}`: the owner identifier

Example manifest usage:

```
transforms:
- type: stringInterpolation
  config:
    fields: ["runAsUserId", "description"]
    failOnMissing: false
```

Example dataset with variables:

```
{ "refName": "adminRule", "runAsUserId": "admin@{tenantId}", "realm": "{realm}" }
{ "refName": "systemRule", "config": { "owner": "{ownerId}", "account": "{accountId}" }
```

After interpolation with tenantId="acme-corp" and realm="acme-realm":

```
{ "refName": "adminRule", "runAsUserId": "admin@acme-corp", "realm": "acme-realm" }
{ "refName": "systemRule", "config": { "owner": "owner-123", "account": "account-456" }
```

Notes and guarantees: - Optional values missing from SeedContext are simply omitted; existing record values are preserved unless you target the same field - Transforms operate on in-memory maps and cannot perform I/O by default; keep them deterministic so re-runs are idempotent - Compose multiple transforms when needed (for example: first tenantSubstitution, then a custom id computation) - To add new transform types, implement SeedTransformFactory and register it during SeedLoader.builder() with registerTransformFactory("myType", new MyFactory()); then declare - type: myType in the manifest - To add custom variables for stringInterpolation, implement SeedVariableResolver and register it with the builder using addVariableResolver(resolver) or make it a CDI bean for automatic discovery

When to use transforms (and why): - Injecting tenant/realm identity: keep datasets source-controlled and generic; inject tenant IDs at apply time (tenantSubstitution) - Deterministic IDs: derive _id or other surrogate keys from naturalKey so upserts remain stable across environments - Normalization and defaults: add missing fields, convert formats, enforce enums before write - Reference remapping: translate human-readable codes in the dataset into datastore-specific identifiers (ObjectIds, UUIDs) in a repeatable way

Practical examples

1) Tenant identity injection (built-in) Manifest snippet:

```
transforms:
- type: tenantSubstitution
  config:
    tenantField: tenantId
    orgField: orgRefName
    ownerField: ownerId
    accountField: accountId
    realmField: realmId
```

Why: keep codeLists.ndjson portable across tenants; provisioning injects the right IDs based on SeedContext.

2) Deterministic _id from natural key (custom) - Goal: ensure stable MongoDB _id across re-applies and environments, derived from codeListName+code - Approach: implement a custom transform that computes a SHA-1/MD5 hash (or any deterministic function) and sets _id

Java registration:

```
SeedLoader loader = SeedLoader.builder()
    .addSeedSource(new FileSeedSource("local", Paths.get("seed-packs")))
    .seedRepository(new MongoSeedRepository(mongoClient))
    .seedRegistry(new MongoSeedRegistry(mongoClient))
    .registerTransformFactory("deterministicId", new DeterministicIdTransform.Factory())
    .build();
```

Manifest usage:

```
transforms:
- type: tenantSubstitution
- type: deterministicId
  config:
    sourceFields: [ codeListName, code ]
    targetField: _id
    algorithm: sha1
```

Why: makes upserts resilient and allows cross-environment joins by a stable key.

3) Foreign key remapping by code (custom) - Goal: dataset uses human-readable statusCode; transform maps it to a canonical statusId - Approach: custom transform with an in-memory map or deterministic derivation

Manifest usage:

```
transforms:
```

```
- type: mapCode
  config:
    field: statusCode
    target: statusId
    mapping:
      NEW: 100
      CLOSED: 900
```

Why: keeps datasets human-friendly while persisting efficient identifiers.

4) Defaulting and sanitization (custom) - Goal: ensure missing fields get defaults and strings are trimmed/lowercased - Approach: simple custom transform that fills defaults and cleans values

Manifest usage:

```
transforms:
- type: defaults
  config:
    defaults:
      isActive: true
      locale: en_US
- type: sanitize
  config:
    trim: [ label ]
    lowercase: [ email ]
```

Why: enforces consistency without editing large datasets.

Testing transforms: - Add integration tests that seed into an ephemeral DB and assert both record shape and `_seed_registry` entries - For custom transforms, add focused unit tests for edge cases (missing fields, nulls, unexpected types)

14.14.1. Creating your own transforms (example: DropIfTransform)

Custom transforms let you implement project-specific shaping logic. You implement two small interfaces and register the type on the SeedLoader builder. Below we walk through a simple "drop the record if a field equals a value" transform used in tests, called DropIfTransform.

Overview of the SPI: - SeedTransform: executes per-record and can return a new map (continue) or null/empty (drop this record). - SeedTransformFactory: builds a SeedTransform instance from the manifest's Transform definition (provides access to type and config map).

Minimal interfaces (simplified for clarity):

```
public interface SeedTransform {
    Map<String, Object> apply(Map<String, Object> record,
                             SeedContext context,
                             SeedPackManifest.Dataset dataset);
}
```

```
public interface SeedTransformFactory {
    SeedTransform create(SeedPackManifest.Transform transformDefinition);
}
```

Implementation: DropIfTransform - Behavior: if record[field] equals a configured value, return null to short-circuit the pipeline and skip writing the record; otherwise, pass the record through unchanged.

```
package com.example.seed.transforms;

import com.e2eq.framework.service.seed.*;
import java.util.Map;
import java.util.Objects;

public final class DropIfTransform implements SeedTransform {
    private final String field;
    private final String equalsValue;

    public DropIfTransform(String field, String equalsValue) {
        this.field = field;
        this.equalsValue = equalsValue;
    }

    @Override
    public Map<String, Object> apply(Map<String, Object> record, SeedContext context,
        SeedPackManifest.Dataset dataset) {
        Object v = record.get(field);
        if (Objects.equals(Objects.toString(v, null), equalsValue)) {
            return null; // short-circuit: drop this record
        }
        return record;
    }

    public static final class Factory implements SeedTransformFactory {
        @Override
        public SeedTransform create(SeedPackManifest.Transform transformDefinition) {
            Map<String, Object> cfg = transformDefinition.getConfig();
            String field = Objects.toString(cfg.get("field"), null);
            String eq = Objects.toString(cfg.get("equals"), null);
            return new DropIfTransform(field, eq);
        }
    }
}
```

Registration on the SeedLoader builder:

```
SeedLoader loader = SeedLoader.builder()
```

```

.addSeedSource(new FileSeedSource("local", Paths.get("seed-packs")))
.seedRepository(new MongoSeedRepository(mongoClient))
.seedRegistry(new MongoSeedRegistry(mongoClient))
.registerTransformFactory("dropIf", new DropIfTransform.Factory())
.build();

```

Manifest usage:

```

datasets:
- collection: codeLists
  file: datasets/codeLists.ndjson
  naturalKey: [ code ]
  upsert: true
  transforms:
  - type: dropIf
    config:
      field: status
      equals: CLOSED

```

Notes and tips: - Validation: your factory should validate required config keys and fail fast with a clear error if missing/invalid. - Determinism: keep transforms pure and deterministic (no I/O) so seeding remains idempotent. - Short-circuit: returning null or an empty map drops the record; otherwise, the next transform in the list will receive the (possibly mutated) map. - Composition: you can chain several transforms; for example, first dropIf, then tenantSubstitution, then a custom deterministicId. - Packaging: test-only transforms can live under test sources; production transforms should be in main sources and registered where you construct the SeedLoader (for example, in a ChangeSet or a provisioning service).

14.14.2. Custom variable resolvers for stringInterpolation

The `stringInterpolation` transform supports custom variable resolution via the `SeedVariableResolver` SPI. This allows you to define application-specific variables that can be used in seed data alongside the built-in context variables.

SeedVariableResolver interface:

```

public interface SeedVariableResolver {
    /**
     * Attempts to resolve a variable by name.
     * @param variableName the name of the variable (without braces)
     * @param context the seed context
     * @return Optional containing the resolved value, or empty if not handled
     */
    Optional<String> resolve(String variableName, SeedContext context);

    /**
     * Returns the priority of this resolver. Higher priority = consulted first.
     * Default is 0. Built-in context resolver has priority -100 (lowest).
     */
}

```

```

    */
    default int priority() {
        return 0;
    }
}

```

Implementation example (environment-based resolver):

```

package com.example.seed;

import com.e2eq.framework.service.seed.*;
import jakarta.enterprise.context.ApplicationScoped;
import java.util.Optional;

@ApplicationScoped // CDI auto-discovery
public class EnvironmentVariableResolver implements SeedVariableResolver {

    @Override
    public Optional<String> resolve(String variableName, SeedContext context) {
        // Resolve variables prefixed with "env."
        if (variableName.startsWith("env.")) {
            String envName = variableName.substring(4);
            return Optional.ofNullable(System.getenv(envName));
        }
        return Optional.empty();
    }

    @Override
    public int priority() {
        return 10; // Higher than context resolver (-100)
    }
}

```

With the above resolver, you can use environment variables in seed data:

```

{"apiEndpoint": "{env.API_BASE_URL}", "adminEmail": "admin@{tenantId}"}

```

Manual registration (alternative to CDI):

```

SeedLoader loader = SeedLoader.builder()
    .addSeedSource(new FileSeedSource("local", Paths.get("seed-packs")))
    .seedRepository(new MongoSeedRepository(mongoClient))
    .seedRegistry(new MongoSeedRegistry(mongoClient))
    .addVariableResolver(new EnvironmentVariableResolver())
    .build();

```

Priority and override behavior: - Resolvers are consulted in priority order (highest first) - The first

resolver that returns a non-empty Optional wins - Custom resolvers can override built-in context values by using a higher priority - The built-in `SeedContextVariableResolver` has priority -100, so any resolver with priority ≥ 0 will be consulted first

Example: overriding a context variable:

```
@ApplicationScoped
public class TenantOverrideResolver implements SeedVariableResolver {
    @Override
    public Optional<String> resolve(String variableName, SeedContext context) {
        if ("tenantId".equals(variableName)) {
            // Override tenantId with a computed value
            return Optional.of("custom-" + context.getTenantId().orElse("unknown"));
        }
        return Optional.empty();
    }

    @Override
    public int priority() {
        return 100; // Higher than context resolver
    }
}
```

14.14.3. Seed record listeners (reacting to seed data)

The `SeedRecordListener` SPI allows application components to react when seed records are applied to the database. This is useful for synchronizing application state with seeded data, such as registering scheduled jobs, updating caches, or triggering other side effects.



The framework provides only the SPI interface. Applications must implement their own listeners for collections they care about. Listeners run within the security context established by the seed operation, so they have access to the tenant/realm context.

SeedRecordListener interface:

```
public interface SeedRecordListener {
    /**
     * Determines whether this listener should receive callbacks for the given
     * collection.
     */
    boolean appliesTo(String collection, SeedContext context);

    /**
     * Called after seed records have been successfully applied to the database.
     */
    void onRecordsApplied(SeedRecordEvent event);
}
```

```

/**
 * Returns the priority of this listener. Higher priority = invoked first.
 */
default int priority() { return 0; }

/**
 * If true, listener runs asynchronously in a separate thread.
 */
default boolean async() { return false; }
}

```

Application-level implementation example

The following example shows how an application can implement a listener to synchronize job configurations when seeds are applied. This code would live in the application, not the framework:

```

@ApplicationScoped
public class JobRunnerSeedListener implements SeedRecordListener {

    @Inject
    JobScheduler jobScheduler;

    @Override
    public boolean appliesTo(String collection, SeedContext context) {
        return "autoPublishConfig".equals(collection) ||
            "scheduledJobs".equals(collection);
    }

    @Override
    public void onRecordsApplied(SeedRecordEvent event) {
        Log.infof("Syncing %d job config records from seed to JobRunner",
            event.getRecordCount());
        for (Map<String, Object> record : event.getRecords()) {
            jobScheduler.registerOrUpdate(record);
        }
    }

    @Override
    public int priority() {
        return 100; // High priority to run early
    }
}

```

With the above listener (implemented in your application), whenever the `autoPublishConfig` collection receives seed data, the JobRunner will automatically sync its internal state with the database.

Security context

Listeners run within the security context established by the seed operation (`SeedStartupRunner` or `SeedLoaderService`). This means:

- The `SecurityContext.getPrincipalContext()` is available with tenant/realm information
- Any repository operations or service calls will execute with appropriate permissions
- Listeners can access tenant-specific data and perform tenant-aware operations

Example accessing security context in a listener:

```
@Override
public void onRecordsApplied(SeedRecordEvent event) {
    Optional<PrincipalContext> principal = SecurityContext.getPrincipalContext();
    if (principal.isPresent()) {
        String tenantId = principal.get().getDataDomain().getTenantId();
        String userId = principal.get().getUserId();
        Log.infof("Processing seeds for tenant %s as user %s", tenantId, userId);
    }
    // Process records with proper tenant context...
}
```

SeedRecordEvent properties

- `getCollection()`: The collection/entity name
- `getSeedPack()`: The seed pack that provided the data
- `getVersion()`: The seed pack version
- `getContext()`: The `SeedContext` with realm/tenant info
- `getRealm()`: Shortcut for `getContext().getRealm()`
- `getRecords()`: List of applied records (transformed, final form)
- `isUpsert()`: Whether upsert (true) or insert-only (false)
- `getRecordCount()`: Number of records applied

Registration

CDI auto-discovery (recommended):

```
// Just annotate your listener with @ApplicationScoped
// It will be automatically discovered and registered
@ApplicationScoped
public class MyAppSeedListener implements SeedRecordListener { ... }
```

Manual registration (alternative):

```
SeedLoader loader = SeedLoader.builder()
    .addSeedSource(new FileSeedSource("local", Paths.get("seed-packs")))
    .seedRepository(new MongoSeedRepository(mongoClient))
    .seedRegistry(new MongoSeedRegistry(mongoClient))
    .addRecordListener(new MyAppSeedListener())
    .build();
```

Best practices

- Listeners are invoked after records have been persisted to the database
- Listener exceptions are caught and logged; they don't interrupt the seeding process
- Async listeners (`async() = true`) run in separate threads and don't block seeding
- CDI-discovered listeners are automatically registered when using `SeedLoaderService`
- Keep listener logic fast and idempotent since seeds may be re-applied
- Use the security context to ensure tenant-aware operations

14.15. Test walkthrough: SeedLoaderIntegrationTest

The `SeedLoaderIntegrationTest` demonstrates end-to-end seeding using the demo seed pack at `src/main/resources/seed-packs/demo-seed`.

What the test does: - Creates a `SeedLoader` backed by `FileSeedSource`, `MongoSeedRepository`, and `MongoSeedRegistry`. - Builds a `SeedContext` populated with tenant/realm details to exercise the `tenantSubstitution` transform. - Applies the pack reference `SeedPackRef.of("demo-seed")`, which resolves the latest version of that pack (1.0.0 in tests). - Asserts that 2 records were inserted into the `codeLists` collection and that the `tenantSubstitution` fields were populated from the context. - Verifies an entry was written to `_seed_registry` with the dataset checksum and `records: 2`. - Re-applies the same pack and asserts the record count remains 2, demonstrating idempotency (thanks to upsert + registry checksum).

Why these design choices: - NDJSON for datasets: allows streaming large datasets and simple line-by-line diffs in code review; arrays are also supported for smaller payloads. - Natural-key upsert: manifests declare `naturalKey` to form the filter for `replaceOne(..., upsert=true)` ensuring idempotent writes and predictable overwrites. - Transform pipeline: keeps dataset files free of environment-specific values; all tenant/realm specifics are injected consistently at apply time. - Registry-based skip: checksums per dataset avoid unnecessary writes when content hasn't changed—fast, safe re-runs during deployments. - Semantic-version selection: when multiple versions of a pack are available, the latest semver is used unless an exact version is requested.

Alternatives considered: - Store seed state in an external table keyed only by version. Rejected in favor of per-dataset checksums to detect content drift without bumping versions. - Hardcode seeding logic inside ad-hoc migrations. Rejected for lack of composability and poor reviewability. - Use inserts only (no upsert). Rejected due to lack of idempotence and difficulty correcting baseline data.

14.16. Archetypes explained

Archetypes are named compositions of seed packs that model a product edition, SKU, or vertical stack. Instead of listing several packs every time you provision a tenant, you define an archetype once in a manifest and then apply it by name.

What an archetype is in this context: - It lives inside a seed pack manifest under `archetypes:`. - It contains a list of includes (same syntax as top-level includes) referring to packs and version ranges. - When applied, the loader resolves those pack refs plus the hosting pack itself (the manifest that defines the archetype) so that local datasets are included as part of the archetype. - Resolution uses semantic version rules and deduplicates by pack name, respecting dependency order and preventing cycles.

When to use archetypes: - To represent product tiers (e.g., Community, Pro, Enterprise) that bundle different combinations of base packs and optional modules. - To group verticalized defaults (e.g., Logistics-Fulfillment, Healthcare-Core) without forcing consumers to know every underlying pack.

14.16.1. Example A: Define and apply an archetype in the same pack

Manifest (logistics-core/manifest.yaml):

```
seedPack: logistics-core
version: 1.4.2

includes:
  - accounting-base@^1.1

datasets:
  - collection: codeLists
    file: datasets/codelists.ndjson
    naturalKey: [ codeListName, code ]
    upsert: true

archetypes:
  - name: FulfillmentPlus
    includes:
      - logistics-core@^1.4      # self + constraints
      - shipping-defaults@~2
```

Applying it in code:

```
SeedLoader loader = SeedLoader.builder()
    .addSeedSource(new FileSeedSource("local", Paths.get("seed-packs")))
    .seedRepository(new MongoSeedRepository(mongoClient))
    .seedRegistry(new MongoSeedRegistry(mongoClient))
    .build();

SeedContext ctx = SeedContext.builder("my-realm").build();
```

```
loader.applyArchetype("FulfillmentPlus", ctx);
```

Notes: - `applyArchetype` looks up the latest manifest that defines an archetype named "FulfillmentPlus" across all discovered packs, resolves the include refs, and then applies the union. - If multiple manifests define the same archetype name, the latest semver manifest wins.

14.16.2. Example B: Cross-pack archetype in a dedicated "editions" pack

You can centralize product definitions into a thin pack that only defines archetypes and forward-references other packs:

Manifest (product-editions/manifest.yaml):

```
seedPack: product-editions
version: 1.0.0

archetypes:
- name: Enterprise
  includes:
    - logistics-core@^1.5
    - shipping-defaults@~2
    - analytics-starter@^0.9
```

Apply in code:

```
loader.applyArchetype("Enterprise", ctx);
```

This keeps edition composition decoupled from individual module packs.

14.16.3. Resolution and ordering details

- Version matching: Each include can be exact (=1.2.3), a semver range (e.g., ^1.5, ~2), or omitted (latest). See `SeedPackRef.parse("name@spec")`.
- Deduplication: If multiple includes select the same pack name (possibly different versions), the highest version that satisfies all constraints is chosen; duplicates are applied only once.
- Dependency order: Includes are recursively resolved depth-first, while the loader guards against cycles and applies datasets in a stable order per resolved pack.

14.16.4. Interaction with `ApplySeedPacksChangeSet`

- The `Apply Seed Packs` change set scans the seed root and applies the latest version of every discovered pack to the realm. It does not automatically choose an archetype.
- Use `applyArchetype` programmatically (e.g., from a `TenantProvisioningService`) when you want to provision only the packs that belong to a specific edition.
- You can combine approaches: let migrations ensure baseline packs are present for all tenants;

then, on tenant onboarding, call `applyArchetype(...)` to add edition-specific content.

14.16.5. Tenant provisioning with archetypes

The tenant provisioning API accepts an optional list of archetype names and will apply the corresponding seed packs during onboarding.

- Endpoint: `POST /admin/tenants`
- Request body fields (subset):
- `tenantEmailDomain`, `orgRefName`, `accountId`, `adminUserId`, `adminUsername`, `adminPassword`
- `archetypes`: optional array of strings (archetype names)
- Behavior:
- After running migrations and creating the admin user, each archetype is resolved across all manifests and applied using the same `SeedLoader` used elsewhere.
- If an archetype name is unknown, the request fails with 409/500 depending on context.

Example request:

```
{
  "tenantEmailDomain": "demo-archetype.example",
  "orgRefName": "demo-archetype.example",
  "accountId": "9999999999",
  "adminUserId": "admin@demo-archetype.example",
  "adminUsername": "admin@demo-archetype.example",
  "adminPassword": "secret",
  "archetypes": ["DemoArchetype"]
}
```

On success, the new realm (`demo-archetype-example`) will have the datasets from the selected archetypes applied and recorded in the `_seed_registry`.

14.17. REST API for seed packs

The `quantum-system-service` exposes admin-only endpoints to inspect and apply seed packs per realm (tenant DB). The service opts into the `com.end2endlogic:quantum-seed-rest` adapter; application backends and SDK-only consumers should omit that module. The endpoint-free runtime lives in `quantum-seed-core`.

The endpoints are disabled to non-admin users via role checks.

Base path: `/admin/seeds`

Endpoints: - `GET /admin/seeds/pending/{realm}` - Lists pending seed packs for the realm. A pack is pending if any dataset checksum differs from the last applied or was never applied. - Optional query parameter: `filter=pack1,pack2` to restrict by pack name. - Response example:

+

```
[
  {
    "seedId": "demo-seed@1.0.0",
    "seedPack": "demo-seed",
    "version": "1.0.0",
    "datasets": [
      { "collection": "codeLists", "file": "datasets/codelists.ndjson", "checksum":
"..."}
    ]
  }
]
```

- POST /admin/seeds/apply/{realm}
- Applies the latest version of all discovered packs (or only those matching filter).
- Query parameter: filter=pack1,pack2
- Response example:

```
{"applied":["demo-seed"]}
```

- POST /admin/seeds/{realm}/{seedPack}/apply
- Applies the latest version of a single pack by name. Idempotent: unchanged datasets are skipped.
- Response example:

```
{"applied":["demo-seed"]}
```

- GET /admin/seeds/history/{realm}
- Returns the per-dataset seeding history as recorded in the `_seed_registry` collection.
- GET /admin/seeds/verify/{realm}
- Verifies all applicable seed packs for the realm.
- GET /admin/seeds/{realm}/{seedPack}/verify
- Verifies one seed pack for the realm.
- GET /admin/seeds/archetypes/{realm}
- Lists the seed archetypes applicable to the realm.
- POST /admin/seeds/archetypes/{realm}/{archetypeName}/apply
- Applies all packs included by the named archetype.

Authentication and roles: - All endpoints require role admin. In integration tests you can use `@TestSecurity(user="admin", roles={"admin"})`.

Configuration: - `quantum.seed.root`: filesystem path to the root folder where seed packs are discovered. In tests this defaults to `src/test/resources/seed-packs`. - `quantum.seed.apply.filter`: optional comma-separated list of pack names to limit automatic application by changeset.

14.18. Using MorphiaSeedRepository (Morphia-backed seeding)

Note about collection vs `modelClass`: - You can now omit collection in a dataset when you specify `modelClass`. The framework will derive the collection name from the Morphia mapping of the model class. - Derivation rules: if the model class has `@Entity` with a non-empty value, that value is used as the collection name; otherwise the simple Java class name is used. - This derived name is used consistently for logging, in the `_seed_registry` entries, and for the Mongo fallback path. - Backwards compatibility: specifying collection is still supported, and will override the derived name.

In addition to the default Mongo-based persistence, you can instruct the seeding pipeline to persist a dataset via a Morphia repository mapped to a concrete `UnversionedBaseModel`. This enables: - Automatic class discriminator fields (for example, Morphia's `_t`) and proper collection mapping. - Population of framework-managed fields (`dataDomain`, audit info, etc.) by the repository layer. - Consistent security filtering and validations applied by Morphia repos in normal runtime.

How to enable Morphia for a dataset: - Add `modelClass` to the dataset in `manifest.yaml` with the fully-qualified class name that extends `UnversionedBaseModel`. - Keep `naturalKey` and transforms as usual. The loader will still compute checksums and idempotently upsert.

Example manifest snippet:

```
seedPack: morphia-demo
version: 1.0.0

datasets:
- collection: CodeList
  file: datasets/codeList.ndjson
  naturalKey: [category, key]
  upsert: true
  modelClass: com.e2eq.framework.model.persistent.base.CodeList
  transforms:
  - type: tenantSubstitution
    config:
      tenantField: tenantId
      orgField: orgRefName
      ownerField: ownerId
      accountField: accountNum
      realmField: realmId
```

Walkthrough (based on the integration test `MorphiaSeedRepositoryIntegrationTest`): - The test builds a `SeedLoader` with `MorphiaSeedRepository` and `MongoSeedRegistry` pointing at `src/test/resources/seed-packs`. - A `SeedContext` is created for a test realm. For repository-layer

behavior (dataDomain, permissions), a minimal SecurityContext is also established in the test. - The loader applies SeedPackRef.of("morphia-demo"), which reads datasets/codeList.ndjson with two CodeList records. - The dataset declares modelClass, so MorphiaSeedRepository resolves the CodeList MorphiaRepo and attempts to save via Morphia. - If Morphia permission rules are not yet configured for the realm, MorphiaSeedRepository automatically falls back to a direct Mongo write, ensuring seeds still apply predictably but without Morphia-only fields. - The test then asserts that two documents exist in the CodeList collection and that _seed_registry has a matching history entry for the dataset checksum with records: 2.

Key behaviors and edge cases: - Indexes: When modelClass is present, ensureIndexes relies on Morphia mapping to enforce annotated indexes for the model. Any requiredIndexes declared in the manifest are still respected by the Mongo fallback. - Transforms: tenantSubstitution adds tenant context fields. When saving via Morphia, the repository adapts top-level tenant fields into dataDomain.* automatically for common models, and removes transient fields like realmId before mapping. - Permissions: If repository permissions prevent writes (for example, missing policies in a brand-new realm), MorphiaSeedRepository will log a warning and fall back to Mongo so seeding is not blocked. As you evolve policies, the Morphia path will be taken automatically on future runs. - Idempotency: Upsert semantics still honor naturalKey for both Morphia and Mongo paths, and _seed_registry records checksums so unchanged datasets are skipped.

See also: - Test source: quantum-framework/src/test/java/com/e2eq/framework/service/seed/MorphiaSeedRepositoryIntegrationTest.java - Manifest and dataset: quantum-framework/src/test/resources/seed-packs/morphia-demo - Admin APIs: /admin/seeds to list pending, apply packs, and inspect history per realm.

14.19. Dataset URLs and routing (file:// and s3://)

Starting with Quantum 1.2.2, dataset files in a manifest can be specified either as:

- Relative paths (backward compatible): resolved relative to the manifest's own location as provided by its SeedSource (filesystem or S3).
- Absolute URLs with a scheme that identifies the source: currently supported schemes are file:// and s3://. The loader automatically routes these URLs to the appropriate SeedSource at runtime.

Notes: - If the dataset value contains "://", it is treated as a URL and routed by scheme. Otherwise it is treated as a relative path. - Using URLs allows you to mix sources inside a single manifest (e.g., some datasets from the local filesystem and others from S3). - This is extensible; additional schemes can be supported by adding optional modules in the future.

Examples:

```
seedPack: mixed-demo
version: 0.1.0

datasets:
  # Relative path (resolved relative to the manifest location)
  - collection: localDefaults
    file: datasets/defaults.ndjson
```

```

naturalKey: [ key ]
upsert: true

# Absolute filesystem URL
- collection: roles
  file: file:///opt/app/seed-packs/roles.ndjson
  naturalKey: [ code ]
  upsert: true

# Absolute S3 URL
- collection: carriers
  file: s3://my-seeds/mixed-demo/0.1.0/carriers.ndjson
  naturalKey: [ code ]
  upsert: true

```

Routing behavior: - Relative path: delegated to the SeedSource that loaded the manifest (unchanged behavior). - file:// URL: handled by FileSeedSource. - s3:// URL: handled by S3SeedSource (requires the optional quantum-seed-s3 module on the classpath).

14.20. S3 Seed Source (optional module)

The framework provides an optional SeedSource backed by Amazon S3, packaged in a separate module so you can include it only when needed:

- Module: `quantum-seed-s3`
- Class: `com.end2endlogic.quantum.seed.s3.S3SeedSource`
- Purpose: Discover seed pack manifests and datasets stored in S3; optionally assume an IAM role via STS if cross-account access is required.

14.20.1. When to use S3 vs. filesystem

Use S3 when any of the following apply: - You want a single canonical location for seed content shared across services and environments. - You need to distribute seed packs across many runtimes (containers, serverless) without baking files into images. - You want to leverage S3 features: versioning, object immutability, access logging, and cross-account sharing. - CI/CD pipelines or content teams publish seed packs independently from application deployments.

Filesystem may be simpler when: - Developing locally with small packs that live inside your project. - You prefer packs bundled within the application JAR/image and don't need central distribution.

Advantages of S3 over filesystem: - Centralized distribution and caching via S3/CloudFront. - Cross-account access with STS AssumeRole. - Object versioning and retention policies for auditability. - Decouples seed content delivery from app build artifacts.

14.20.2. S3 layout conventions

S3SeedSource expects the same on-disk structure as FileSeedSource, just hosted under a bucket + optional prefix. Each seed pack version is a folder containing a manifest file and any referenced

datasets. For example:

- s3://my-seeds/seed-packs/customer/1.0.0/manifest.yaml
- s3://my-seeds/seed-packs/customer/1.0.0/customers.ndjson
- s3://my-seeds/seed-packs/logistics-core/1.4.2/manifest.yaml

Notes: - The manifest file name defaults to `manifest.yaml` but can be overridden. - Dataset file paths in the manifest are resolved relative to the manifest's key (folder).

14.20.3. Configuration and credentials

Constructor parameters for `S3SeedSource`: - `id`: Human-friendly source id (e.g., "s3-main"). - `bucket`: S3 bucket name. - `prefix`: Optional prefix under which seed packs live (e.g., "seed-packs/"). May be empty. - `manifestFileName`: Optional. Defaults to "manifest.yaml". - `region`: Optional AWS region. If omitted, AWS SDK v2 region resolution applies. - `roleArn`: Optional ARN of a role to assume via STS. If omitted/blank, the default credentials chain is used. - `roleSessionName`: Optional session name when assuming a role. Defaults to "quantum-seed". - `externalId`: Optional `ExternalId` for `AssumeRole` if the target role trust policy requires it. - `roleDuration`: Optional session duration for `AssumeRole`; defaults to 30 minutes.

Credential modes: - Default credentials chain (no `roleArn`): The AWS SDK v2 uses environment variables, profile, ECS/EC2/Lambda task role, etc. - `AssumeRole` (`roleArn` provided): The source uses STS to assume the specified role before calling S3.

IAM considerations: - The credentials (base or assumed) must allow `s3:ListBucket` on the bucket (scoped to the prefix) and `s3:GetObject` for objects under the prefix. - For cross-account usage, grant the calling account `sts:AssumeRole` on the target role, and in the role's policy grant the S3 permissions above.

Minimal target role policy example:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3:::my-seeds",
      "Condition": {"StringLike": {"s3:prefix": ["seed-packs/*"]}}
    },
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::my-seeds/seed-packs/*"
    }
  ]
}
```

14.20.4. Usage examples

Create a loader that discovers packs from S3 with the runtime IAM role or local credentials:

```
import com.e2eq.framework.service.seed.*;
import com.end2endlogic.quantum.seed.s3.S3SeedSource;
import software.amazon.awssdk.regions.Region;

SeedLoader loader = SeedLoader.builder()
    .addSeedSource(new S3SeedSource(
        "s3-main",
        "my-seeds",
        "seed-packs/",
        "manifest.yaml",
        Region.US_EAST_1,
        null, // roleArn null => use default creds / runtime role
        null,
        null,
        null))
    .seedRepository(new MongoSeedRepository(mongoClient))
    .seedRegistry(new MongoSeedRegistry(mongoClient))
    .build();
```

Assuming a cross-account role:

```
SeedLoader loader = SeedLoader.builder()
    .addSeedSource(new S3SeedSource(
        "s3-cross-account",
        "partner-seeds",
        "prod/",
        "manifest.yaml",
        Region.US_WEST_2,
        "arn:aws:iam::123456789012:role/SeedReadOnly",
        "quantum-seed-session",
        "external-id-abc123",
        java.time.Duration.ofMinutes(45)))
    .seedRepository(new MongoSeedRepository(mongoClient))
    .seedRegistry(new MongoSeedRegistry(mongoClient))
    .build();
```

Using the builder helper:

```
import com.end2endlogic.quantum.seed.s3.S3SeedSourceBuilder;

S3SeedSource s3 = new S3SeedSourceBuilder()
    .id("s3-main")
    .bucket("my-seeds")
    .prefix("seed-packs/");
```

```
.region(Region.US_EAST_1)
.build();
```

14.20.5. Defining datasets in the manifest (S3)

Manifests can reference S3 in two ways:

- Relative paths (backward compatible): when a manifest itself is loaded from S3, dataset files are resolved relative to the manifest's S3 key (folder), just like the filesystem source.
- Absolute URLs: you can specify `s3://bucket/key` URLs directly in any manifest. These are routed to S3 regardless of where the manifest was discovered from.

Example S3 manifest and dataset layout:

```
s3://my-seeds/seed-packs/customer/1.0.0/manifest.yaml
s3://my-seeds/seed-packs/customer/1.0.0/customers.ndjson
s3://my-seeds/seed-packs/customer/1.0.0/roles.json
```

Manifest file (manifest.yaml):

```
seedPack: customer
version: 1.0.0

datasets:
  - collection: customers
    file: customers.ndjson      # resolved relative to the manifest's folder in S3
    naturalKey: [ accountNumber ]
    upsert: true

  - collection: roles
    file: roles.json           # also relative to the manifest's folder
    naturalKey: [ code ]
    upsert: true

archetypes:
  - name: CustomerBase
    includes:
      - customer@=1.0.0
```

No additional configuration is required in the manifest to indicate S3; the SeedSource you configure at runtime determines where manifests are discovered and how dataset paths are resolved.

14.20.6. Troubleshooting

- Access denied: Verify credentials or the assumed role's permissions for ListBucket/GetObject on the bucket/prefix.
- Region mismatch: Provide an explicit region if your runtime's default region doesn't match the

bucket's region.

- Object not found: Confirm the prefix and that the manifest file name matches (default is manifest.yaml).
- Slow discovery: Use a tighter prefix to avoid scanning a large bucket namespace; consider S3 inventory or object tagging strategies if your repository grows significantly.

Chapter 15. 10. Migrations

Problem: Evolving schemas and data safely across environments.

Why for SaaS: Continuous delivery means frequent, incremental change with strict uptime and audit needs.

How Quantum helps: Opinionated patterns and hooks for applying safe migrations.

Walkthrough: Add and run a simple migration.

Chapter 16. Database Migrations and Index Management

This guide explains Quantum’s MongoDB migration subsystem (quantum-morphia-repos), how migrations are authored and executed, and how to manage indexes. It also documents the REST APIs that trigger migrations and index operations.

16.1. Overview

Quantum uses a simple, versioned change-set mechanism to evolve MongoDB schemas and seed data safely across realms (databases). Key building blocks:

- **ChangeSetBean**: a CDI bean describing one migration step with metadata (from/to version, priority, etc.) and an execute method.
- **ChangeSetBase**: convenience base class you can extend; provides logging helpers and optional targeting controls.
- **MigrationService**: discovers pending change sets, applies them in order within a transaction, records execution, and bumps the **DatabaseVersion**.
- **DatabaseVersion** and **ChangeSetRecord**: stored in Mongo to track current schema version and previously executed change sets.

16.2. Semantic Versioning

Semantic Versioning (SemVer) expresses versions in the form MAJOR.MINOR.PATCH (for example, 1.4.2):

- **MAJOR**: increment for incompatible/breaking schema changes.
- **MINOR**: increment for backward-compatible additions (new collections/fields that don’t break existing code).
- **PATCH**: increment for backward-compatible fixes or small adjustments.

Why this matters for migrations: - **Ordering**: migrations must apply in a deterministic order that reflects real compatibility. SemVer provides a natural ordering and clear intent for authors and reviewers. - **Compatibility checks**: the application can assert that the current database is “new enough” to run the code safely.

How `semver4j` is used: - **Parsing and validation**: version strings are parsed into a SemVer object. Invalid strings fail fast during parsing, ensuring only compliant versions are stored and compared. - **Introspection and comparison**: the parsed object exposes major/minor/patch components and supports comparisons, enabling safe ordering and “greater than / less than” checks. - **Consistent string form**: the canonical string is retained for display, logs, and API responses.

How **DatabaseVersion** leverages SemVer: - **Single source of truth**: **DatabaseVersion** stores the canonical SemVer string alongside a parsed SemVer object for logic and comparisons. - **Efficient ordering**: for fast sorting and tie-breaking, **DatabaseVersion** also keeps a compact integer encoding

of MAJOR.MINOR.PATCH as (major*100) + (minor*10) + patch (e.g., 1.0.3 → 103). This makes numeric comparisons straightforward while still recording the exact SemVer string. - Migration flow: when migrations run, successful execution records the new database version in DatabaseVersion. Startup checks compare the stored version to the required quantum.database.version to prevent the app from running against an older, incompatible schema.

Recommendations: - Always bump MAJOR for breaking data changes, MINOR for additive changes, and PATCH for backward-compatible fixes. - Keep change sets small and target a single to-version per change set to make intent clear. - Use SemVer consistently in getDbFromVersion/getDbToVersion across all change sets so ordering and compatibility checks remain reliable.

16.3. Configuration

The following MicroProfile config properties influence migrations:

- quantum.database.version: target version the application requires (SemVer, e.g., 1.0.3). MigrationService.checkDataBaseVersion compares this to the stored version.
- quantum.database.migration.enabled: feature flag checked by resources/services when running migrations. Default: true.
- quantum.database.migration.changeset.package: package containing change sets (CDI still discovers beans via type, but this property documents the intended package).
- quantum.migration.apply.realms: optional comma-separated list of additional realms to initialize during application startup. These startup migrations are evaluated before `SeedStartupRunner` runs, so realms listed here can be safely included in `quantum.seed-pack.apply.realms`.
- quantum.realmConfig.systemRealm, quantum.realmConfig.defaultRealm, quantum.realmConfig.testRealm: well-known realms used by MigrationResource when running migrations across environments. On startup, MigrationService checks whether the *system realm* database exists and is initialized; if missing or uninitialized, it runs all pending change sets and applies indexes (creating all entity collections, including credentials). Set `quantum.realmConfig.systemRealm` to your application's system database name (e.g. `system-com`) in `application.properties` so the correct database is created and initialized when dropped or empty.
- quantum.defaultSystemPassword: initial password for the system user credential when created by the AddSystemUserCredential change set (default: `test123456`). Set in `application.properties`; the system user (quantum.realmConfig.systemUserId) is created in the credentials collection so login with that user works.
- quantum.database.version: target schema version (e.g. 1.0.4). Must be at least 1.0.4 for the system user credential to be created during migration.

16.4. How change sets are discovered and executed

- Discovery: MigrationService#getAllChangeSetBeans locates all CDI beans implementing ChangeSetBean.

- **Ordering:** change sets are sorted by `dbToVersionInt`, then by priority (ascending). That ensures lower target versions apply before higher ones; priority resolves ties.
- **Pending selection:** For the target realm, `MigrationService#getAllPendingChangeSetBeans` considers a change set pending if either (a) it has never run, (b) the bean's `changeSetVersion` is greater than the last recorded one, or (c) the bean's checksum is non-null and differs from the last recorded checksum in `ChangeSetRecord`. It also compares each change set's `dbToVersion` against the stored `DatabaseVersion`.
- **Locking:** A distributed lock (Mongo-backed Sherlock) is acquired per realm before applying change sets to prevent concurrent execution.
- **Transactions:** Each change set runs within a `MorphiaSession` transaction; on success the change is recorded in `ChangeSetRecord` and `DatabaseVersion` is advanced (if higher). On failure the transaction is aborted and the error returned.
- **Realms:** Migrations run per realm (Mongo database). A change set can optionally be restricted to certain database names or even override the realm it executes against (see below).

16.5. Authoring a change set

Implement `ChangeSetBean`; most change sets extend `ChangeSetBase`.

Required metadata methods:

- `getId()`: a string id for human tracking (e.g., 00003)
- `getDbFromVersion()` / `getDbFromVersionInt()`: previous version you are migrating from (SemVer and an int like 102 for 1.0.2)
- `getDbToVersion()` / `getDbToVersionInt()`: target version after running this change (SemVer and int)
- `getPriority()`: integer priority when multiple change sets have same toVersion
- `getAuthor()`, `getName()`, `getDescription()`, `getScope()`: informational fields recorded in `ChangeSetRecord`
- `getChangeSetVersion()` [optional]: an integer you bump when the definition/logic of this change set evolves but its target database version does not. Defaults to 1.
- `getChecksum()` [optional]: a stable checksum string representing the content/logic of the change set. If provided and it changes, the change set will re-run even if versions did not change.

Execution method:

- `void execute(MorphiaSession session, MongoClient mongoClient, MultiEmitter<? super String> emitter)`
- Perform your data/index changes using the provided session (transaction).
- Use `emitter.emit("message")` to stream log lines back to SSE clients.

Optional targeting controls (provided by `ChangeSetBase`):

- `boolean isOverrideDatabase()`: return true to execute against a specific database instead of the

requested realm.

- `String getOverrideDatabaseName()`: the concrete database name to use when overriding.
- `Set<String> getApplicableDatabases()`: return a set of database names to which this change set should apply. Return null or an empty set to allow all.

Logging helper:

- `ChangeSetBase.log(String, MultiEmitter)` emits to both Quarkus log and the SSE stream.

16.6. Example change sets in the framework

Package: `com.e2eq.framework.model.persistent.morphia.changesets`

- `InitializeDatabase`
- Seeds foundational data in a new realm: counters (e.g., `accountNumber`), system Organization and Account, initial Rule and Policy scaffolding, default user profiles and security model. Uses `EnvConfigUtils` and `SecurityUtils` to derive system `DataDomain` and defaults.
- `AddAnonymousSecurityRules`
- Adds a `defaultAnonymousPolicy` with an allow rule for unauthenticated actions such as registration and contact-us in the website area.
- `AddRealms`
- Creates the system and default Realm records based on configuration, if missing.

These are typical examples of idempotent change sets that can be safely re-evaluated.

16.7. REST APIs to trigger migrations (MigrationResource)

Base path: `/system/migration`

The deployable `quantum-system-service` owns this API by selecting the `com.end2endlogic:quantum-migration-rest` adapter. Application backends and SDK-only consumers should omit that artifact so they do not expose database administration endpoints.

Security: Most endpoints require admin role; `dbversion` is `PermitAll` for introspection.

- `GET /system/migration/dbversion/{realm}`
- Returns the current `DatabaseVersion` document for the given realm, or 404 if not found.
- Example: `curl -s http://localhost:8080/system/migration/dbversion/system-com`
- `POST /system/migration/indexes/applyIndexes/{realm}`
- Admin only. Calls `MigrationService.applyIndexes(realm)` which invokes `Morphia Datastore.applyIndexes()` for all mapped entities. Use this after adding `@Indexed` annotations.
- Example: `curl -X POST -H "Authorization: Bearer $TOKEN" http://localhost:8080/system/migration/indexes/applyIndexes/system-com`

- POST /system/migration/indexes/applyIndexes/{realm}/{collection}
- Admin only. Applies indexes for one mapped collection in the realm.
- POST /system/migration/indexes/applyAllIndexes/{realm}
- Admin only. Applies all declared indexes across mapped collections, bypassing data scoping for this system-level operation.
- POST /system/migration/indexes/dropAllIndexes/{realm}
- Admin only. Drops all indexes on all mapped collections in the realm. Useful before re-creation or when changing index definitions.
- Example: `curl -X POST -H "Authorization: Bearer $TOKEN" http://localhost:8080/system/migration/indexes/dropAllIndexes/system-com`
- POST /system/migration/indexes/drop/{realm}/{collection}
- Admin only. Drops indexes for one collection in the realm.
- POST /system/migration/changeSet/execute/{realm}/{beanRefName}
- Admin only. SSE stream that executes the named change-set bean for the realm.
- POST /system/migration/initialize/{realm}
- Admin only. Server-Sent Events (SSE) stream that executes all pending change sets for the specific realm.
- Example (note -N to keep connection open): `curl -N -X POST -H "Authorization: Bearer $TOKEN" http://localhost:8080/system/migration/initialize/system-com`
- GET /system/migration/start
- Admin only. SSE stream that runs pending change sets across test, system, and default realms from configuration.
- Example: `curl -N -H "Authorization: Bearer $TOKEN" http://localhost:8080/system/migration/start`
- GET /system/migration/start/{realm}
- Admin only. SSE for a specific realm.
- Example: `curl -N -H "Authorization: Bearer $TOKEN" http://localhost:8080/system/migration/start/my-realm`

SSE responses stream human-readable messages produced by MigrationService and your change sets. The connection ends with "Task completed" or an error message.

16.8. Per-entity index management (BaseResource)

Every entity resource that extends `BaseResource<T, R extends BaseMorphiaRepo<T>>` exposes a convenience endpoint to (re)create indexes for a single collection in a realm.

- POST <entity-resource-base-path>/indexes/ensureIndexes/{realm}?collectionName=<collection>
- Admin only. Invokes `R.ensureIndexes(realm, collectionName)`.
- Use this when you want to reapply indexes for one collection without touching others.

- Example (assuming a ProductResource at /products): `curl -X POST -H "Authorization: Bearer $TOKEN" \ "http://localhost:8080/products/indexes/ensureIndexes/system-com?collectionName=product"`

16.9. Global index management (MigrationService)

MigrationService also exposes programmatic index utilities used by the MigrationResource endpoints:

- `applyIndexes(realm)`: calls Morphia Datastore.applyIndexes() for the realm.
- `dropAllIndexes(realm)`: iterates mapped entities and drops indexes on each underlying collection.

16.10. Validating versions at startup

- `MigrationService.checkDataBaseVersion()` compares the stored DatabaseVersion in each well-known realm to `quantum.database.version` and throws a `DatabaseMigrationException` when lower than required. This prevents the app from running against an incompatible schema.
- `MigrationService.checkInitialized(realm)` is a convenience that asserts DatabaseVersion exists and is $\geq$ required version; helpful for preflight checks.

16.11. Notes and best practices

- Make change sets idempotent: Always check for existing records before creating/updating indexes or documents.
- Use SemVer consistently for from/to versions. The framework computes an integer form (e.g., `1.0.3` $\rightarrow$ `103`) for ordering.
- Prefer small, focused change sets with clear descriptions and authorship.
- Use the MultiEmitter in `execute(...)` to provide progress to operators consuming the SSE endpoint.
- Apply new indexes with `applyIndexes` after deploying models with new `@Indexed` annotations; optionally `dropAllIndexes` then `applyIndexes` when changing index definitions across the board.
- Limit scope: use `getApplicableDatabases()` to constrain execution to specific databases, or `isOverrideDatabase/getOverrideDatabaseName` to target a different database when appropriate.

16.11.1. Checksum vs. from/to versions

ChangeSetBean introduces two optional mechanisms that influence whether a change set runs again after it has already executed:

- `changeSetVersion (int)`: A manual counter you increment when you intentionally want the same change set (same to-version) to run again because its logic changed or needs to reapply. Defaults to 1.

- `checksum` (String): A content-derived fingerprint of the change set's implementation. If provided, the framework will re-run the change set whenever the checksum differs from the last recorded value, even if `versions` and `changeSetVersion` are unchanged.

How they complement `getDbFromVersion`/`getDbToVersion`:

- `getDbFromVersion`/`getDbToVersion` define the database version boundary the change set moves the realm across. They control ordering and whether the database version should advance on success.
- `changeSetVersion`/`checksum` control repeatability of a change set at a fixed to-version. They do not change ordering; they only indicate that the change set should be applied again.

Typical usage patterns:

- Minor logic refactor that should re-apply: bump `changeSetVersion`, or update `checksum` if you compute it from code/resources.
- Data or index definition tweaked without a version bump: provide `getChecksum()` that reflects the relevant definitions (e.g., hash of DDL/index specs or embedded dataset), so the system auto-detects changes and re-runs.
- Actual schema version change: bump `getDbToVersion` (and possibly from) as usual; you may leave `changeSetVersion` and `checksum` alone.

Recording and behavior in `ChangeSetRecord`:

- On each execution, `MigrationService` stores `changeSetVersion` and `checksum` alongside from/to versions.
- A change set is considered pending if no record exists, or the bean's `changeSetVersion` is greater than the recorded one, or the bean's `checksum` is non-null and differs from the recorded `checksum`.
- After a successful run, `DatabaseVersion` is updated to `dbToVersion` if it is higher than current.

Caveats:

- If you return null from `getChecksum()`, only first-run and `changeSetVersion` increases can trigger re-execution.
- Ensure your `checksum` is stable across processes and deterministic for the same logic; do not include timestamps or environment-specific data.
- When migrating large datasets, prefer idempotent logic so repeated runs are safe even if `checksum` forces a re-run.

Chapter 17. 11. Testing

Problem: Validating behavior quickly and reliably, including tenant-specific behavior.

Why for SaaS: Isolation, entitlements, and data variance require strong test coverage.

How Quantum helps: Test utilities, runtime hooks, and integration testing guidance.

Walkthrough: Write integration tests for APIs and seed packs.

17.1. Testing in Quantum: Security Contexts, Repos, and REST APIs

17.1.1. Testing Framework

This guide explains how to write effective tests in the Quantum framework, with a focus on setting the security context and choosing the right Quarkus testing patterns for repository logic vs. REST APIs.

It covers four practical patterns you can use to establish the security context in tests: - Extending `BaseRepoTest` - Using a try-with-resources `SecuritySession` - Using a scoped-call pattern to wrap work under a `SecuritySession` - Using the `@TestSecurity` annotation

In addition, it outlines how to test REST endpoints vs. repository logic and highlights useful features of the Quarkus Test Framework.

17.1.2. Prerequisites and glossary

- `RuleContext`: the in-memory rule engine configuration used by authorization checks.
- `PrincipalContext` (`pContext`): who is performing the action (`userId`, roles, realms, data domain).
- `ResourceContext` (`rContext`): what is being acted upon (`area/domain`, resource, action).
- `SecuritySession`: a small utility that binds `PrincipalContext` and `ResourceContext` to `SecurityContext` for the current thread, and clears them on close.
- `SecurityIdentity`: Quarkus' current identity (used by `@TestSecurity` and HTTP request security).

17.1.3. Pattern 1 — Extend `BaseRepoTest`

For repository tests (no HTTP), the simplest approach is to extend `BaseRepoTest`.

What `BaseRepoTest` does for you: - Initializes `RuleContext` with default rules. - Builds default `pContext` and `rContext` for a system/test user. - Ensures test database migrations are applied. - Gives you protected fields `pContext` and `rContext` you can use in your test.

Example:

```
@QuarkusTest
```

```

class MyRepoTest extends com.e2eq.framework.persistent.BaseRepoTest {

    @Inject
    com.e2eq.framework.model.persistent.morphia.UserProfileRepo userProfileRepo;

    @Test
    void canReadUnderTestUser() {
        // Activate the security context for this block
        try (final com.e2eq.framework.securityrules.SecuritySession ignored =
            new com.e2eq.framework.securityrules.SecuritySession(pContext, rContext))
        {
            var list = userProfileRepo.list(testUtils.getTestRealm());
            org.junit.jupiter.api.Assertions.assertNotNull(list);
        }
    }
}

```

Notes: - BaseRepoTest prepares contexts but does not keep them permanently active. Wrap repo calls that require authorization in a SecuritySession (see Pattern 2), or activate it in @BeforeEach if many test methods use it. - BaseRepoTest also runs migrations once using your test principal, which avoids authorization failures during initialization.

17.1.4. Pattern 2 — Try-with-resources SecuritySession

Use SecuritySession explicitly to scope work that should run under a specific PrincipalContext and ResourceContext. This is the most explicit pattern and works in both repository and service-level tests.

Example:

```

@QuarkusTest
class CredentialRepoTest extends com.e2eq.framework.persistent.BaseRepoTest {

    @Inject
    com.e2eq.framework.model.persistent.morphia.CredentialRepo credRepo;

    @Test
    void findByUserId_asTestUser() {
        try (final com.e2eq.framework.securityrules.SecuritySession s =
            new com.e2eq.framework.securityrules.SecuritySession(pContext, rContext))
        {
            var op = credRepo.findByUserId(testUtils.getTestUserId(), testUtils
                .getSystemRealm());
            org.junit.jupiter.api.Assertions.assertTrue(op.isPresent());
        }
    }
}

```

Tips: - Prefer try-with-resources so contexts are always cleared, even when assertions fail. - You can

construct custom `PrincipalContext/ResourceContext` for specific scenarios (e.g., different roles or realms) and pass them to `SecuritySession`.

17.1.5. Pattern 3 — Scoped-call wrapper (`ScopedCallScope`)

If you prefer not to repeat `try-with-resources` blocks, wrap your work in a helper that creates a `SecuritySession`, runs your logic, and ensures cleanup. This is sometimes called a “scoped call” pattern, often referred to as a `ScopedCallScope`.

Example helper (placed in test sources):

```
public final class SecurityScopes {
    private SecurityScopes() {}

    public static <T> T call(
        com.e2eq.framework.model.securityrules.PrincipalContext p,
        com.e2eq.framework.model.securityrules.ResourceContext r,
        java.util.concurrent.Callable<T> work) {
        try (final com.e2eq.framework.securityrules.SecuritySession s =
            new com.e2eq.framework.securityrules.SecuritySession(p, r)) {
            try { return work.call(); }
            catch (Exception e) { throw new RuntimeException(e); }
        }
    }

    public static void run(
        com.e2eq.framework.model.securityrules.PrincipalContext p,
        com.e2eq.framework.model.securityrules.ResourceContext r,
        Runnable work) {
        try (final com.e2eq.framework.securityrules.SecuritySession s =
            new com.e2eq.framework.securityrules.SecuritySession(p, r)) {
            work.run();
        }
    }
}
```

Usage:

```
var result = SecurityScopes.call(pContext, rContext, () -> repo.getByUserId(realms,
userId));
SecurityScopes.run(pContext, rContext, () -> repo.save(realms, entity));
```

This achieves the same effect as `try-with-resources` but centralizes the pattern.

17.1.6. Pattern 4 — `@TestSecurity` annotation (`Quarkus`)

For tests that run through HTTP (and in some repo tests), you can use Quarkus’ `@io.quarkus.test.security.TestSecurity` to set the `SecurityIdentity` without creating a `SecuritySession`.

Quantum integrates with this in two places: - `SecurityFilter`: when a request has a `SecurityIdentity` but no JWT, it builds a `PrincipalContext` from the identity (user and roles). You can also pass `X-Realm` to control the realm. - `MorphiaRepo`: when repo methods are invoked under `@TestSecurity` with no active `SecuritySession`, `MorphiaRepo` lazily builds `PrincipalContext` from `SecurityIdentity` and sets a safe default `ResourceContext` to enable rule evaluation.

Example (HTTP):

```
@QuarkusTest
class SecureResourceTest {

    @Inject com.e2eq.framework.util.TestUtils testUtils;

    @Test
    @io.quarkus.test.security.TestSecurity(user = "test@system.com", roles = {"user"})
    void listProfiles_asUser() {
        io.restassured.RestAssured.given()
            .header("X-Realm", testUtils.getTestRealm())
            .when().get("/user/userProfile/list")
            .then().statusCode(200);
    }
}
```

Example (repo call under `@TestSecurity` fallback, no `SecuritySession`):

```
@QuarkusTest
class RepoFallbackTest {

    @Inject com.e2eq.framework.util.TestUtils testUtils;
    @Inject com.e2eq.framework.model.persistent.morphia.CredentialRepo credentialRepo;

    @Test
    @io.quarkus.test.security.TestSecurity(user = "test@system.com", roles = {"user"})
    void repoUsesIdentityWhenNoSecuritySession() {
        // Internally, MorphiaRepo will ensure PrincipalContext exists using
        // SecurityIdentity
        credentialRepo.findById("nonexistent@end2endlogic.com", testUtils.
            getTestRealm(), false);
        // Optionally assert that SecurityContext has been initialized
        org.junit.jupiter.api.Assertions.assertTrue(
            com.e2eq.framework.model.securityrules.SecurityContext.getPrincipalContext()
                .isPresent());
    }
}
```

Notes: - `@TestSecurity` is perfect for authorizing requests in HTTP tests without generating JWTs. - For repo tests that require precise `ResourceContext` (area/domain/action), prefer `SecuritySession`; `MorphiaRepo` sets a generic default `ResourceContext` when needed.

17.1.7. Testing REST APIs vs. Repository Logic

When to prefer REST (HTTP) tests: - End-to-end authorization: validate request filters, identity mapping, realm headers, and JWT handling. - Request/response shape and status codes. - Role-based access checks via `@TestSecurity`.

How to test REST APIs: - Use `@QuarkusTest` and `RestAssured`: [source,java] ---- `var resp = io.restassured.RestAssured.given().header("Content-Type", "application/json").header("X-Realm", testUtils.getTestRealm()).when().get("/user/userProfile/list").then().statusCode(200).extract().response();` ---- - To test JWT-protected endpoints end-to-end, first call the login API to obtain a token, then pass `Authorization: Bearer <token>`. See `SecurityTest.testGetUserProfileRESTAPI` for a complete example.

When to prefer repository/service tests: - You want precise control over `PrincipalContext/ResourceContext` and rule evaluation without HTTP overhead. - You are asserting persistence logic, query filters, or domain rules.

How to test repository logic: - Extend `BaseRepoTest` (Pattern 1) for ready-to-use `pContext/rContext` and migrations. - Wrap calls with `SecuritySession` (Pattern 2) or use a scoped-call helper (Pattern 3).

17.1.8. Useful Quarkus Test features

- `@QuarkusTest`: boots the app for integration tests with CDI, config, and persistence.
- `RestAssured`: fluent HTTP client baked into Quarkus tests; supports JSON assertions and extraction.
- `@TestSecurity`: set `SecurityIdentity` (user, roles) for tests.
- `@InjectMock/@InjectSpy` (quarkus-junit5-mockito): replace beans with mocks/spies for isolation.
- `@QuarkusTestResource`: manage external resources (e.g., starting/stopping containers) for a test class or suite.
- `@TestHTTPEndpoint` and `@TestHTTPResource`: convenient endpoint URI injection.

17.1.9. Real-world tips

- Clearing thread locals: If you manipulate `SecurityContext` directly in advanced tests, clear it in `@AfterEach` to avoid cross-test leakage: [source,java] ---- `@AfterEach void cleanup() { com.e2eq.framework.model.securityrules.SecurityContext.clear(); }` ----
- Realm routing: pass X-Realm in REST tests to select the target realm. `SecurityFilter` also validates realm access against user credentials when present.
- Data prep: If your test needs specific users/roles, create them under a `SecuritySession` beforehand (see `SecurityTest.ensureTestUserExists()`).
- Logging: enable `DEBUG` for `com.e2eq` to inspect rule evaluation and identity resolution during tests.

17.1.10. Summary

- Use `BaseRepoTest` for repository tests and migrations, and wrap work in `SecuritySession`.

- For less ceremony, create a simple scoped-call helper to run code under a `SecuritySession`.
- For REST/API tests and quick identity setup, use `@TestSecurity`, realm headers, and `RestAssured`.
- For full e2e security, obtain a JWT via the login API and include it in requests.

Chapter 18. 12. Next steps

- Explore the Supply Chain sample tutorial for an end-to-end scenario.
- Use the Reference Guide to quickly jump to deeper explanations of any topic above.